

Nozioni di base

GSTRUMSVIL - Guida allo sviluppo in BC - © SISTEMI S.p.A.

- [Elementi sintattici di base](#)
- [I Tipi Dato](#)
- [Le Classi e gli Oggetti](#)
- [La struttura dei programmi](#)
- [Utilizzo di commenti nel sorgente](#)
- [Creazione primo programma BC](#)
- [Strumenti di analisi e di debug](#)
- [Esercitazioni di approfondimento \(1\)](#)
- [Legacy](#)

Elementi sintattici di base

Il linguaggio BC è un metalinguaggio che storicamente nasce come derivazione dal BASIC con lo scopo di semplificare la produzione del codice.

Consente un livello di astrazione rispetto ai linguaggi più comuni presenti sul mercato e al DBMS(Database Management System) utilizzato, semplificando lo sviluppo e dando la possibilità di scrivere sorgenti che danno vita ad applicazioni multiplatforma.

È un linguaggio orientato alla programmazione ad oggetti ed è in costante evoluzione.

Il linguaggio BC consente la gestione di tutti gli elementi di cui un programma deve poter fa uso: gestione dell'interfaccia utente e dei controlli a video che la possono comporre:

- accesso alla base dati;
- accesso al file system;
- produzione di stampe grafiche e tabulari
- interfacciamento con scanner
- interazione con Word ed Excel
- utilizzo di protocolli FTP e HTTP

Il sorgente BC non è altro che un file di testo con estensione .BC. Questo, una volta completato, deve essere tradotto in sorgente nativo (C o C#) e successivamente compilato. Il successivo Link del Main o della DLL a cui è associato dà origine al programma eseguibile vero e proprio.

Le specifiche BC

Le specifiche BC sono le istruzioni del linguaggio e hanno il seguente formato:

BC
<code>'@SPECIFICA PAR1[] PAR2[] PAR3[] ... PARN[]</code>

dove:

'@ è fisso ed obbligatorio

SPECIFICA indica il nome della specifica BC

PAR1 [] ... PARN [] sono i parametri previsti dalla specifica per il suo utilizzo, per ogni specifica sono disponibili parametri di input e/o di output e per ogni parametro è definito se si tratta di un dato obbligatorio o facoltativo. L'indicazione di un parametro facoltativo viene indicata sul manuale racchiudendo tra quadre il parametro stesso.

Per le specifiche che prevedono questa sintassi i parametri devono essere indicati nella forma e sequenza prevista; diversamente possono essere segnalati errori in traduzione del sorgente oppure malfunzionamenti in esecuzione.

Un specifica BC può essere scritta su più righe del sorgente utilizzando il carattere di continuazione "_" (underscore), è molto utile quando una specifica prevede molti parametri.

In questo modo, con una scrittura del sorgente più ordinata, viene migliorata la leggibilità del programma scritto in BC.

Esempio

BC

```
'@SPECIFICA PAR1 [ ] _
      PAR2 [ ] _
      PAR3 [ ] _
      ... _
      PARN [ ]
```

La linea di continuazione può essere indentata a piacimento poiché i caratteri " " (spazio) a sinistra del parametro sono considerati come non significativi sono ignorati. Per questo motivo il carattere di continuazione "_" (underscore) non può essere inserito nel nome di una costante alfanumerica come ultimo carattere.

Concatenare più istruzioni

Due specifiche BC non possono essere inserite sulla stessa riga di codice.

È invece possibile farlo con istruzioni e funzioni base, solo in tale caso è possibile concatenare più istruzioni sulla stessa riga separate dal carattere ":" (due punti).

Esempio

BC - CBAS_67

```
' Concatenare più istruzioni
varA = 0 : varB = 0
varC = varA + 1 : Elabora()
```

Questa forma non è facilmente leggibile e non consente un uso efficace dei commenti, è quindi consigliabile un utilizzo limitato.

Le variabili e i tipi di dato base

Nel linguaggio BC, una variabile è descritta mediante il nome che la identifica, il tipo del dato destinata a contenere e la sua dimensione.

I nomi delle variabili devono iniziare con una lettera alfabetica e devono contenere soltanto caratteri alfabetici, numerici o il carattere "_" (underscore) purché non in ultima posizione.

Dichiarazione e inizializzazione

In BC la dichiarazione di una variabile ha la seguente sintassi:

```
DIM <NomeVar> [<Tipo>]
```

dove <NomeVar> è il nome della variabile e <Tipo> è il tipo della variabile.

I tipi delle variabili gestite dal BC sono i seguenti:

- Stringa
- Intero
- Double
- Periodo
- Data
- Blob (obsoleto)



Per ulteriori informazioni sulla sintassi è possibile consultare la scheda [DIM](#) del Manuale di riferimento del Linguaggio BC.

L'inizializzazione della variabile può essere fatta direttamente sulla dichiarazione della variabile. Ma non è obbligatoria.

L'ambito di utilizzo della variabile può essere locale o globale di sorgente in base all'utilizzo. Lo vedremo nel dettaglio più avanti.

Esempi di dichiarazione e inizializzazione delle variabili:

BC - CBAS_67

```
DIM stringaUnno[STRING] = ""      ' Alfabetico inizializzato a stringa vuota
DIM posiz[INT]           = 0      ' Numerico intero inizializzato a 0
DIM importo[DOUBLE]     = 1.2    ' Numerico double inizializzato a 1,2
DIM perCorr[PERIOD]     = 22007   ' Periodo inizializzato a 02/2007
DIM dataOr[DATE]        = 26071966 ' Data inizializzata a 26/07/1966
DIM stringaDue[STRING]   ' Alfabetico non inizializzato
```

Esempi di nomenclatura delle variabili non ammessa:

- CodCl_ (carattere "_" (underscore) in ultima posizione non consentito)
- Rag-Soc (carattere "-" (trattino) non consentito)
- 2ndoFile (primo carattere numerico non consentito)

 L'assegnazione di un valore con decimali ad una variabile numerica (es: PercIVA=11,5) deve sempre prevedere l'indicazione della parte intera del valore, anche quando si tratta di un valore inferiore ad 1 (es: PercIVA =0,5). Non è consentito omettere lo zero della parte intera del numero (es: PercIVA =,25 non è consentito).



Per maggiori informazioni sugli standard da applicare al nome delle variabili è possibile consultare la scheda "Variabili" degli standard di sviluppo sulla *Guida allo sviluppo in BC*.

Variabili di tipo Data

Le date sono espresse con 8 cifre quando sono previste 4 cifre per definire l'anno.

Esempio

20/12/1968 data a 8 cifre

Tutte le variabili di tipo Data devono contenere date espresse nella forma GGMMAAAA.

Per le variabili di tipo Data derivate dalla lettura di un record l'operazione di conversione dal formato AAAAMMGG al formato GGMMAAAA è eseguita automaticamente dall'ambiente di esecuzione al momento della lettura del campo a cui la variabile fa riferimento.

 È consentito assegnare un campo di tipo Data con un valore costante, il valore numerico da assegnare deve essere composto necessariamente nel formato GGMMAAAA.

Esempio di assegnazione della data 20/12/1968 alla variabile varData1:

```
DIM varData1[DATE] = 20121968
```

Ma è preferibile utilizzare l'apposito metodo di creazione della data:

```
DIM varData1[DATE] = BCDATE.CreateDate(1968, 12, 20)
```

Data corrente

Per ottenere la data di sistema è disponibile il metodo Getdate() della classe statica di linguaggio BCSystemInfo.

Esempio

BC - CBAS_67

```
DIM varData1[DATE] = BCSystemInfo.GetDate()  
DIM varDataStr[STRING] = varData1.ToString()
```

Variabili di tipo Periodo

Allo stesso modo che per i campi di tipo Data sono gestiti i campi di tipo Periodo che rappresentano una data espressa nel formato MMAAAA (mese+anno).

Esempio

```
DIM Periodo1[PERIOD]=122013
```

Le interazioni tra le variabili di tipo Periodo e le specifiche BC di manipolazione delle date sono assimilabili a quanto detto in precedenza sulle variabili di tipo Data.

Confronto e test sulle variabili di tipo Data e Periodo

In tutti i test (*If/Then/Else*) tra variabili di tipo Data, non è necessario invertire il formato della data da GGMMAAAA in AAAAMMGG prima dell'esecuzione del test in quanto ciò viene fatto automaticamente dall'ambiente di esecuzione.

Esempio

BC - CBAS_67

```
If varData1 <= varData2 Then  
  
EndIf
```

Operazioni su dati numerici

Nel linguaggio BC le operazioni aritmetiche hanno la stessa sintassi del linguaggio BASIC, le operazioni consentite sono:

- prodotto (*)
- divisione (/)
- somma (+)
- sottrazione (-)
- incremento (+=)
- decremento (-=)
- moltiplicatore (*=)
- divisore (/=).

Come già per il linguaggio BASIC esiste una priorità di esecuzione ereditata dalla matematica classica, sono prima effettuate moltiplicazioni e/o divisioni per poi essere eseguite le somme e le sottrazioni.

Durante la stesura del codice è ovviamente possibile stabilire con quale priorità eseguire più operazioni aritmetiche utilizzando la coppia di caratteri "(" e ")" (parentesi aperta e chiusa) per racchiudere porzioni di un calcolo con diversi argomenti.

 L'espressione aritmetica ammessa dal BC può essere un numero, una variabile numerica o un'espressione tra numeri, variabili e funzioni annidate con eventuali parentesi.

 Gli operatori di assegnazione di addizione (incremento) e di sottrazione (decremento) hanno la particolarità di poter scrivere in maniera contratta un'espressione. Non è possibile utilizzare questi operatori con variabili di tipo Data e Periodo.

Esempio

BC - CBAS_67

```

DIM varX[INT] = 1
DIM varY[INT] = 1

'                               equivalente
varX = varX + 1                :   varX += 1
varX = varX - (varY + 1)      :   varX -= (varY + 1)
varX = varX * varY            :   varX *= varY
varX = varX / varY            :   varX /= varY

```

Operazioni su dati alfanumerici

Le variabili di tipo alfanumerico (stringhe e blob) possono essere tra loro concatenate e possono essere manipolate per modificarne i contenuti.

Nel concatenamento di variabili di tipo stringa è possibile indicare anche un'espressione complessa, purché il risultato di ciascun addendo sia una stringa e non compaiano, all'interno dell'espressione, istruzioni che iniziano con '@'.

E' possibile utilizzare la forma contratta utilizzando l'operatore di assegnazione di addizione (+=).

Esempio

BC - CBAS_67

```

varStringa1 = varStringa1 + "ABC" + varStringa2
varStringa1 += "ABC" + varStringa2

```

I Metodi

La dichiarazione di una variabile comporta non solo la creazione di una porzione di memoria in cui andare a memorizzare il valore, ma anche la possibilità di poter manipolare questo valore tramite dei metodi associati al tipo di dato base cui è riferita la variabile.

Vedremo più avanti cosa intendiamo per metodo, ora prendiamo atto che esistono e che ci aiutano a manipolare il nostro dato.

Questi metodi sono delle funzioni fornite dall'Ambiente di sviluppo e richiamabili tramite la sintassi <variabile>.<Metodo>.

Esistono metodi differenti per ogni tipologia di tipo di dato base.

Un esempio di metodi può essere:

- per tipi di dato STRING:
 - .SubString(Start, [Count]): restituisce una sottostringa a partire da una determinata posizione e, opzionalmente, per un certo numero di caratteri.
 - .Trim(): rimuove gli spazi iniziali e finali della stringa di partenza
- per i tipi di dato INT:
 - .ToString(): converte il numero in stringa usando la formattazione indicata

Tutti i metodi non modificano direttamente il valore, ma è necessario assegnare il risultato ad una variabile.

Esempio

BC - CBAS_67

```
' Dichiarazione variabile di tipo STRING
DIM varStringa[STRING] = "Abcd"

' Estraggo 2 caratteri a partire dal secondo della variabile VarStringa
DIM varSubString[STRING] = varStringa.SubString(2,2)

MESSAGEBOX(#INFO, "Metodo SubString", varSubString)

' Verrà visualizzato "bc"
```



Per ulteriori informazioni sull'elenco dei metodi dei tipi di dato base è possibile consultare le schede relative ai metodi dei tipi base del *Manuale di riferimento del linguaggio del BC*:

- [metodi per la gestione delle date](#)
- [metodi per la gestione delle stringhe](#)
- [metodi per la gestione dei numerici](#)

Gestione informazioni di contesto

Il linguaggio BC rende disponibile un insieme di classi e metodi statici di Linguaggio che servono a fornire al programma alcune informazioni utili legate al contesto di esecuzione, al sistema o all'accesso alla base dati.

Le classi al momento disponibili sono: [BCContextInfo](#), [BCSytemInfo](#) e [BCDatabaseInfo](#).

I metodi disponibili sono elencati nella specifica scheda di manuale e possono essere richiamati usando la sintassi <Classe>.<Metodo>.

 Queste classi sono disponibili con la versione di ambiente 31.1

I Tipi Dato

Nel linguaggio BC, oltre ai tipo di dato base (Numerico, Carattere, Data, ...), esistono anche i Tipi Dato.

Lo scopo principale del Tipo Dato è quello di centralizzare in un unico punto la definizione di determinate informazioni utili a semplificare e velocizzare lo sviluppo e la manutenzione delle applicazioni.

Esistono due tipologie di Tipi Dato:

- Tipo Dato di Ambiente;
- Tipo Dato applicativo.

I Tipi Dato di Ambiente

I Tipi Dato di Ambiente sono messi a disposizione dall'Ambiente di Sviluppo e sono dei tipi generici utili per la dichiarazione immediata di variabili riferite ad un tipo base senza dover quindi usare variabili semplici.

Alcuni esempi di Tipi Dato di Ambiente possono essere:

- BCBOOL: per poter gestire i booleani;
- BCFILE: per la gestione di un file ;
- BCFOLDER: per la gestione della cartella

Per poter utilizzare questi Tipi Dato è necessario importarli nell'area di sviluppo (prodotto/progetto) di riferimento creando i relativi oggetti in AGIS tramite la funzione "Importa tipo dato d'ambiente".



Per conoscere quali sono i Tipi Dato di ambiente disponibili è possibile effettuare una ricerca "tipi dato" (comprese le "") sul *Manuale di riferimento del linguaggio BC*.

Il Tipo Dato booleano

Nel linguaggio BC, non esiste un tipo di dato base che rappresenti il valore logico di una preposizione (true o false) per cui è stato definito direttamente dall'Ambiente di sviluppo.

Tecnicamente, il booleano, fa riferimento ad un intero, ma nella scrittura dei sorgenti è ammessa la sintassi che contraddistingue questo tipo di dato.

La dichiarazione è del tutto simile a quella vista in precedenza:

BC

```
' Dichiarazione
DIM esisteCliente[TIPO[BCBOOL]]
```

Una volta dichiarata può essere utilizzata per definire espressioni utilizzabili ad esempio in costrutti If/Then/Else:

BC

```
' Test
If esisteCliente Then
```

Come si può notare si può usare la forma contratta, senza specificare quindi il termine di confronto. Il test sarà verificato se la variabile assume il valore 1 (Vero).

Si può usare anche la forma negata utilizzando la keyword Not:

BC

```
' Test
If Not esisteCliente Then
```


Oppure assegnazioni dirette utilizzando i valori enumerate #True e #False:

BC

```
' Assegnazione
esisteCliente = #False
esisteCliente = #True

' Inversione di valore
esisteCliente = Not esisteCliente
```

Tipo Dato applicativo

Il Tipo Dato applicativo è una entità che viene creata dal programmatore partendo da un tipo di dato di base e su cui è possibile aggiungere delle caratteristiche e delle funzionalità che altrimenti non sarebbe possibile avere.

Questa entità si crea in AGIS tramite il tipo oggetto 45 e presenta una anagrafica simile a:

Ogni Tipo Dato deve essere relazionato ad un contenitore di Tipi Dato.

Il contenitore è un altro oggetto di AGIS identificato dal tipo oggetto 46. L'azione di link del contenitore consente la generazione automatica di codice, a partire dall'anagrafica dei tipi dato relazionati, che da origine ad una DLL necessaria all'esecuzione dei programmi che fanno uso dei tipi dato.

Avendo quindi isolato in una specifica DLL il codice generato per la gestione del Tipo Dato, un'eventuale evoluzione dello stesso non comporta modifiche al codice BC che ne fa uso ma soltanto la rigenerazione della DLL contenitore

Una delle funzioni che vediamo in questo capitolo è la possibilità di creare un enumerato.

Il resto del Tipo Dato lo scopriremo negli argomenti successivi.

Gli enumerati

Un enumerato è un Tipo Dato che presenta un elenco di valori ben definito associabile ad una descrizione parlante.

Esempio

L'enumerato "Frutta" può avere i seguenti valori:

1. Mela
2. Pera
3. Arancia
4. Fragola

Per cui se mi riferisco a Frutta.Arancia all'interno del mio sorgente identifico in maniera unica il valore 3.

Per creare un enumerato in BC è necessario indicare il tipo di base (1) del valore e l'opzione "Enumerazione" (2) sulla gestione del Tipo Dato:

L'elenco dei valori va indicato utilizzando il relativo bottone.

La gestione dei valori ammessi per l'enumerato è simile a quanto segue:

Ogni valori deve essere univoco e deve essere indicato come segue:

1. è il valore che assume l'enumerato in base al tipo base indicato in precedenza;
2. è l'identificativo descrittivo del valore che verrà utilizzato all'interno del sorgente per riferirsi al valore dell'enumerato. Questo identificativo deve essere univoco, come per il valore.

Utilizzo del Tipo Dato nei sorgenti BC

Una volta definito in AGIS l'oggetto Frutta (tipo oggetto 45), è possibile utilizzarlo all'interno dei sorgenti. Può essere utilizzato in due modi differenti:

1. per assegnazioni o test
2. come tipo dato di una variabile.

Vediamo i due casi.

Nel primo caso, supponiamo che il nostro programma debba gestire una variabile che identifica la tipologia di un frutto:

BC - SUB07_ENUMERATO

```
' Dichiarazione della variabile che identifica un frutto
DIM varFrutta[TIPO[FRUTTA]]
```

Questa variabile all'interno del nostro sorgente viene manipolata in modo da ottenere dei valori associabili ai vari frutti. Posso usare il Tipo Dato per associargli un valore oppure per eseguire espressioni utilizzabili ad esempio in costrutti If/Then/Else.

BC - SUB07_ENUMERATO

```
' Assegnazione
varFrutta = FRUTTA.Fragola

' Test
If varFrutta = FRUTTA.Mela Then

    ...

EndIf
```

 **Attenzione:** la variabile varFrutta deve essere dello stesso tipo di dato base del nostro Tipo Dato Frutta. Per fare riferimento ad uno dei valori che può assumere l'enumerato si utilizza la notazione punto indicando <TIPO DATO>. <Identificativo enumerato>. In BCDevelop l'intellisense propone l'elenco definito nell'anagrafica del tipo dato per agevolare la scrittura del codice.

L'utilizzo degli enumerati rende più leggibile il codice, esplicitando meglio le operazioni che sono svolte senza dover risalire alla definizione dei valori sparsa per il sorgente.

Nel secondo caso, possiamo usare una variabile dichiarata utilizzando come tipo di dato base proprio il Tipo Dato enumerato creato:

BC - SUB07_ENUMERATO

```
' Dichiarazione
DIM varFrutto[TIPO[FRUTTA]]
```

In questo caso si introduce anche un maggior controllo sulla corretta valorizzazione delle variabili.

Infatti la traduzione del sorgente evidenzia con un warning le seguenti situazioni:

- Assegnazione ad un enumerato di un valore costante. Ad esempio varFrutto = 1
- Assegnazione ad un enumerato di un tipo base. Ad esempio varFrutto = flTipoFrutto, dove flTipoFrutto è un intero non enumerato.

Collezione di Tipi Dato

Fino ad ora abbiamo visto variabili che possono contenere un solo valore. Queste variabili sono definite scalari. Esistono variabili che possono contenere più di un valore. Queste variabili le possiamo definire come *collezione* di elementi.

La collezione può contenere un numero non finito di elementi tutti dello stesso tipo. Ogni elemento della collezione è una variabile scalare, quindi avrà tutte le funzioni che abbiamo descritto fino ad ora (possibilità di definire tipi personali, metodi per la manipolazione del dato, e così via).

Definizione di una collezione di Tipi Dato

Per definire una collezione è necessario utilizzare la keyword *COLLEZIONE* all'interno della dichiarazione della variabile:

BC

```
DIM collVariabile[TIPO[BCSTRING] COLLEZIONE]
```

Abbiamo dichiarato una variabile *collVariabile* che è una collezione di BCSTRING, ovvero un elenco di stringhe.

Accesso alla collezione di Tipi Dato

Essendo un elenco di valori, per accedere al singolo elemento è necessario effettuare un ciclo di lettura che scorra tutti gli elementi. Il metodo che permette di effettuare il ciclo della collezione è il *.ForEach()* che, come dal nome si può intendere, scorre i singoli elementi ritornando l'i-esimo elemento.

Esempio

BC

```
DIM singVariabile[TIPO[BCSTRING]]

collVariabile.ForEach(singVariabile, LetturaSingoloElemento(singVariabile [IN]))
```

```
'@SRP LetturaSingoloElemento(inVar[TIPO[BCSTRING]] [IN])
```

```
' La variabile interna inVar conterrà il valore i-esimo della collezione
' in questa SRP posso effettuare qualsiasi operazione con il valore letto
```

```
Return
```

All'interno della SRP interna è possibile andare ad utilizzare il nostro elemento. Da notare che il singolo elemento è una variabile di tipo BCSTRING, per cui ha a disposizione tutte le funzionalità del Tipo Dato di ambiente.

La modifica del singolo elemento all'interno della SRP richiamata dal metodo *.ForEach()* non si riflette sulla collezione.

Se è necessario modificare il singolo elemento bisogna utilizzare il ciclo *For/Next*. In questo caso dobbiamo sapere a priori di quanti elementi è composta la nostra collezione. Ci viene in aiuto il metodo *.Count()* che ritorna il numero totale di valori presenti nella collezione. Per accedere all'i-esimo elemento bisogna utilizzare il metodo *.Item(n)* dove n è l'elemento i-esimo del ciclo.

BC

```

For indColl[INT]=1 To collVariabile.Count()

    collVariabile.Item(indColl) = "TESTO_MODIFICATO"

Next indColl

```

Metodi

Anche i Tipi Dato di ambiente e applicativi hanno a corredo i loro metodi per poter manipolare il dato. Come per i tipi di dato base, i metodi sono richiamabili <variabile>.<Metodo>.

Esistono diverse tipologie di metodi:

- metodi dei Tipi Dato applicativi;
- metodi del tipo dato di base;
- metodi definiti dall'utente.

Vediamoli nel dettaglio.

Metodi dei Tipi Dato applicativi

I metodi dei Tipi Dato applicativi sono generati in automatico dall'Ambiente di sviluppo quando viene linkata la DLL del contenitore e sono differenti in base alla tipologia del Tipo Dato: singolo, collezione e statico.

Esempio

Utilizzo del metodo .Clear() per ripristinare il valore di default della variabile varFrutto:

BC - SUB07_ENUMERATO

```

' Pulisco il contenuto della variabile
varFrutto.Clear()

```

Possono esserci dei metodi che agiscono sulla singola variabile, come il .Clear() dell'esempio precedente, ma anche su collezioni di tipi Dato, come per esempio il .Count(), metodo che permette di avere il numero di elementi contenuto nella collezione.



E' possibile consultare l'elenco completo dei metodi per i Tipi Dato applicativi nelle schede del *Manuale di riferimento del linguaggio BC*:

- Metodi dei tipi dato applicativi
- Metodi delle collezioni di tipi dato applicativi

Metodi del tipo di dato base

Anche questi metodi sono generati in automatico dall'Ambiente di sviluppo e dipendono dalla tipologia del dato di base utilizzato nella definizione del Tipo di Dato applicativo.

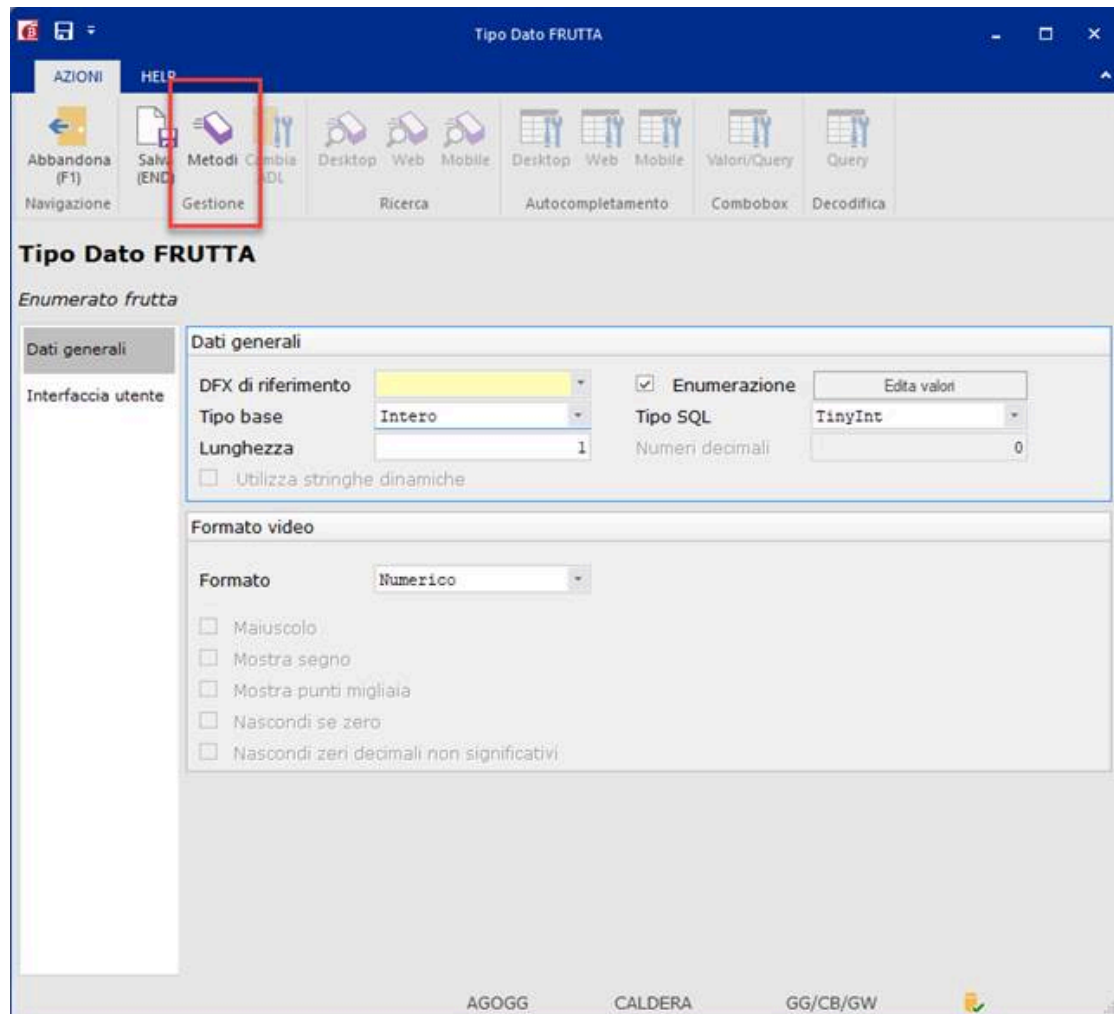
Questi metodi li abbiamo già incontrati in relazione alla dichiarazione di variabili semplici.

L'elenco completo di questi metodi si trova nella scheda "Elementi sintattici di base"

Metodi definiti dall'utente

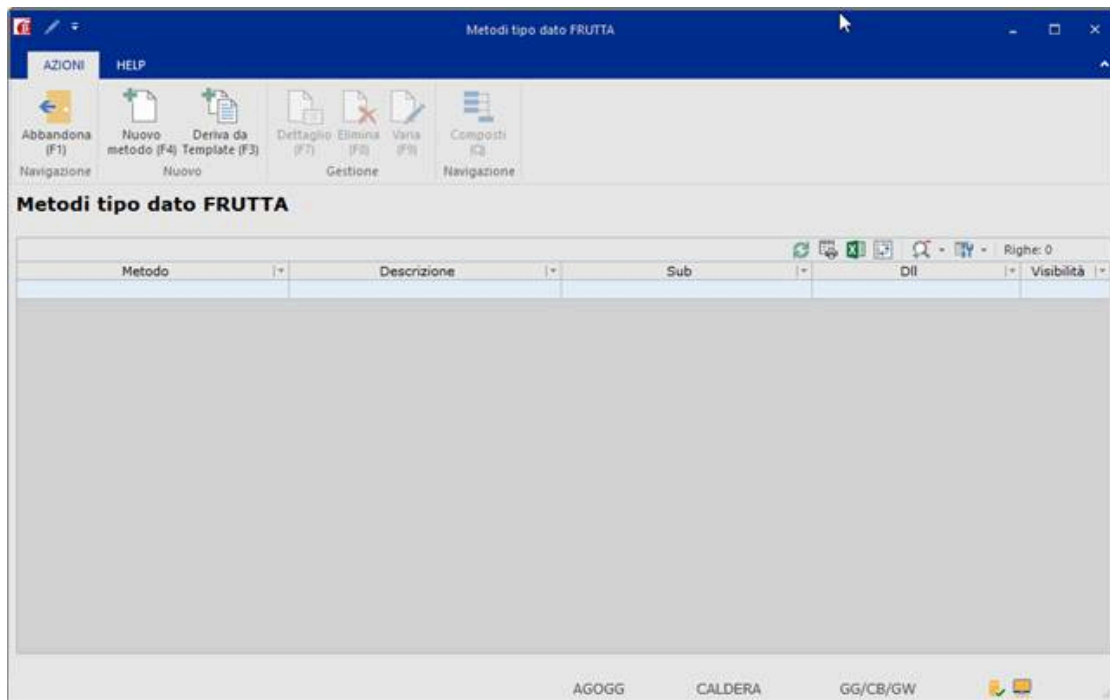
Infine, è possibile creare per ogni Tipo Dato applicativo dei metodi riferiti allo specifico Tipo Dato. Questi metodi vengono chiamati *metodi applicativi*.

I metodi applicativi sono a tutti gli effetti dei sorgenti BC collegati al Tipo Dato. Si creano dalla gestione del Tipo Dato premendo il bottone Metodi:



I metodi sono a tutti gli effetti delle subroutine esterne che contengono del codice BC.

E' disponibile sia il bottone "Nuovo metodo (F4)" per creare singolarmente i metodi necessari, che il bottone "Deriva da Template (F3)" che permette di derivare i metodi da un elenco di Template standard distribuiti da SISTEMI.



La sub viene battezzata automaticamente con <Nome Tipo Dato>_<Nome metodo>, questa dovrà essere inserita in una DLL.

💡 Come linea guida è preferibile inserire le SUB dei metodi in DLL riferite ad un ambito applicativo di riferimento, stesso criterio utilizzato per la definizione delle normali sub.

Lo schema di riferimento è: <Sigla prodotto>_<Nome Contenitore>_BL, dove il suffisso BL sta per Business Logic.

Nel caso fosse necessario è possibile integrare con parametri aggiuntivi l'interfaccia della SUB in base alle esigenze, mantenendo ferma la definizione del primo parametro come indicato in precedenza.

Creazione nuovo metodo

Tralasciamo per un momento la creazione del metodo derivato da template e concentriamoci sul nuovo metodo F4.

Premendo il bottone ci vengono chieste le informazioni di base del nostro metodo:

Una volta indicato il nome, la descrizione e la DLL in cui dovrà esser inserito, è necessario andare a definire altre due proprietà:

- visibilità: indica chi può utilizzarlo e può valere:
 - o pubblico: lo possono usare tutti, anche prodotti/progetti differenti da quello in cui viene sviluppato;
 - o interno: lo può utilizzare solo il prodotto/progetto in cui viene sviluppato;
 - o privato: può essere utilizzato solo all'interno del tipo dato stesso, quindi nei suoi metodi.
- ambito di utilizzo: definisce a cui si riferisce il metodo e può valere:
 - o oggetto (si riferisce a se stesso): viene inserito come primo parametro della sub dichiarato come `This[TIPO[<Tipo Dato>]]`;
 - o collezione (si riferisce ad una collezione di Tipi Dato): viene inserito come primo parametro della sub dichiarato come `This[TIPO[<Tipo Dato>] COLLEZIONE]`;
 - o statico (si riferisce al Tipo Dato e non alla singola variabile istanziata): non viene definito alcun parametro sulla sub.

Confermati i dati anagrafici del metodo, viene presentata l'anagrafica della subroutine esterna che verrà creata come metodo.

Esempio

Sul nostro Tipo Dato enumerato Frutta andiamo a creare un metodo per controllare se il nostro frutto è corretto. Creiamo un metodo Check il cui sorgente sarà:

BC - FRUTTA_CHECK

```
'@PG FRUTTA_CHECK STRICT[1]          ' Frutta - Controllo presenza Frutta

'@SBP This[TIPO[FRUTTA]]              [IN] _ ' Enumerato frutta
    AS FruttaOk[TIPO[BCBOOL]]

    FruttaOk = #True

    If This=0 Then FruttaOk=#False

End
```

E il suo richiamo:

BC - SUB07_ENUMERATO

```
' Pulisco il contenuto della variabile
varFrutto.Clear()
If (Not varFrutto.Check()) Then

    MESSAGEBOX(#INFO, "Esercitazione 7", "Il frutto è vuoto" )

EndIf
```

Le Classi e gli Oggetti

Introduzione allo sviluppo OOP

Gli ambienti di sviluppo che permettono di programmare ad oggetti (Object Oriented Programming, da cui l'acronimo OOP che utilizzeremo in seguito) consentono la definizione di oggetti software in grado di interagire gli uni con gli altri.

La programmazione ad oggetti è senza dubbio il modello più diffuso ed utilizzato per via degli innumerevoli vantaggi che ne derivano. Questo paradigma ha reso quindi superate le precedenti metodologie strutturate e procedurali.

Anche nell'ambiente di sviluppo Sistemi questa metodologia si affianca alla precedente (quella procedurale) introducendo concetti già previsti in altri ambienti di sviluppo, modellandoli al meglio rispetto alle specificità dell'ambiente proprietario Sistemi.

Nell'approccio procedurale possiamo distinguere due mondi diversi e separati: il mondo dei dati e il mondo del codice. Il mondo dei dati è definito da variabili di diverso tipo e scollegate tra loro, mentre il mondo del codice è definito da funzioni che consentono il riutilizzo del codice. Le funzioni sono in grado di usare i dati, ma non viceversa.

L'approccio della OOP suggerisce un modo di pensare completamente diverso: i dati e il codice sono racchiusi nello stesso mondo, contraddistinto dalla definizione di *classi*.

Che differenza c'è tra classi e oggetti

Una classe è un modello che consente di creare oggetti aventi attributi e metodi che saranno condivisi da tutti gli oggetti creati. Un oggetto è di fatto l'istanza di una classe.

Un *oggetto* prevede un insieme di tratti (*proprietà*) ed è in grado di eseguire un insieme di attività (*metodi*). Possono interagire tra di loro scambiando dati e/o attivando i loro metodi: non c'è più un confine tra dato e codice. Essi convivono all'interno di un oggetto.

La fase di progettazione iniziale dovrà dare quindi risposta alle domande: "*Quali proprietà sono necessarie a compiere le azioni da svolgere?*", "*Cosa si vuole che sia in grado di fare?*".

Le classi in BC

A differenza di quanto avviene in altri linguaggi di programmazione, nell'ambiente di sviluppo Sistemi, la definizione delle classi avviene a design-time. È quindi AGIS il punto in cui vengono definite le classi identificate con lo specifico tipo di oggetto "40: Classe".

A livello di sorgente viene invece definito l'oggetto da utilizzare riferito ad una specifica classe.

Diremo quindi che un *oggetto* fa riferimento ad una specifica *classe* e costituisce un'*istanza della classe* stessa. All'interno dello stesso sorgente è possibile creare due o più istanze facenti riferimento alla stessa classe, ma ciò non significa che tutte queste istanze eseguano le stesse azioni o che siano correlate tra di loro.

Definizione di una classe

Usiamo l'entità Cliente come esempio di definizione della nostra classe. Un Cliente può essere descritto con le seguenti caratteristiche:

- un identificativo univoco: CodCliente;

- una denominazione: RagioneSociale;
- una informazione che attesti che abbia la partita IVA: TitPIVA
- una Partita IVA che identifica il Cliente per il pagamento dell'IVA: PIVA;
- un elenco di indirizzi (fatturazione, spedizione, sede legale, ecc) che costituisce a tutti gli effetti ad un'altra entità che chiameremo Indirizzi.

La classe degli Indirizzi sarà quindi costituita da:

- un identificativo univoco: NumProgressivo;
- l'indirizzo comprensivo del numero civico (via, corso, viale, ecc...): Indirizzo;
- codice di avviamento postale: CAP;
- il comune: Comune;
- Provincia: Provincia.

Vediamo come creare le due classi su AGIS.

L'anagrafica è simile a:

Una volta definito il "Nome oggetto", la descrizione e inserita in un contenitore di classi, possiamo andare a creare fisicamente la nostra classe.

Per il momento ignoriamo le altre informazioni, le vedremo più avanti.

Come per il Tipo Dato, anche la classe deve essere inserita all'interno di un contenitore di classi identificato con il tipo oggetto 41. L'azione di link del contenitore consente la generazione automatica di codice, a partire dall'anagrafica delle classi relazionati, che da origine ad una DLL necessaria all'esecuzione dei programmi che fanno uso delle classi.

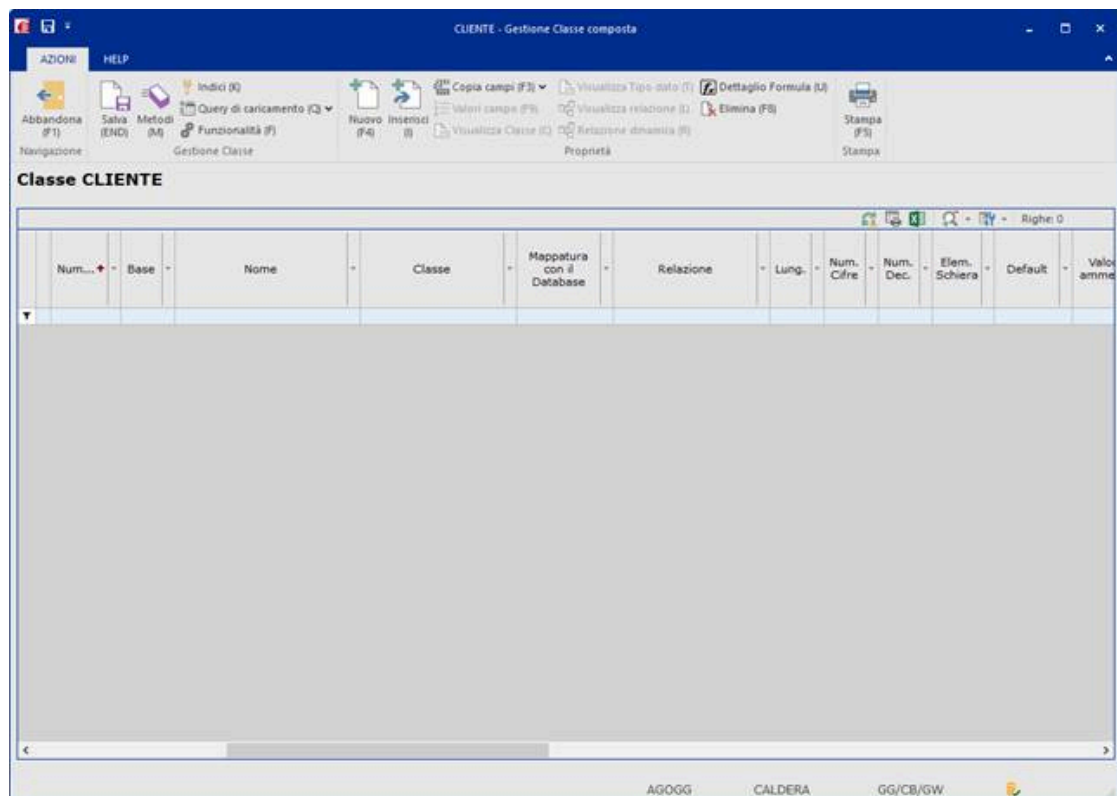


Come linea guida è preferibile battezzare i contenitori raggruppando le classi per ambito applicativo di riferimento, stesso criterio utilizzato per la definizione delle librerie.

Lo schema di riferimento è: <Sigla prodotto>_<ambito applicativo>_CL.

Per maggiori informazioni riguardo la nomenclatura degli oggetti si può fare riferimento alla scheda "Nomenclatura oggetti AGIS" degli *Standard di sviluppo*.

La gestione della classe si presenta come segue:



Tramite il bottone "Nuovo" possiamo andare ad inserire le proprietà e tramite "Metodi" le sue azioni.

Proprietà

Per ogni proprietà vanno definite una serie di informazioni quali descrizione, tipo di dato della proprietà, dimensione e altro ancora.

Su ogni proprietà possiamo andare ad indicare la sua visibilità (pubblica o interna) in modo da proteggere quei dati che non devono essere manipolati dall'esterno se non dalla classe stessa.

Il tipo di dato della proprietà può essere scelto tra i tipi di dato base, oppure tra un Tipo Dato (di ambiente o applicativo) oppure tra una classe.

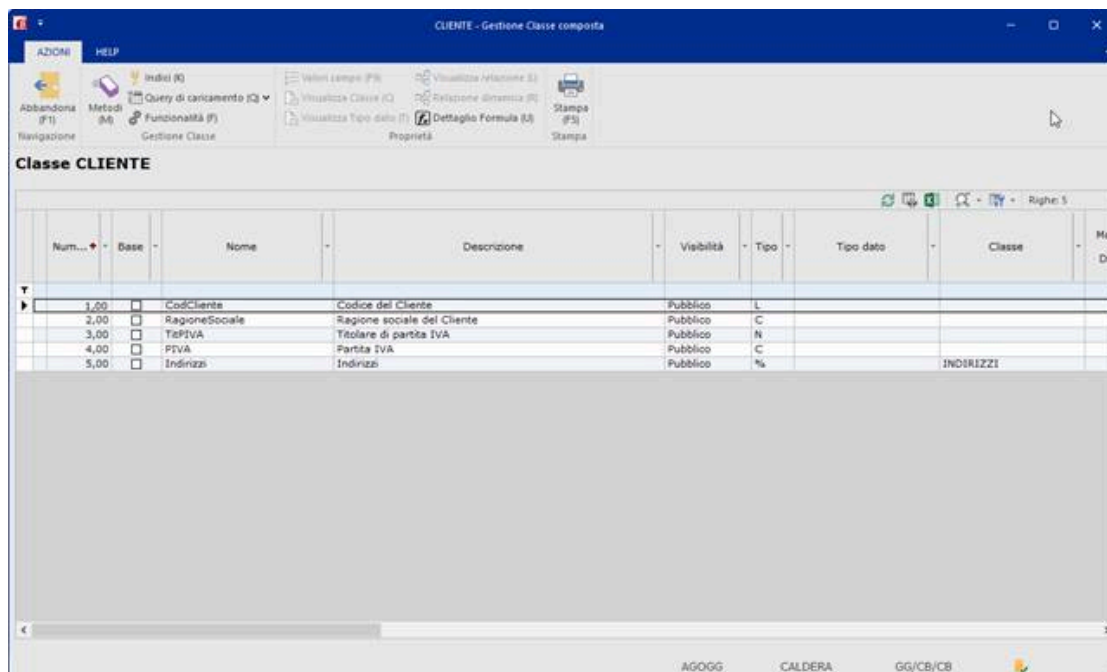
⚠ Per poter gestire il booleano è stato inserito un tipo di dato base F - Booleano. Di seguito, l'elenco con i tipi di dato gestibili:

Tipo	Descrizione	Tipo SQL	Valori ammessi
I	Intero	Smallint	Da -32.768 a 32.767
L	Long	Integer	Da -2.147.483.648 a 2.147.483.647
S	Singola Precisione	Decimal	Precisione appross.: valori con 5 interi e 2 decimali (7 cifre totali)
D	Doppia Precisione	Decimal	Precisione appross.: valori con 13 interi e 8 decimali
C	Carattere	Varchar	
N	Numerico (N1,N2)	TinyInt	Valori numerici interi da 0 a 99
N	Numerico (N3,N4)	Smallint	Valori numerici interi da 0 a 9999
N	Numerico (N>4)	Integer	Valori numerici interi da 0 a 2.147.483.647
A	Data	Datetime o Date	Su archivio e' un campo di tipo Long in formato AAAAMMGG (Su database datetime o date)
E	Periodo	Integer	Su archivio e' un campo di tipo Long in formato AAAAMM
Z	Numerico	-Non gestito	Valori numerici da 0 a 61 memorizzati su 1 byte con codifica interna
R	Numerico	-Non gestito	Valori numerici con decimali ed eventuale segno in formato ASCII su archivio allineato a Sx
Y	Percentuale	Smallint	(Obsoleto) Utilizzato per memorizzare valori percentuali su archivio
X	Asci	Varchar	Utilizzato per memorizzare il valore ASCII di un carattere
B	Blob	Text o Varchar(Max)	Lunghezza variabile
T	Tipo dato	Tipo Dato	
Q	Collezione di Tipo dato	Collezione di Tipo dato	
F	Booleano	Booleano	Su archivio è un campo N1 (Può assumere i valori 0-Falso;1-Vero)
#	Classe	Classe	
%	Collezione di Classi	Collezione di Classi	

La particolarità della classe Clienti è dovuta dagli indirizzi. L'indirizzo è una entità a sé stante e, soprattutto, può essere più di uno.

Una volta creata la classe degli Indirizzi è possibile collegarla alla classe dei Clienti utilizzando come tipologia di dato **% - collezione di classi**.

Ecco come si presenta la classe dei Clienti dopo aver caricato le proprietà che la definiscono:



Livello di visibilità delle proprietà

In fase di definizione di una classe è possibile attribuire ad ogni singola proprietà due possibili livelli di visibilità:

- Interno
- Pubblico

Impostando il livello di visibilità come interno, nella fase di importazione di una classe in AGIS, queste proprietà non saranno considerate e quindi non potranno essere utilizzate. In questo modo si protegge la proprietà da eventuali letture e modifiche indesiderate.

Al contrario, se si necessita di mettere a disposizione una proprietà è necessario indicarla come Pubblica.

Metodi

Per le classi esistono due tipologie differenti di metodi:

1. metodi base
2. metodi definiti dall'utente

I metodi base

I metodi base sono tutti i metodi autogenerati dall'Ambiente di sviluppo in fase di generazione della DLL contenitore, predisposti per la manipolazione dei dati.

La sintassi per utilizzare i metodi è <Oggetto>.<Metodo>.

Esempio:

BC

```
' Pulisco il contenuto della variabile
Cliente.Clear()
```

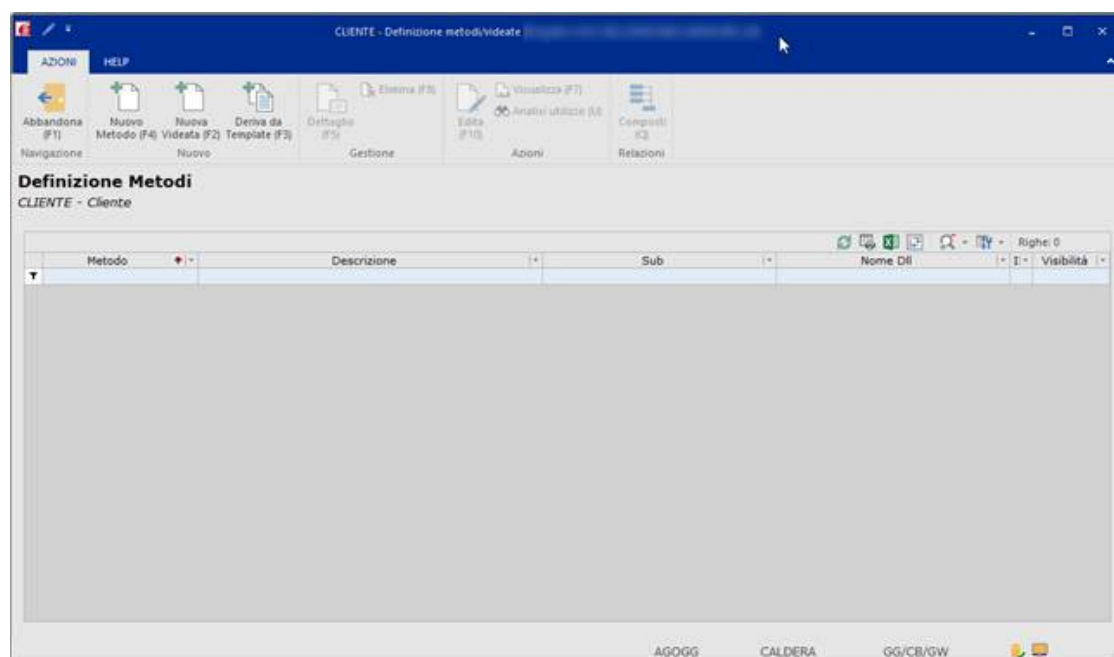


Per i metodi base disponibili è possibile fare riferimento alle schede del *Manuale di riferimento del linguaggio BC*:

- metodi base delle classi
- metodi base delle collezioni di classi

I metodi definiti dall'utente

La loro creazione avviene tramite la gestione simile a quella sotto riportata:



Per il momento tralasciamo i bottoni "Nuova videata" e "Deriva da template" che vedremo più avanti, e vediamo nel dettaglio "Nuovo metodo".

I metodi sono dei sorgenti BC vengono creati con il nome automatico <Nome classe>_<Nome metodo> e devono essere contenuti in DLL.



Come linea guida è preferibile inserire le SUB dei metodi in DLL riferite ad un ambito applicativo di riferimento, stesso criterio utilizzato per la definizione delle normali sub.

Lo schema di riferimento è:

- <Sigla prodotto>_<Nome Contenitore>_BL, dove il suffisso BL sta per Business Logic per i metodi normali;
- <Sigla prodotto>_<Nome Contenitore>_UI, dove il suffisso UI sta per User Interface per i metodi di videata.

La creazione del metodo normale ha bisogno di un'anagrafica sul quale sono richieste le caratteristiche di base del metodo:

Una volta indicato nome e descrizione, è necessario andare a definire altre due proprietà:

- ambito di utilizzo: definisce a cui si riferisce il metodo e può valere:
 - o oggetto (si riferisce a se stesso): viene inserito come primo parametro della sub dichiarato come `This[CLASSE[<Nome classe>]];`
 - o collezione (si riferisce ad una collezione della classe): viene inserito come primo parametro della sub dichiarato come `This[CLASSE[<Nome classe>] COLLEZIONE];`
 - o statico (si riferisce alla classe e non alla singola oggetto istanziato): non viene definito alcun parametro sulla sub;
- visibilità: indica chi può utilizzarlo e può valere:
 - o pubblico: lo possono usare tutti, anche prodotti/progetti differenti da quello in cui viene sviluppato;
 - o interno: lo può utilizzare solo il prodotto/progetto in cui viene sviluppato;
 - o privato: può essere utilizzato solo all'interno della classe stessa, quindi nei suoi metodi.

L'opzione "Sostituzione del metodo base" lo vediamo più avanti quando affronteremo i metodi di override collegati alla base dati.

Definite queste informazioni, viene presentata la videata della sub che verrà creata e si aprirà l'editor dei sorgenti BCDevelop per permettere la stesura del codice.

Il sorgente generato in automatico prevede come unico parametro l'oggetto riferito alla classe. I parametri possono essere integrati in base alle specifiche esigenze. Al suo interno va implementata la logica applicativa specifica che il metodo deve eseguire.

Vediamo un esempio di metodo che carica una collezione:

In anagrafica abbiamo indicato l'ambito collezione e visibilità interna. Il sorgente BC generato è il seguente:

BC

```
'@PG CLIENTE_CARICACLIENII STRICT[1]      ' Inserimento clienti in collezione

'@SBP This[CLASSE[CLIENTE] COLLEZIONE] [INOUT]  ' Cliente

End
```

Come detto in precedenza il primo argomento è sempre un oggetto riferito alla classe per cui viene predisposto il metodo. Per convenzione la variabile viene denominata sempre This identificando l'oggetto per il quale viene eseguito il metodo.

A livello di codice BC i metodi standard vengono richiamati con la stessa sintassi di quelli base: `varCliente.<Metodo>()`.

Vediamo come deve essere richiamato il nostro metodo di collezione:

BC

```
' Dichiarazione di una variabile di tipo collezione
DIM varCliente[CLASSE[CLIENTE] COLLEZIONE]

' Richiamo del metodo di caricamento della collezione
varCliente.CaricaClienti()
```

Se la variabile non fosse stata definita di collezione, avrebbe dato errore in compilazione. Già l'intellisense non avrebbe proposto il metodo perché non facente parte dell'oggetto.

Come si può notare anche se la sub prevede un parametro, ovvero l'oggetto, la chiamata non lo esplicita ma viene autoreferenziato direttamente dall'oggetto per il quale deve essere eseguito il metodo.

Se il metodo prevede ulteriori parametri aggiuntivi, questi vanno indicati tra le parentesi tonde.

Quando riferiti a Oggetto o Collezione possono essere richiamati da una specifica istanza di un oggetto o di una collezione con la sintassi <Nome oggetto>.<Nome metodo>() oppure <Nome collezione>.<Nome metodo>().

Per i metodi statici invece non è necessario istanziare un oggetto ma possono essere richiamati con la sintassi <Nome classe>.<Nome Metodo>().



La sintassi per il richiamo dei metodi utilizza gli stessi formalismi utilizzati per la chiamata tramite CALL delle sub. Possono quindi essere utilizzati direttamente valori costanti o variabili preventivamente dichiarate. A differenza del richiamo dei metodi base va indicato per i parametri anche il qualificatore IN/OUT definito sul prototipo della sub di riferimento del metodo.

Gli oggetti e il loro utilizzo in BC

La definizione delle classi in AGIS e la generazione della DLL del contenitore consentono la definizione nel codice BC di specifiche istanze riferite ad una classe. Per farlo, è necessario utilizzare la keyword *DIM[]* già vista per la dichiarazione di variabili semplice e Tipi Dato.

Esempi

BC

```
' Sintassi
' DIM <nome oggetto>[CLASSE[<nome classe>]]

' Definizione oggetto varCliente
DIM varCliente[CLASSE[CLIENTE]]
```

Con la parola chiave *COLLEZIONE* è invece possibile istanziare una collezione/lista di oggetti dello stesso tipo:

BC

```
' Sintassi
' DIM <nome collezione>[CLASSE[<nome classe>] COLLEZIONE]

' Definizione oggetto varClienti
DIM varClienti[CLASSE[CLIENTE] COLLEZIONE]
```

Una volta dichiarato l'oggetto è possibile utilizzare le sue proprietà attraverso la notazione punto:

BC

```
' Assegnazione (SET) della proprietà CodCliente
varCliente.CodCliente = nuovoCliente

' Utilizzo (GET) proprietà RagioneSociale
DIM varStringa[STRING] = "Cliente " + varCliente.RagioneSociale.Trim()
```

 L'utilizzo della notazione punto per utilizzare nei programmi BC le proprietà dell'oggetto **non** prevede che nella sintassi sia utilizzato l'indicatore che normalmente determina il tipo di dato base.

Operazioni di confronto

Due istanze di una classe possono essere confrontate fra loro direttamente con i semplici operatori di confronto:

BC

```
' Esegue una copia dell'oggetto cliente
oldCliente = varCliente

' Viene eseguito il confronto tra i due oggetti
If oldCliente <> varCliente Then ...
```

L'operazione esegue il confronto di tutte le proprietà previste dalla classe di riferimento degli oggetti. L'operazione può essere eseguita anche tra due collezioni.

Utilizzo nella definizione dei parametri di una sub

Allo stesso modo degli altri tipi di variabile, gli oggetti e le collezioni possono essere passati come parametro ad una sub.

Definizione oggetto come parametro della sub:

BC

```
'@PG BLD_STAMPA_CLIENTE      ' Stampa Cliente

'@SBP SigoloCliente[CLASSE[CLIENTE]] [INOUT] ' Singolo cliente
```

Chiamata alla sub:

BC

```
DIM varCliente[CLASSE[CLIENTE]]

. . .

CALL BLD_STAMPA_CLIENTE(varCliente [INOUT])
```

Definizione collezione come parametro della sub:

BC

```
'@PG BLD_ELABORA_CLIENTI      ' Elabora clienti

      '@SBP ClientiDaElaborare[CLASSE[CLIENTE] COLLEZIONE] [INOUT] ' Elenco clienti
```

Chiamata alla sub:

BC

```
DIM ClientiSelezionati[CLASSE[CLIENTE] COLLEZIONE]

. . .

CALL BLD_ELABORA_CLIENTI(ClientiSelezionati [INOUT])
```

⚠ A differenza delle semplici variabili, oggetti e collezioni **non** possono essere passati per valore (parametro [IN]). Nel caso servisse preservare lo stato dell'oggetto al momento della chiamata alla sub è necessario eseguire esplicitamente la copia prima di effettuare la chiamata.

Da prestare particolare attenzione quando si trattano collezioni con un elevato numero di oggetti contenuto la cui copia può creare un'eccessiva occupazione di memoria.

Assegnazione per valore e assegnazione per riferimento

Assegnando un oggetto (o una collezione) tramite l'operatore uguale (A=B), viene fatta *un'assegnazione per valore* e quindi il contenuto di B viene interamente copiato in A.

Questo implica una duplicazione di memoria e può essere un'operazione costosa ed esosa se si tratta di un oggetto molto grosso o di una collezione con molti elementi.

Per l'assegnazione tra oggetti/collezioni è disponibile l'operatore '=' che effettua *un'assegnazione per riferimento* (A=>B).

In questo modo l'oggetto/collezione A **punta** all'oggetto/collezione B e di fatto ne condivide il contenuto anziché duplicarlo. Quindi una qualsiasi modifica fatta su A si rifletterà anche su B.

Per sganciare il riferimento è possibile utilizzare il metodo .Release().

L'assegnazione per riferimento è indispensabile per modificare un elemento della collezione, come vedremo più avanti, ma è anche molto utile in tanti altri contesti per non duplicare memoria.

Concludendo, possiamo dire che l'assegnazione per riferimento è da preferirsi all'assegnazione per valore, che invece andrebbe usata soltanto quando si intende preservare il contenuto dell'oggetto da copiare.

Ricerca di un elemento in una collezione

La ricerca di un elemento all'interno di una collezione può essere eseguita utilizzando i metodi base generati in automatico nel momento in cui viene generata la DLL del contenitore delle classi.

Esempio

Viene illustrato come ricercare il primo oggetto cliente titolare di partita IVA e con Partita IVA non valorizzata all'interno di una collezione di clienti:

BC

```
' Dichiarazione oggetto cliente titolare di partita IVA
DIM ClienteTitPIVA[CLASSE[CLIENTE]]

' Dichiarazione collezione contenente l'elenco dei clienti
DIM ElencoClienti[CLASSE[CLIENTE] COLLEZIONE]

' Ricerca primo cliente titolare di partita IVA con P.IVA non valorizzata
ClienteTitPIVA = ElencoClienti.FindFirst(CLIENTE.TitPIVA=1 And CLIENTE.PIVA="")

If ClienteTitPIVA.IsEmpty() Then ExitSr ' Oggetto non trovato
```

Il metodo .FindFirst() restituisce il primo oggetto della collezione che rispetta le caratteristiche indicate.

Il risultato del metodo è quindi un oggetto dello stesso tipo su cui si basa la collezione. Nell'esempio riportato sopra l'oggetto restituito dal metodo, se trovato, viene copiato nell'oggetto *ClienteTitPIVA*.

Quest'ultimo non ha nulla a che fare con quanto contenuto nella collezione. Per cui successive modifiche all'oggetto *ClienteTitPIVA* non si riflettono sulla collezione.

Con la notazione punto è anche possibile scrivere le due operazioni con un'unica istruzione:

BC

```
' Se non c'è un cliente titolare di partita IVA con P.IVA non valorizzata allora esco
If ElencoClienti.FindFirst(CLIENTE.TitPIVA=1 And CLIENTE.PIVA="").IsEmpty() Then ExitSr
```

Modifica di un elemento di una collezione

Riprendendo l'esempio precedente vediamo ora come modificare l'oggetto della collezione restituito dal metodo .FindFirst().

BC

```

' Dichiarazione oggetto cliente titolare di partita IVA
DIM ClienteTitPIVA[CLASSE[CLIENTE]]

' Dichiarazione collezione contenente l'elenco dei clienti
DIM ElencoClienti[CLASSE[CLIENTE] COLLEZIONE]

' Ricerca primo cliente titolare di partita IVA con P.IVA non valorizzata
ClienteTitPIVA => ElencoClienti.FindFirst(CLIENTE.TitPIVA=1 And CLIENTE.PIVA="")

' Modifica dell'oggetto se trovato
If Not ClienteTitPIVA.IsEmpty() Then

    ClienteTitPIVA.TitPIVA = 1
    ClienteTitPIVA.PIVA = "1234567890"

EndIf

```

Rispetto al precedente esempio cambia l'operatore di assegnazione. L'utilizzo dell'operatore "=>" indica che non deve essere fatta una copia dell'oggetto bensì l'oggetto *ClienteTitPIVA* diventa un **puntatore** all'oggetto estratto dal metodo *.FindFirst()*.

In questo caso quindi le successive modifiche all'oggetto si riflettono anche sulla collezione. Per scollegare l'oggetto *ClienteTitPIVA* dalla collezione sarà necessario utilizzare il metodo *.Release()*. Dopo l'esecuzione di questo metodo l'oggetto torna ad essere un'altra istanza della classe CLIENTI vuota.

Nell'esempio viene evidenziato anche l'utilizzo del metodo *.IsEmpty()* utilizzato per sapere se l'oggetto risultante dal metodo *.FindFirst()* non sia vuoto.

Esecuzione azione su tutti gli elementi della collezione

Per eseguire una modifica a tutti gli elementi della collezione è possibile utilizzare il metodo *.ForEach()* come riportato qui di seguito:

BC

```

' Dichiarazione oggetto di appoggio per ForEach
DIM tmpCliente[CLASSE[CLIENTE]]

ElencoClienti.ForEach(tmpCliente, AzzerPartitaIVA(tmpCliente [INOUT]))

. . .

'@SRP AzzerPartitaIVA(tmpCliente[CLASSE[CLIENTE]] [INOUT])

    tmpCliente.PIVA = ""

Return

```

in questo caso l'oggetto *tmpCliente* viene utilizzato come puntatore alla collezione *ElencoClienti*. Questo significa che ogni modifica all'oggetto si riflette in automatico sulla collezione stessa.

La release dell'oggetto avviene in automatico alla fine del `.ForEach()`.

⚠ All'interno di un `.ForEach()` non è possibile eliminare oggetti dalla collezione. Se necessario va utilizzato un ciclo `For/Next` con metodo `.Remove()` riferito all'elemento dell'iterazione.

Il caso esposto sopra può essere semplificato nel seguente modo:

BC

```
' Dichiarazione oggetto di appoggio per ForEach
DIM tmpCliente[CLASSE[CLIENTE]]

ElencoClienti.ForEach(tmpCliente, tmpCliente.PIVA = "")
```

Il metodo `.ForEach()` con più statement

La sintassi del metodo `.ForEach()` consente che nel suo secondo parametro siano indicati anche più di uno statement separati dal carattere ":" (due punti) come nell'esempio di seguito riportato:

BC

```
' Dichiarazione oggetto di appoggio per ForEach
DIM tmpCliente[CLASSE[CLIENTE]]

ElencoClienti.ForEach(tmpCliente, tmpCliente.PIVA="" : tmpCliente.RagioneSociale="")
```

Utilizzo di metodi concatenati

Volendo modificare solo una parte di oggetti della collezione, si può utilizzare in modo concatenato il metodo `.Find()`:

BC

```
' Dichiarazione oggetto di appoggio per ForEach
DIM tmpCliente[CLASSE[CLIENTE]]

ElencoClienti.Find(CLIENTE.TitPIVA=1 And CLIENTE.PIVA="").ForEach(tmpCliente, tmpCliente.PIVA = "")
```

il metodo `.Find()` infatti restituisce una collezione di oggetti che rispettano le selezioni indicate. In questo modo solo i clienti che soddisfano l'espressione indicata saranno modificati.

Ordinamento di una collezione

Per ordinare una collezione sono disponibili i metodi `Sort()` e `OrderBy()`.

Il primo ordina la collezione sul posto, ovvero ordina la collezione dalla quale è chiamato.

Il secondo invece non cambia l'ordinamento della collezione ma ne restituisce una nuova ordinata come richiesto (gli elementi sono gli stessi copiati per riferimento).

BC

```
ElencoClienti.Sort(CLIENTE.Nome) ' Ordina la collezione per nome cliente
```

```
ElencoClientiOrd => ElencoClienti.OrderBy(CLIENTE.Nome) ' Restituisce una collezione di  
clienti ordinata per nome cliente
```

Le classi statiche

A differenza delle classi che abbiamo visto fino ad ora, le classi statiche non rappresentano alcuna entità, per cui non possono essere istanziate per creare degli oggetti ad essa associata. Questo vuol dire che di una classe statica avrà solo metodi e nessuna proprietà.

In AGIS una classe statica si attiva andando a valorizzare l'apposito flag nell'anagrafica della classe:

The screenshot shows the 'Variazione dati oggetto' window with the 'Classi' section selected. The 'Dati oggetto' is 'CLASSE_STATICA'. In the 'Dati generali' section, the 'Classe statica' checkbox is checked and highlighted with a red box. Other fields include 'Nome oggetto' (CLASSE_STATICA), 'DFX di riferimento', 'Descrizione' (Esempio di classe statica), 'Descrizione estesa', 'Categoria', 'Importato' (unchecked), and 'Data validita' da'.

Non avendo proprietà, la videata successiva è quella dell'elenco dei metodi:

The screenshot shows the 'CLASSE_STATICA - Definizione metodi/Videate' window. The 'Definizione Metodi' section is active for 'CLASSE_STATICA - Esempio di classe statica'. The table below lists methods:

Metodo	Descrizione	Sub	Nome Oll	Visibilità
NuovoMetodo	Descrizione del nuovo metodo		CLASSE_STATICA_NUOVOMETODO	Interno

I metodi che vengono creati su questa classe, saranno tutti metodi statici che lavorano solo su parametri di input e quindi senza nessun legame con la classe stessa. Non dovendo istanziare alcun oggetto, il metodo verrà richiamato dalla classe stessa:

BC

```
' Richiamo di un metodo di una classe statica
CLASSE_STATICA.NuovoMetodo()
```

Nell'esempio si può vedere che non è stato istanziato nessun oggetto riferito alla classe CLASSE_STATICA ma si sta usando direttamente la classe per sfruttare i suoi metodi.

In definitiva, si può dire che una classe statica sia un contenitore di metodi non associabili ad alcuna entità.

Le classi di ambiente

Oltre alle classi definite dall'utente, ci sono una serie di classi definite a livello di ambiente che possono essere utilizzate nella progettazione dei programmi.

Alcuni esempi di classi di ambiente possono essere:

- BC_VID_ERROR: classe che permette di gestire gli errori
- BC_DA_VALUE: classe che permette di gestire un valore generico

Per poter utilizzare queste classi è necessario importarle nell'area di sviluppo (prodotto/progetto) di riferimento creando i relativi oggetti in AGIS tramite la funzione "Importa classe d'ambiente".

La struttura dei programmi

Ogni programma per essere avviato ha bisogno di un <Nome Main>.exe, chiamato ***eseguibile***.

L'***eseguibile*** è un contenitore di programmi. I programmi sono a tutti gli effetti dei file memorizzati su file system che contengono del codice sorgente (che chiameremo ***sorgenti***), nel nostro caso il BC.

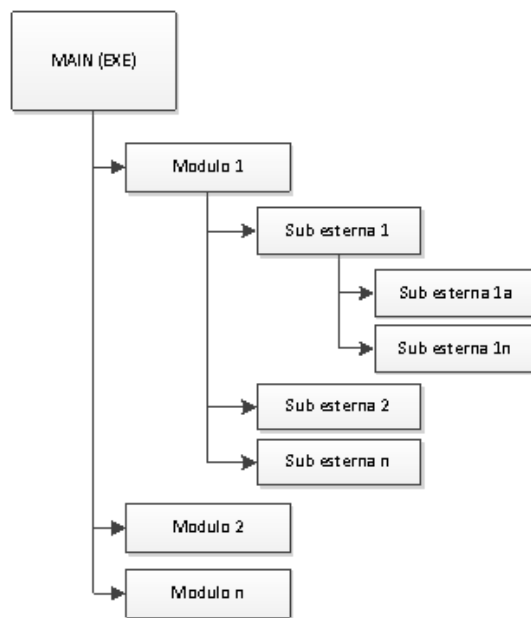
I ***sorgenti*** si suddividono principalmente in due tipologie:

- moduli: sono il punto di ingresso dei programmi e collegati direttamente al Main (eseguibile);
- subroutine (che chiameremo ***sub***): contengono parti di codice che può essere condiviso in più programmi.

Ogni sorgente inizia con la specifica '@PG e termina con la keyword End. I costrutti previsti per il controllo del flusso del programma sono:

- GoSub / '@SR / '@SRP / ExitSr / Return
- If / Then / Elseif / Else / EndIf
- Switch / Case / Case Else / EndSwitch
- For / To / Step / Continue / ExitFor / Next
- While / ContinueWHhile / ExitWhile / EndWhile
- '@BS / GoTo

Un programma può essere più o meno complesso, ma la struttura non si discosta da quella mostrata nell'esempio sottostante:



In generale, le regole per strutturare un programma sono:

- un eseguibile può contenere uno o più moduli;
- un modulo non può essere richiamato da un altro modulo;
- un modulo può richiamare una o più sub;
- una sub può richiamare altre sub.

Per eseguire un Main è necessario scegliere il modulo che deve essere avviato. Per cui possiamo dire che il modulo è il nostro vero programma BC e il Main è il mezzo per poterlo eseguire.



Tutti gli esempi utilizzati sono consultabili sui sorgenti di BuilderDEMO.

Moduli

Il Modulo (tipo oggetto 1) può contenere direttamente il codice al suo interno o richiamare sub esterne.

Nella pratica difficilmente il modulo prevede molta logica al suo interno, ma esegue direttamente il richiamo a sub esterne in modo da avere una maggiore strutturazione del codice ed un'eventuale riutilizzo in più programmi delle sub.

Il comando di esecuzione del programma dovrà quindi specificare <Nome Main> <Nome Modulo> per riferire ad uno specifico punto di ingresso. Se non specificato il nome modulo viene avviato il primo modulo presente nel Main.

Esempio

Esempio di un modulo che visualizza un messaggio a video con il classico "Hello, World!"

BC - CBAS_01

```
'@PG CBAS_01 STRICT[1]      ' Esercitazione 1: Hello + Main'

DIM varTesto[STRING]="Hello, World!"

' Emissione a video della variabile varTesto
MESSAGEBOX (#INFO, "CBAS_01", varTesto)

End
```

Lo spazio tra '*@PG* ed *End* è chiamato **corpo** del sorgente. Al suo interno è possibile inserire tutto il codice che ci serve per scrivere il nostro programma: dichiarazione di variabili, costrutti, ecc.

Il parametro *STRICT* permette di applicare delle regole di sintassi più strutturata e adeguata ai nuovi standard.

Il programma termina subito dopo l'*End*.



Per maggiori informazioni riguardo il parametro *STRICT* è possibile consultare la scheda della specifica '*@PG* del Manuale di riferimento del BC.

Subroutine esterne (sub)

Le sub esterne contengono parti di codice che possono essere richiamate da più programmi, il codice della sub esterna è un sorgente autonomo e fisicamente diverso da quello del programma chiamante.

 Una sub esterna deve essere sempre contenuta all'interno di una DLL.

La sub esterna può ricevere dei parametri (di input [*IN*], output [*OUT*] o input/output [*INOUT*]) e può avere un valore di ritorno, a differenza del modulo che non ha parametri.

Parametri in ingresso

L'elenco dei parametri avviene tramite la specifica '*@SBP* e deve essere indicata subito dopo la '*@PG*. Ogni parametro deve essere corredato da:

- un nome univoco;
- un tipo e una dimensione;
- un attributo

Se la subroutine non prevede parametri, la specifica '*@SBP* rimane vuota.

Il programma chiamante deve utilizzare la keyword *CALL* ed indicare l'elenco dei parametri nello stesso ordine e tipo della dichiarazione sulla '*@SBP*.

Nel caso in cui la subroutine da richiamare sia pubblicata da un altro Prodotto, allora il richiamo può avvenire tramite la keyword *CALL* <nome DLL>::<nome sub>, previa importazione dell'interfaccia (tipo oggetto 12) che contiene l'oggetto in questione.

Fanno eccezione alcune funzioni particolari per il quale non è possibile avere a disposizione l'interfaccia, per cui il richiamo avviene tramite la specifica '*@SBL*.



Per ulteriori informazioni più approfondite riguardo le funzioni di interfaccia ed interfaccia è possibile consultare la scheda "La gestione delle interfacce di prodotto" del manuale di *AG/S*.

Esempio

Esempio di un sub esterna richiamata dal modulo CBAS_02:

BC - CBAS_02

```
'@PG CBAS_02 STRICT[1]      ' Esercitazione 2: Hello + Sub

DIM varTesto[STRING]=" Hello, World!"

' Chiamata alla sub SUB_002 con passaggio della variabile varTesto
CALL SUB02_MSGSUB(varTesto)  ' Esercitazione 2: Hello + Sub

End
```

Nel modulo viene dichiarata la variabile `varTesto` e inizializzata con il testo "Hello, World!". Questa variabile viene poi passata alla sub esterna per poterla utilizzare.

BC - SUB02_MSGSUB

```
'@PG SUB02_MSGSUB STRICT[1]      ' Esercitazione 2: Hello + Sub

'@SBP varTesto[STRING] [IN]  ' Testo da visualizzare

' Emissione a video della variabile ricevuta in interfaccia (varTesto)
MESSAGEBOX(#INFO,"CBAS_02", varTesto)

End
```

L'utilizzo degli attributi `[IN]`, `[OUT]` e `[INOUT]` è obbligatorio poiché forniscono un maggiore controllo sui parametri in ingresso e in uscita dalle sub; il loro utilizzo consente, ad esempio, di non contaminare il modulo chiamante con eventuali valori dei parametri in input alla sub chiamata alterati dalla sub stessa al suo interno.

Dopo l'esecuzione della keyword `End`, il programma torna al chiamante subito dopo il punto in cui è stata chiamata la sub.

Valore di ritorno: le funzioni

Le *funzioni* in BC sono delle sub esterne con il parametro del valore di ritorno.

Come per i parametri in ingresso, il valore di ritorno va indicato sulla specifica `'@SBP` tramite la parola chiave `AS`.

E' possibile definire un solo valore di ritorno e può essere indicare tutte le tipologie di variabili: semplici, variabili di tipo oggetto e collezione.

Esempio

BC

```
'@PG SUB01 STRICT[1]      ' Funzione per la somma di due numeri

'@SBP Numero1[INT] [IN] _ ' Numero 1
      Numero2[INT] [IN] _ ' Numero 2
      AS Risultato[INT]

      ' Somma dei due valori
      Risultato = Numero1 + Numero2

End
```

La funzione, nel programma chiamante, può essere richiamata in due modi differenti:

1. utilizzare il valore di ritorno della funzione:

BC

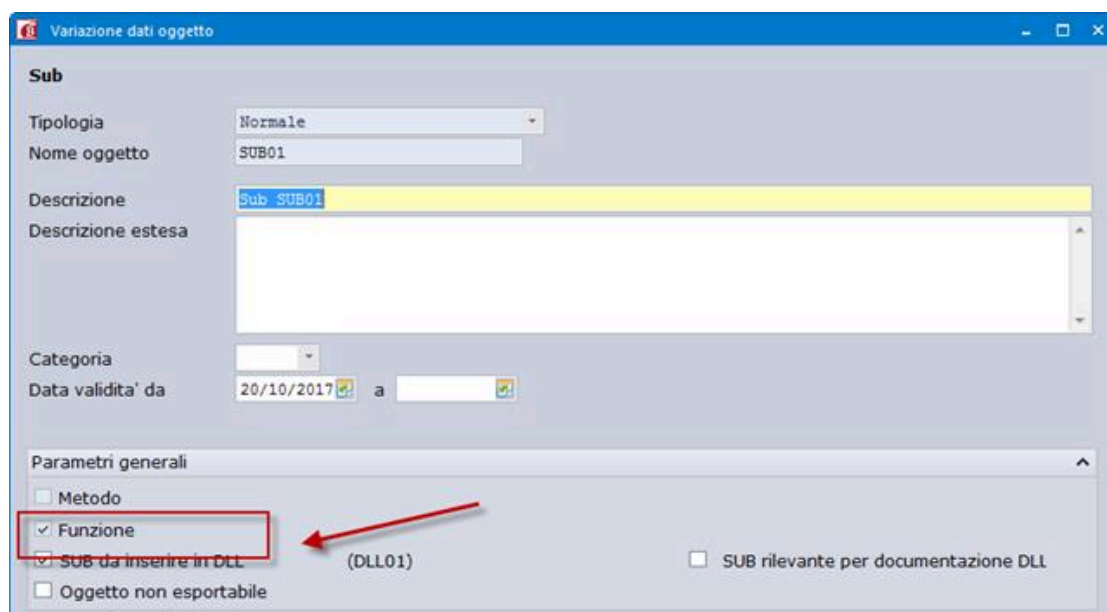
```
DIM Risultato[INT] = SUB01(PAR1, 1)
```

2. senza utilizzare il valore di ritorno:

BC

```
CALL SUB01(PAR1, 1)
```

L'utilizzo di una sub esterna come sopra riportato fa in modo che in AGIS la sub venga definita in modo automatico come una funzione:



Stesura del codice

La scrittura del codice è sicuramente influenzata da aspetti di tipo soggettivo e l'introduzione di alcuni accorgimenti che possono diventare standard di scrittura migliora sicuramente la leggibilità del codice, la sua manutenzione e la sua implementazione.

Insieme ad una attenzione nella scrittura del sorgente, facciamo un uso razionale dei commenti (evitare di essere troppo prolissi o troppo sintetici), definiamo variabili dal nome intuitivo, facendo un limitato uso di *GoTo*, rendiamo più agevole e qualitativamente migliore il lavoro a tutti i collaboratori del gruppo di lavoro coinvolto nello sviluppo.

Un altro modo di agevolare il lavoro del programmatore è di definire una struttura dei nostri programmi con un *corpo* molto semplice e organizzare l'algoritmo senza ripetizioni di pezzi di sorgente sfruttando le subroutine interne.

Subroutine interne

Il codice della subroutine interna si trova nel medesimo file sorgente che contiene il codice del programma e viene quindi compilato con esso, insieme danno origine ad un unico sorgente.

La subroutine interna non riceve parametri definiti in modo esplicito, può fare uso di qualsiasi variabile definita nel sorgente in cui è presente.

Viene richiamata con la parola chiave *GoSub*, mentre il ritorno al corpo del programma avviene affidato alle keywords *Return* o *ExitSr*.

Di fatto la subroutine interna consente una migliore organizzazione del codice all'interno del sorgente evitando di avere tutto il flusso gestito nel corpo principale.

Subroutine interne parametriche

La subroutine interna parametrica può ricevere parametri definiti in modo esplicito e può avere un valore di ritorno, come una sub esterna.

Al suo interno può fare uso soltanto delle variabili Shared e dei parametri in '@SBP. BCDevelop ci viene in aiuto proponendoci solo le variabili che possiamo effettivamente utilizzare.

Le variabili dichiarate all'interno di essa sono locali e non possono essere usate in altre subroutine interne.

Non è possibile dichiarare, al suo interno o nei suoi parametri, variabili che abbiano lo stesso nome di quelle Shared.

Può essere chiamata senza la keyword *GoSub* (a differenza della subroutine non parametriche) oppure essere usata come una funzione, nel caso avesse un valore di ritorno.

Come possiamo vedere nell'esempio riportato in seguito, nella sub SUB59_USO_SRP richiama la subroutine interna parametrica *LeggiDataSistema()*, terminata la sua elaborazione la sub restituisce il controllo al corpo del programma che riprende l'elaborazione dalla specifica successiva alla chiamata della sub interna.

Esempio

Caso 1: richiamo normale

```
BC - SUB59_USO_SRP
```

```

' Caso 1: viene utilizzata la subroutine interna parametrica normale
' riceve in ingresso un parametro di tipo stringa e non ha parametri di ritorno

DIM varTesto[STRING] = "La data di oggi è "           ' Variabile locale del corpo del
programma
LeggiDataSistema(varTesto)

'@SRP LeggiDataSistema(inTesto[STRING] [IN])

' inTesto è una variabile locale di questa SRP

' Utilizzo la variabile globale DataOdierna per ricevere il valore della data di
oggi
'@GETSYSDATE GMA8[DataOdierna]

' Visualizzo la data di oggi formatta in modo da cambiare il tipo di dato da DATE a
STRING
MESSAGEBOX(#INFO, "CBAS_59", " Caso 1: la data di oggi è
" + DataOdierna.FormatDate("gX, G mX AX" )

Return

```

L'SRP di questo esempio riceve un solo parametro in input e non ha valori di ritorno. Viene richiamata come una normale subroutine interna, ma senza il GoSub.

Va notato che la variabile utilizzata come parametro della SRP (inTesto) è locale alla subroutine stessa: viene creata in ingresso della SRP ed eliminata in uscita. Per cui non è possibile utilizzare inTesto dopo l'esecuzione della SRP.

Viene usata la variabile globale DataOdierna per effettuare la lettura della data di sistema. Essendo globale, al termine dell'esecuzione della SRP mantiene il suo valore modificato nella subroutine.

Caso 2: richiamo come funzione

BC - SUB59_USO_SRP

```

' Caso 2: viene utilizzata la subroutine interna parametrica utilizzata come funzione
' riceve in ingresso due parametri di tipo intero e ritorna un valore di tipo intero

DIM varAddendo1[INT] = 10                                ' Variabile locale del corpo del
programma                                                ' Variabile locale del corpo del
DIM totale[INT]                                          programma
totale = SommaFunzione(varAddendo1, 20)

' Visualizzo il risultato tramite MESSAGEBOX andando a cambiare il formato dei tipi di dato
INT in STRING
MESSAGEBOX(#INFO, "CBAS_59", " Caso 2: il risultato della somma di
" + varAddendo1.ToString() + " + 20 è " + totale.ToString() )

'@SRP SommaFunzione(inAddendo1[INT] [IN], inAddendo2[INT] [IN]) AS Risultato[INT]

' inAddendo1 e inAddendo2 sono variabili locali di questa SRP

Risultato = inAddendo1 + inAddendo2

Return

```

Questa SRP ha un parametro di ritorno, per cui si presta ad essere richiamata come una funzione.

Anche in questo caso, le due variabili inAddendo1 e inAddendo2 sono locali alla SRP, per cui non sono disponibili dopo l'esecuzione della subroutine.



Per ulteriori dettagli riguardo la dichiarazione di variabili, consultare la scheda [DIM - Istruzione per la dichiarazione di variabili](#) del *Manuale di riferimento del BC*.



Per ulteriori dettagli sulla strutturazione del codice è possibile consultare la scheda [Regole generali](#) della *Guida allo sviluppo*.

Versionamento sub

Il versionamento delle sub consente di aggiungere nuovi parametri all'interfaccia di una funzione, senza doverne creare una nuova, e soprattutto mantenendo la compatibilità con il pregresso.

Versionare una sub vuol dire aggiungere un set di parametri in interfaccia, che verranno rilasciati con una versione del prodotto, etichettandoli con un valore via via crescente.

Ogni nuova versione conterrà un set di parametri che si andrà ad aggiungere a quelli già esistenti.

E' importante precisare che il versionamento può essere utilizzato solo nei casi in cui si aggiungano nuovi parametri all'interfaccia, e questi vengano accodati a quelli esistenti; in tutti gli altri casi (eliminazione parametri, inserimento in altre posizioni) occorre procedere con la creazione di una nuova sub.

Il versionamento è supportato per le chiamate in *CALL*, '@SBI, '@SBC, e tramite l'utilizzo di metodi, ma NON è supportato per le chiamate in '@SBL.

Il riferimento alla versione va indicato vicino al parametro con la sintassi *[VER=Valore]*. Per ogni parametro è anche possibile indicare un valore di default da assegnare al parametro, che sarà utilizzato solo nei chiamanti delle versioni precedenti a quella cui fa riferimento il parametro. L'indicazione del parametro va effettuata con la sintassi

[DEFAULT=Valore].

Di seguito un esempio di come evolvono le chiamate a seguito della nascita di nuove versioni:

Versione base

BC - SUB60_VERSIONAMENTO

```
'@PG SUB60_VERSIONAMENTO STRICT[1]      ' Esercitazione 60: Utilizzo di sub versionate

'@SBP ParametroIniziale1[INT]            [IN]                _ ' Primo
parametro iniziale
    ParametroIniziale2[STRING]           [IN]                ' Secondo
parametro iniziale

End
```

e relativa chiamata

BC - CBAS_60

```
'@PG CBAS_60 STRICT[1]      ' Esercitazione 60: Utilizzo di sub versionate

    DIM inParametro1[INT]
    DIM inParametro2[STRING]

    CALL SUB60_VERSIONAMENTO(inParametro1, inParametro2)

End
```

Aggiunta di nuovi parametri e nascita della Versione 1

BC - SUB60_VERSIONAMENTO

```
'@PG SUB60_VERSIONAMENTO STRICT[1]      ' Esercitazione 60: Utilizzo di sub versionate

'@SBP ParametroIniziale1[INT]            [IN]                _ ' Primo
parametro iniziale
    ParametroIniziale2[STRING]           [IN]                _ ' Secondo
parametro iniziale
        ParStringa[STRING]               [IN]    [VER=1] [DEFAULT="TEST"] _ ' Stringa
        ParDouble[DOUBLE]                [IN]    [VER=1] [DEFAULT=1000000] _ ' Double
        ParData[DATE]                    [IN]    [VER=1] [DEFAULT=01012019] _ ' Data
        ParIntero[INT]                   [IN]    [VER=1] [DEFAULT=1234]    _ ' Intero

        ...

End
```

e relativa chiamata

BC - CBAS_60


```
'@PG CBAS_60 STRICT[1]      ' Esercitazione 60: Utilizzo di sub versionate

DIM inParametro1[INT]
DIM inParametro2[STRING]

DIM inVer1ParStringa[STRING]
DIM inVer1ParDouble[DOUBLE]
DIM inVer1ParData[DATE]
DIM inVer1ParIntero[INT]

CALL SUB60_VERSIONAMENTO(inParametro1, inParametro2, _
                        inVer1ParStringa, inVer1ParDouble, inVer1ParData, inVer1ParIntero)

End
```

Aggiunta di nuovi parametri e nascita della Versione 2

```
BC - SUB60_VERSIONAMENTO

'@PG SUB60_VERSIONAMENTO STRICT[1]      ' Esercitazione 60: Utilizzo di sub versionate

'@SBP ParametroIniziale1[INT]           [IN]           _ ' Primo
parametro iniziale
ParametroIniziale2[STRING]             [IN]           _ ' Secondo
parametro iniziale
ParStringa[STRING]                     [IN]   [VER=1] [DEFAULT="TEST"] _ ' Stringa
ParDouble[DOUBLE]                      [IN]   [VER=1] [DEFAULT=1000000] _ ' Double
ParData[DATE]                          [IN]   [VER=1]                _ ' Data
ParIntero[INT]                         [IN]   [VER=1] [DEFAULT=1234] _ ' Intero
ParCollStringhe[TIPO[BCSTRING] COLLEZIONE] [INOUT] [VER=2]          _ '
Collezione di stringhe
ParPeriodo[PERIOD]                     [OUT]   [VER=2]                ' Periodo

...

End
```

e relativa chiamata

```
BC - CBAS_60
```

```
'@PG CBAS_60 STRICT[1]      ' Esercitazione 60: Utilizzo di sub versionate

DIM inParametro1[INT]
DIM inParametro2[STRING]

DIM inVer1ParStringa[STRING]
DIM inVer1ParDouble[DOUBLE]
DIM inVer1ParData[DATE]
DIM inVer1ParIntero[INT]

DIM inVer2ParCollStringhe[TIPO[BCSTRING] COLLEZIONE]
DIM inVer2ParPeriodo[PERIOD]

CALL SUB60_VERSIONAMENTO(inParametro1, inParametro2, _
                        inVer1ParStringa, inVer1ParDouble, inVer1ParData, inVer1ParIntero, _
                        inVer2ParCollStringhe [INOUT], inVer2ParPeriodo [OUT])

End
```

A fronte della nascita di nuove versioni, le chiamate effettuate alle versioni precedenti continuano a essere supportate, quindi anche a fronte della nascita della versione 2, la chiamata alla versione base continuerà a funzionare. In questi casi, i parametri delle versioni successive saranno valorizzati automaticamente con il valore impostato nel default, qualora questo sia stato indicato.

La versione da richiamare viene determinata in fase di traduzione in base al numero di parametri indicati. Di conseguenza è sempre necessario specificare tutti i parametri previsti da una determinata versione.

Parametri passati sulla riga di comando

I programmi BC, quando sono eseguiti, possono ricevere dalla riga di comando delle variabili utili alla loro esecuzione.

Sulla riga di comando possono essere passate variabili "pilota" per l'esecuzione oppure dei parametri definiti dal programma chiamante.

Lo stesso SisMenu utilizza la riga di comando per il passaggio di variabili ai moduli richiamati e anche quando un modulo viene eseguito da AGIS è possibile indicare tra i parametri in esecuzione eventuali valori:

Operatore CALDERA [CB] - Profilo di stazione per esecuzione programma Windows

Avvio programma main MAIN_CBASE modulo CBAS_01

Area BLD_DEMO Prodotto BLD_DEMO

Coordinate di esecuzione

Area di Lavoro: BUILDER_DEMO ☐ Modifica coordinate manualmente

Rilevanza Windows: Win32

Cartelle

PHB - Archivi di base

PHD - Archivi dati

PHP - Programmi

PHG - Archivi DR

PHE - Dati tabelle

PHSXN - Dizionari SXN

PHSXF - Formati SXF

Database

Nome server

Tipo autenticazione

Nome utente server

Password utente server

Nome database

Contesto di esecuzione

Gruppo in elaborazione

Anno IVA/Dichiarazioni

Codice ditta: 0

Ragione sociale

Stazione principale (WSP): W1

Stazione esecuzione (WS): W1

Sigla supermenu' (PRP): BLDD

Sigla procedura (PX): \$W1

Operatore: ADMIN

Profilo

Altri parametri di esecuzione

Parametri esecuzione

☐ Esegui in locale

☐ Trace Mode

☐ Controllo protezioni

Profiling mode: Applicativo

☐ Applica configurazione progetti di personalizzazione

DLL ambiente

Cartella debugger C

OK (Invio) Abbandona (F1) Opzioni esecuzione locale (F2) Esporta profilo esecuzione (F3)

Se selezionato, il programma verrà lanciato nella cartella C:\AGISEXE CALDERA GG/CB/CB

All'interno del modulo è possibile avere a disposizione tali valori all'interno di variabili di programma utilizzando la specifica '@GETARG.

Utilizzo di commenti nel sorgente

Il linguaggio BC, come tutti i linguaggi di programmazione, permette al programmatore di inserire nel testo commenti di carattere documentativo, con lo scopo di descrivere indifferentemente aspetti applicativi e tecnici, utili alla comprensione del codice.

Esistono anche altri due tipi di commenti documentativi, che integrati con l'ambiente di sviluppo e opportunamente utilizzati, sono in grado di fornire in modo automatico e sempre aggiornato, una documentazione applicativa espressa sotto forma di diagramma di flusso.

Riassumendo, nel linguaggio BC è possibile avere tre tipologie diverse di commenti:

- Commenti semplici
- Commenti Speciali, primari e secondari
- Metadati

I commenti speciali permettono a BCDevelop di presentare e navigare in tempo reale il flow chart del programma, mentre i metadati permettono ad AGIS di raccogliere e pubblicare in modo organizzato i flowchart prodotti anche al di fuori dello sviluppo.

I commenti semplici

Tutte le righe, anche quelle che continuano sulla riga successiva concatenate tramite l'underscore, accettano una descrizione di commento che deve essere preceduta dal carattere " ' " (singolo apice).

Esempio

```
'@BS VIDEO   ' Videata dati
...
'@SR CALCOLI ' Calcolo totali parziali
...

' Semplice riga di commento
If MESSAGEBOX(#WARNING,"", "Riga messaggio<br>Altra riga") = BCTD_MESSAGEBOX_AZIONE.Conferma Then

    ...

EndIf
```

È buona norma corredare il sorgente di commenti significativi per rendere più chiaro e facilmente leggibile il codice scritto. Prima o poi infatti qualcun altro dovrà modificare o estendere il codice e la mancanza di commenti renderà il lavoro nettamente più difficile.



La convenzione utilizzata nella scrittura dei sorgenti standard dei prodotti Sistemi prevede di iniziare a scrivere le righe di sorgente dalla colonna 7 di ogni riga; si tratta di una eredità dal vecchio basic che in passato dava al programmatore la possibilità di indicare label numeriche nelle 5 colonne iniziali della riga per eventuali salti di esecuzione con istruzioni *GOTO* e *GOSUB* che è stata mantenuta sul BC.

Questo tipo di convenzione è da ritenersi obsoleta poiché ormai si è passati a scrivere le righe del sorgente a partire dalla colonna 1

I Commenti speciali primari

I commenti che già oggi arricchiscono il sorgente NON sono automaticamente trasformati in commenti validi ai fini della costruzione dei flow chart, ma possono esserne la base di partenza.

Quelli significativi vanno puntualmente "attivati" e quasi sicuramente "migliorati" per essere resi comprensibili anche all'esterno del sorgente da un Applicativo.

Per rendere attivo un commento è sufficiente posporre il simbolo (^) al simbolo classico che identifica l'inizio di un commento (').

Perciò i commenti validi per il flow chart saranno solo quelli così composti:

BC

```
'^ Commento valido per la generazione del flow chart  
  
' Commento non valido per la generazione del flow chart
```

Commenti speciali su righe consecutive originano un commento **multilinea**:

BC

```
'^ Riga 1 Commento multilinea  
'^ Riga 2 Commento multilinea
```



In senso generale un diagramma di flusso può arrivare a dettagliare ogni singola operazione fatta dal sorgente, ma NON è questo l'obiettivo.

L'obiettivo è «sintetizzarne» la logica generale per renderla «compatta» e facilmente comprensibile.

Il rapporto di «compressione» deve essere ALTO.

Deve emergere la logica generale, il flusso, NON i dettagli implementativi.



Tutti i sorgenti di esempio BCFLOW_*.BC sono consultabili all'interno del Prodotto BUILDER/DEMO.

Di seguito un esempio di codice con commenti speciali e il flow chart generato.

BC

```
'@PG STAMPA_ESTRATTOCONTO      ' Stampa estratto conto

'@SBP  MOD%[1]      ' I O Commento

Condizione[1]=0
'@BS NEXT_CLIENTE '^ Ciclo di lettura sui Clienti
  '@BS NEXT_PARTITA '^ Ciclo di lettura sulle partite aperte
    '^La Partita è in scadenza entro data limite?
    If Condizione = 1 Then
      '^ Elabora saldo partita

      '^ Inserisco identificativo e saldo partira su collezione
    EndIf

    '^Ci sono altre partite da analizzare?
    If Condizione=1 Then
      GoTo NEXT_PARTITA '^Vai alla partita successiva
    Else
      '^Prepara stampa Estrattoconto
      GoSub PreparaStampaEstrattoconto
    EndIf

    '^Ci sono altri Clienti da analizzare?
    If Condizione=1 Then
      '^ Vai al cliente successivo
      GoTo NEXT_CLIENTE
    Else
      GoSub StampaEstrattoconto '^ Stampa estrattoconto Clienti
    endif

'@SR PreparaStampaEstrattoconto
  Return

'@SR StampaEstrattoconto
  Return

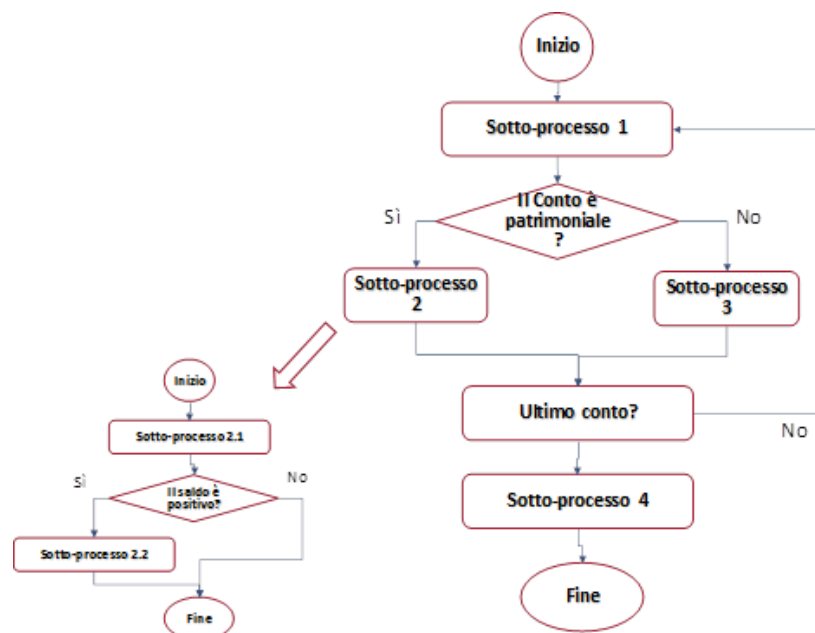
'@BS FINE
END
```



Gli elementi di base

Ogni processo può essere scomposto in:

- Un inizio
- Una fine
- Dei sotto-processi
- Delle scelte
- Delle iterazioni (o loop)
- Delle sequenze.



Inizio e fine di un processo

Ogni SUB è assimilabile ad un Processo

Il Processo è rappresentato solamente da **punto di inizio** e **punto di fine** se al suo interno non esistono sezioni documentate mediante commento speciale.

Il blocco di inizio contiene

- Nome della sub (Valore di PG)
- Commento laterale, semplice o speciale, della specifica PG

Esempio

BC

```
'@PG BLD_FLOW_INIZIO_FINE      ' Commento laterale specifica PG
'@SBP ARG[1]

GoSub Inizializza
GoSub Elabora
GoSub Stampa

End
```

Processo sequenziale

Ogni commento speciale ('^') documenta il codice compreso tra se stesso e

- il commento speciale seguente
- la fine di un blocco di codice (END, RETURN, ENDIF, ELSE...)

Un commento speciale ('^'):

- Può precedere o affiancare una riga di codice
- Può essere multilinea.

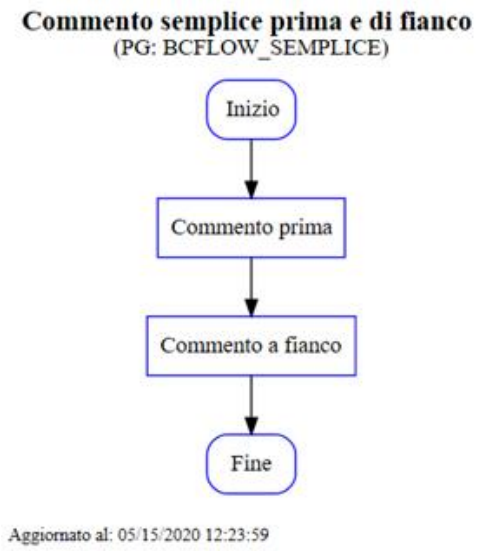
Esempio

BC

```
'@PG BLD_FLOW_SEMPlice      ' Commento semplice prima e di fianco
'@SBP ARG[1]

'^Commento prima
X[1]=0
Y[1]=1 '^ Commento a fianco

End
```

Sotto-Processo (GOSUB)

Un commento speciale, mono o multilinea, che precede o affianca una parola chiave GOSUB, descrive sinteticamente il processo definito dal codice contenuto sulla SR corrispondente.

Il dettaglio del processo deve essere poi descritto dai commenti speciali contenuti nel blocco SR, raggiungibile con un click.

Esempio

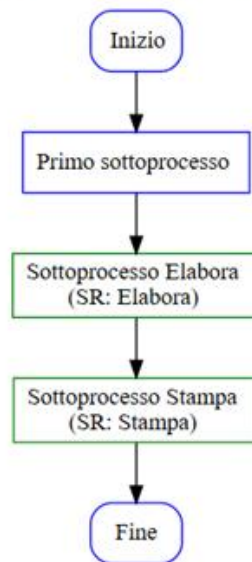
BC

```
'@PG BLD_FLOW_INIZIO_GOSUB      ' Commento laterale specifica PG
'@SBP ARG[1]

'^ Primo sottoprocesso
X[1]=1
Y[1]=1
Z[1]=1
X=Y+Z

'^ Sottoprocesso Elabora
GoSub Elabora
GoSub Stampa '^ Sottoprocesso Stampa

End
```

Commento laterale specifica PG
(PG: BCFLOW_INIZIO_GOSUB)

Aggiornato al: 05/15/2020 13:32:59

Può succedere che si inserisca un commento speciale su una GOSUB, ma che la SR corrispondente non contenga commenti speciali.

Poiché è presumibile che questa sia una situazione incoerente, il flow chart della SR la evidenzia con un blocco specifico che evidenzia l'incongruenza.

Esempio

BC

```

'@PG BLD_FLOW_INIZIO_GOSUB2      ' Commento laterale specifica PG
'@SBP ARG[1]

'^ Primo sottoprocesso
X=Y+Z
GoSub Elabora
GoSub Stampa '^ Sottoprocesso Stampa

End

'=====
'@SR Elabora ' Elaborazione
'=====

'^ Terzo sottoprocesso
Y[1]=1

Return

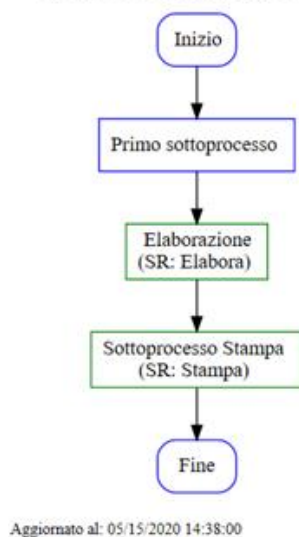
'=====
'@SR Stampa ' Esecuzione stampa
'=====

W[1]=X+Y+Z

Return

```

Commento laterale specifica PG (PG: BCFLOW_INIZIO_GOSUB2)



Esecuzione stampa (SR: Stampa)



Un processo GOSUB compare ugualmente nel flusso se la SR riferita contiene un sotto-processo documentato al suo interno.

Sotto-Processo (SBC-SBI-SBL)

A differenza dei sottoprocessi interni al sorgente, non viene analizzata la presenza di commenti speciali nel sorgente esterno:

Perciò un processo esterno è:

- Rappresentato se documentato mediante commento speciale (^)
- Ignorato se non documentato.

Esempio

BC

```
'@PG BLD_FLOW_SBC      ' Commento laterale Sorgente BCFLOW_SBC
'@SBP ARG[1]

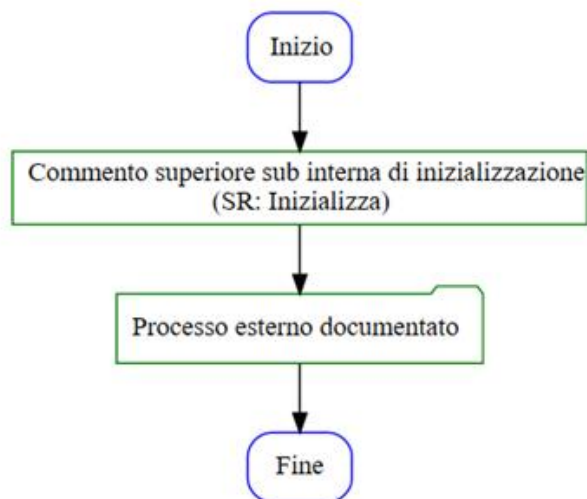
'^ Commento superiore sub interna di inizializzazione
GoSub Inizializza

'^ Processo esterno documentato
'@SBC BCFLOW_FOR _ ' Sorgente - Inizio Fine
    ARG[1]

' Richiamo a processo esterno (non documentato)
'@SBC BLD_FLOW_FOR _ ' Sorgente 1 - Inizio Fine
    ARG[1]      '

End
```

Commento laterale Sorgente BCFLOW_SBC (PG: BCFLOW_SBC)



Aggiornato al: 05/15/2020 15:01:54

Scelta (IF)

Un punto di scelta viene rappresentato se almeno uno dei sottoprocessi che origina è documentato.

Altrimenti viene ignorato perché non significativo.

Esempio

BC

```
'@PG BLD_FLOW_IFELSE      ' Commento laterale specifica PG
'@SBP ARG[1]

'^ Primo sottoprocesso
X[1]=1
Y[1]=1
Z[1]=1

If X=2 then X=Y+Z

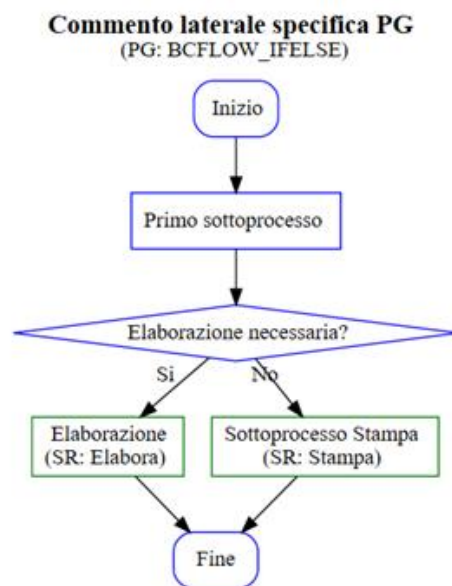
'^ Elaborazione necessaria?
If X=1 then
  GoSub Elabora
else
  GoSub Stampa '^ Sottoprocesso Stampa
endif

End

'=====
'@SR Elabora ' Elaborazione
'=====

'^ Terzo sottoprocesso
Y[1]=1

Return
```



Aggiornato al: 05/15/2020 15:05:50

Scelta (IF-ELSEIF)

Un punto di scelta viene rappresentato se almeno uno dei sottoprocessi che origina è documentato.

Altrimenti viene ignorato perché non significativo.

Esempio

BC

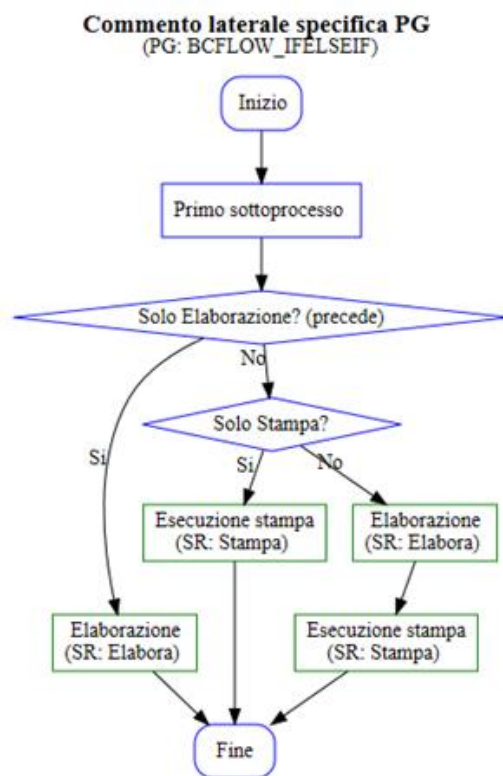
```
'@PG BLD_FLOW_IFELSEIF      ' Commento laterale specifica PG
'@SBP ARG[1]

'^ Primo sottoprocesso

If X=2 then X=Y+Z

'^ Solo Elaborazione? (precede)
If X=1 then
  GoSub Elabora
elseif X=2 then  '^ Solo Stampa?
  GoSub Stampa
else
  GoSub Elabora
  GoSub Stampa
endif

End
```



Aggiornato al: 05/15/2020 15:12:52

Scelta (SWITCH)

Una condizione SWITCH viene rappresentata se almeno uno dei sottoprocessi che origina è documentato, altrimenti viene ignorato perché non significativo.

La condizione SWITCH può essere descritta con un commento precedente o di fianco

I singoli CASE possono essere descritti con un commento speciale di fianco, e in assenza di commenti speciali la descrizione sarà calcolata di default

Esempio

BC

```
'@PG BLD_FLOW_SWITCH      ' Stampa estratto conto
'@SBP ARG[1]

'^ Primo sottoprocesso  ' Commento laterale specifica PG
X[1]=1
Y[1]=1
Z[1]=1

If X=2 then X=Y+Z

Switch X '^ Descrizione seguente switch

Case 1 ' ^ Solo Elaborazione?
    GoSub Elabora

Case 2 '^ Solo Stampa?
    '^ Entro nel caso 2

    GoSub Stampa

Case 3 '^ Solo caso 3?

    '^ Entro nel caso 3

    GoSub Stampa

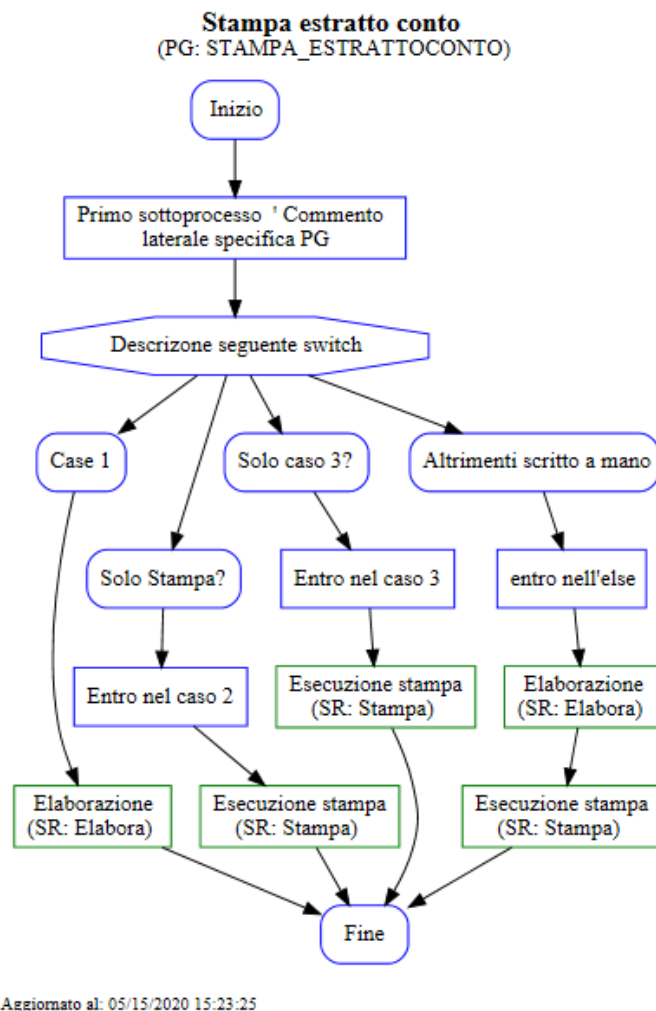
Case else '^ Altrimenti scritto a mano

    '^ entro nell'else

    GoSub Elabora
    GoSub Stampa

endswitch

End
```



Ciclo (FOR)

Un ciclo (FOR) viene rappresentato se il sottoprocesso iterato è documentato.

Il sottoprocesso iterato è documentato, se:

- È presente almeno un commento speciale
- Se contiene almeno un sottoprocesso documentato.

La descrizione del ciclo è:

- Automatica
- Acquisita dal commento speciale che precede immediatamente o che segue la parola chiave FOR.

Esempio

BC


```
'@PG BLD_FLOW_FOR      ' Commento laterale specifica PG
'@SBP ARG[1]

'^ Primo sottoprocesso
X[1]=1
Y[1]=1
Z[1]=1

For Y=1 To 5
    X=Y+Z
Next X

'^ Per tutti gli elementi
For X=1 To 5
    GoSub Elabora
    GoSub Stampa
Next X

'^ Con commento di fianco

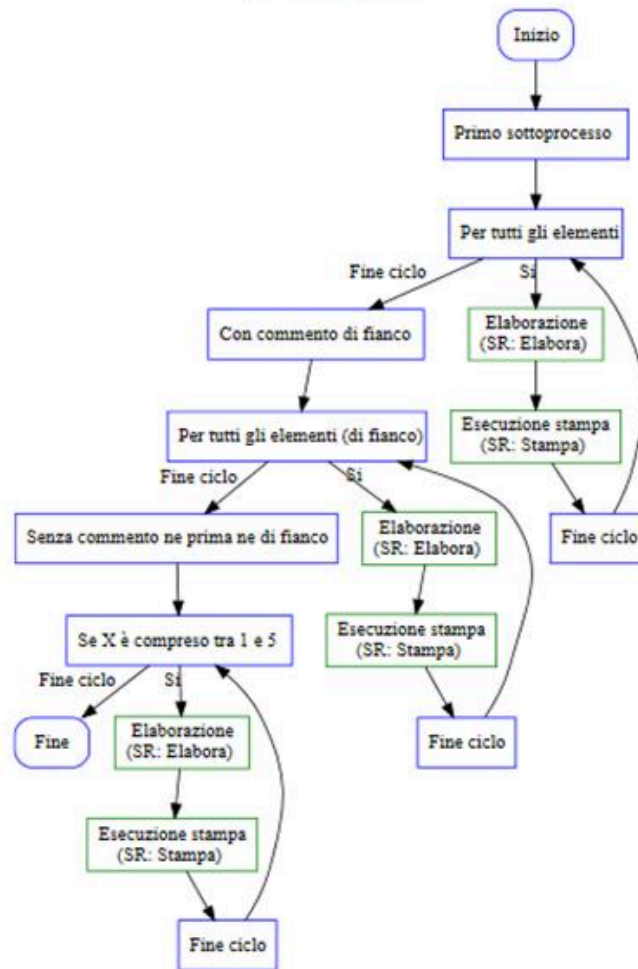
For X=1 To 5 '^ Per tutti gli elementi (di fianco)
    GoSub Elabora
    GoSub Stampa
Next X

'^ Senza commento ne prima ne di fianco

For X=1 To 5
    GoSub Elabora
    GoSub Stampa
Next X

End
```

Commento laterale specifica PG
(PG: BCFLOW_FOR)



Aggiornato al: 05/15/2020 15:40:15

Ciclo (FOREACH)

Un ciclo (FOREACH) viene rappresentato se il sottoprocesso iterato è documentato.

Il sottoprocesso iterato è documentato, se:

- È presente almeno un commento speciale
- Se contiene almeno un sottoprocesso documentato.

La descrizione del ciclo è:

- Automatica
- Acquisita dal commento speciale che precede immediatamente o che segue la parola chiave FOREACH.

Esempio

BC

```

'@PG BLD_FLOW_FOREACH      ' Commento laterale specifica PG
'@SBP ARG[1]

'^ Primo sottoprocesso
X[1]=1
Y[1]=1
Z[1]=1

DIM ELE_CLIENTI[CLASSE[BLD_CLIFOR] COLLEZIONE]
DIM TMP_CLIENTE[CLASSE[BLD_CLIFOR]]

'^ 1 Elabora tutti gli elementi
ELE_CLIENTI.FOREACH(TMP_CLIENTE, GOSUB Elabora)
ELE_CLIENTI.FOREACH(TMP_CLIENTE, GOSUB Elabora) '^ 2 Elabora tutti gli elementi
ELE_CLIENTI.FOREACH(TMP_CLIENTE, GOSUB Elabora)
GoSub Elabora

End

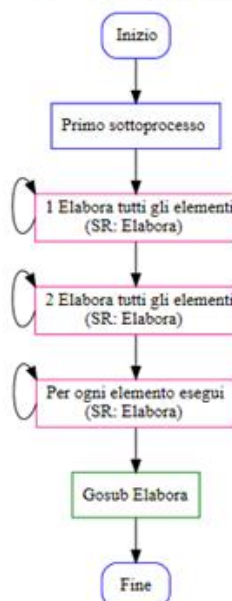
'=====
'@SR Elabora
'=====

'^ Terzo sottoprocesso
Y[1]=1

Return

```

Commento laterale specifica PG (PG: BCFLOW_FOREACH)



Aggiornato al: 05/15/2020 15:49:01

Ciclo (WHILE)

Un ciclo (WHILE) viene rappresentato se il sottoprocesso iterato è documentato.

Il sottoprocesso iterato è documentato, se:

- È presente almeno un commento speciale
- Se contiene almeno un sottoprocesso documentato.

La descrizione del ciclo è:

- Automatica
- Acquisita dal commento speciale che precede immediatamente o che segue la parola chiave WHILE.

Esempio

BC

```
'@PG BLD_FLOW_WHILE      ' Commento laterale specifica PG
'@SBP ARG[1]

'^ Primo sottoprocesso
X[1]=1
Y[1]=1
Z[1]=1

WHILE Y<5
    X=Y+Z
ENDWHILE

'^ La condizione è ancora vera?
WHILE Y<5
    GoSub Elabora
    GoSub Stampa
ENDWHILE

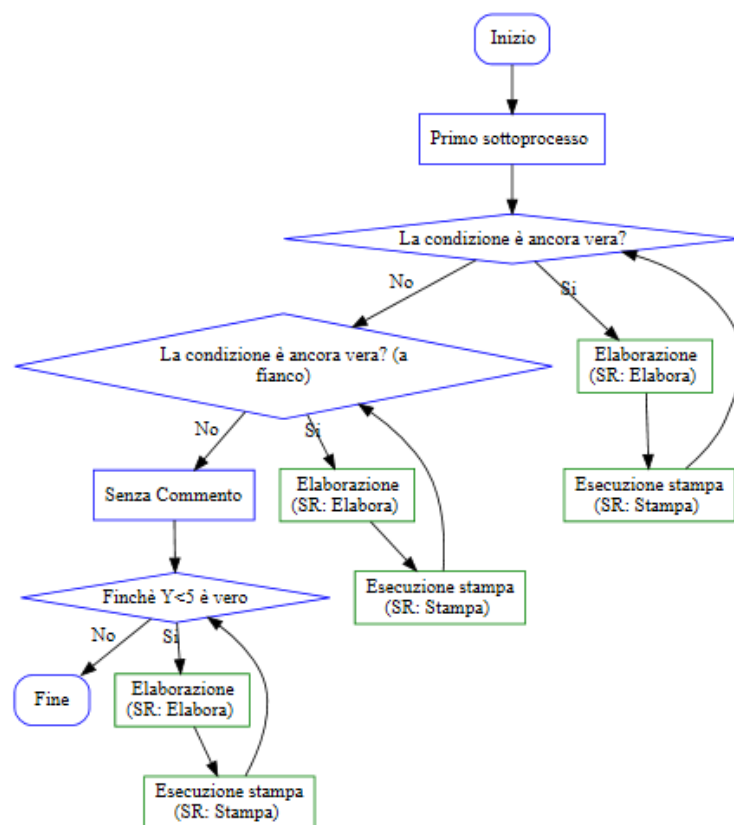
WHILE Y<5  '^ La condizione è ancora vera? (a fianco)
    GoSub Elabora
    GoSub Stampa
ENDWHILE

'^ Senza Commento

WHILE Y<5
    GoSub Elabora
    GoSub Stampa
ENDWHILE

End
```

Commento laterale specifica PG (PG: BCFLOW_WHILE)



Aggiornato al: 05/15/2020 15:57:31

Ciclo (BS-GOTO)

Un ciclo Bs-GOTO viene rappresentato se il sottoprocesso iterato è documentato.

Il sottoprocesso iterato è documentato, se:

- È presente almeno un commento speciale
- Se contiene almeno un sottoprocesso documentato.

Esempio

BC

```

'@PG BLD_FLOW_BS_GOTO      ' Commento laterale specifica PG
'@SBP ARG[1]

'^ Primo sottoprocesso
X[1]=1

'@BS CICLO_1

If ARG=0 And X=1 Then
    X=2
    GoTo CICLO_1
endif

'@BS CICLO2 '^ Ciclo documentato

If X=1 Then '^ Scelta valida?

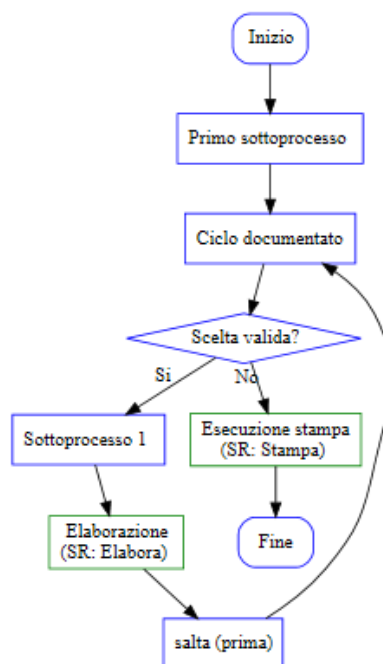
    '^ Sottoprocesso 1

    GoSub Elabora
    X=2
    '^ salta (prima)
    GoTo CICLO2
endif
GoSub Stampa

End

```

Commento laterale specifica PG (PG: BCFLOW_BS_GOTO)



Aggiornato al: 05/15/2020 16:10:48

Ciclo con continuazione ed uscita (EXITFOR-CONTINUE)

In presenza di interruzioni e salti nel ciclo.

Esempio

BC

```
'@PG BLD_FLOW_FORCONTINUEBREAK      ' Commento laterale specifica PG
'@SBP ARG[1]

'^ Primo sottoprocesso
X[1]=1
Y[1]=1
Z[1]=1

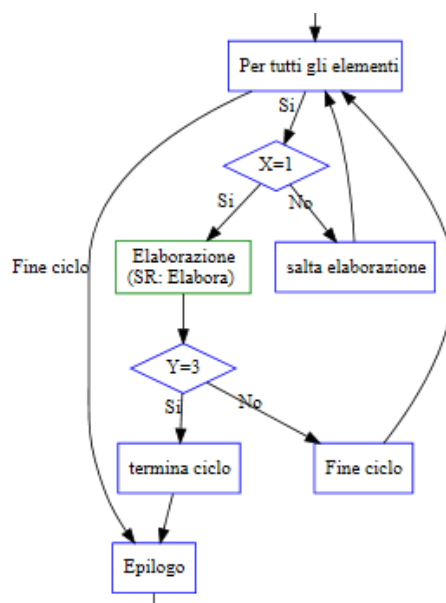
'^ Per tutti gli elementi
For X=1 To 5
    If X=1 then
        GoSub Elabora
    else
        Continue '^ salta elaborazione
    endif

    if Y=3 then
        exitfor '^ termina ciclo
    endif
Next X

'^ Epilogo

Z=5

End
```



Specifica con Sotto-Processo

Vengono rappresentate solo le specifiche BC che tra i valori dei parametri presentano sottoprocessi documentati (GOSUB documentate).

Il sottoprocesso specificato nel parametro è documentato, se:

- È presente almeno un commento speciale
- Se contiene almeno un sottoprocesso documentato.

Esempio

BC

```
'@PG BLD_FLOW_SPECIFICA      ' Commento laterale Sorgente BCFLOW_SPECIFICA
'@SBP ARG[1]

'@DEFVIDPROGRESSBAR  TIT["Titolo progress bar inizializzazione"] _
                     OGGETTO[PBAR] _
                     AZIONE[GOSUB Inizializza] _
                     DETTAGLIO[1]

'@DEFVIDPROGRESSBAR  TIT["Titolo progress bar di stampa"] _
                     OGGETTO[PBAR] _
                     AZIONE[GOSUB Stampa] _
                     DETTAGLIO[1]

End

'=====
'@SR Inizializza ' Inizializzazioni
'=====

'^ Commenti superiore Primo sottoprocesso
X[1]=1
Y[1]=1
Z[1]=1
DIM PBAR[CLASSE[BC_VID_PROGRESS] ]

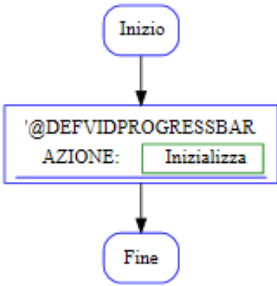
Return

'=====
'@SR Stampa ' Inizializzazioni
'=====

X=Y+Z

Return
```


Commento laterale Sorgente BCFLOW_SPECIFICA
(PG: BCFLOW_SPECIFICA)



Aggiornato al: 05/15/2020 16:35:29

Videata (DEFVID-RUNVID)

La logica di funzionamento di una videata non è sequenziale ma ad eventi

Difficile conciliare il funzionamento di una videata con questo tipo di rappresentazione

Al momento viene semplicemente rappresentata la sua presenza nel flusso.

Esempio

BC

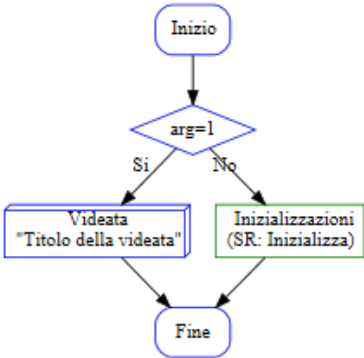
```
'@PG BLD_FLOW_VIDEATA      ' Commento laterale Sorgente BCFLOW_VIDEATA
'@SBP ARG[1]

If arg=1 Then
    '@DEFVID DIM[24,105] TIT["Titolo della videata"]

    '@RUNVID
else
    GoSub Inizializza
endif

End
```

Commento laterale Sorgente BCFLOW_VIDEATA
(PG: BCFLOW_VIDEATA)



Aggiornato al: 05/15/2020 16:39:18

Commenti speciali secondari

Sono dei commenti speciali (^) che permettono di inserire informazioni aggiuntive per la rappresentazione del flusso sul flowchart all'interno di un commento speciale multilinea.

I commenti secondari previsti sono i seguenti:

- '^ <C>
- '^ <E>
- '^ <G>
- '^ <S>

Commento secondario '^<C>

Mentre il **commento principale** è volto a spiegare il **cosa** fa il programma, in taluni casi può essere utile aggiungere commenti aggiuntivi volti a spiegare il **perché** lo fa o il **come** lo fa.

La sua rappresentazione sul flowchart è un callout del blocco cui si riferisce.

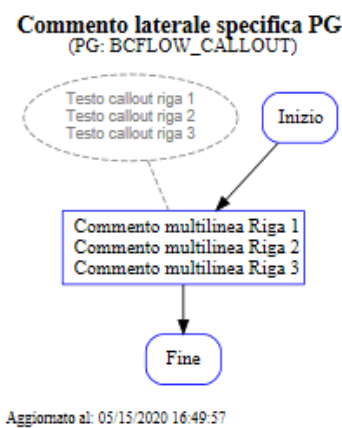
Esempio

BC

```
'@PG BLD_FLOW_CALLOUT      ' Commento laterale specifica PG
'@SBP ARG[1]

'^ Commento multilinea Riga 1
'^ Commento multilinea Riga 2
'^ Commento multilinea Riga 3
'^ <C> Testo callout riga 1
'^ <C> Testo callout riga 2
'^ <C> Testo callout riga 3

GoSub Inizializza
GoSub Elabora
GoSub Stampa
End
```



Aggiornato al: 05/15/2020 16:49:57

Commento secondario '^<E>

E' un commento secondario di formattazione.

Permette di evidenziare il blocco a cui si riferisce colorando il suo sfondo

Commento secondario '^<G>

E' un commento secondario di formattazione.

Permette di evidenziare il testo del blocco a cui si riferisce usando il carattere in grassetto

Commento secondario '^<S>

E' un commento secondario di formattazione.

Permette di evidenziare il testo del blocco a cui si riferisce usando il carattere sottolineato

Esempio

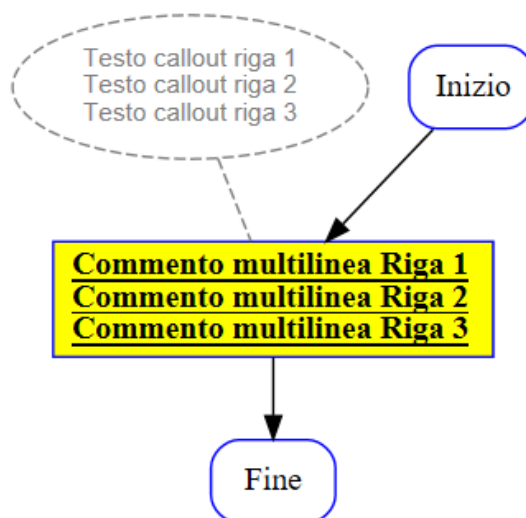
BC

```
'@PG BLD_FLOW_CALLOUT      ' Commento laterale specifica PG
'@SBP ARG[1]

'^ Commento multilinea Riga 1
'^ Commento multilinea Riga 2
'^ Commento multilinea Riga 3
'^ <C> Testo callout riga 1
'^ <C> Testo callout riga 2
'^ <C> Testo callout riga 3
'^ <S>
'^ <E>
'^ <G>

GoSub Inizializza
GoSub Elabora
GoSub Stampa
End
```

Commento laterale specifica PG (PG: BCFLOW_CALLOUT)



Aggiornato al: 05/15/2020 16:58:01

I Metadati

Anche in BC è stata introdotta la possibilità di inserire metadati all'interno del sorgente.

I metadati attualmente previsti sono:

- AREA
- SOTTOAREA
- DESCRIZIONE

Di seguito un esempio di utilizzo

BC

```

AREA["AREA AMMINISTRATIVA"]
SOTTOAREA["ELABORAZIONE BILANCIO"]
DESCRIZIONE["Stampa estratto conto"]

'@PG STAMPA_ESTRATTOCONTO      ' Commento laterale Sorgente BCFLOW_VIDEATA
'@SBP ARG[1]
  
```

I metadati sono opzionali, quando impiegati, in presenza di almeno un commento speciale, vengono utilizzati per l'organizzazione e la pubblicazione dei flow chart al di fuori dei tools di sviluppo.

Le etichette dei metadati sono fisse, mentre il loro valore è una stringa di testo libero.

Nomenclatura standard

L'obiettivo è arrivare ad avere una metodologia di rappresentazione dei processi **comune a tutti**, così da rendere la lettura dei flow il più semplice ed intuitivo possibile.

Per fare questo è sufficiente adottare una **nomenclatura standard** da applicare alla stesura dei commenti.

Di seguito vengono riportate le regole di base di composizione dei commenti.

Condizioni IF, ELSEIF, WHILE

Vanno sempre espresse come domanda esplicita (es.: La partita risulta saldata?)

Cicli di Lettura su DB e Strutture:

- Inizio ciclo: **"Ciclo su Xxxxx"** + eventuali info aggiuntive significative per il contesto, come ad esempio l'ordinamento e/o il posizionamento
- GOTO di riposizionamento: **"Xxxxx successivo"**
- Condizione di uscita: **"Ultimo Xxxxx?"**

Cicli FOR

- Inizio ciclo: **"Ciclo su Xxxxxxx"** + eventuali info aggiuntive significative per il contesto, come ad esempio valore iniziale e finale
- Next : **"Xxxxx successivo"**
- Continue: **"Xxxxx successivo"**

Integrazione con l'ambiente di sviluppo

La gestione dei flowchart è completamente integrata con AGIS e BCDevelop.

BCDevelop prevede una finestra di visualizzazione dedicata al proprio interno.

Il posizionamento del cursore sul sorgente effettua il posizionamento sul flowchart così come il clic sul flowchart effettua il posizionamento del cursore sul sorgente.

La generazione avviene su richiesta, mediante apposita funzione a menu, e automaticamente sia in fase di precompilazione che di compilazione, garantendo l'allineamento continuo col sorgente.

AGIS invece prevede una funzione di pubblicazione dei flowchart appartenenti ad un'area prodotto, in modo che la fruizione possa avvenire anche all'esterno dell'ambiente di sviluppo.

Creazione primo programma BC

Nello sviluppo di un prodotto Sistemi il sorgente BC passa attraverso diverse fasi fino ad arrivare alla sua forma definitiva di programma di prodotto.

Inizialmente si procede alla sua creazione andando a definire l'oggetto in AGIS e la sua collocazione in un main, se si tratta di un modulo, oppure in una dll se si tratta di una sub.

Successivamente seguirà la parte di sviluppo dove oltre all'editazione del sorgente BC si procederà all'eventuale definizione di ulteriori componenti per poi arrivare alla fase di lavoro in cui saranno eseguiti i primi test del programma.

Le fasi che il programma BC seguirà sono:

- editazione dei sorgenti che lo compongono
- traduzione e compilazione dei sorgenti
- link di main e dll
- esecuzione

Creazione di un nuovo programma

Il programma più semplice da creare è quello composto da un modulo collegato ad main. Inseriamo nel modulo il messaggio "Hello, World!".

Per prima cosa creiamo il modulo e, dopo aver inserito la descrizione, lo colleghiamo ad un main. Se il main non esiste è possibile crearlo in quel momento.

Terminata la creazione delle due anagrafiche in Agis si apre l'editor BCDevelop che ci consente di editare la nostra istruzione:

```
BC

'@PG CBAS_01 STRICT[2]      ' Esercitazione 1: Hello + Main'

    DIM varTesto[STRING]="Hello, World!"

    ' Emissione a video della variabile varTesto
    MESSAGEBOX(#INFO,"CBAS_01",varTesto)

End
```

Terminato di scrivere il nostro pezzo di codice dobbiamo tradurlo e compilarlo.

Traduzione e compilazione dei sorgenti

Terminata l'editazione, è necessario compilare il sorgente e linkare la DLL o il Main per far sì che venga generato l'eseguibile da poter eseguire.

Sono due azioni separate (compilazione e link), ma che possono essere effettuate anche insieme tramite opportune configurazioni di AGIS.



Per maggiori informazioni sulla configurazione di Agis è possibile visualizzare la scheda Parametri stazione di lavoro del manuale di AGIS.

La compilazione avviene tramite il compilatore Windows (come abbiamo visto nella prima parte del corso) che sicuramente non conosce il linguaggio BC.

C'è una fase molto importante che viene fatta prima della compilazione, ossia la traduzione del sorgente da linguaggio BC a linguaggio C.

La fase di traduzione prende il sorgente (modulo o sub) e crea il suo parallelo .c in modo che possa esser interpretato dal compilatore.

La traduzione potrebbe generare degli errori di tipo sintattico, ovvero di scrittura errata del sorgente BC. Ci viene in aiuto BCDevelop che, grazie al controllo sintattico, ci previene da eventuali possibili errori durante la fase di scrittura del sorgente.



Per maggiori informazioni sull'attivazione del controllo sintattico è possibile consultare la scheda [Controllo sintattico](#) del sorgente del manuale di *BCDevelop*.

Finita la traduzione, il sorgente viene compilato, generando il .obj che serve per generare la DLL o il main. Se la compilazione va in errore, allora l'.obj non viene creato e il sorgente verrà visualizzato nel pannello di AGIS (on in BCDevelop) con un colore rosso.

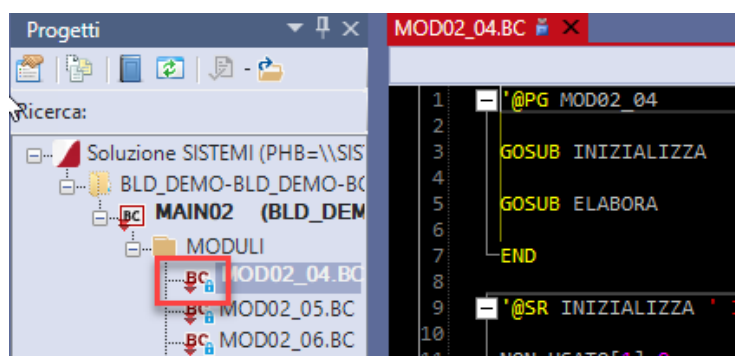


Sul pannello di AGIS è possibile visualizzare gli oggetti che devono essere compilati o linkati perché avranno una "A" nella colonna "Stato":

	Tipo	Nome	Descrizione	Stato
Mod	+	MOD01_08	Modulo esercizio 8 main 1	
Mod	+	MOD01_09	Modulo esercizio 9 main 1	
Mod	+	MOD02_04	Modulo esercizio 4 main 2	A
Mod	+	MOD02_05	Modulo esercizio 5 main 2	A

Per maggiori informazioni sugli stati degli oggetti è possibile consultare la scheda [Stati degli oggetti](#) del manuale di *AGIS*.

In BCDevelop lo stato dell'oggetto è visibile sull'albero del progetto dalla freccia rossa:



Per maggiori informazioni circa l'albero dei progetti è possibile consultare la scheda [Albero dei progetti](#) del manuale di *BCDevelop*.

Generazione DLL o eseguibile

Una volta eseguita la compilazione del sorgente, è necessario procedere con la generazione della DLL e/o del Main.

Se non venisse generata non si vedrebbero le modifiche apportate al programma.

Una volta ottenuto l'eseguibile o la DLL sarà possibile procedere con l'esecuzione vera e propria.

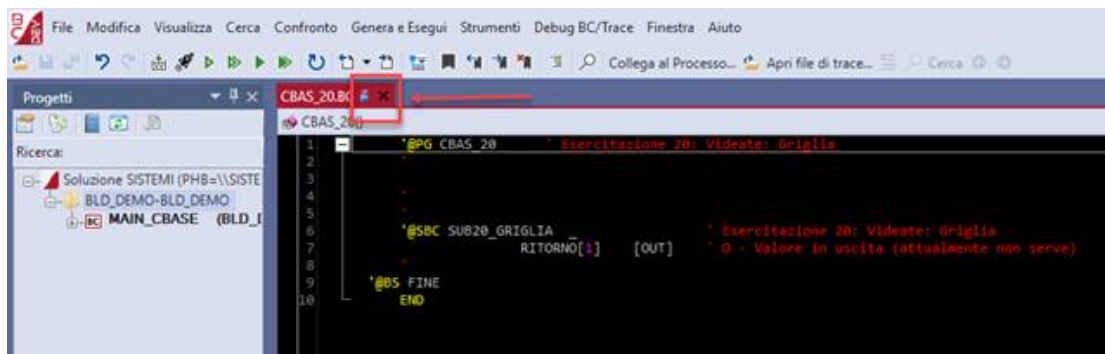
Esecuzione da BCDevelop

E' possibile eseguire ogni singola azione sugli oggetti tramite AGIS, su ogni oggetto fornisce un menù di azioni disponibili su cui agire e con cui il programmatore completa le varie fasi di sviluppo di un programma BC.

L'editor di sorgenti BCDevelop è stato notevolmente implementato e potenziato nel tempo tanto da renderlo fortemente integrato con gli strumenti dell'ambiente di sviluppo Builder e soprattutto con AGIS.

Dall'interno di BCDevelop è possibile eseguire molte operazioni e praticamente quasi tutte le azioni previste su un sorgente BC.

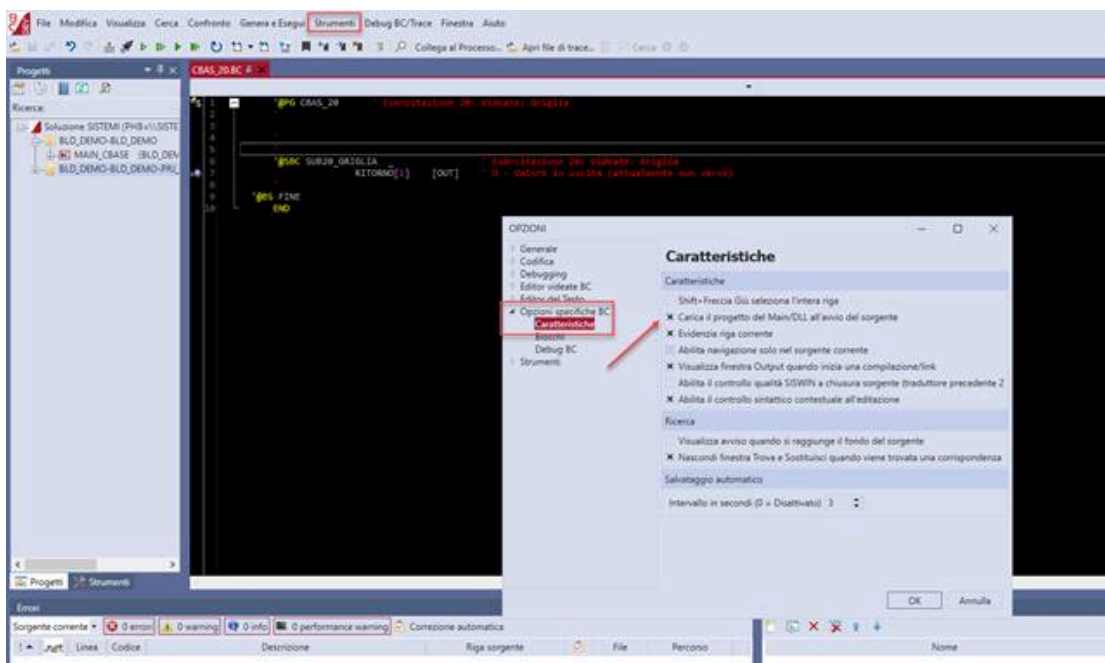
Ad esempio, entrando in visualizzazione di un sorgente:



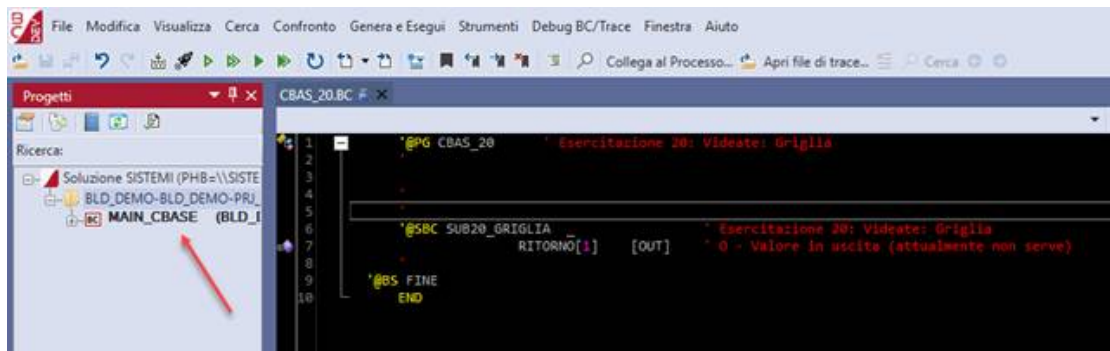
possiamo poi procedere alla sua editazione:



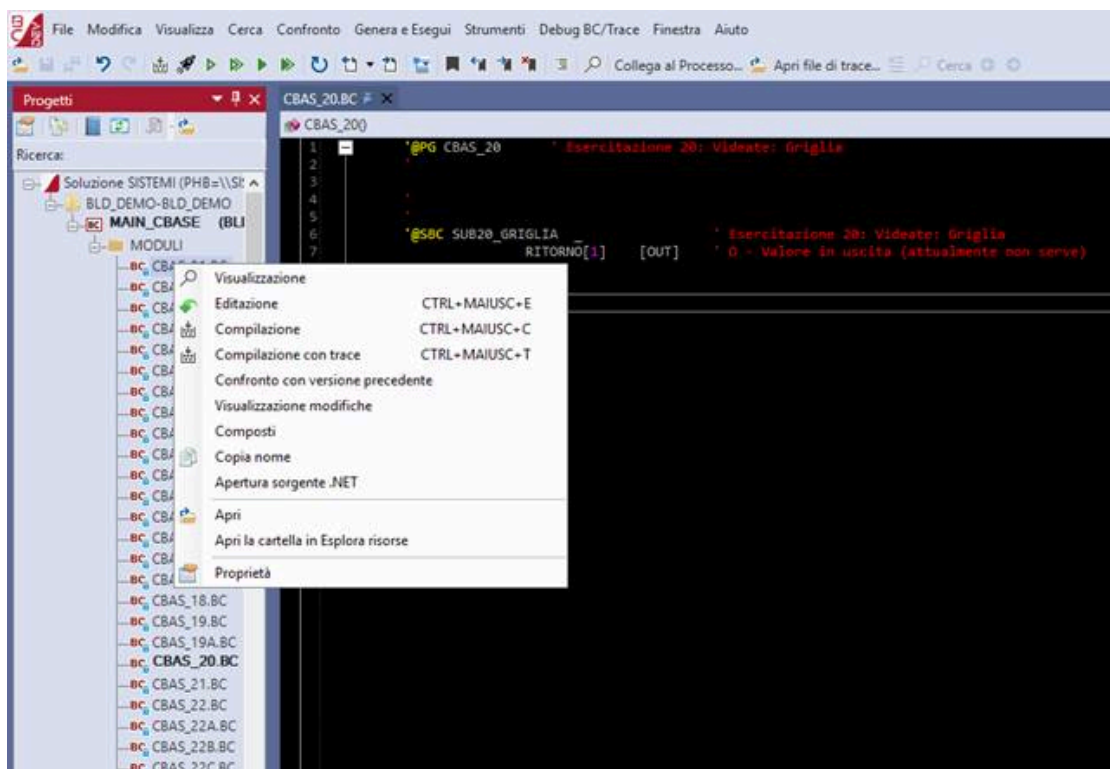
E' consigliabile impostare a 'si' l'opzione per caricare il progetto all'avvio del sorgente all'interno del menù strumenti:



in questo modo il progetto viene sempre caricato in modo automatico:



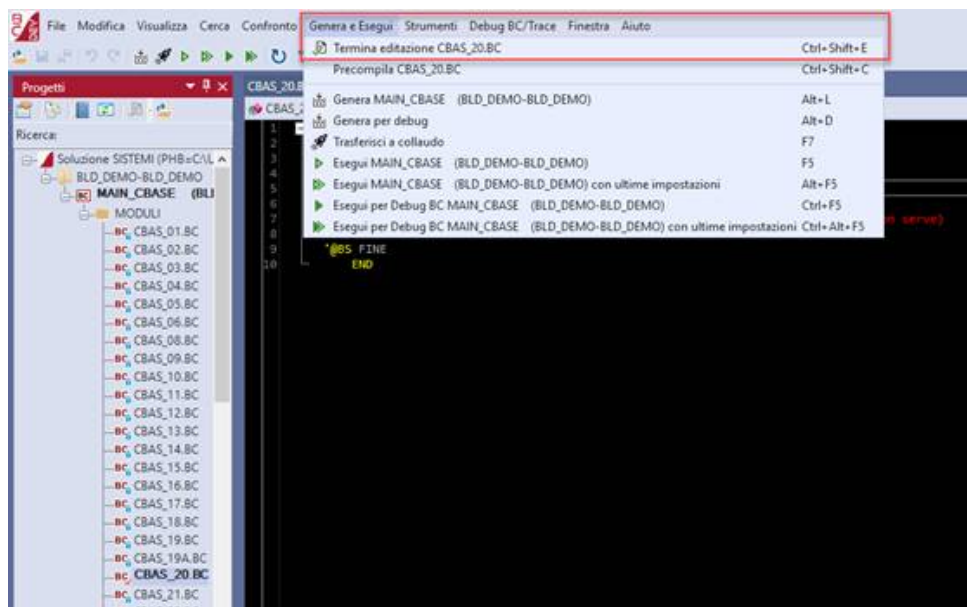
e attraverso il progetto è poi possibile lavorare sugli oggetti:



con le azioni fornite dal menù visualizzato premendo il tasto dx del mouse sul singolo oggetto.

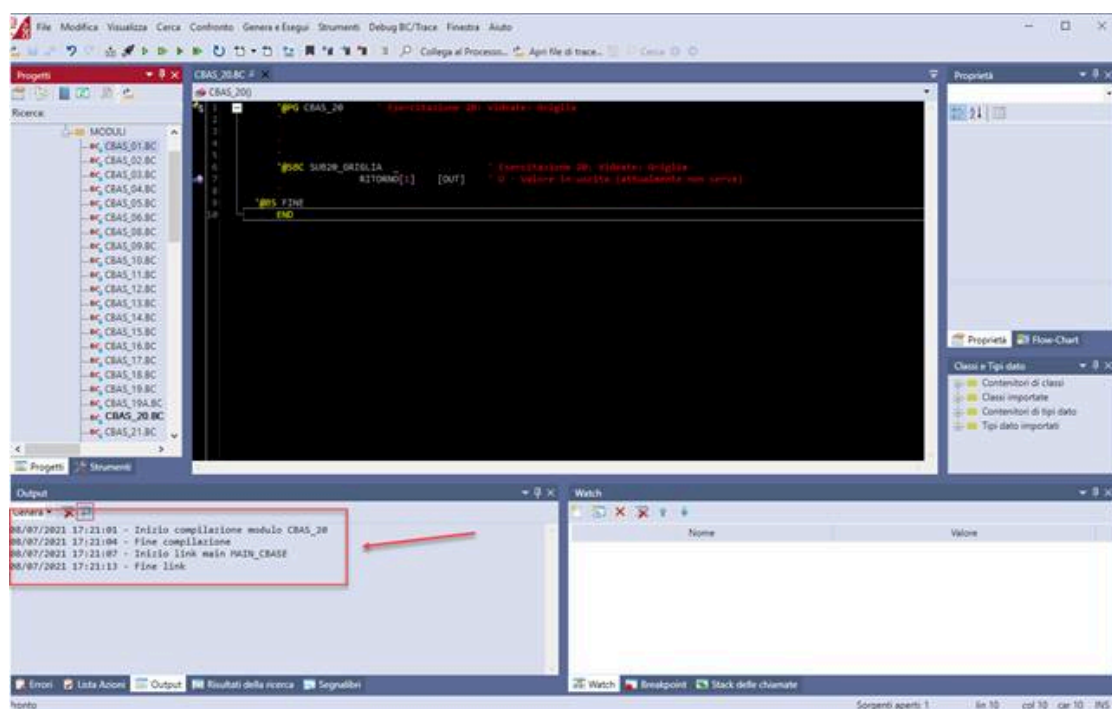
In pratica una volta entrati in BCDevelop non abbiamo bisogno di chiuderlo per tornare in AGIS e completare eventuali operazioni necessarie.

Ad esempio, dopo aver modificato un sorgente e aver completato le modifiche possiamo terminare l'editazione:

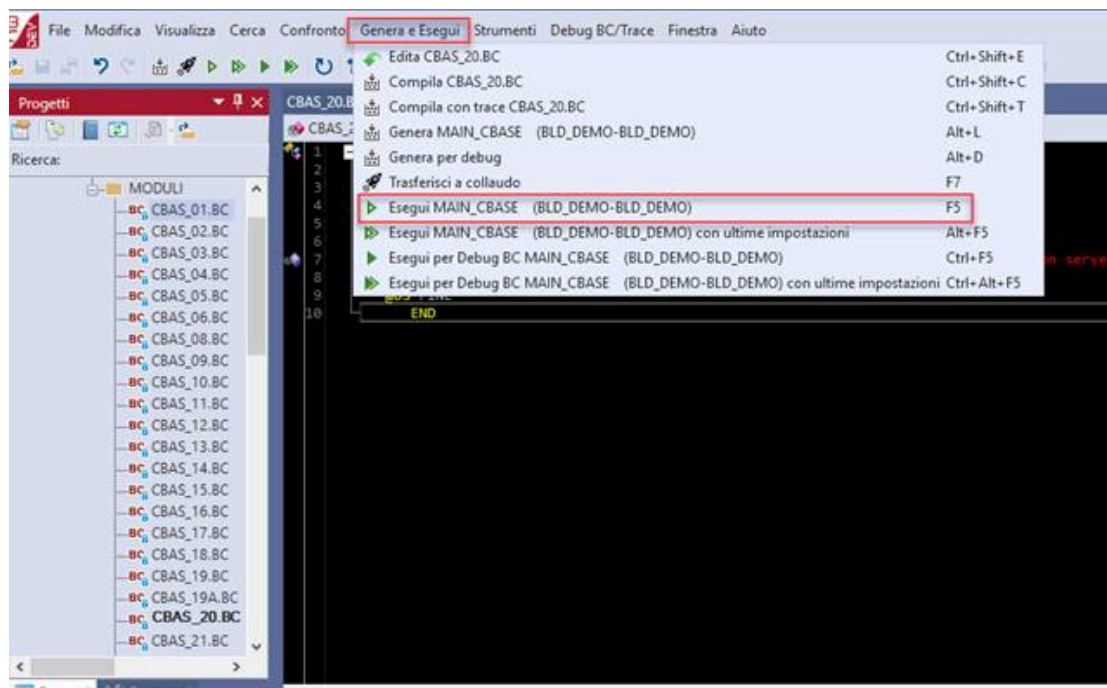


Terminando l'edizione sono eseguite le medesime operazioni, che sono state descritte sopra, che sono svolte da AGIS all'uscita da BCDevelop.

Successivamente BCDevelop esegue le varie operazioni evidenziando nella box output l'eventuale esito delle stesse:



Dopo aver completato la generazione dell'eseguibile è possibile eseguire il nostro programma:



Lavorando in questo modo è possibile fare tutte le operazioni da BCDevelop e non serve uscire e rientrare dall'editor per tornare in AGIS e selezionare eventuali operazioni da eseguire.



Si rimanda direttamente al [manuale di BCDevelop](#) per visualizzare tutte le potenzialità dell'editor consigliato.

L'editazione di un sorgente BC

Ogni sorgente che compone il nostro programma BC (modulo e/o sub) dovrà essere editato per poter inserire il codice che ci serve.

Quando si esegue l'azione di "editazione" di un sorgente BC vengono eseguite diverse operazioni dal pannello di Agis. L'azione dovrà essere eseguita su ogni sorgente che si intende modificare.

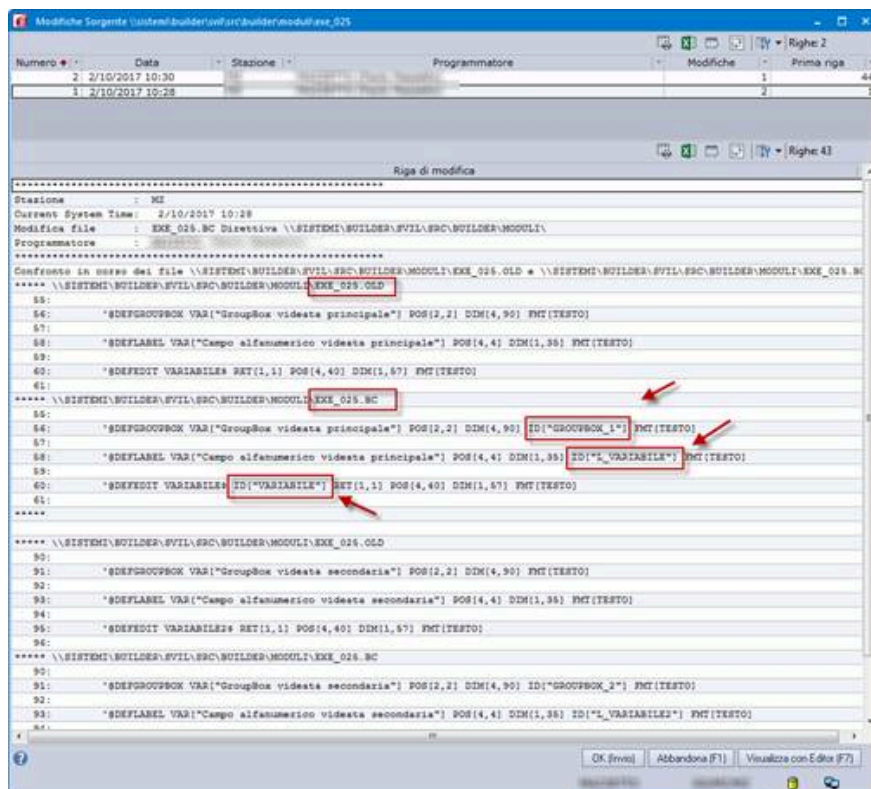
Oltre ad aprire il sorgente con BCDevelop, per poterlo modificare, vengono effettuate diverse operazioni:

- AGIS genera una copia fisica del sorgente .BC creando un file con estensione .OLD nella stessa cartella di collocazione del sorgente stesso;
- provvede al blocco dell'oggetto in AGIS per gestire la concorrenza
- procede con l'apertura del sorgente in BCDevelop;

Una volta completate le modifiche al sorgente, alla chiusura dell'editazione da BCDevelop, vengono eseguite le seguenti operazioni:

- Sblocco dell'oggetto in AGIS ed aggiornamento dell'anagrafica con la firma di ultima modifica
- Creazione del file .MOD risultante dal confronto tra file .BC aggiornato e file .OLD
- aggiornamento delle relazioni tra sorgente modificato e i suoi componenti

In questo modo AGIS gestisce la rilevazione di tutte le modifiche che sono apportate ad un sorgente e consente la loro visualizzazione con l'azione 'visualizza modifiche':



L'esecuzione di un programma BC

Quando un programma raggiunge una certa stabilità, possono essere eseguiti i primi test in esecuzione.

Un programma BC per poter essere eseguito ha bisogno dell'*ambiente di esecuzione* Sistemi e dell'eventuale base dati a lui necessaria.

Nella pratica un programma deve essere eseguito in un contesto di prodotto, dove sono disponibili l'ambiente, i dati e la configurazione del prodotto a cui il programma appartiene.

Deve quindi essere predisposta un'area di lavoro, prodotta tramite l'installazione di un prodotto Sistemi, che viene utilizzata come area di sviluppo utile per i test da effettuare sui programmi in corso di sviluppo.

L'area di lavoro

L'area di lavoro rappresenta quindi l'insieme di elementi utili all'esecuzione del programma come ad esempio la cartella base di riferimento e le coordinate del database.

Le aree di lavoro fanno parte della base dati di Builder e possono essere definite come globali, di prodotto, di progetto e di stazione(personali).

Quando poi si esegue un programma AGIS tramite l'azione "Esecuzione Windows" o direttamente da BCDevelop, viene richiesta l'area di lavoro da utilizzare per derivare tutte le informazioni necessarie alla corretta esecuzione:

AGXEXEC - Operatore CALDERA (0C) - Profilo di stazione per esecuzione programma Windows

Avvio programma main MAIN_CBAS modulo CBAS_20

Area BLD_DEMO Prodotto BLD_DEMO

Coordinate di esecuzione

Area di Lavoro: **BLD_DEMO** ☐ Modifica coordinate manualmente

Rilevanza Windows: Win32

Cartelle

PHB - Archivi di base: \\SISTEMI\BUILDER\COLL\VERIFICHE\BUILDERDEMO\

PHD - Archivi dati: \\SISTEMI\BUILDER\COLL\VERIFICHE\BUILDERDEMO\DATI\

PHP - Programmi: \\SISTEMI\BUILDER\COLL\VERIFICHE\BUILDERDEMO\PROG32\

PHG - Archivi DR

PHE - Dati tabelle

PHSXN - Dizionari SXN

PHSXF - Formati SXF

Database

Nome server: NISQL020

Tipo autenticazione: SQL Server

Nome utente server: lettore

Password utente server: *****

Nome database: BUILDERDEMO

Contesto di esecuzione

Gruppo in elaborazione: GG

Anno IVA/Dichiarazioni: 2021

Codice ditta: 0

Ragione sociale:

Stazione principale (WSP): 0C

Sigla supermenu' (PRP): BLDS

Operatore:

Stazione esecuzione (WS): 0C

Sigla procedura (PR): 40C

Profilo:

Altri parametri di esecuzione

Parametri esecuzione:

☐ Esegui in locale ☐ Trace Mode ☐ Controllo protezioni

Profiling mode: Applicativo

DLL ambiente:

Cartella debugger C:

OK (Invio) Abbandona (F1) Opzioni esecuzione locale (F2) Esporta profilo esecuzione (F3)

Selezione Area di Lavoro su cui operare GG/0C/0C

I dati possono essere eventualmente ulteriormente rettificati rispetto a quanto definito in anagrafica dell'area di lavoro per la singola esecuzione.

Oltre alle informazioni relative all'area di lavoro devono essere indicati alcuni dati utili all'esecuzione come il gruppo archivi e l'operatore. Possono essere inoltre impostati eventuali parametri da accodare alla riga di comando per l'esecuzione, quelli intercettabili tramite la specifica '@GETARG'.

Se non viene selezionata l'esecuzione in locale il programma viene eseguito sull'area di lavoro indicata, in tale situazione il main eseguito risiede nel prodotto/progetto di AGIS e l'ambiente di esecuzione utilizzato è quello dell'area di lavoro.

Se invece si seleziona l'esecuzione in locale può essere anche indicata la cartella in cui reperire l'ambiente di esecuzione da utilizzare, non specificato viene preso direttamente dalla cartella base indicata.

In questa modalità AGIS effettua una copia del programma da eseguire e di tutti gli elementi utili alla sua esecuzione sul client nella cartella C:\AGISEXE, e lancia la sua esecuzione da questa specifica cartella.

L'esecuzione in locale è molto utile perché permette di utilizzare l'area di lavoro scelta con un ambiente diverso senza alterare il contenuto dell'area stessa inoltre, andando a copiare il programma in una cartella sul proprio client, non vengono utilizzati fisicamente gli eseguibili del prodotto/progetto e si evitano in questo modo eventuali conflitti con altri operatori che lavorano sullo stesso prodotto/progetto.

Il trasferimento a collaudo

Quando il programmatore ha terminato gli interventi su un programma BC e dopo aver effettuato tutte le verifiche necessarie può procedere al trasferimento del programma per il collaudo.

Il collaudo è una verifica approfondita delle funzionalità del programma e può essere eseguito da una figura all'interno del gruppo di sviluppo che molto probabilmente ha gestito l'analisi applicativa degli interventi da sviluppare.

Chi si occupa del collaudo dei programmi lavora con aree di lavoro specifiche, diverse dalle aree di lavoro disponibili per i test iniziali che sono utilizzate dai programmatori.

Per gestire tali esigenze all'interno di AGIS sarà frequente avere la definizione di diverse aree di lavoro utili alle diverse necessità delle persone che compongono il gruppo di sviluppo.

Il "trasferimento a collaudo" di main e dll è una azione messa a disposizione di AGIS utile alla copia fisica degli eseguibili all'interno dell'area di destinazione.

Coloro che si occupano del collaudo di un programma non eseguono i programmi passando da AGIS ma dall'interno di un prodotto, passando dal menù; eseguono quindi i programmi allo stesso modo con cui saranno poi eseguiti dall'utente finale in una situazione ovviamente ancora in sviluppo ma già in tale prospettiva.

Strumenti di analisi e di debug

Introduzione

Lo sviluppo di un programma comprende, oltre alla sua progettazione, una fase di verifica e di controllo utile alla validazione del programma.

Nella verifica di un programma possono essere individuate delle anomalie oppure dei funzionamenti anomali, possono essere visualizzati errori di diverso tipo oppure, in alcuni casi, si ha anche l'interruzione del programma in esecuzione.

L'ambiente di sviluppo BUILDER/Sistemi mette a disposizione del programmatore BC diversi strumenti utili all'analisi ed al controllo dei programmi BC; strumenti che permettono una esecuzione controllata del programma, di visualizzare serie complete di informazioni presenti in memoria oppure di analizzare la situazione della base dati.

Gli stessi strumenti e gli stessi metodi di analisi possono ovviamente essere utilizzati nella fase di verifica di una eventuale segnalazione di anomalia riportata da un utente esterno su un programma di prodotto, da qui l'esigenza di verificare la funzionalità di un sorgente nel dettaglio per correggere l'anomalia.

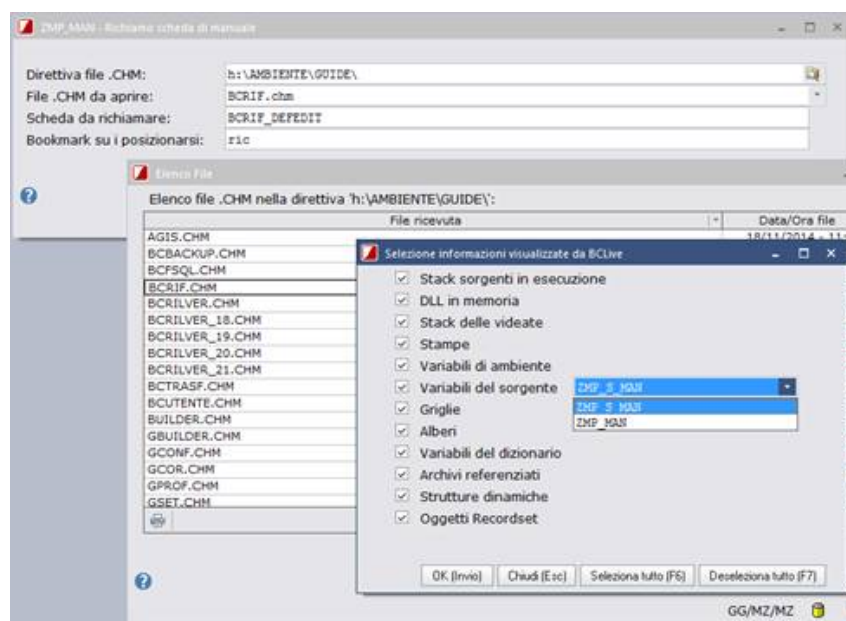
Strumenti di analisi disponibili:

- BCLive
- BCProfiler
- Debug BC

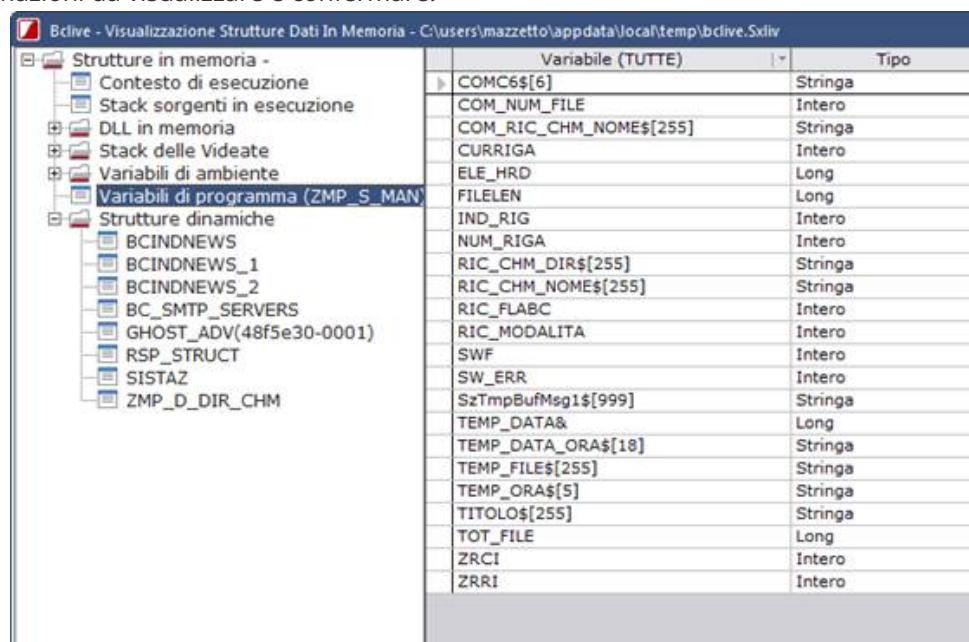
BCLive

Il BCLive è una utility che permette la visualizzazione di una serie completa di informazioni sullo stato di esecuzione di un programma BC in un dato istante. Per attivare il BCLive è sufficiente avere il focus su una videata BC in esecuzione e premere la combinazione di tasti <Shift+F9>:

Esempio



selezionare le informazioni da visualizzare e confermare.



⚠ Il BCLive può essere richiamato solo quando il programma è in attesa di un input dall'utente, quindi normalmente quando in esecuzione abbiamo una videata che richiede l'intervento da parte dell'operatore.

BCProfiler

Con l'ambiente di sviluppo viene distribuito il BCProfiler, si tratta di un sistema di Profiling totalmente locale che analizza l'esecuzione dei soli processi BC locali della macchina su cui è stato eseguito.

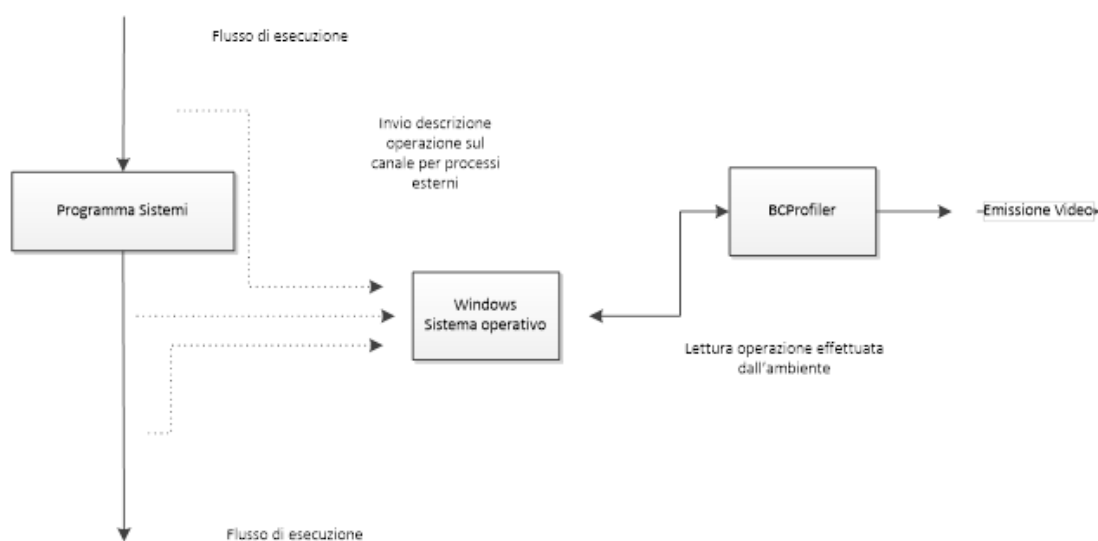
L'esecuzione dei programmi di prodotto non cambia e non viene in nessun modo influenzata dall'utility, questo perché il BCProfiler ha la funzione di monitorare determinate operazioni effettuate dall'ambiente di esecuzione e non interagisce con i programmi in esecuzione.

Durante l'esecuzione di un programma di prodotto l'ambiente effettua una moltitudine di operazioni, per determinate operazioni (errori, stringhe sql, ecc...) l'ambiente invia la descrizione di quanto fatto su un canale esterno al suo processo.

Il BCProfiler, essendo sintonizzato su questo canale, intercetta tale messaggio e lo visualizza a video.

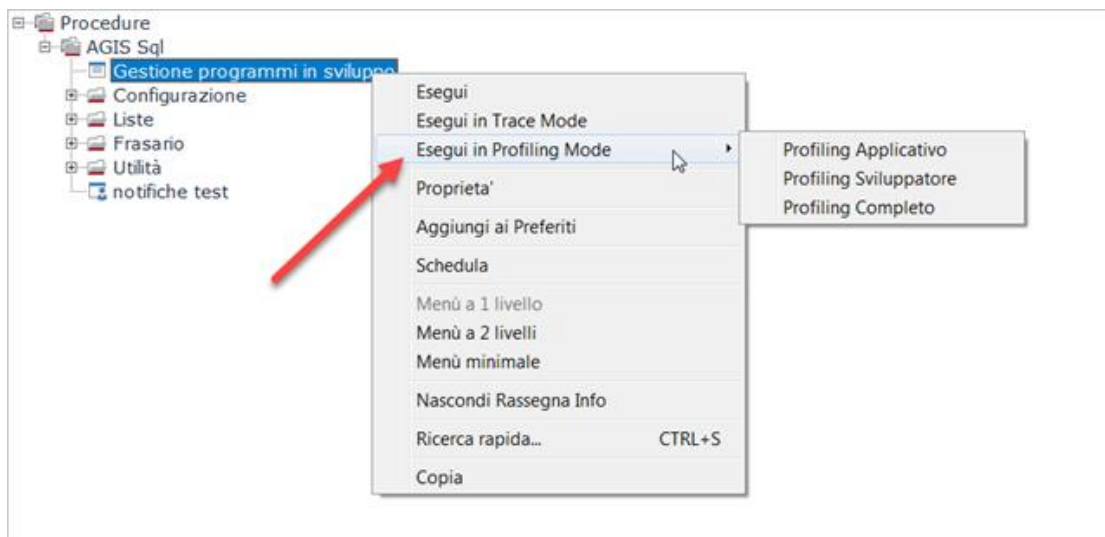
The screenshot shows the BCProfiler application window with a table of operations. The table has columns: Prog#, Date / Ora, Man, Dr, Subroutine, Operazione, and Descrizione. The 'Man' column contains 'ZMP_TEST' and 'ZMP_S_MAN'. The 'Subroutine' column contains 'VNTPERF1-SISTEM@BULO'. The 'Operazione' column contains various system calls like 'STRUCTURE', 'IN SRC', 'OUT SRC', and 'AGS'. The 'Descrizione' column contains detailed descriptions of the operations, such as 'GHOST_ADVANCE1250001 GETDY', 'GHOST_ADVANCE1250001 GETDY', 'GH_S_LEGGI_IND_SCHEDULE_3', 'SM_S_VERIFICA_PRESENZA_WEB', 'GCS_S_GES_IN', 'GCS_S_GES_IN', 'SM_S_VERIFICA_PRESENZA_WEB', 'GH_S_LEGGI_CONF', 'SM_S_SISPROO_MENU_ANNO', 'SM_S_PRODO_O3', 'SM_S_PRODO_O3', 'SM_S_SISPROO_MENU_ANNO', 'BCRONNEWS_COUNTINSTRUCT', 'SM_S_KILLINSTRUCT', 'BCRONNEWS_FACTINSTRUCT', 'GH_S_LEGGI_IND_SCHEDULE_3', 'GHOST_ADVANCE1250001 GETDY', 'AG_S_GEST_RECORD_LOO_S2', 'SELECT TOP 1 NumePig IPPhone', 'AG_S_GEST_RECORD_LOO_S2', 'SM_S_RUN_PGM_SCHED', 'SM_S_VERIFICA_PRESENZA_WEB', 'GCS_S_GES_IN', 'GCS_S_GES_IN', 'SM_S_VERIFICA_PRESENZA_WEB', 'AG_S_RICHAMA_NOTIFICA_CAS', 'AG_S_RICHAMA_NOTIFICA_CAS', 'BC_S_GEST_MESSAGGI_SGM', 'SM_S_SEL_PROCEDURA', 'SM_S_SEL_PROCEDURA_S2', 'PROCFARE COUNTINSTRUCT', 'PROCFARE GETDYINSTRUCT', 'SM_S_SEL_PROCEDURA_S2', 'SM_S_SEL_PROCEDURA', 'PROCFARE GETDYINSTRUCT', 'BC_S_GEST_MESSAGGI_03', 'BC_S_GEST_MESSAGGI_03'.

Schema di riferimento:

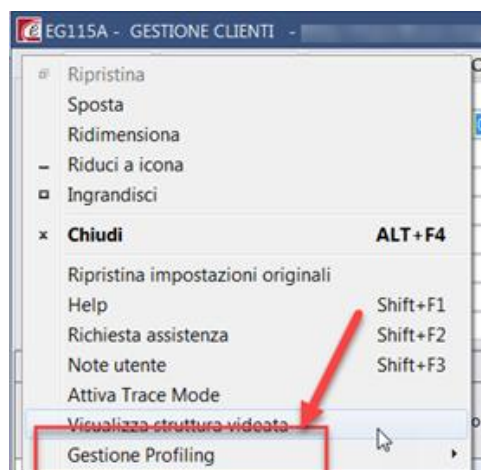


Attivazione del profiling

L'esecuzione del BCProfiler può avvenire in diverse modalità. La prima è l'esecuzione di un programma presente a menù utilizzando la voce "Esegui in profiling Mode" attiva su tutti i prodotti Sistemi.



Attivando invece a livello di CFG di prodotto il parametro per l'esecuzione in modalità "sviluppatori", il BcProfiler viene attivato automaticamente all'avvio di qualsiasi programma lanciato da menù, configurando gli elementi da tracciare direttamente del menù attivabile dall'icona in alto a sinistra:



L'attivazione della modalità "sviluppatori" deve essere effettuata nel seguente modo:

- inserire la sezione di cui sotto all'interno del file CFG di prodotto (ad esempio sul *CFGPRF3.INI* in cartella <Base>):
[Configurazione]

SUBARG=1

Con l'impostazione sopra descritta l'esecuzione dei programmi lanciati da menù attiva in automatico il BCProfiler se non ancora in esecuzione.

E' poi possibile inserire sulla riga di comando il parametro *-PROFILING=* per attivare il BCProfiler nelle modalità: 1- Applicativo, 2-Sviluppatore e 3-Completo. Questa possibilità è utile anche per il lancio dei programmi da AGIS perché utilizzabile come argomento nella videata di esecuzione programmi.

Profiling

Il BCProfiler è un eseguibile che permette la lettura di quello che avviene durante l'esecuzione di un programma BC e permette di visualizzare sei tipi di operazioni distinte:

SISERR Profiler:

Durante il flusso di esecuzione del programma, tiene traccia di tutti gli errori tracciati sul SISERR.LOG.

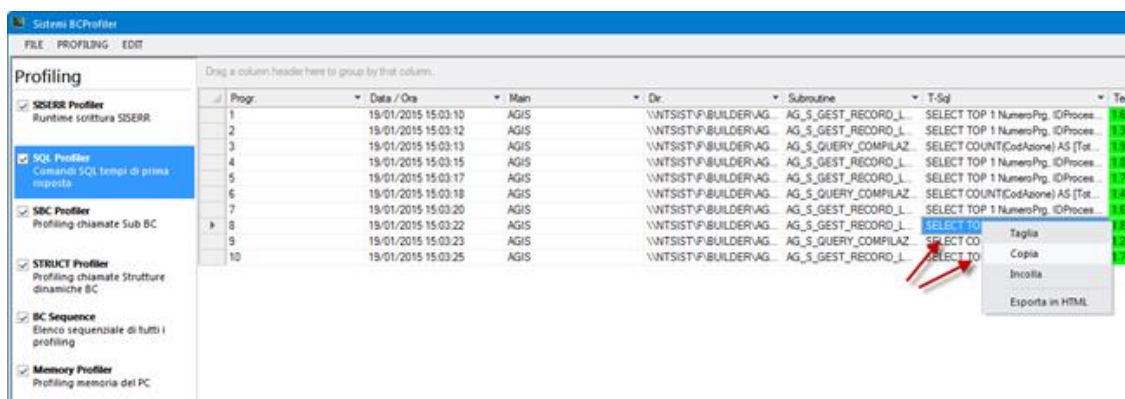
Lo scopo è quello di prevenire eventuali errori di programmazione dove all'interno della specifica viene gestito il parametro *ERRORE[]* ma successivamente non viene testato il valore della variabile di ambiente *ZBCERR* dal programma BC.

SQL Profiler:

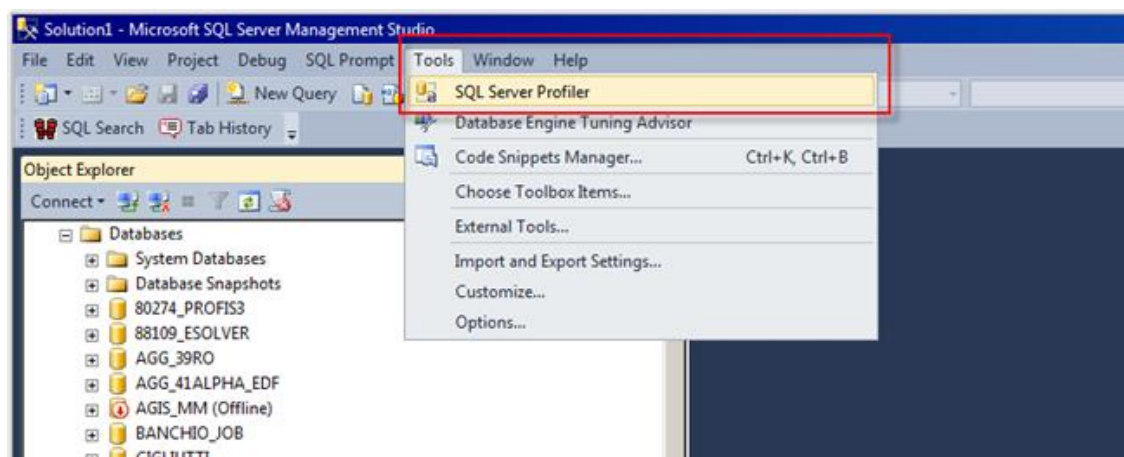
Sono tracciati tutti i comandi SQL inviati al server.

Le tempistiche indicate non sono quelle complessive di esecuzione di una query, ma solamente quelle di prima risposta.

Lo scopo è capire se le query eseguite sono corrette e/o ottimizzabili.



💡 Dall'SQL Profiler è inoltre possibile effettuare la copia del comando SQL inviato al server (click su tasto destro del mouse), operazione molto più comoda da effettuare invece che passare da SQL Server Management Studio (SSMS) e utilizzare lo strumento SQL Server Profiler (SSP).

**SBC Profiler:**

Sono tracciate tutte le chiamate effettuate alle subroutine BC.

Viene analizzato il numero di volte che una sub viene chiamata, il tempo di esecuzione della sub e dei suoi figli, il tempo medio per chiamata e la percentuale che occupa all'interno dell'intero flusso operativo.

Lo scopo è quello di riuscire ad analizzare il flusso di esecuzione di un programma BC per capire se è possibile ridurre il numero di chiamate di alcune SUB ottimizzando il flusso di esecuzione del programma.

STRUCT Profiler:

Sono tracciate tutte le macro operazioni effettuate sulle strutture dinamiche.

Lo scopo è l'analisi del flusso di operazioni effettuate sulle strutture dinamiche per analizzare eventuali lentezze rilevate in esecuzione.

Viene inoltre effettuata l'analisi a livello di macro operazioni ('@DEFDYNSTRUCT, '@ADDYDYNSTRUCT), questo per evitare di aumentare in maniera sproporzionata la mole di passaggio di dati tra applicazioni.

BC Sequence:

E' la cronologia dell'esecuzione dei programmi, racchiude tutte le operazioni precedenti (SISERR, SQL, '@SBC, STRUCT) e le elenca in ordine di esecuzione, creando una vera e propria storia dell'esecuzione del programma.

Lo scopo è quello di avere una visione d'insieme della sequenza delle operazioni fatte dal singolo programma.

Memory Profiler:

Sono tracciati tutti i processi eseguiti analizzando la memoria occupata (Ram e Virtuale), i suoi picchi e gli oggetti GDI creati e gestiti.

 Se il sistema di profiling è attivato, ed esiste almeno un BCProfiler in esecuzione, il flusso di esecuzione del programma di prodotto potrebbe avere un sensibile calo delle prestazioni.

Debug BC

BCDevelop non è un semplice editor, è bensì un ambiente di sviluppo evoluto che racchiude in sé molte funzionalità e facilitazioni che vanno oltre la stesura del codice.

Le attività compiute da Sistemi sono state quelle di "snellire" BCDevelop dalle funzionalità non necessarie, dopodiché personalizzarlo ed estenderlo integrando in esso funzionalità che potessero, da un lato, agevolare la stesura del codice BC e, dall'altro, limitare le volte in cui il programmatore BC avesse necessità di "tornare" in AGIS o accedere al manuale per acquisire informazioni di contorno (ad esempio la documentazione delle specifiche, la dimensione dei campi delle tabelle, etc).

BCDevelop inoltre permette di eseguire i programmi BC in modalità Debug BC oppure in modalità Trace: con funzionalità piuttosto avanzate come l'utilizzo di breakpoint ed il watch delle variabili.

La modalità Debug BC permette di eseguire un programma BC con BCDevelop per effettuare una analisi ed un controllo dell'esecuzione "step by step", è normalmente una funzione utilizzata nello sviluppo di prodotto per individuare eventuali comportamenti anomali o per visualizzare nel dettaglio il flusso di un programma in sviluppo; in questa modalità si esegue fisicamente il programma e si procede passo dopo passo nel suo flusso di esecuzione.

La modalità Trace, se attivata durante l'esecuzione di un programma di prodotto, permette di generare un file di testo con al suo interno la registrazione delle operazioni fatte dal programma BC; BCDevelop permette la visualizzazione di un file così prodotto per analizzare in maniera semplice ed intuitiva il flusso del programma e tutte le informazioni raccolte durante l'esecuzione del programma BC visualizzandone il flusso eseguito.

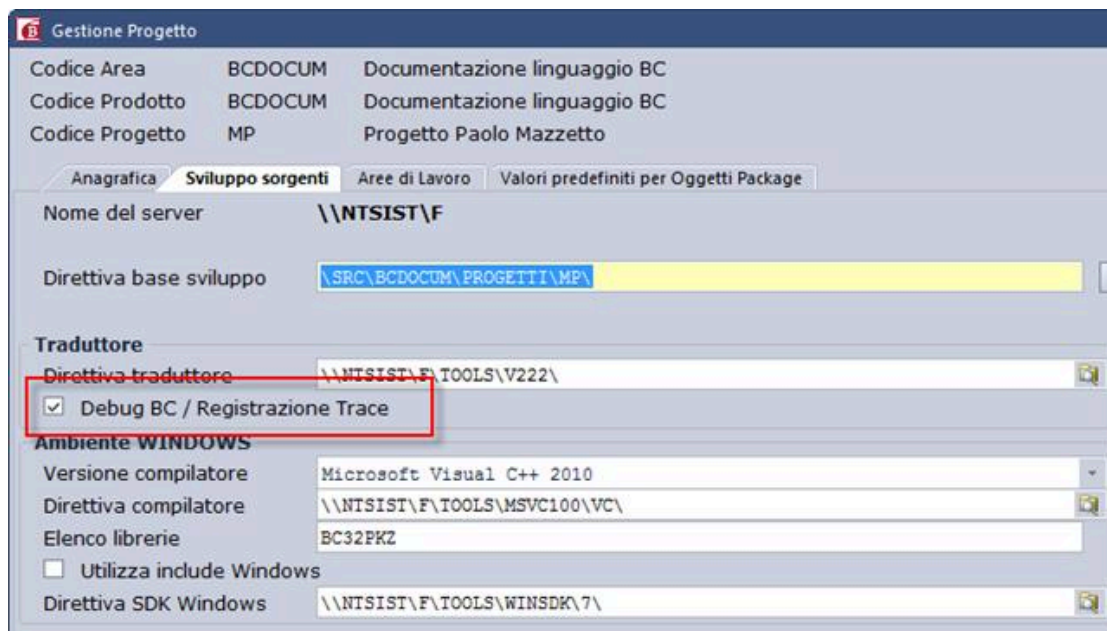
In pratica il Debug BC è una esecuzione in tempo reale del nostro programma, mentre la visualizzazione di un file di Trace è la consultazione, in un secondo momento, dell'esecuzione del nostro programma.

 Le funzionalità "Debug BC" e "Registrazione Trace" sono disponibili dalla versione 21.0 VS2010 dell'ambiente di sviluppo BUILDER/Sistemi.

Prerequisito per l'utilizzo del Debug BC e del Trace

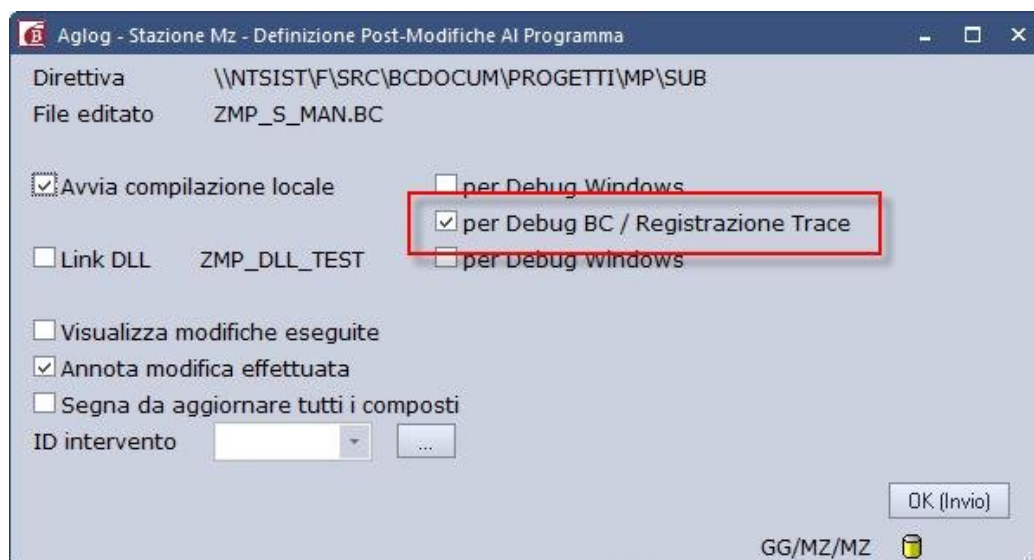
Per poter eseguire in BCDevelop un sorgente BC in modalità Debug BC o per visualizzare un file di Trace prodotto dall'esecuzione di un programma BC è necessario che i programmi siano stati compilati con l'opzione *"Debug BC / Registrazione Trace"*.

Sui dati anagrafici del Prodotto\Progetto, nella sezione sviluppo sorgenti, è stata aggiunta una nuova opzione abilitata solo se la versione di traduttore è maggiore uguale a 21.0.



Questa opzione fa in modo che tutte le compilazioni fatte dal prodotto/progetto prevedano l'istrumentazione del codice C generato affinché possa generare in esecuzione il file di trace.

Se non attivo a livello di prodotto/progetto è possibile compilare il singolo sorgente utilizzando l'azione "Compilazione con Trace" oppure selezionando l'opzione direttamente dalla videata di chiusura editazione (AGLOG):



I programmi compilati in questo modo sono evidenziati in AGIS con la specifica colonna "Trace" nella griglia di elenco oggetti.

Esecuzione in modalità Debug BC

L'esecuzione con Debug BC permette di avviare l'esecuzione di un Main / DLL con tracciatura delle righe di codice BC eseguite; questo permette di avere visibilità del flusso eseguito dal programma e, per ogni riga, avere a disposizione tutte le informazioni collegate (valore variabili, comando SQL eseguito, etc).

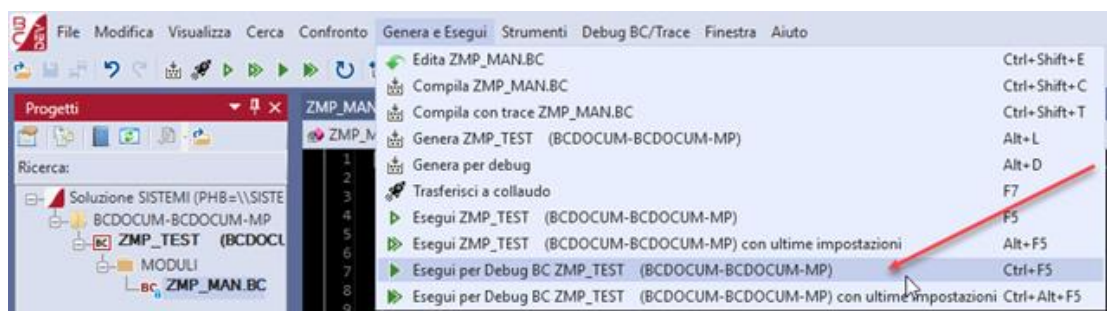
L'esecuzione in modalità Debug BC permette inoltre di interrompere il programma a proprio piacimento attraverso l'inserimento dei breakpoint nel codice BC e BCDevelop mette a disposizione la vista dei breakpoint per avere una visione di insieme.

E' quindi possibile compiere le seguenti operazioni:

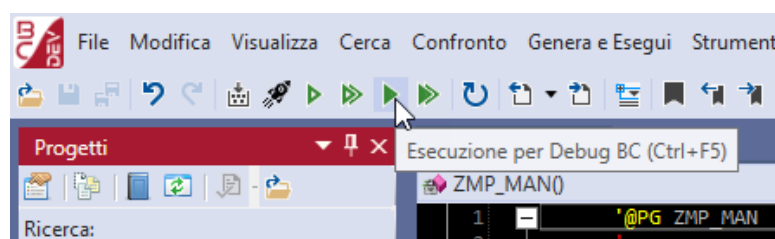
- inserire nel sorgente BC dei breakpoint, in modo che il programma BC si fermi in automatico a proprio piacimento;

- eseguire il programma in Debug BC;
- seguire il flusso di esecuzione del programma;
- visualizzare le informazioni collegate ad ogni riga di programma BC eseguita.

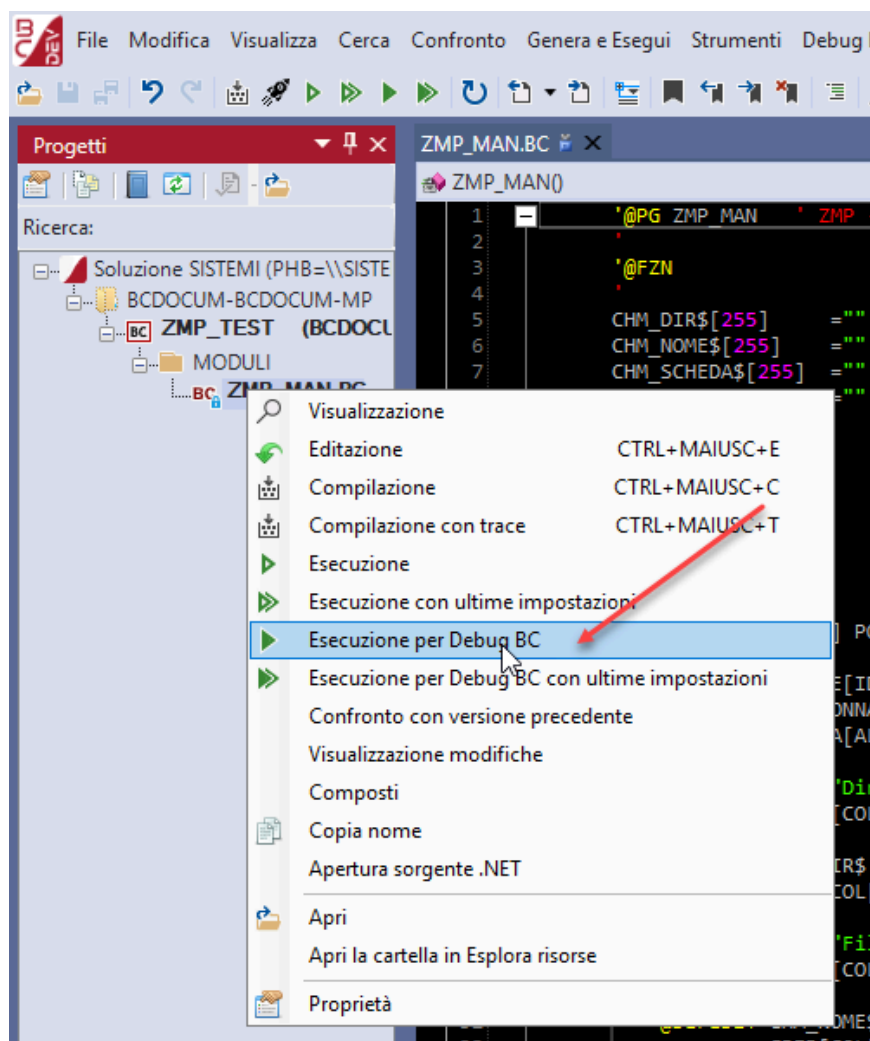
E' possibile avviare l'esecuzione in modalità Debug BC dal menù "Genera e Esegui".



dal bottone sulla barra:



oppure cliccando con il tasto destro sul programma nella vista "Progetti":

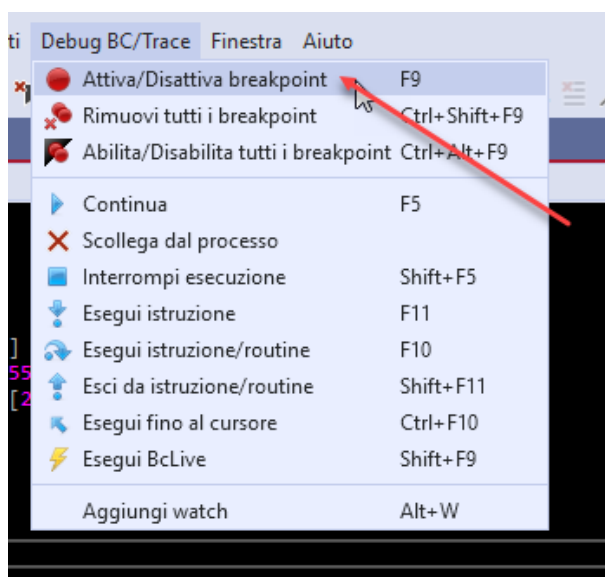


Inserimento di un breakpoint

Il breakpoint è un "punto di interruzione" e permette appunto di interrompere l'esecuzione di un programma BC in qualsiasi momento.

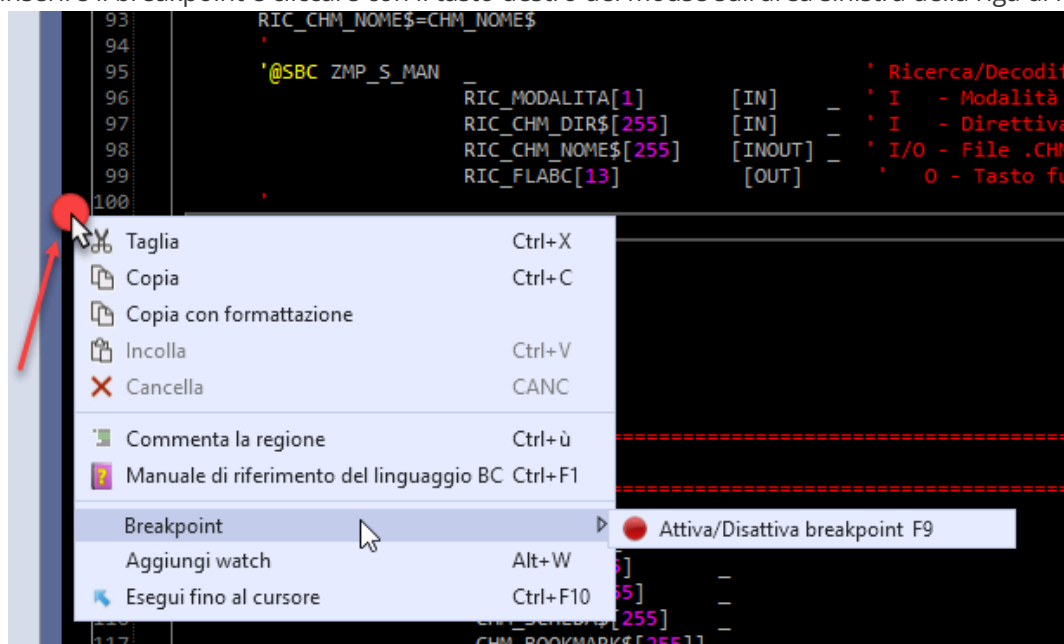
In BCDevelop è possibile aggiungere i breakpoint sia prima che durante l'esecuzione con Debug BC del programma. Durante l'esecuzione è però possibile farlo solo se l'esecuzione è ferma in attesa di input dall'operatore (su una videata) oppure se l'elaborazione è stata fermata da un altro breakpoint precedentemente inserito.

Per inserire un breakpoint su una riga è necessario posizionarsi sulla riga specifica ed effettuare l'inserimento tramite il menù "Trace":



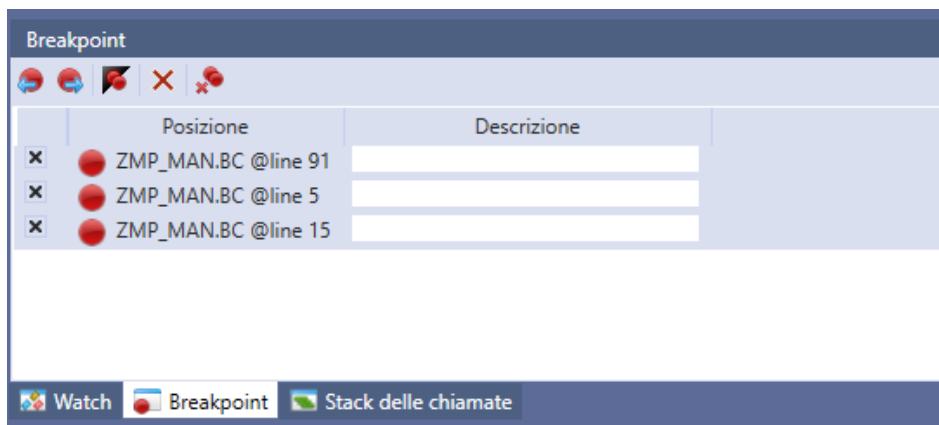
oppure premendo <F9>.

Altro modo per inserire il breakpoint è cliccare con il tasto destro del mouse sull'area sinistra della riga di riferimento:



⚠ I breakpoint devono essere inseriti su righe di codice BC con un contenuto significativo, non possono essere inseriti su righe vuote o su righe commentate.

tutti i breakpoint inseriti sono gestibili e navigabili dalla vista breakpoint visualizzata in basso a sinistra:



Dalla vista BreakPoint è possibile compiere le seguenti operazioni:

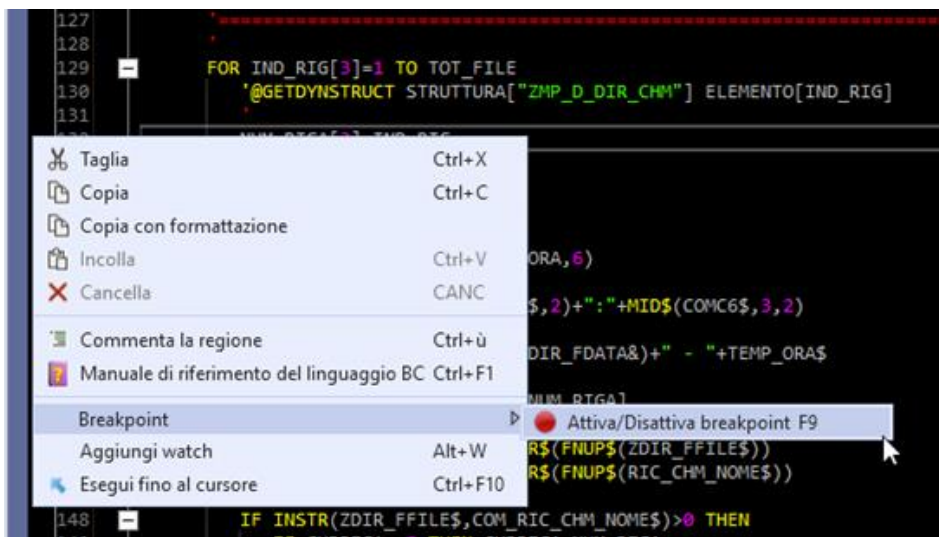
- Doppio click: posizionamento automatico sul punto del sorgente in cui è presente il breakpoint;
- Check/Uncheck: abilitare/disabilitare il breakpoint (disabilitare non vuol dire rimuovere ma solo fare in modo che non sia tenuto in considerazione dall'esecuzione con Debug BC);
- e : posizionamento sul breakpoint precedente o successivo;
- : abilitare/disabilitare tutti i breakpoint;
- : rimuovere il breakpoint;
- : rimuovere tutti i breakpoint.

Inserimento di un breakpoint condizionale

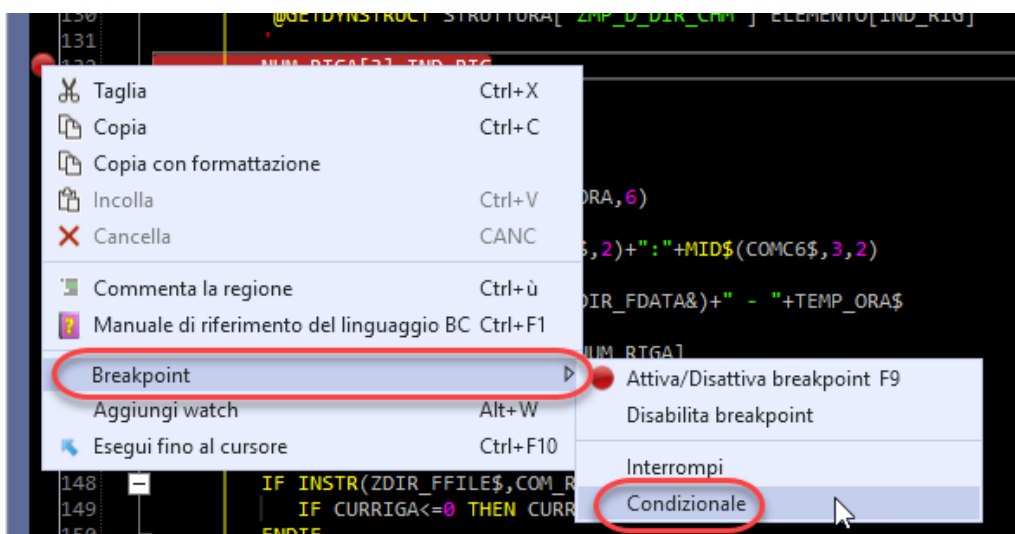
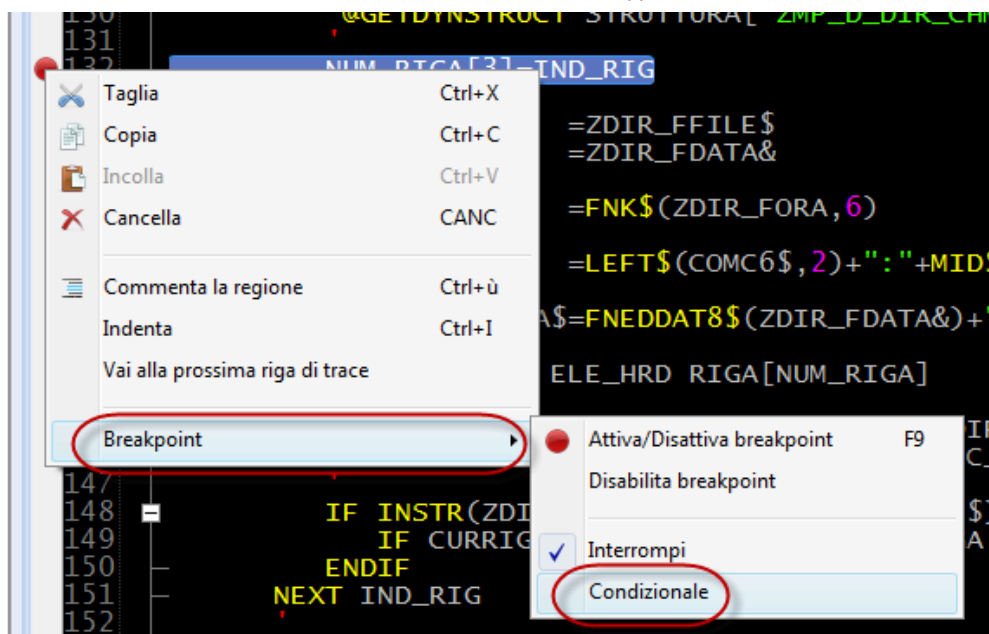
Il breakpoint inserito può essere definito come breakpoint condizionale, in questo modo l'esecuzione del programma si interrompe sul breakpoint quando si verifica la condizione definita su di esso.

Di seguito viene riportato un esempio della definizione di un breakpoint condizionale:

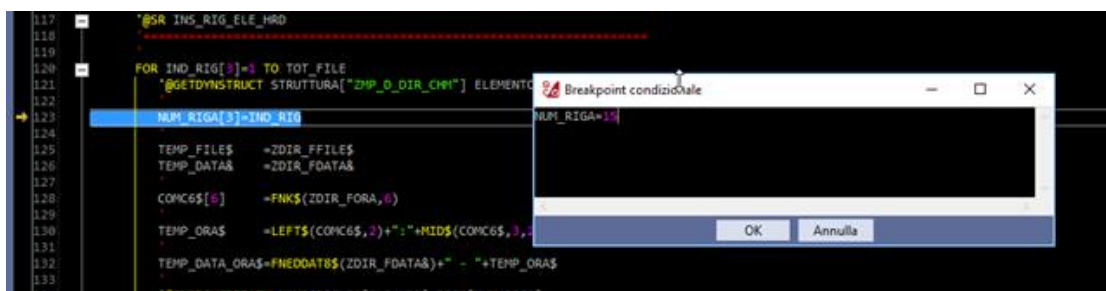
attivazione del breakpoint:



definizione del breakpoint condizionale:

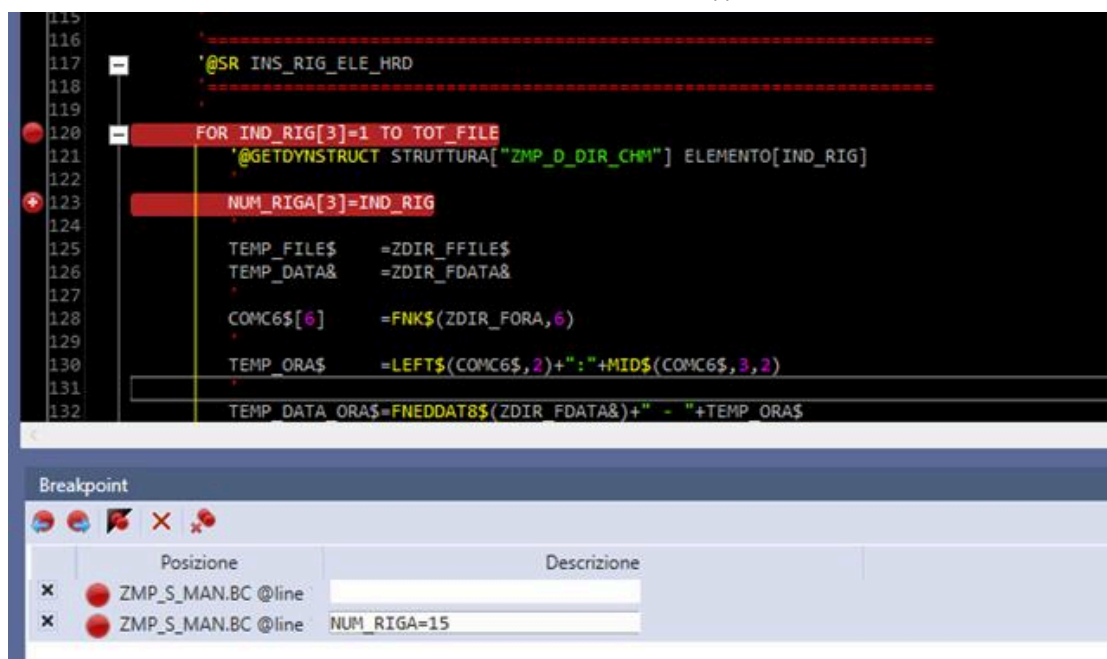


inserimento della condizione nella box che viene proposta da BCDevelop:



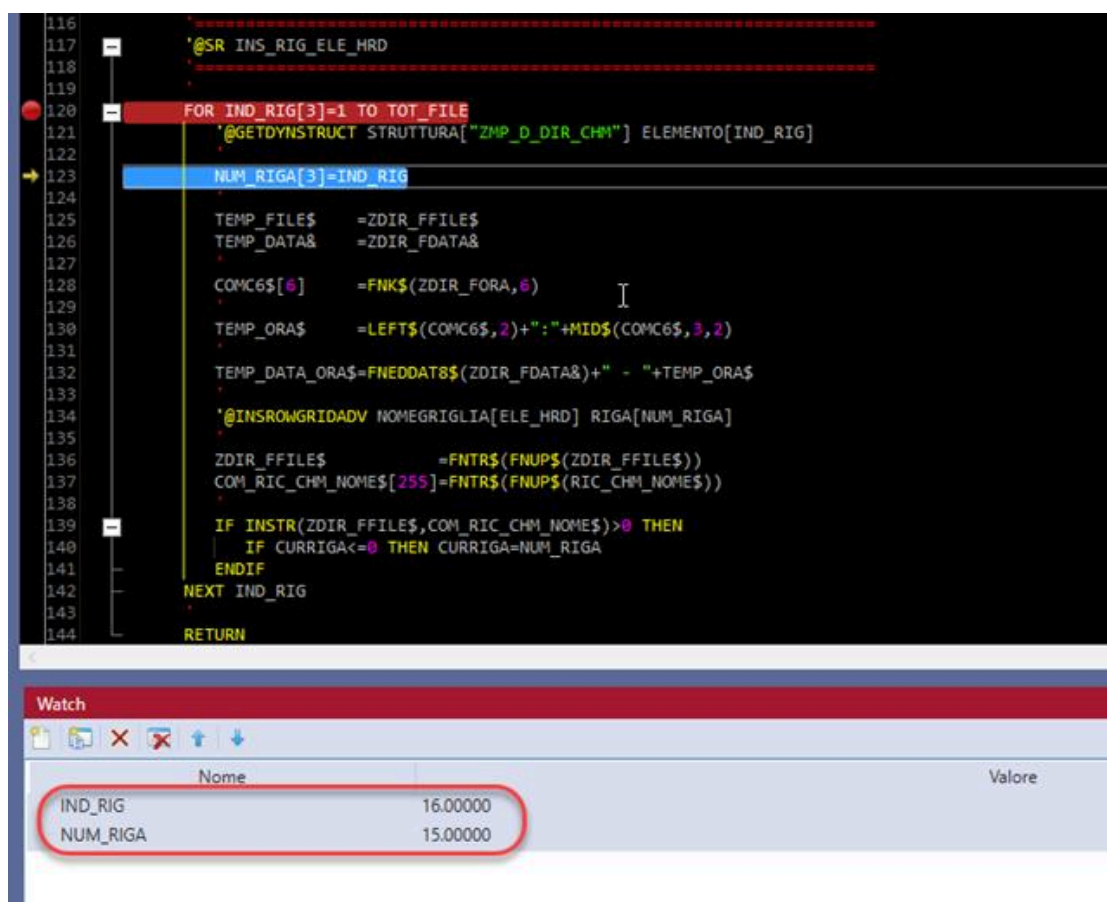
nel caso specifico il breakpoint è stato inserito alla riga 123 del sorgente BC che si trova all'interno di un ciclo *FOR-NEXT* e si vuole interrompere l'esecuzione quando la variabile interna *NUM_RIGA* assume il valore '15' (quindici).

notare le icone differenti del breakpoint e del breakpoint condizionale:



⚠ La condizione indicata è riportata all'interno della descrizione del breakpoint per facilitare l'esecuzione in debug del programma. Per modificare la condizione inserita è necessario effettuare i passaggi sopra descritti.

L'esecuzione del programma procede fino al verificarsi della condizione indicata:

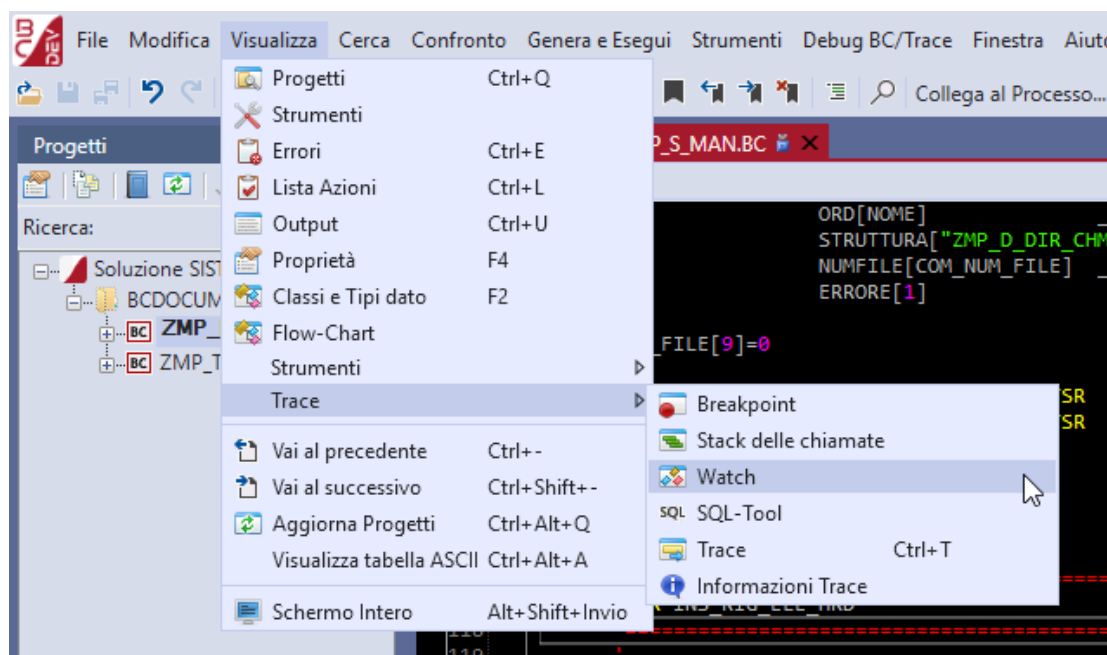


⚠ La condizione indicata deve avere una sintassi simile a quella riportata nell'esempio di cui sopra, non può contenere parole chiave del linguaggio BC (es: *IF/THEN*) e non può contenere funzioni BC (es: *VAL()*, *FNTR\$*).

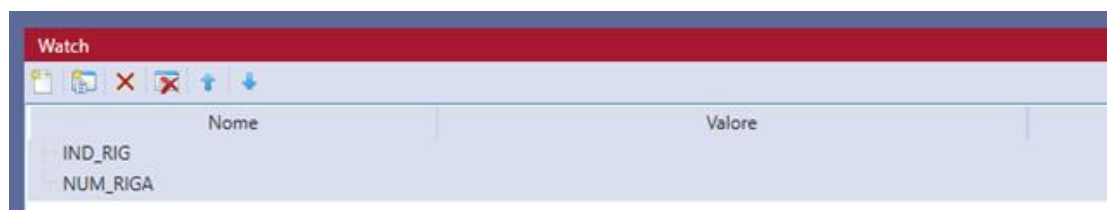
Inoltre, ad oggi, non è possibile indicare nella condizione valori di array, variabili di un oggetto e variabili di una collezione. Per utilizzare i breakpoint condizionali è necessario utilizzare la versione 22.7 (o successiva) del traduttore e dell'ambiente di sviluppo BC.

Il watch delle variabili

La vista del watch delle variabili permette di avere sempre a disposizione, durante un'esecuzione con Debug BC, il valore delle variabili aggiunte a questa vista; è attivabile dal menu "Visualizza->Trace" come nell'esempio di seguito riportato:



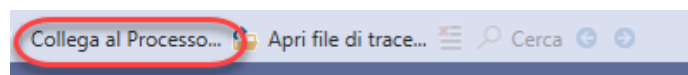
esempio della vista Watch:



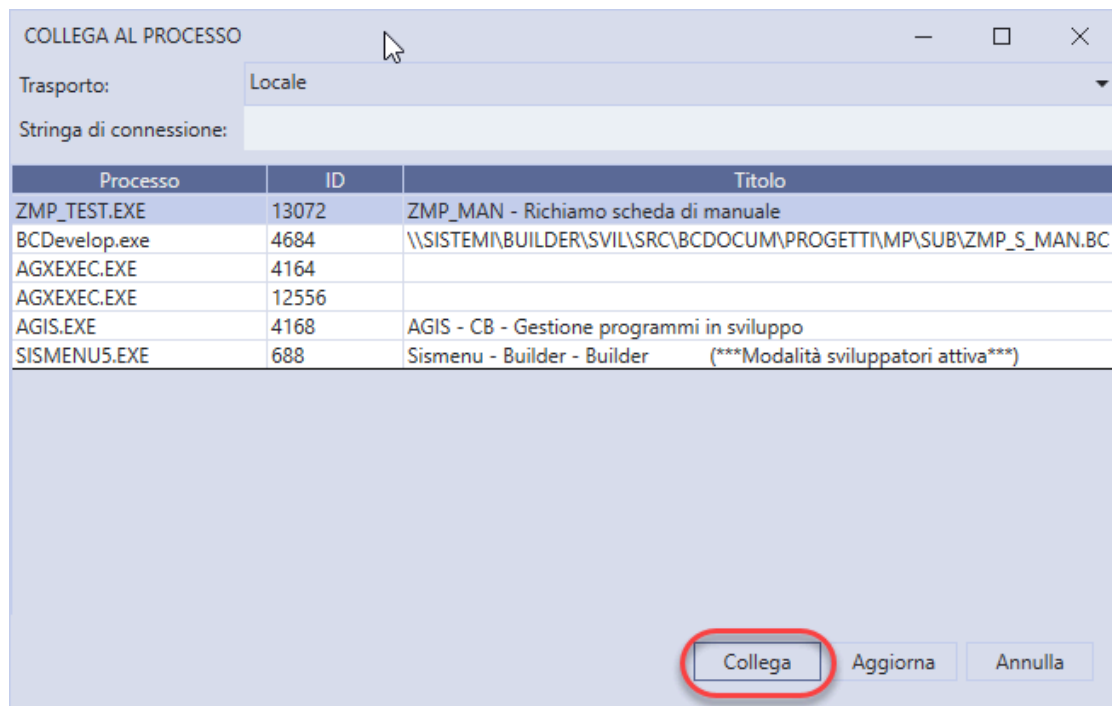
Collega al processo

A partire dalla versione 25.0 dell'ambiente (4.2.25.0001 di BCDevelop) è possibile eseguire un debug BC collegandosi ad un processo BC già in esecuzione.

È possibile collegarsi ad un processo con la combinazione di tasti CTRL+ALT+P oppure attraverso il bottone "Collega al processo" posto nella toolbar di BCDevelop.



Selezionando il comando, viene visualizzata una finestra con l'elenco dei processi BC disponibili.



Selezionando il processo desiderato e premendo il bottone Collega si potrà seguire il debug BC del programma, dal momento del collegamento in poi.

Il debug remoto

È possibile collegarsi ad un processo che non è in esecuzione sulla propria macchina.

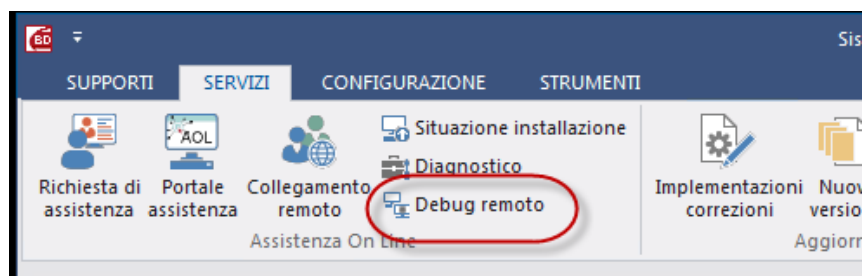
Sono disponibili due protocolli diversi per collegarsi in remoto ad un processo:

- Rete locale
- Web Service

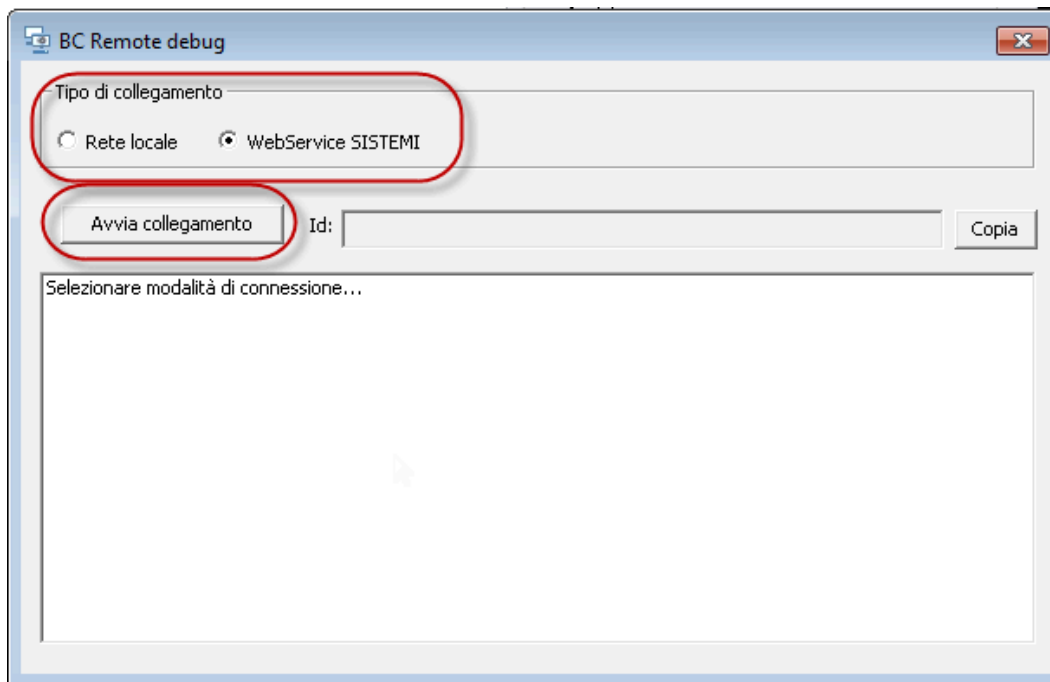
Il primo permette di collegarsi ad un processo presente su una macchina della rete locale, il secondo permette di collegarsi ad un processo fuori dalla rete locale.

In entrambi i casi, la prima operazione da effettuare è quella di attivare un client sul PC remoto dove risiede il processo in esecuzione.

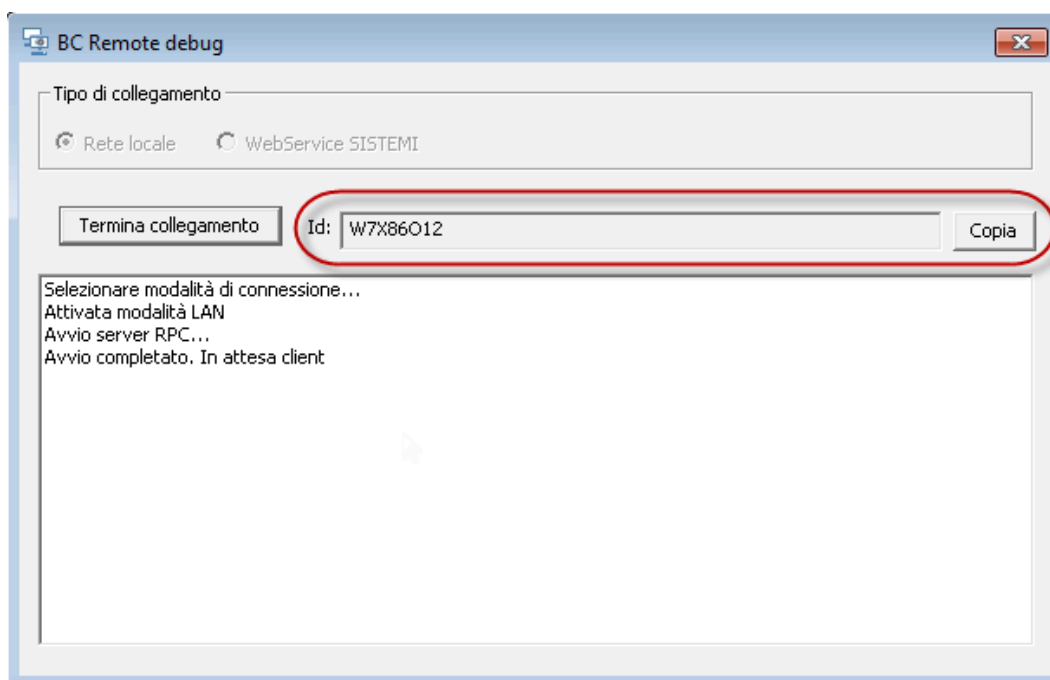
Il client è attivabile dal menu Servizi - Debug remoto



Cliccando sul bottone viene avviato il client. Da qui occorre selezionare il tipo di protocollo che si desidera utilizzare e poi premere sul bottone "Avvia collegamento".

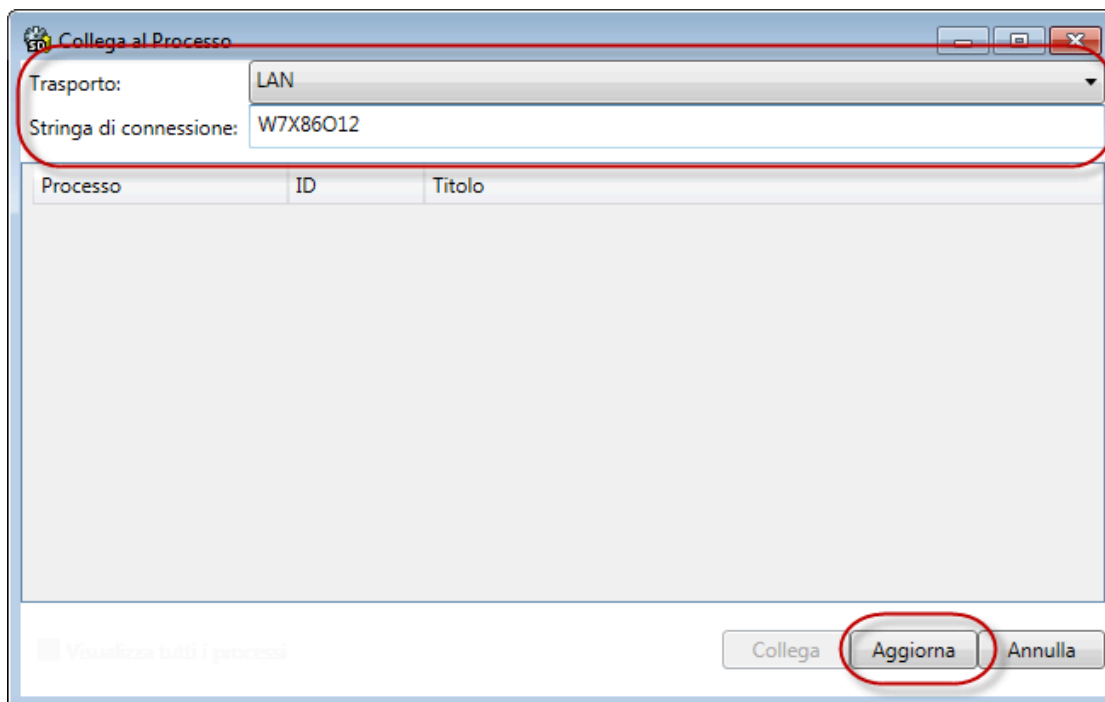


Una volta avviato il collegamento, nel campo Id verrà visualizzato un codice che serve a BCDevelop per determinare univocamente il client al quale collegarsi; nel caso di Rete locale l'Id altro non sarà che il nome della macchina, mentre per il Web Service sarà un GUID. Premendo il bottone Copia è possibile inserire nella clipboard il valore dell'ID.

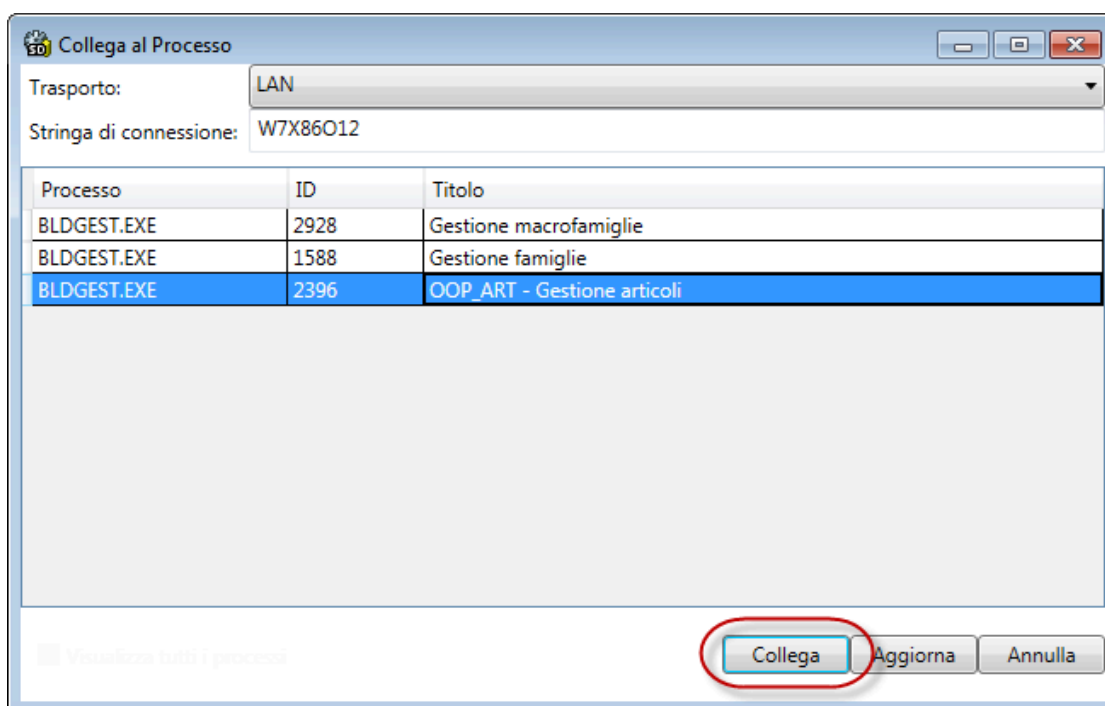


Una volta avviato il client, è possibile andare su BCDevelop e avviare il debug remoto, selezionando la funzione "Collega al processo".

Dalla finestra di collegamento al processo è possibile selezionare il protocollo da utilizzare e la stringa di connessione, che corrisponde all'Id visualizza sul client.



Una volta indicati questi valori, premendo su Aggiorna si avrà l'elenco dei processi BC attivi sulla macchina remota.

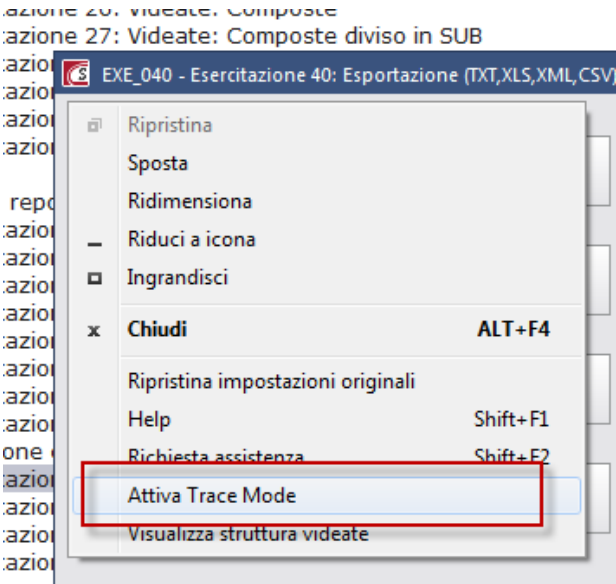


Selezionando il processo desiderato e premendo il bottone Collega si potrà seguire il debug BC del programma, dal momento del collegamento in poi, come un normale debug eseguito in locale.

Registrazione Trace

Attivazione della modalità di esecuzione in Trace Mode

L'attivazione della modalità Trace Mode è disponibile sul Sistem Menù (click sull'icona di prodotto in alto a sinistra della finestra del programma in esecuzione):



Durante l' esecuzione in modalità Trace è visualizzato sulla toolbar del programma il simbolo di "registrazione in corso";



Parametri di esecuzione in modalità "Trace"

L'attivazione dell'esecuzione in Trace Mode propone una videata che richiede all'utente quali tipologie di informazioni aggiuntive intende tracciare.

Al momento le categorie disponibili sono le seguenti:

CATEGORIE	Informazioni tracciate	Default
ACCESSO ALLA BASE DATI	Registra: • Stringa SQL inviata al DBMS • Numero di record selezionati dall'interrogazione • Primi 5 record letti dalla base dati (quando disponibili)	Attivato
ACCESSO ALLE STRUTTURE DINAMICHE	Registra • Nome struttura • Operazione effettuata • Record interessati dall'operazione	Attivato
REGISTRAZIONE EVENTI	Tutte le informazioni raccolte dalla Registrazione eventi	Disattivato
MESSAGGI DEL SISERR.LOG	Tutte le informazioni raccolte dall'ambiente per l'alimentazione del file SISERR.LOG	Attivato

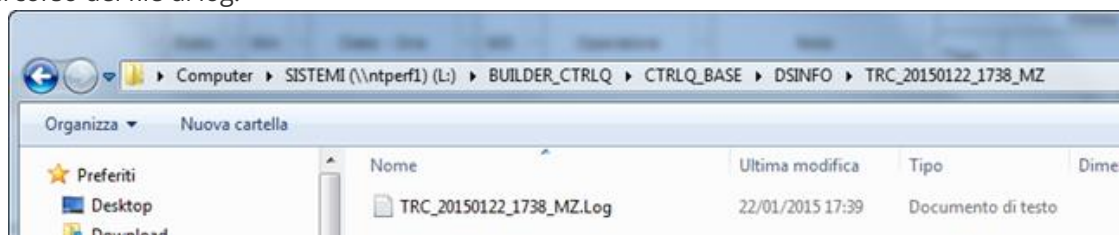
Inoltre qui, è anche possibile visualizzare e impostare la cartella nella quale verrà generato il file di registrazione trace.

L'esecuzione in Trace mode si occuperà da questo istante in avanti di generare ed alimentare il file *<trace>.log*; la cartella di destinazione del file di trace dipende dalla modalità di attivazione del trace mode;

Attivazione Trace mode	Percorso file di trace
Da BCDevelop	<Direttiva Files temporanei di Windows>\SISTEMI\TRACE\
Da Sismenu	<Direttiva Base>\DSINFO\TRC<AnnoMeseGiorno>_<OraMinuti>_<Stazione>\, oppure impostata manualmente
Da AGIS	<Direttiva Base>\DSINFO\TRC<AnnoMeseGiorno>_<OraMinuti>_<Stazione>\, oppure impostata manualmente

Il file verrà alimentato fino a quando l'esecuzione in trace mode non verrà disattivata, poi in un secondo momento tale file può essere visualizzato con BCDevelop.

esempio di percorso del file di log:

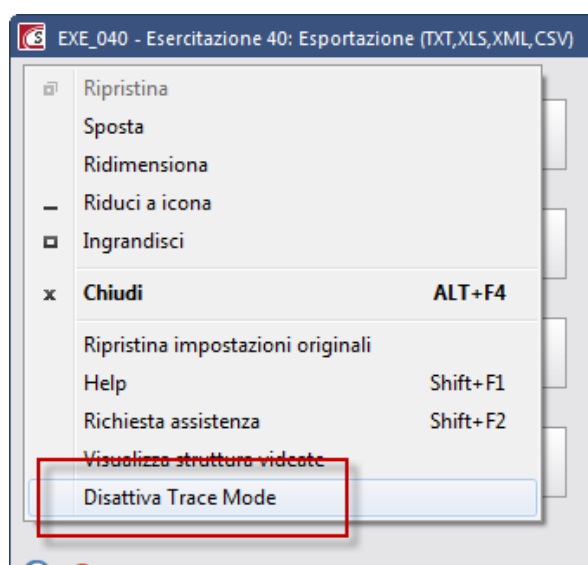


⚠ L'esecuzione in modalità Trace può sempre essere eseguita senza particolari restrizioni; se però vengono attraversati dei sorgenti che non sono stati compilati con l'azione "Compilazione con registrazione Trace", sul file *<trace>.log* questi sorgenti compariranno come se fossero una sola riga BC senza ulteriori dettagli.

Disattivazione della modalità di esecuzione in Trace Mode

L'esecuzione in modalità Trace può essere terminata solo dal programma BC in esecuzione, oppure quando l'esecuzione del programma stesso termina.

Se il programma è in esecuzione in trace Mode sul menù di sistema è disponibile la nuova voce "Disattiva Trace Mode" che consente di disattivare l'esecuzione in Trace Mode:

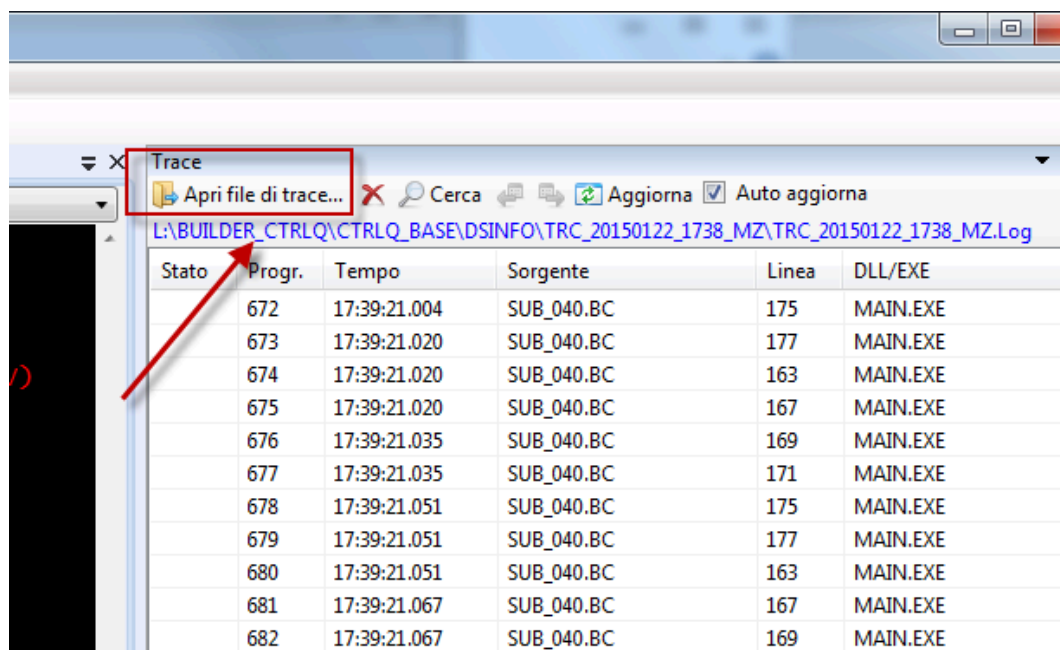


Contestualmente viene rimosso il simbolo di "Registrazione in Corso" dalla toolbar.

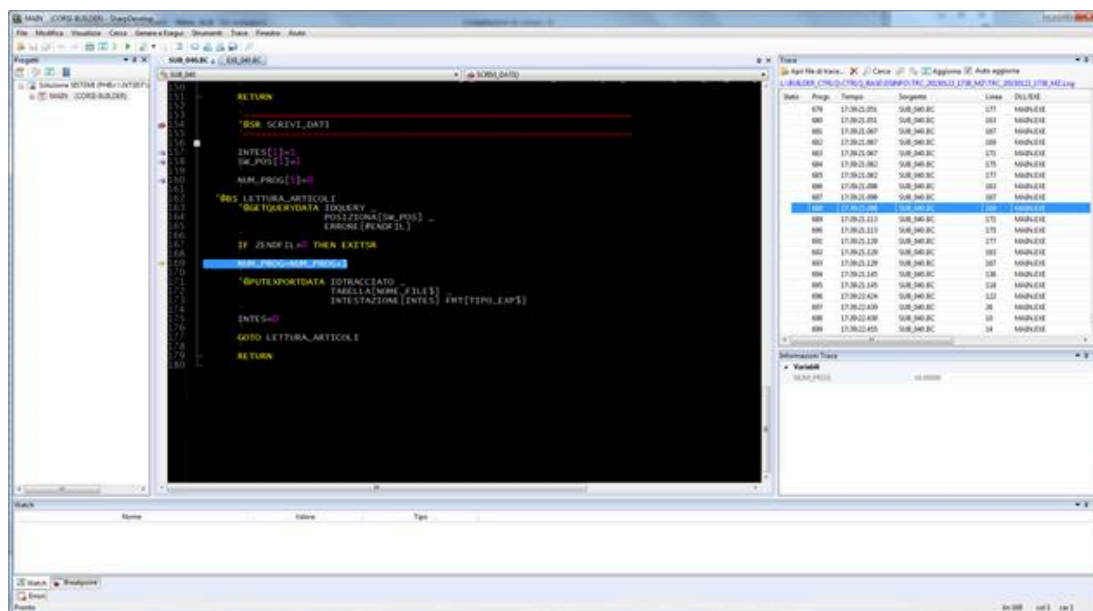
Visualizzazione di un file di Trace

Per visualizzare un file di Trace è necessario che con BCDevelop sia visualizzato il sorgente BC del programma che è stato "registrato" ed è indispensabile che si tratti della medesima versione del programma.

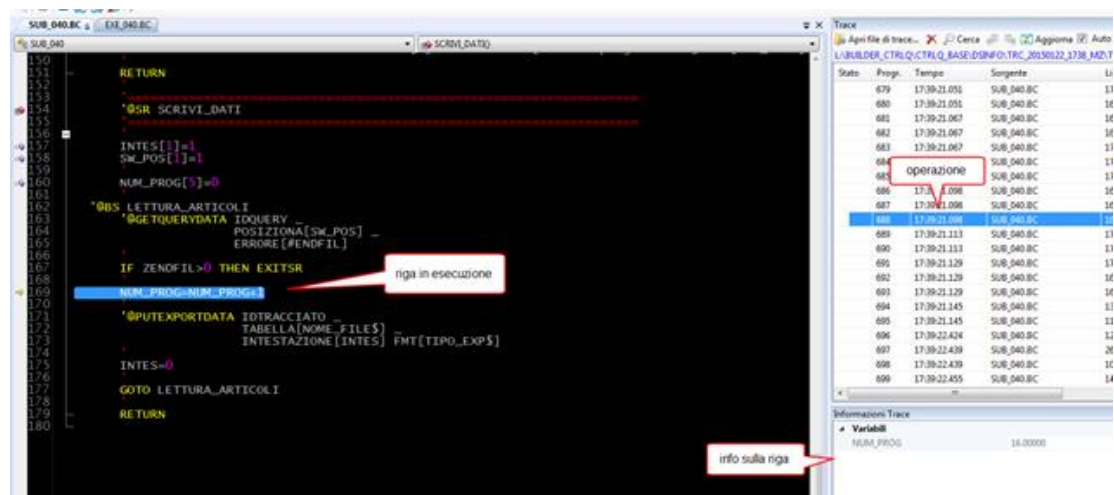
Dalla finestra "Trace" selezionare il file di log:



BCDevelop visualizza tutte le operazioni rilevate nella finestra di Trace, ogni riga presentata in questa vista corrisponde ad una istruzione BC eseguita e per ognuna di queste, se presenti, vengono mostrate anche le informazioni collegate nella vista delle Informazioni Trace:



posizionandosi su una operazione viene in automatico aperto il sorgente di riferimento posizionati sulla riga corrispondente in modo da avere una idea immediata della riga BC di riferimento. Così facendo è possibile scorrere le righe del trace in BCDevelop e avere con immediatezza la conoscenza del flusso avvenuto a runtime, con tanto di informazioni a corredo:



L'opzione "Auto aggiorna" nella vista Trace è molto utile se combinata con l'impostazione dei breakpoint, viene effettuato in automatico il posizionamento sull'ultima riga del trace vale a dire quelle in cui sono stati inseriti i breakpoint.

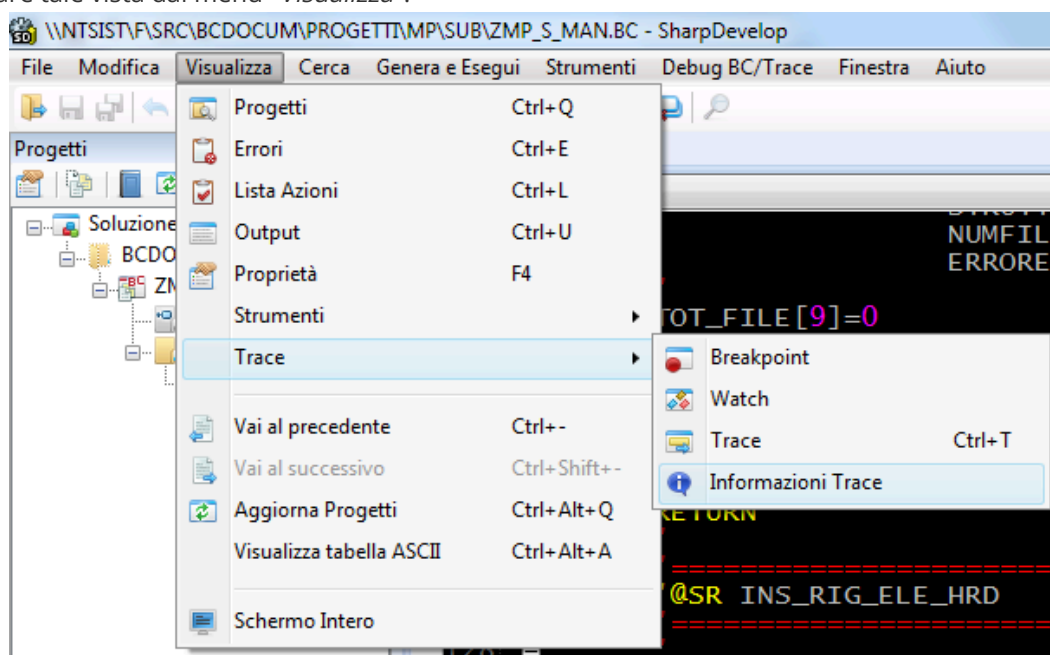
La logica di posizionamento in fase di "Auto aggiorna" è la seguente:

- se non sono presenti righe o si era precedentemente posizionati sull'ultima riga: sono caricate le nuove righe e viene effettuato in automatico il posizionamento sulla nuova ultima riga;
- se erano presenti delle righe e non si era posizionati sull'ultima riga: viene mantenuto il posizionamento sulla riga.

Informazioni del trace

La vista "Informazioni del trace" è strettamente collegata alla vista del "Trace", contiene le informazioni relative alla riga correntemente selezionata nella vista del Trace.

E' possibile attivare tale vista dal menu "Visualizza":



Dump di un programma

L'ambiente di esecuzione BUILDER/Sistemi permette inoltre di gestire errori di accesso alla memoria o simili mediante la creazione di un file di DUMP.

Il file di DUMP creato contiene lo stato della memoria del programma al momento in cui si è verificato l'errore che ha causato l'interruzione dell'elaborazione e racchiude una serie di informazioni diagnostiche utili per identificare rapidamente la causa dell'errore.

Il file di DUMP può essere poi consultato utilizzando programmi di analisi avanzata come WINDBG, su internet, è possibile trovare diverse info utili su come consultare file di questo tipo.

All'interno di un prodotto Sistemi, la generazione del file di DUMP è subordinata alla sua attivazione tramite la definizione sul file di configurazione CONFIG<stazione>.PRO della sezione "[File stato memoria]" con la variabile "Livello dettagli".

Di seguito un esempio del contenuto del file di configurazione:

```
[File stato memoria]
```

```
Livello dettagli=2
```

dove il livello di dettaglio delle informazioni acquisite dal processo in esecuzione può assumere due valori:

Livello dettagli	Descrizione
0 (default)	Il file di DUMP contiene solo informazioni di base (file DUMP di alcuni KByte);
2	Il file di DUMP comprende l'immagine completa della memoria del processo (file DUMP dell'ordine di centinaia di MByte).

Data la natura del file di DUMP e dei suoi contenuti è necessario possedere delle competenze sistemiche o di programmazione avanzata per poter analizzare quanto riportato al suo interno.

Esercitazioni di approfondimento (1)

Esercitazione "Variabili e strutturazione del codice"

Requisiti per lo svolgimento delle esercitazioni

Per eseguire l'esercitazione, è necessario aver installato i sorgenti del corso Builder e creare un nuovo progetto di personalizzazione (es.: "TUT_AP") all'interno dell'area/prodotto installata.

È necessario l'uso della guida e una buona conoscenza degli argomenti: main, moduli e sub, strutturazione del codice, variabili, variabili di ambiente e passaggio parametri.

Durata prevista dell'esercitazione 8h.

Modalità di esecuzione dell'esercitazione

Creare un main in cui andare ad inserire i moduli dei singoli esercizi.

Il nome del main deve includere il riferimento all'esercitazione.

Il nome del modulo deve includere il riferimento dell'esercitazione e dell'esercizio che si sta svolgendo.

Il nome della sub deve includere il riferimento dell'esercitazione e dell'esercizio.

Il nome della dll in cui inserire le sub sviluppate deve contenere il riferimento dell'esercitazione.

Se non esplicitamente richiesto, il modulo deve contenere esclusivamente la sub che svolge l'esercizio.

Nel caso in cui l'esercizio richieda di aggiungere delle funzionalità all'esercizio precedente, si dovrà procedere con la duplicazione di tutti gli oggetti che compongono il programma (sub, classi, tipi dato) il quale avranno nel nome l'indicazione del nuovo esercizio.

Esempi

Nome del main: ESE01 si riferisce al main della prima esercitazione

Nome del modulo: MOD01_02 si riferisce al secondo esercizio della prima esercitazione

Nome della sub: SU01_03_** si riferisce all'esercizio 3 della prima esercitazione

Nome della DLL: DLL01 si riferisce alla prima esercitazione

Esercizio 1

Prendere come riferimento il modulo "CBAS_01" del prodotto e creare un nuovo programma in modo che il messaggio a video:

- sia acquisito da linea di comando del programma (dimensione massima 80 caratteri);
- se superiore a 40 caratteri, sia suddiviso su più righe;
- presenti come ultima riga le informazioni di data e ora corrente in grassetto.

Esercizio 2

Prendere come riferimento quanto sviluppato nell'esercizio 1 e creare un nuovo programma in modo da soddisfare le seguenti caratteristiche:

- inserite sulle finestra del messaggio i tasti <F1> ed <F4>;
- inserite un nuovo messaggio che evidenzi il tasto premuto;
- se premuto <F1> il programma deve visualizzare il messaggio "Premuto F1" ed esce;
- se premuto <F4> il programma deve incrementare un contatore, visualizzarne il valore e rieseguire il primo messaggio.

Esercizio 3

Prendere come riferimento il modulo "CBAS_02" del prodotto e creare un nuovo programma in modo da soddisfare le seguenti caratteristiche:

- il testo del messaggio sia acquisito da riga di comando (dimensione massima 80 caratteri);
- se premuto <F1> il programma deve visualizzare il messaggio "Premuto F1" ed esce;
- se premuto <F4> il programma deve visualizzare il valore del contatore incrementato in una sub esterna, inserita in DLL.

Esercizio 4

Prendere come riferimento il modulo "CBAS_02" del prodotto e creare un nuovo programma in modo da soddisfare le seguenti caratteristiche:

- scomponga la stringa "gennaio,febbraio,marzo,aprile,maggio,giugno,luglio,agosto,settembre,ottobre,novembre,dicembre", ricevuta come riga di comando, nei singoli elementi caricando una collezione;
- in un sub esterna, inserita in DLL, utilizzare la collezione dei mesi passato in interfaccia per creare un'altra collezione che ne inverta l'ordine;
- inserire, a fine elaborazione, un messaggio a video con il testo "Utilizzare il BCLIVE" per controllare la correttezza della nuova collezione.

Per la manipolazione delle stringhe è possibile fare riferimento alle schede del Manuale di riferimento del linguaggio BC:

- "Tipi dato di ambiente"
- "Metodi dei Tipi dato applicativi"
- "Metodi dei tipi base"

Esercizio 5

Prendere come riferimento il modulo "CBAS_02" del prodotto e creare un nuovo programma in modo da soddisfare le seguenti caratteristiche:

- esporre con un messaggio a video il valore della somma di due numeri interi;
- i due numeri devono essere ricevuti tramite riga di comando;

- la somma deve essere eseguita in una sub esterna, inserita in DLL, che riceve in input i due numeri e restituisca il risultato come valore di ritorno (Funzione).

Esercizio 6

Prendere come riferimento il modulo "CBAS_02" del prodotto e creare un nuovo programma in modo da soddisfare le seguenti caratteristiche:

- dichiarare una variabile data e valorizzarla con la data (fissa da programma) 13/10/2019;
- controllare in una sub esterna, in DLL, che il giorno sia un Mercoledì. La sub deve ricevere in interfaccia la data e ritornare il risultato booleano come variabile di ritorno (Funzione). Per effettuare il confronto utilizzare una variabile di tipo dato enumerato con l'elenco dei giorni della settimana (Lunedì, Martedì, ecc...);
- visualizzare con un messaggio il risultato: "Il giorno è un mercoledì" in caso positivo oppure "Il giorno non è un mercoledì" in caso negativo.

Per la manipolazione delle stringhe è possibile fare riferimento alle schede del Manuale di riferimento del linguaggio BC:

- "Tipi dato di ambiente"
- "Metodi dei Tipi dato applicativi"
- "Metodi dei tipi base"

Esercizio 7

Prendere come riferimento il modulo "CBAS_61" del prodotto e creare un nuovo programma in modo da soddisfare le seguenti caratteristiche:

- modificare il testo del contenuto del file in modo da andare a capo ad ogni parola utilizzando come caratteri di fine riga e a capo i seguenti CHR\$(13) e CHR\$(10). Per eseguire questa operazione rileggere il file;
- sovrascrivere il file con il risultato ottenuto inserendo in una subroutine interna parametrica la scrittura e controllo del file;
- aprire il file, da programma, dopo averlo creato per controllare la modifica.

Per la manipolazione delle stringhe è possibile fare riferimento alle schede del Manuale di riferimento del linguaggio BC:

- "Tipi dato di ambiente"
- "Metodi dei Tipi dato applicativi"
- "Metodi dei tipi base"

Esercizio 8

Creare un nuovo programma che visualizzi il prezzo al netto dello sconto di un articolo. Per svolgere questo esercizio è necessario creare una classe CLS01_08_ARTICOLO composta da:

- Codice, di 15 caratteri;
- Descrizione, di 100 caratteri;
- Prezzo, numerico di tipo double di 13 interi di cui 2 decimali;
- Sconto, numerico di tipo intero di 2.

Il calcolo del prezzo scontato deve essere un metodo della classe.

Visualizzare il risultato tramite un messaggio video:

Il prezzo scontato dell'articolo <codice> - <descrizione> è <prezzo scontato>.

Libera scelta per i valori che devono assumere le proprietà della classe. L'unico vincolo è che il prezzo sia maggiore di zero.

Esercizio 9

Creare un nuovo programma in modo che soddisfi le seguenti caratteristiche:

- modificare la classe INDIRIZZI aggiungendo la proprietà TipoIndirizzo di tipo numerico di 1 che accetti i valori: 1-Fatturazione; 2-Spedizione; 3-Altro. La proprietà deve essere inserita tra "NumProgressivo" e "Indirizzo";
- modificare il metodo di caricamento della collezione della classe CLIENTE (.CaricaClienti()) andando a valorizzare per ogni indirizzo di ogni cliente la nuova proprietà, scegliendo liberamente tra i tre valori che può assumere, rispettando i seguenti vincoli:
- il primo indirizzo deve essere quello di fatturazione;
- è possibile avere più indirizzi di spedizione, ma solo uno di fatturazione;
- cercare tutti i clienti che abbiano almeno un indirizzo di spedizione e che non sia titolare di partita IVA;
- modificare i clienti estratti dalla precedente ricerca aggiungendo la P.IVA e impostando il flag di titolare di P.IVA. Impostare la P.IVA con il valore "0123456789" più un ultimo carattere con valore random tra 5 e 9;
- eliminare il Cliente con Codice=2 per cessata attività;
- inserire, a fine elaborazione, un messaggio a video con il testo "Utilizzare il BCLIVE" per controllare la correttezza collezione modificata.

Legacy

- [Nozioni di base sul linguaggio BC](#)
- [La struttura dei programmi](#)
- [Strumenti di analisi e debug](#)
- [Variabili di ambiente BC](#)

Nozioni di base sul linguaggio BC

Nel tempo, il Linguaggio BC si evolve aggiornandosi di nuovi costrutti sempre più simili ai linguaggi di programmazione esterni a Sistemi. Questo permette di essere sempre più flessibile nella progettazione di nuovi programmi.

Ne consegue che man mano si vadano a rendere obsolete vecchie logiche ormai non più supportate dai nuovi standard di programmazione. Questa scheda ha il compito di riassumere i vecchi costrutti utilizzati nei vecchi sorgenti.

Nozioni di base sul linguaggio BC - versione 1

Nel tempo, il Linguaggio BC si evolve aggiornandosi di nuovi costrutti sempre più simili ai linguaggi di programmazione esterni a Sistemi. Questo permette di essere sempre più flessibile nella progettazione di nuovi programmi.

Ne consegue che man mano si vadano a rendere obsolete vecchie logiche ormai non più supportate dai nuovi standard di programmazione. Questa scheda ha il compito di riassumere i vecchi costrutti utilizzati nei vecchi sorgenti.

Dichiarazione variabili

Nella vecchia sintassi la dichiarazione di una variabile aveva la seguente sintassi:

```
<NomeVar><Indicatore tipo>[<Dimensione>]
```

dove <NomeVar> è il nome della variabile, <indicatore tipo> è il carattere che identifica il tipo della variabile e <Dimensione> la dimensione data alla variabile.

	TIPO	Indicatore tipo	NOTE
Alfanumerico	Stringa	\$ (*)	
	Blob	? (*)	
Numerico	Intero	%	Numerici interi su 32 bit (da -2147483648 a +2147483647)
	Long		Numerici interi su 32 bit (da -2147483648 a +2147483647)
	Double	#	Numerici decimali con virgola mobile
Data e ora	Data	& (*)	Una variabile di tipo Data contiene una data espressa nella forma GGMMAAAA
	Periodo	! (*)	Una variabile di tipo Periodo contiene un periodo espresso nella forma MMAAAA

(*) *identificatore obbligatorio*

Le variabili numeriche definite nel sorgente BC sono tradotte in una delle tipologie sopra indicate e sono sempre dotate di segno algebrico, non esistono tipi di dato booleano per i quali si utilizzano variabili di tipo integer numerico di 1 carattere.

Esempi dichiarazione variabili:

STRINGA\$[10]	=""	Alfanumerico di 10 caratteri inizializzata a stringa vuota
TESTO?	="Valore di inizio"	Blob inizializzata con un valore costante
POSIZ%[2]	=0	Numerico intero inizializzato a 0, tra le quadre è indifferente indicare un valore da 1 a 4
NUMDOC[6]	=0	Numerico long inizializzato a 0, tra le quadre è indifferente indicare un valore da 5 a 9
IMPORTO#[10]	=0	Numerico double che non prevede decimali
PERC_IVA[5,2]	=19	Numerico singola precisione che prevede 5 cifre di cui 2 decimali
PER_CORR!	=22007	Periodo inizializzato a 02/2007
DATA_OR&	=26071966	Data inizializzata a 26/07/1966.

Le variabili alfanumeriche devono essere dichiarate indicando la dimensione massima che possono avere.

 Il numero di cifre indicato nel dimensionamento non è vincolante in assoluto, le dichiarazioni $A\%[1]=0$ e $A\%[3]=0$ definiscono entrambe una variabile di tipo intero. Indicare tra parentesi quadre una cifra piuttosto che un'altra può servire come documentazione del sorgente, per manifestare in modo immediato il range previsto per la suddetta variabile, oppure per guidare la realizzazione delle formattazioni (come numero di cifre) per l'input e l'output della variabile stessa.

 Con questa sintassi è obbligatoria l'inizializzazione della variabile contestualmente alla sua dichiarazione.

Vettori e matrici

È possibile definire vettori mono dimensionali e matrici bidimensionali tramite la specifica *DIM*.

- gli elementi del vettore devono essere tutti uguali e dello stesso tipo;
- il primo elemento del vettore ha indice 1;
- per azzerare o inizializzare tutti gli elementi di un vettore è disponibile la specifica *'@BKS*.

Gli elementi che caratterizzano un vettore sono:

- l'identificatore (il nome dato all'array);
- l'indicatore di tipo di dato (obbligatorio in alcuni casi);
- la dimensione del singolo elemento;
- la dimensione del vettore o della matrice.

 I vettori hanno il limite di avere dimensioni fisse. Se ne sconsiglia l'utilizzo soprattutto laddove non si sa a priori il numero di elementi che dovranno contenere. L'alternativa consigliata, è quella di creare una collezione di tipi dato di ambiente BCSTRING, BCINT, BCDOUBLE, BCDATE, BCPERIOD. Le collezioni sono interrogabili e manipolabili molto più

facilmente e la loro occupazione di memoria varia dinamicamente in base al numero di elementi aggiunti. La definizione e l'utilizzo di classi e collezioni verrà trattato in seguito.

Esempio

Definizione di array in BC:

<i>DIM A%[3] (10)</i>	vettore di 10 elementi, tipo di dato numerico intero
<i>DIM VETTL[8] (15, 3)</i>	matrice numerica di 15 righe x 3 colonne, tipo di dato numerico long
<i>DIM MESI\$[20] (12)</i>	vettore di 12 elementi, tipo di dato alfanumerico di 20 caratteri
<i>'@BKS MESI\$</i>	inizializza a "" (spazi) tutti gli elementi del vettore MESI\$

Le strutture dinamiche

Gli array hanno caratteristiche statiche, nella loro definizione deve essere definito in modo esplicito il numero massimo di elementi e tutti gli elementi devono essere dello stesso tipo e della stessa dimensione.

La struttura dinamica ha caratteristiche che sono assimilabili a quelle di una tabella, può contenere campi di tipo diverso, è possibile accedere ad un elemento della struttura in modo diretto oppure accedere effettuando una lettura con indice, può essere creata partendo da un DFX (descrittore archivi) derivando i nomi dei campi dal descrittore utilizzato ed esula dal concetto di visibilità delle variabili tipico delle altre entità del BC.

Una volta definita è utilizzabile in qualsiasi punto del programma successivo alla sua definizione. Per chiarezza, tuttavia, è opportuno che il programmatore definisca la struttura dinamica in tutti i sorgenti che ne fanno uso. La struttura dinamica muore insieme al programma in esecuzione e non al sorgente che l'ha definita e popolata; inoltre è possibile effettuare su di essa operazioni di svuotamento e cancellazione.

La struttura dinamica comunica con il programma attraverso due variabili d'ambiente:

- **ZDSELEM:** posizione elemento corrente.
- **ZDSELEM_TOT:** numero totale elementi in struttura.

⚠ Le strutture dinamiche sono "globali", e come tali possono essere usate in sorgenti differenti anche senza un passaggio esplicito di parametri. Anche se questo può a volte sembrare un vantaggio, il più delle volte porta alla poca comprensione del codice scritto. Ci si trova infatti nelle condizioni di dover rispondere a domande del tipo: Chi ha definito la struttura? Dove è stata popolata? AGIS in parte tiene traccia dell'utilizzo nei sorgenti ma viene penalizzata la lettura e comprensione del codice. Una soluzione più strutturata si è invece ottenuta con l'introduzione della programmazione ad oggetti, che verrà approfondita nello specifico capitolo.

Definizione di una struttura

La struttura viene definita per mezzo della specifica *'@DEFDYNSTRUCT*, indicando il parametro *IND[]* è possibile definire le chiavi di ordinamento diversamente sarà definita come una struttura non ordinata. È possibile definire in modo esplicito tutti i campi della struttura oppure è possibile definire la struttura in modo implicito utilizzando un DFX (descrittore archivi) come sorgente della definizione.

Esempio

Descrittore DFX:	BLDART.DFX
Tipo Record:	AR

CAMPO	NOME CAMPO	TIPO DI DATO	NOME VARIABILE
Codice	COD	C (Alfanumerico)	ARTCOD\$
Descrizione	DES	C (Alfanumerico)	ARTDES\$
Prezzo	PREZZO	S (Numerico Singola precisione)	ARTPREZZO

CAMPO	NOME CAMPO	TIPO DI DATO	NOME VARIABILE
Quantità ordine	QUA	L (Numerico Long)	ARTQUA
Codice IVA	CIVA	I (Numerico Intero)	ARTCIVA%
Vendita (0-No; 1-Sì)	SNOK	N,1 (Numerico Intero)	ARTVENDITA%
Data Creazione	CDAT	A (Data)	ARTCDAT&
Mese inizio vendita	INVEN	E (Periodo)	ARTINVEN!

La struttura dinamica descritta come:

```
'@DEFDYNSTRUCT STRUTTURA["STR_ART"] _
    DFX[BLDART] _
    TIPOREC[ART] _
    CAMPIDFX[COD, DES, PREZZO]
```

equivale a:

```
'@DEFDYNSTRUCT STRUTTURA["STR_ART"] _
    CAMPI[ARTCOD$, ARTDES$, ARTPREZZO#]
```

purché le variabili ARTCOD\$, ARTDES\$, ARTPREZZO# siano definite alla stessa maniera nel sorgente.

Nella definizione, è possibile anche combinare campi di un DFX (parametro *CAMPIDFX*[]) con altri campi aggiuntivi (parametro *CAMPI*[]). Nel caso in cui sia omesso il parametro *CAMPIDFX*[], tutti i campi del DFX vengono implicitamente presi in considerazione.

Strutture indicizzate

La struttura può avere uno o più indici per l'ordinamento, in questo caso l'accesso posizionale non avrà più senso e la lettura sequenziale andrà a scorrere la struttura attraverso l'indice specificato, possono essere definiti più indici e il primo di questi (quello identificato con numero indice = 0) costituisce l'indice fisico, ovvero l'indice di ordinamento degli elementi della struttura.

Ogni ulteriore indice (identificato da numero indice ≥ 1) definisce un indice secondario ed è gestito attraverso una struttura dinamica di "appoggio" che fisicamente punta agli stessi dati di quella in oggetto. Queste strutture secondarie vengono mantenute allineate a seguito dell'inserimento di un nuovo record o della modifica dei campi che compongono la chiave. Nel caso di strutture la cui definizione deriva da un DFX, gli indici vengono di default derivati da questo a meno che non sia espressamente utilizzato il parametro IND sulla definizione della struttura. Bisogna quindi prestare attenzione alla definizione degli indici in quanto il loro allineamento ha comunque un costo di elaborazione aggiuntivo.

 Sulle strutture indicizzate non sono ammesse funzioni di sort.

Lettura di un elemento

Ci sono due modi di lettura: puntuale e sequenziale.

In entrambi i casi è da distinguere il comportamento per la struttura ordinata con indice e per la struttura senza indice.

Nella tabella che segue si riportano i casi possibili:

CASO	SPECIFICA E SINTASSI	SIGNIFICATO
Puntuale non indicizzata	'@GETDYNSTRUCT STRUTTURA[...] _ ELEMENTO[posizione]	Viene preso l'elemento specificato dal parametro ELEMENTO
Puntuale indicizzata	'@FINDDYNSTRUCT STRUTTURA[...] _ FILTRO[campo1=valore1, campo2=valore2..]	Viene trovato il primo elemento che rispetta il FILTRO (che equivale alla definizione della chiave)
Sequenziale non indicizzata	'@GETDYNSTRUCT STRUTTURA[...] _ INDICEINIZIALE[posizione iniziale]	Partendo dall'INDICEINIZIALE scorre tutte le posizioni

CASO	SPECIFICA E SINTASSI	SIGNIFICATO
Sequenziale indicizzata	'@GETDYNSTRUCT STRUTTURA[...] _ IND[...]	Partendo dall'indice IND scorre tutte le posizioni

Esempio

Data la struttura dinamica definita dalla seguente '@DEFDYNSTRUCT:

```
'@DEFDYNSTRUCT STRUTTURA["CLIENTI"] _
    DFX[CLIENTI] _
    TIPOREC[CL] _
    CAMPI[DES_TESTATA$, RIGHE_TESTATA%, PROGRESSIVO%[2]]
'@GETDYNSTRUCT STRUTTURA["CLIENTI"] _
    ELEMENTO[2]
```

La specifica seleziona l'elemento della struttura "CLIENTI" presente in posizione 2, ed assegna tutte le variabili con il contenuto dei campi omonimi appartenenti alla struttura:

```
'@GETDYNSTRUCT STRUTTURA["CLIENTI"] _
    ELEMENTO[POS]
    DFX[CLIENTI] _
    TIPOREC[CL] _
    CAMPI[D_TST$=DES_TESTATA, PROGRESSIVO]
```

La specifica seleziona l'elemento *POS* appartenente alla struttura dinamica identificata da "CLIENTI".

Il parametro DFX indica che le variabili associate al DFX vengono valorizzate con il contenuto dei campi contenuti nella struttura, definiti tramite DFX dalla '@DEFDYNSTRUCT.

Il parametro *CAMPI* [] invece:

- assegna alla variabile D_TST\$ il contenuto del campo della struttura DES_TESTATA\$
- assegna alla variabile PROGRESSIVO, (definita altrove) il valore del campo PROGRESSIVO.



Il *Manuale di riferimento del linguaggio BC* contiene la definizione dettagliata delle specifiche di accesso alle "Strutture Dinamiche".

Scrittura di un elemento

La scrittura di un elemento, inserimento di una nuova riga in struttura o aggiunta di una nuova riga avviene per mezzo delle specifiche '@INSDYNSTRUCT e '@ADDSDYNSTRUCT, la variazione di uno, alcuni o tutti i campi di uno specifico elemento della struttura è gestito con la specifica '@SETDYNSTRUCT.

⚠ Come per le tabelle è possibile aggiornare solo alcuni campi ed è inoltre possibile indicare anche la scrittura di più elementi tramite opzioni di filtro

Cancellazione di una struttura

L'eliminazione di uno o più elementi di una struttura dinamica è possibile effettuarla utilizzando la specifica '@DELDYNSTRUCT che tramite il parametro *FILTRO* [] permette di agire sugli elementi che soddisfano la condizione specificata nel parametro.

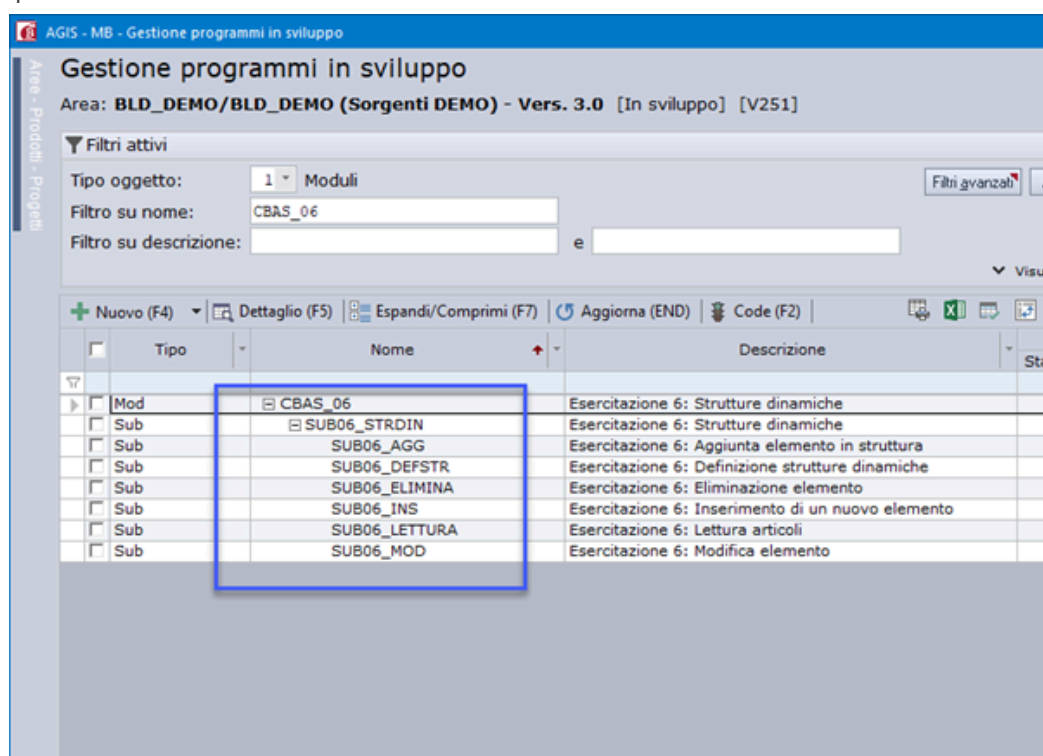
È possibile svuotare completamente una struttura dinamica mantenendo la definizione in memoria (specifica '@ZAPDYNSTRUCT') oppure eliminare completamente una struttura dinamica (specifica '@KILLDYNSTRUCT').

Una struttura dinamica eliminata dalla memoria con la specifica '@KILLDYNSTRUCT' deve essere eventualmente ridefinita ('@DEFDYNSTRUCT') per poter essere nuovamente utilizzata dal programma BC in esecuzione.

Altre specifiche utili alla gestione delle strutture dinamiche

'@FINDDYNSTRUCT	ricerca di un elemento nella struttura (tramite FILTRO)
'@COUNTDYNSTRUCT	conteggio del numero di elementi nella struttura
'@SORTDYNSTRUCT	ordina la struttura secondo uno o più campi specificati
'@ZAPDYNSTRUCT	svuota la struttura ma resta in memoria la sua definizione
'@KILLDYNSTRUCT	elimina la struttura dalla memoria, anche la sua definizione.

Nei sorgenti di esercitazione distribuiti insieme all'ambiente di sviluppo BUILDER/Sistemi è disponibile un programma di prova come esempio di utilizzo delle strutture dinamiche:



Variabili di configurazione

Il linguaggio BC rende disponibile un insieme di variabili di sistema, queste variabili sono utili alla corretta esecuzione del prodotto e forniscono al programma parecchie informazioni utili come le coordinate di accesso alla base dati.

Elenco di alcune delle variabili di sistema disponibili:

NOME VARIABILE	DEFINIZIONE
PHP	Direttiva di collocazione dei programmi
PHB	Direttiva di collocazione dei dati di BASE
PHD	Direttiva di collocazione dei DATI di gruppo
PHT	Direttiva di collocazione di tabelle temporanee (è valorizzata come PHB+"TMP")
GR	Gruppo in elaborazione (sigla di 2 caratteri)
WS	Nome della stazione o sessione di lavoro (sigla di 2 caratteri)
WSP	Nome della stazione principale rispetto alle sessioni virtuali in ambiente Windows (sigla di 2 caratteri)

NOME VARIABILE	DEFINIZIONE
ANNOEXT	Anno solare di riferimento definito per il gruppo in elaborazione

Tutte queste variabili sono deprecate sui nuovi sorgenti che fanno uso di STRICT.

Di seguito, una tabella riepilogativa che indica con quali metodi sono state sostituite:

NOME VARIABILE	Nuovo/i metodo/i
GR	BCContextInfo.GetGruppo / SetGruppo
WS	BCContextInfo.GetStazione
WSP	BCContextInfo.GetStazionePrincipale
ZOPERATORE	BCContextInfo.GetOperatore
ZOPERATOREDOM	BCContextInfo.GetOperatoreDominio
ANNOEXT	BCContextInfo.GetAnnoDF
ZBCERR	BCContextInfo.GetLastError
ZBCMSG ZBCMSGEX	BCContextInfo.GetLastErrorMessage
ZPROFILO	BCContextInfo.GetProfilo
PHB	BCContextInfo.GetCartellaBase
PHD	BCContextInfo.GetCartellaDati
PHP	BCContextInfo.GetCartellaProgrammi
PHT PHTEX	BCContextInfo.GetCartellaTemporanea
PR	BCContextInfo.GetSiglaProdotto
PRP	BCContextInfo.GetSiglaProcedura
DITTA	BCContextInfo.GetDitta
IDSCODCLI	BCContextInfo.GetIdsCodiceUtente
IDSNDISCO	BCContextInfo.GetIdsNumeroDisco
ZSRCSTACK	BCContextInfo.GetCallStack
ZNOMEEXE	BCContextInfo.GetProgramName
ZNOMSBC	BCContextInfo.GetSubroutineName
ZNOMMOD	BCContextInfo.GetModuleName
ZNUMDEC	BCContextInfo.GetDecimalCategory
ZNUMDEC	BCContextInfo.SetDecimalCategory
ZSYSPROCID	BCContextInfo.GetProcessId
ZSYSDIRSTART	BCContextInfo.GetStartupDir
IDS8DEMO	BCContextInfo.IsIDS Demo
ZWEB	BCContextInfo.IsWeb
ZSIR	BCContextInfo.IsSir
PHSTD	BCContextInfo.GetCartellaDatiStandard
ZDBTLPROD	BCDatabaseInfo.GetName
ZDBSERVER	BCDatabaseInfo.GetServer
ZDBUSER	BCDatabaseInfo.GetUser
ZDBPWD	BCDatabaseInfo.GetPassword
ZDBAUT	BCDatabaseInfo.GetAuthenticationType
ZDBRETRY	BCDatabaseInfo.SetRetryTransaction
ZDATAZERO	BCSystemInfo.GetDateMin
SYSDATEXT	BCSystemInfo.GetDate
ZSYSUSERNAME	BCSystemInfo.GetUserName
ZSYSPCNAME	BCSystemInfo.GetComputerName
ZSYSDIRTEMP	BCSystemInfo.GetTempDir
ZSYSIPADDRESS	BCSystemInfo.GetIPAddress

La struttura dei programmi

Nel tempo, il Linguaggio BC si evolve aggiornandosi di nuovi costrutti sempre più simili ai linguaggi di programmazione esterni a Sistemi. Questo permette di essere sempre più flessibile nella progettazione di nuovi programmi.

Ne consegue che man mano si vadano a rendere obsolete vecchie logiche ormai non più supportate dai nuovi standard di programmazione. Questa scheda ha il compito di riassumere i vecchi costrutti utilizzati nei vecchi sorgenti.

La struttura dei programmi - versione 1

Nel tempo, il Linguaggio BC si evolve aggiornandosi di nuovi costrutti sempre più simili ai linguaggi di programmazione esterni a Sistemi. Questo permette di essere sempre più flessibile nella progettazione di nuovi programmi.

Ne consegue che man mano si vadano a rendere obsolete vecchie logiche ormai non più supportate dai nuovi standard di programmazione. Questa scheda ha il compito di riassumere i vecchi costrutti utilizzati nei vecchi sorgenti.

I sorgenti include

Un altro modo di strutturare un programma è costituito dalle include, le include permettono di gestire un programma come se contenesse fisicamente una parte di codice che, di fatto, è stata memorizzata in un diverso file sorgente. In fase di generazione del codice C, la parte di codice incluso, viene inserita fisicamente all'interno del sorgente nella posizione indicata dalla richiesta di inclusione.

Le include possono essere di due tipi:

- **normali:** sono parti di programma che vengono copiate nel sorgente BC così come sono state scritte, senza subire alcuna variazione. Chiaramente è necessario che tutte le variabili, i tipi di record e, in generale, le strutture utilizzate nell'include siano state precedentemente dichiarate e inizializzate dal sorgente che le include;
- **parametrizzati:** sono parti di programma che contengono dei parametri al posto delle strutture utilizzate.

Nella fase di generazione del codice C, dopo aver copiato l'include nel sorgente, vengono sostituiti i parametri con gli oggetti specificati all'atto del richiamo. In questo modo, il codice contenuto nell'include ed il sorgente che la contiene possono anche utilizzare oggetti individuati in modo distinto o oggetti distinti ma compatibili, senza alcun vincolo sugli identificatori.

Come le subroutine esterne, anche le include possono essere utilizzate in più programmi e quindi, prima di apportare delle modifiche al codice, è bene verificare tutti i possibili effetti negativi che ne possono derivare.

Esempio

Modulo CBAS_04.bc

BC

```
'@PG CBAS_04      ' Esercitazione 4: Hello + Include

' Valorizzazione variabile varTesto
DIM varTesto[STRING]="HELLO, WORLD!"

' Richiamo dell'include INC_CB004
'@IN INC_CB04

End
```

Include INC_CB04.bc

BC

```
' include INC_CB004      ' Esercitazione 4: Hello + Include
',
' La funzione MESSAGEBOX va in sostituzione della specifica @DEFVIDMSG,
' semplificandone l'utilizzo e standardizzando la configurazione dei tasti.
',
MESSAGEBOX(#INFO,"CBAS_04", varTesto)
',
' '@DEFVIDMSG MSG[TESTO$] _
'          T[]
```



I sorgenti include sono da considerarsi costrutti obsoleti e pertanto vanno sostituiti ove possibile con delle subroutine esterne che permettono una maggiore leggibilità e comprensione del codice.

I moduli di inizializzazione

Attraverso la definizione di un modulo di inizializzazione, viene data la possibilità di associare a ciascun main di un Prodotto/Progetto un modulo BC, che venga richiamato quando si esegue un qualsiasi programma appartenente al prodotto/progetto nel quale il modulo è stato definito.

Il modulo di inizializzazione è quindi un modulo BC all'interno del quale è possibile:

- scrivere del codice BC per eseguire letture di tabelle, controlli, ecc.
- definire delle variabili globali che possano essere utilizzate anche da tutti i moduli appartenenti al main al quale è associato.

Un modulo di inizializzazione deve essere definito all'interno di un Prodotto/Progetto, nell'apposita box di AGIS per la definizione delle caratteristiche di un Prodotto/Progetto. Può altresì essere ridefinito per ciascun main all'interno dei dati oggetto del main stesso.

Il modulo di inizializzazione eventualmente definito, viene eseguito prima di richiamare un qualsiasi altro modulo definito all'interno del main, dopo che sono state eseguite le seguenti operazioni:

- reperimento delle informazioni "base", per rendere disponibili al programma tutte le variabili che vengono normalmente definite e valorizzate dalla specifica '@FZ;
- reperimento delle informazioni sull'installazione, attraverso la lettura dei file di configurazione sul file system, per rendere disponibili al programma le informazioni sulle caratteristiche dell'installazione (installazione demo, prodotti abilitati, ecc.).

Struttura e caratteristiche del modulo di inizializzazione

Un modulo di inizializzazione è un normale modulo BC, che deve quindi iniziare con una specifica '@PG e terminare con una specifica END. Essendo un modulo BC può quindi contenere qualsiasi istruzione supportata, compresi richiami a subroutine esterne.

Possono essere definite sia variabili locali (dichiarate all'interno del modulo), che variabili common (dichiarate in un'istruzione COMMON) che particolari variabili globali visibili a tutto il main per le quali il modulo di inizializzazione nasce (NCOMMON [GLOBAL] o NCOMMON [SHARED]).

All'interno di un modulo di inizializzazione possono essere eseguiti dei controlli, per verificare, ad esempio, se tutti i parametri definiti consentono la corretta esecuzione di un programma; nel caso in cui qualcuno di questi controlli fallisca, bisogna fare in modo che il programma termini, senza richiamare i moduli definiti all'interno del main. Per fare ciò si

introduce sulla specifica `END` un parametro che, se indicato con un valore diverso da 0, costituisce un Error Level del modulo e consente la terminazione del programma in esecuzione.

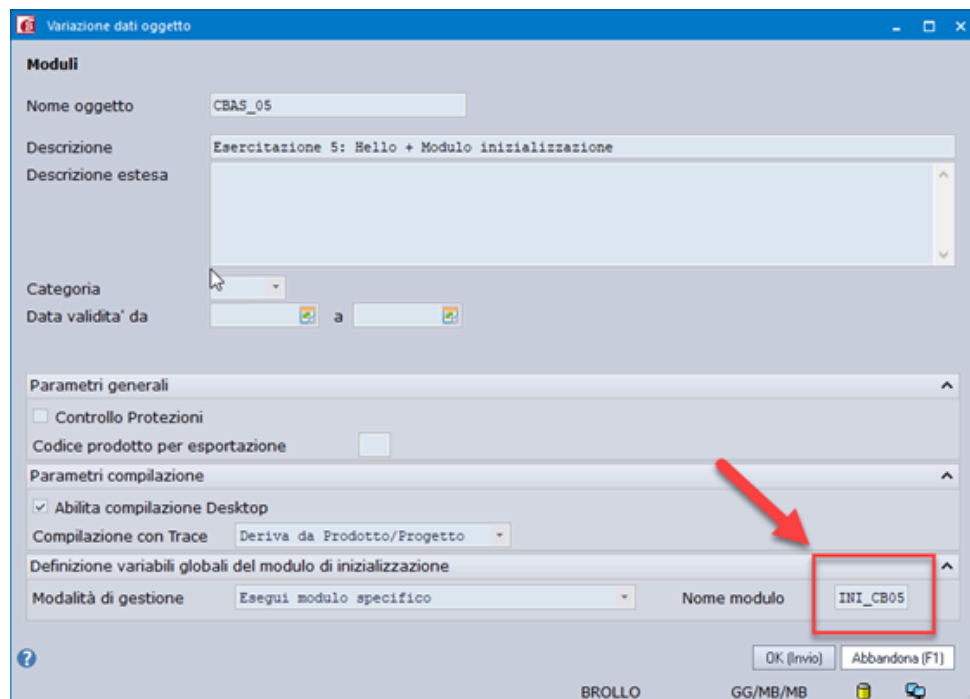
BC

```
End ERROR%
```

Se `ERROR%` è uguale a 0, allora il programma prosegue nell'esecuzione, altrimenti il programma termina e non sono richiamati i moduli presenti all'interno del main.

Ovviamente se non si intende gestire tale meccanismo di controllo degli errori, è possibile utilizzare `END` senza parametri, così come per un qualsiasi altro modulo BC.

Esempio



Modulo di inizializzazione INI_CB05.bc

BC

```
'@PG INI_CB05      ' Esercitazione 5: Modulo inizializzazione
'
NCOMMON [SHARED]
    TESTO$[50]
END NCOMMON
'
TESTO$="HELLO, WORLD!"
ERROR% [1]=0
'
END ERROR%
```

Modulo CBAS_05.bc

BC

```
'@PG CBAS_05      ' Esercitazione 5: Hello + Modulo inizializzazione
'
' Settare il modulo INI_005 come modulo d'inizializzazione
' all'interno della gestione dati prodotto/progetto
'
' EMISSIONE A VIDEO DELLA VARIABILE TESTO DEFINITA NEL MODULO D'INIZIALIZZAZIONE
'
MESSAGEBOX (#INFO, "CBAS_05", TESTO$)
'
END
```

Definizione variabili globali NCOMMON

Nel linguaggio BC è possibile definire delle variabili globali che sono condivise tra i moduli e le sub di un eseguibile, normalmente sono definite all'interno di un modulo di inizializzazione.

Per definire questo tipo di variabili, si deve utilizzare un'istruzione *NCOMMON*, come descritto nel seguente esempio:

BC

```
NCOMMON [SHARED]
  A% [2]
  B$ [10]
  C# [10, 2]
END NCOMMON
```

La sintassi segue le regole definite per l'istruzione *NCOMMON*, con la particolarità che non è necessario indicare un nome di *NCOMMON*, per identificare l'insieme di variabili utilizzare l'attributo *[Named Common]*.

In fase di traduzione di un qualsiasi modulo BC (programma o subroutine esterna) che utilizzi il modulo di inizializzazione, verrà analizzata la struttura *NCOMMON [SHARED]* e verranno dichiarate e rese disponibili le variabili in essa contenute. Quindi un modulo di inizializzazione può, ad esempio, dichiarare come globali le variabili indicate nella struttura *NCOMMON*, e poi impostare tali variabili. Un qualsiasi modulo richiamato dopo il modulo di inizializzazione avrà a disposizione tali variabili valorizzate, e ovviamente potrà a sua volta modificarle.

Le variabili dichiarate in *NCOMMON [SHARED]* saranno valorizzate in tutti i moduli richiamati in *CHAIN* o '@SBC, sino a che:

- si richiama un modulo appartenente ad un altro main
- si richiede l'esecuzione in background di un modulo.

In queste due situazioni, infatti, prima di eseguire l'operazione richiesta, viene eseguito il modulo di inizializzazione definito per il main.

Nel caso in cui venga variata la struttura *NCOMMON [SHARED]*, aggiungendo o togliendo variabili, non viene persa la compatibilità con gli altri moduli tradotti in precedenza.

Per rendere effettive le modifiche sarà sufficiente compilare il modulo di inizializzazione e tutti i moduli che fanno riferimento alle variabili inserite o eliminate, eseguendo poi un link del main.

Nel caso in cui, invece, venga variata la dimensione o il tipo di una variabile già presente in *NCOMMON* [SHARED] è necessario ritradurre tutti i moduli del main associato al modulo di inizializzazione variato.

Integrazione tra prodotti e NCOMMON di prodotto

La definizione di variabili globali tramite la specifica *NCOMMON* all'interno di un modulo di inizializzazione rende di fatto un modulo o una sub di un prodotto **dipendente** dal modulo di inizializzazione.

Eventuali variabili globali utilizzate dal modulo o dalla sub sono definite all'ingresso del main, all'inizio dell'esecuzione, e sono sempre disponibili anche in tutte le sub richiamate.

Nella pratica, le variabili globali sono sempre correttamente valorizzate e disponibili ai moduli e alle sub di un prodotto quando questi sono eseguiti attraverso il loro stesso prodotto dove all'interno dei main è stato inserito il modulo di inizializzazione.

Tuttavia il richiamo di una sub che fa uso di variabili globali non può comportarsi nel modo corretto se richiamata da un eseguibile esterno che non esegue quanto previsto dal modulo di inizializzazione.

Infatti se all'interno di un prodotto viene richiamata una sub in dll appartenente ad un altro prodotto è possibile non avere delle variabili globali necessarie correttamente valorizzate, e di conseguenza non è garantito il corretto funzionamento della funzione stessa.

Per questo motivo sui prodotti standard Sistemi si sta cercando di dismettere l'uso di variabili globali definite in *NCOMMON*.

Queste sono un'arma a doppio taglio, sebbene comode perché non devono essere passate come parametri tra le varie sub, nascondono la loro dichiarazione e la loro valorizzazione, soprattutto non rendono la sub autonoma per la sua esecuzione.

Come vedremo nei capitoli relativi allo sviluppo Web, queste risultano un limite e non possono essere utilizzate.

Strumenti di analisi e debug

Nel tempo, il Linguaggio BC si evolve aggiornandosi di nuovi costrutti sempre più simili ai linguaggi di programmazione esterni a Sistemi. Questo permette di essere sempre più flessibile nella progettazione di nuovi programmi.

Ne consegue che man mano si vadano a rendere obsolete vecchie logiche ormai non più supportate dai nuovi standard di programmazione. Questa scheda ha il compito di riassumere i vecchi costrutti utilizzati nei vecchi sorgenti.

Nozioni di base sul linguaggio BC - versione 1

Nel tempo, il Linguaggio BC si evolve aggiornandosi di nuovi costrutti sempre più simili ai linguaggi di programmazione esterni a Sistemi. Questo permette di essere sempre più flessibile nella progettazione di nuovi programmi.

Ne consegue che man mano si vadano a rendere obsolete vecchie logiche ormai non più supportate dai nuovi standard di programmazione. Questa scheda ha il compito di riassumere i vecchi costrutti utilizzati nei vecchi sorgenti.

Debug C - Microsoft Visual C++ 2010

L'ambiente di sviluppo BUILDER/Sistemi supporta due versioni diverse del compilatore:

- Microsoft Visual C++ 6;
- Microsoft Visual C++ 2010.

l'utilizzo di Microsoft Visual Studio C++ 2010 è stata resa disponibile con la versione di ambiente 22.1 e ad oggi i prodotti Sistemi sono stati tutti aggiornati a tale versione di compilatore.

In questo capitolo descriviamo le funzionalità e le modalità di esecuzione del Debug C per la versione 2010 di Visual Studio.



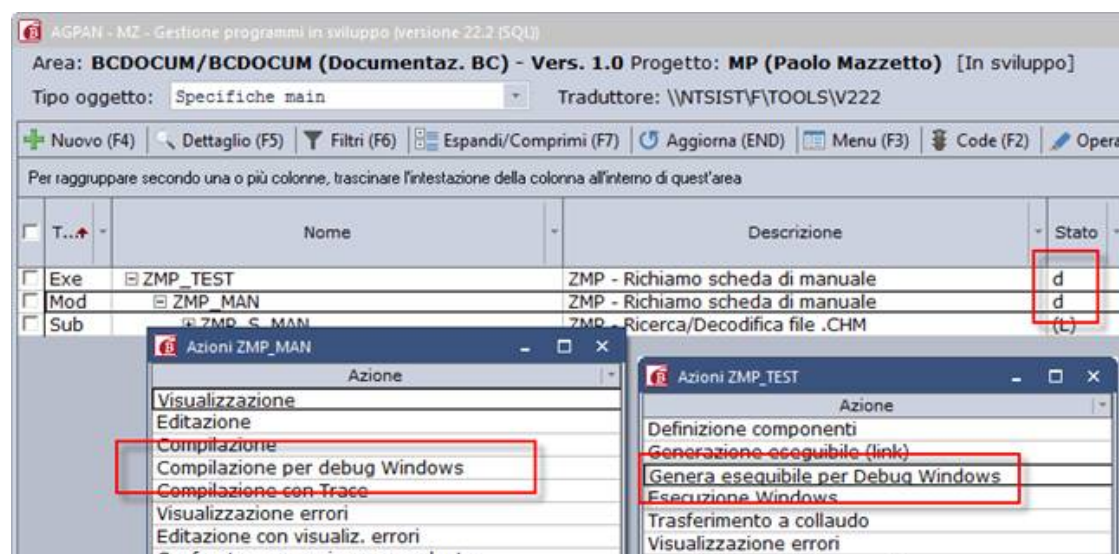
verificare che siano correttamente inseriti la versione del compilatore, la direttiva di installazione del compilatore, l'elenco librerie e la direttiva SDK Windows.

Direttiva SDK Windows:

È la direttiva che contiene il Software Development Kit di Windows, è necessario indicarla se è impostato Microsoft Visual C++ 2010 come versione del compilatore.

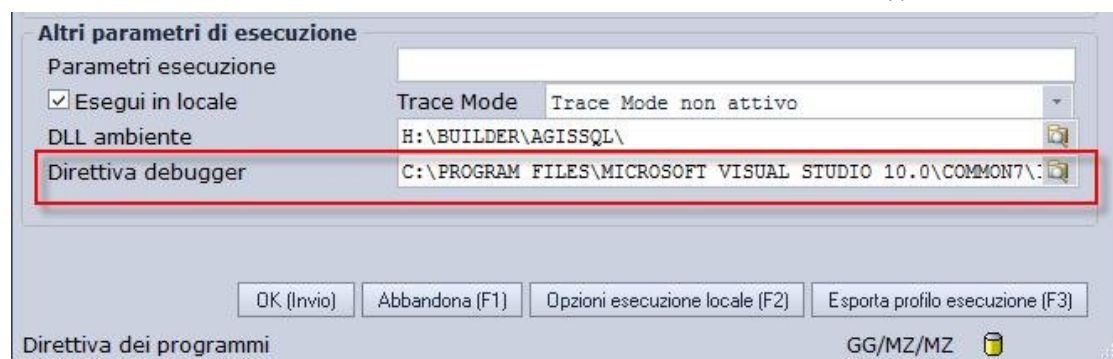
Normalmente si trova sotto "*C:\Program Files\Microsoft Visual Studio 10.0\SDK\vn.n*" se Visual Studio è installato sul client; invece in una installazione in rete il percorso potrebbe essere come quello dell'esempio sopra riportato.

E' necessario compilare e linkare il programma (o i programmi) che devono essere eseguiti in debug attraverso la specifica azione messa a disposizione da AGIS e, allo stesso, modo devono essere linkati i main e le dll che li contengono:



Al termine della compilazione verrà visualizzata una "d" all'interno della colonna stato di AGIS per indicare che l'oggetto è stato compilato per debug.

Dopo aver effettuato queste operazioni eseguire il programma indicando correttamente la "*Direttiva debugger*".



Normalmente il debugger lo si trova nel seguente percorso: "*C:\Program Files\Microsoft Visual Studio 10.0\Common7\IDE*".

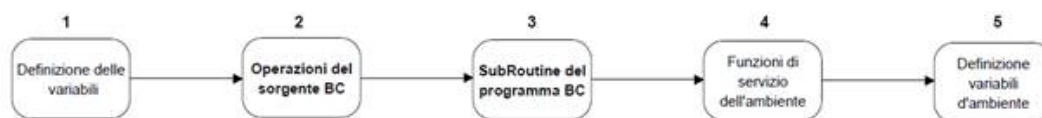
⚠ Attenzione! Negli esempi riportati in questo manuale il compilatore di Microsoft Visual Studio C++ 2010 è stato installato in rete mentre il debugger è stato installato sul client; normalmente Microsoft Visual Studio è installato sul client.

Struttura del sorgente C

Prima di passare all'utilizzo di Visual Studio 2010 per eseguire il debug, analizziamo il sorgente C che viene creato dalla traduzione / compilazione.

La struttura

Un sorgente C derivante dalla traduzione BC è strutturato nel seguente modo:



1. Nella prima parte del sorgente vengono dichiarate le variabili del sorgente BC e le strutture C che le conterranno;
2. successivamente sono riportate tutte le operazioni del sorgente presenti al di fuori delle subroutine BC;
3. dopo vengono inserite tutte le subroutine del programma BC;
4. in seguito, vengono rilasciate le funzioni specifiche per il funzionamento dell'ambiente;
5. infine l'inizializzazione delle variabili d'ambiente.

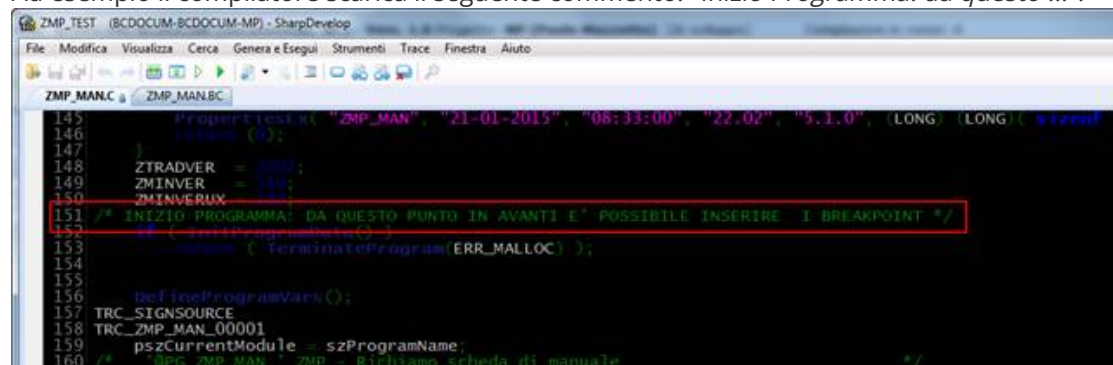
Per quanto riguarda l'utilizzo del debug, le sezioni del sorgente C interessate sono quelle indicate al punto 2 e al punto 3, nella quale sono riportate tutte le istruzioni del programma BC.

I commenti

La "*compilazione per debug windows*" oltre a predisporre il sorgente per la fase di debug con il Visual Studio 2010 aggiunge all'interno del sorgente C dei commenti.

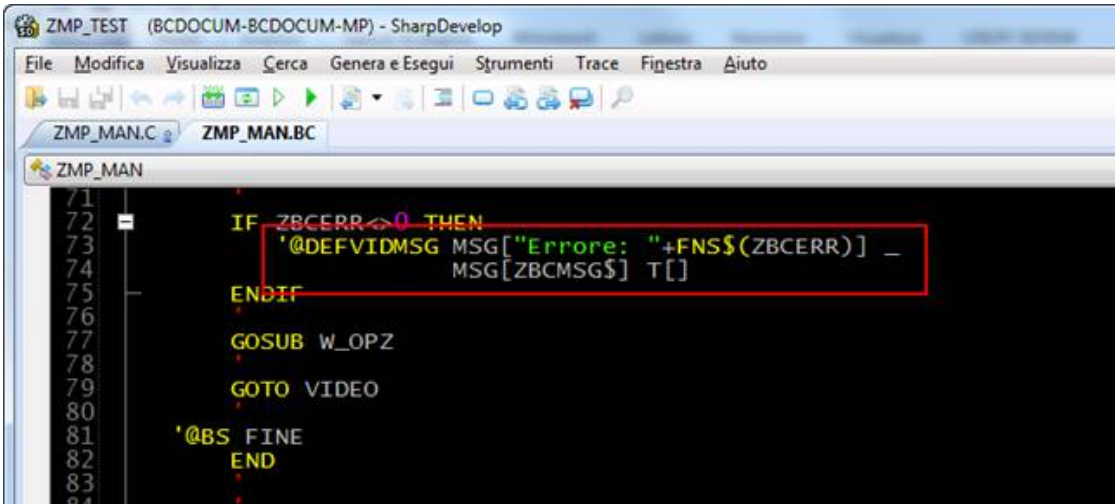
I commenti sono utili per facilitare la navigazione all'interno del sorgente e per determinare da quale punto in avanti è possibile inserire un breakpoint per iniziare il debug del programma.

Ad esempio il compilatore scarica il seguente commento: "*Inizio Programma: da questo ...*".

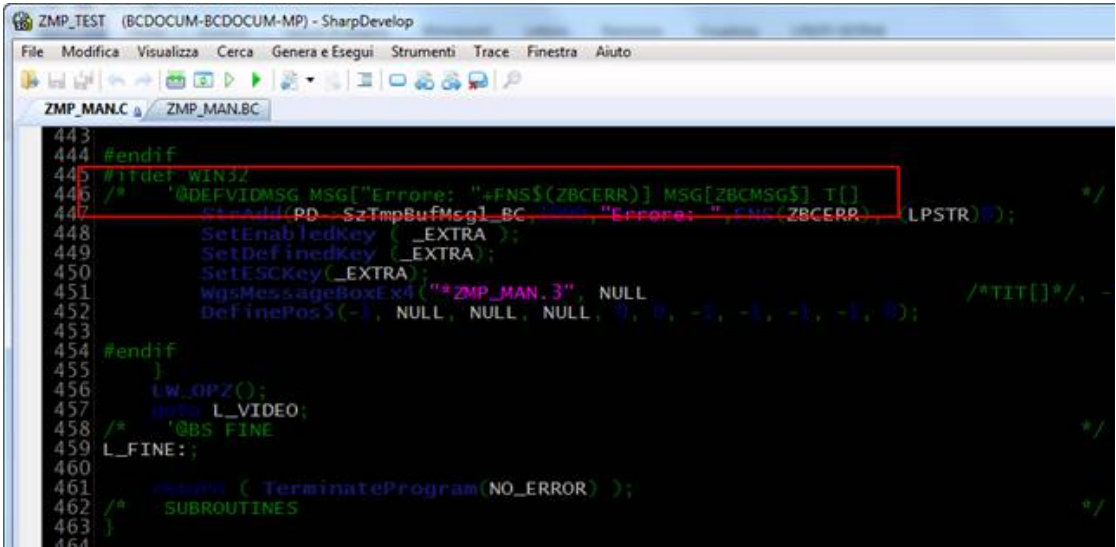


Inoltre, tutte le funzioni presenti nel sorgente BC ('@SBC', '@GETDBDATA', '@DEFVID', '@PRINTFORM, etc.) vengono riportate con codifica BC, come commento all'interno del sorgente C, prima della relativa funzione tradotta in C.

Esempio sorgente BC:



Esempio sorgente C:



Grazie a questo accorgimento è molto più facile e comprensibile navigare all'interno di un sorgente C. Per ricercare una specifica parte del programma BC all'interno del sorgente C, sarà sufficiente ricercare la funzione BC senza dover conoscere come viene codificata in C.

Le variabili

La fase di traduzione/compilazione modifica in parte il nome delle variabili utilizzate nel programma BC:

- viene aggiunto un prefisso;
- viene aggiunto il suffisso "_BC";
- sono rimossi i simboli che identificano il tipo della variabile (\$, %, &, #, ?, !).

Prefissi:

PD->	Prefisso per tutte le variabili dichiarate nei sorgenti e per tutti campi dei descrittori
PA->	Prefisso per tutti gli array dichiarati nel sorgente
PC->	Prefisso per tutte le variabili COMMON del sorgente
PS->	Prefisso per tutte le variabili presenti nell'SBP del sorgente

⚠ Se le variabili di programma o gli array sono inseriti nei parametri della specifica '@SBP' viene applicato loro il prefisso PS->.

Suffisso:

Di seguito viene riportata la tabella di conversione dei tipi variabile da BC a C:

BC C		
BC	Tipo	C
A[1 .. 4]	INT	A_BC
B[5 .. 9]	LONG	B_BC

IMP[10 ..]	DOUBLE	IMP_BC
IMP2[5.2]	DOUBLE	IMP2_BC
DATA	LONG	DATA_BC
PERIODO!	LONG	PERIODO_BC
TESTO[20]	CHAR	TESTO_BC[21]

Da come si può notare in tabella, in caso di conversione di una variabile di tipo testo viene modificato il dimensionamento aggiungendo un carattere in più sulla dichiarazione C, nell'esempio specifico la variabile TESTO di 20 caratteri viene portata a 21 caratteri in C.

Il motivo dell'aumento del dimensionamento per le variabili di tipo alfanumerico è che il testo nella variabile viene terminato con il carattere C "\0" (codice ASCII); si tratta di una proprietà del linguaggio C.

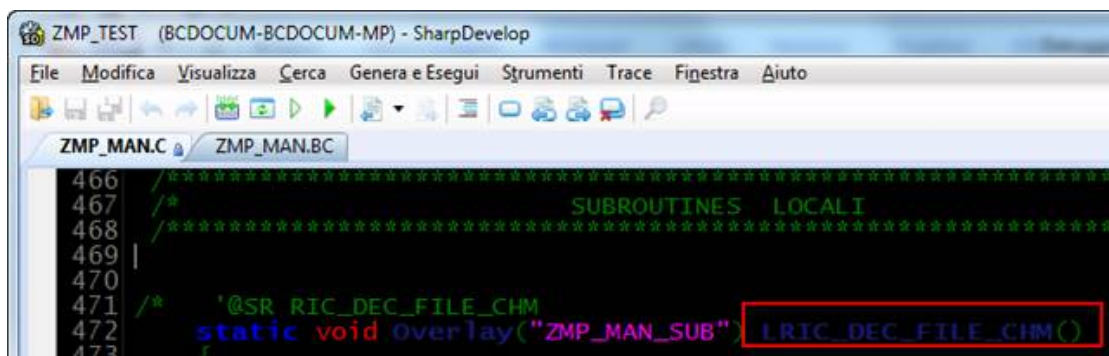
Le subroutine

La fase di traduzione/compilazione modifica in parte il nome delle sub interne aggiungendo come prefisso il carattere "L" prima del nome.

Esempio sorgente BC:



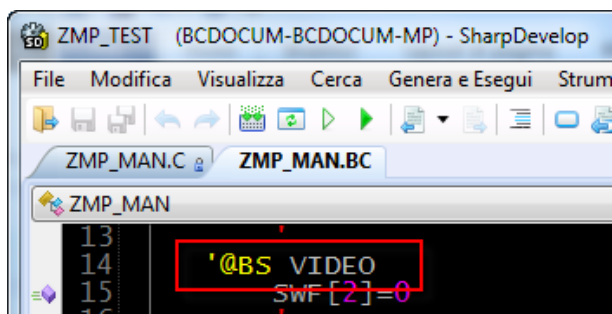
Esempio sorgente C:



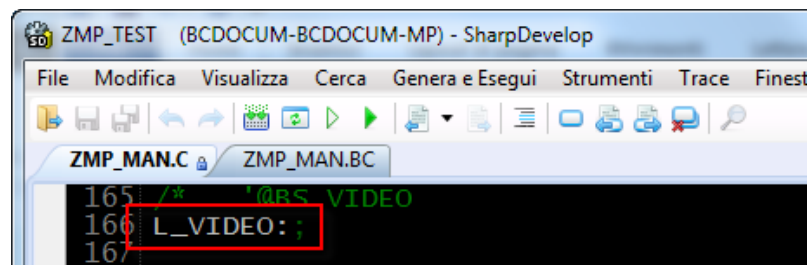
Le label

Allo stesso modo sono modificate le label a cui si aggiungono come prefisso i caratteri "L_" prima del nome:

Esempio sorgente BC:

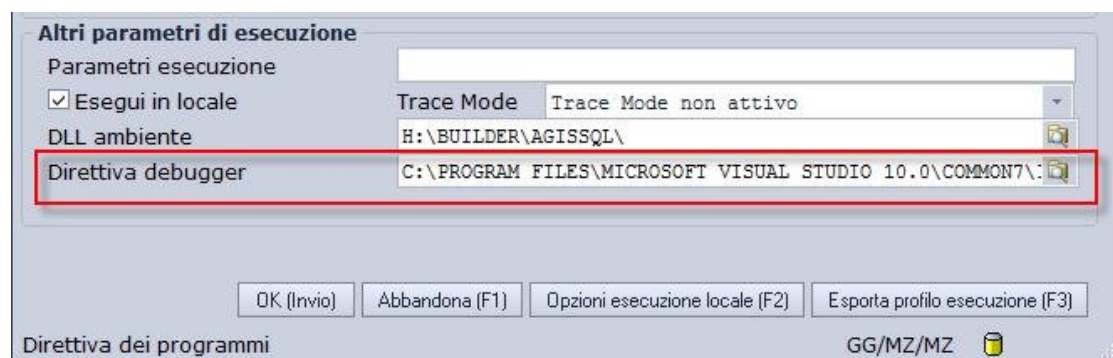


Esempio sorgente C:



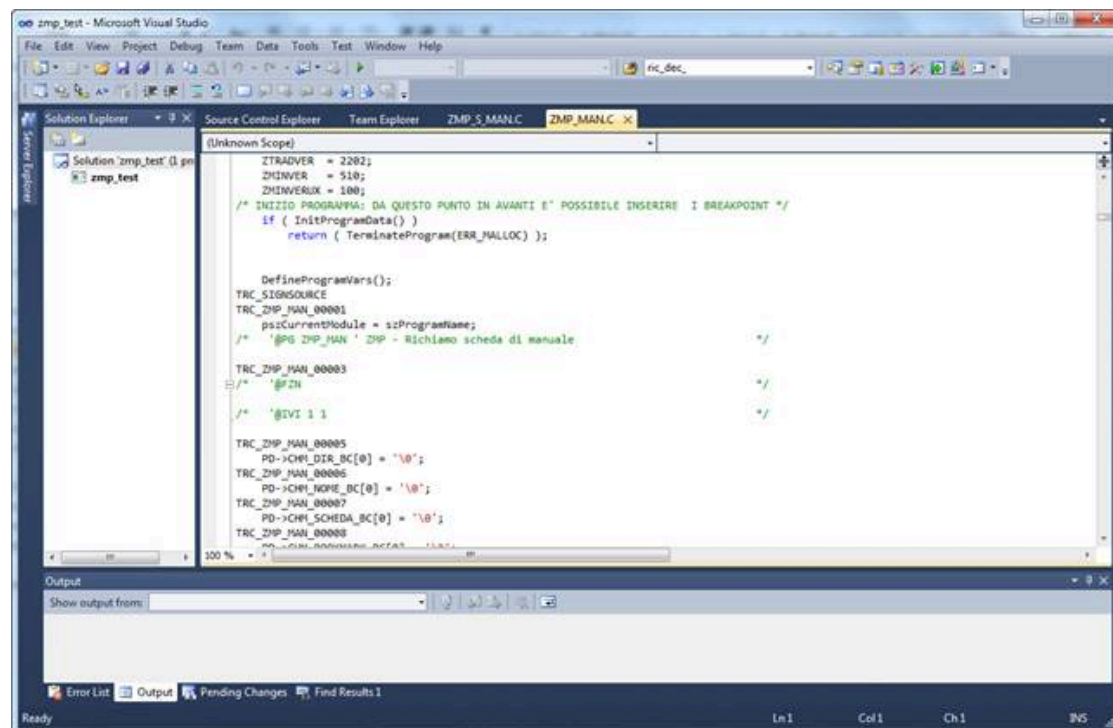
Esecuzione di un modulo con Debug

In AGIS selezionare il programma che deve essere eseguito ed eseguire l'azione di AGIS *"Esecuzione Windows"*, facendo attenzione che tra gli altri parametri di esecuzione sia correttamente indicata la *"Direttiva debugger"*:



In questo campo occorre inserire il percorso nel quale è installato Visual Studio 2010 per poter eseguire il debug.

Proseguendo sarà eseguito il programma utilizzando il debugger di Visual Studio 2010, l'esecuzione sarà gestita Step-by-Step sul sorgente "C" tradotto:



A questo punto è possibile:

- Inserire breakpoint sul sorgente con <F9>;
- avviare il debug con <F5>;

- interrompere in modo forzato il debug <Shift+F5>;
- eseguire il programma Step-by-step con <F10>;
- utilizzare tutti gli strumenti disponibili su Visual Studio quali:
 - breakpoint condizionali;
 - watch;
 - ...

⚠ La fine forzata di un programma è un'operazione delicata che potrebbe lasciare attivi archivi temporanei o danneggiare la struttura dei dati a causa di operazioni lasciate a metà.

Pertanto, sebbene sia possibile interrompere il processo in qualsiasi momento, è auspicabile avere una buona conoscenza del funzionamento del programma per evitare danni alla base dati.

Utilizzo del debug

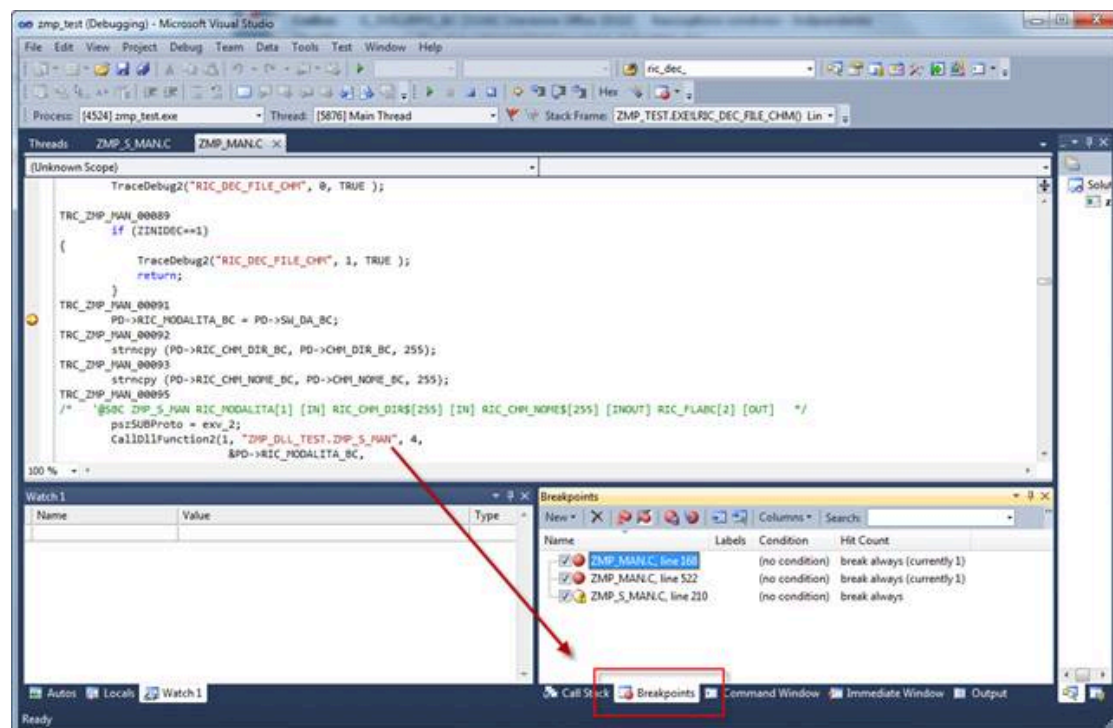
Terminate tutte le operazioni di compilazione e di configurazione dei sorgenti è possibile utilizzare il debug.

Inserire un breakpoint

I Breakpoint consentono al debug di fermarsi ad un determinato punto del sorgente e possono essere inseriti a partire dal commento presente sul sorgente C: *"Da questo punto è possibile inserire i breakpoint"*.

Posizionandosi sulla linea di codice sulla quale si desidera inserire un breakpoint premere il tasto <F9>, premere nuovamente <F9> per rimuoverlo.

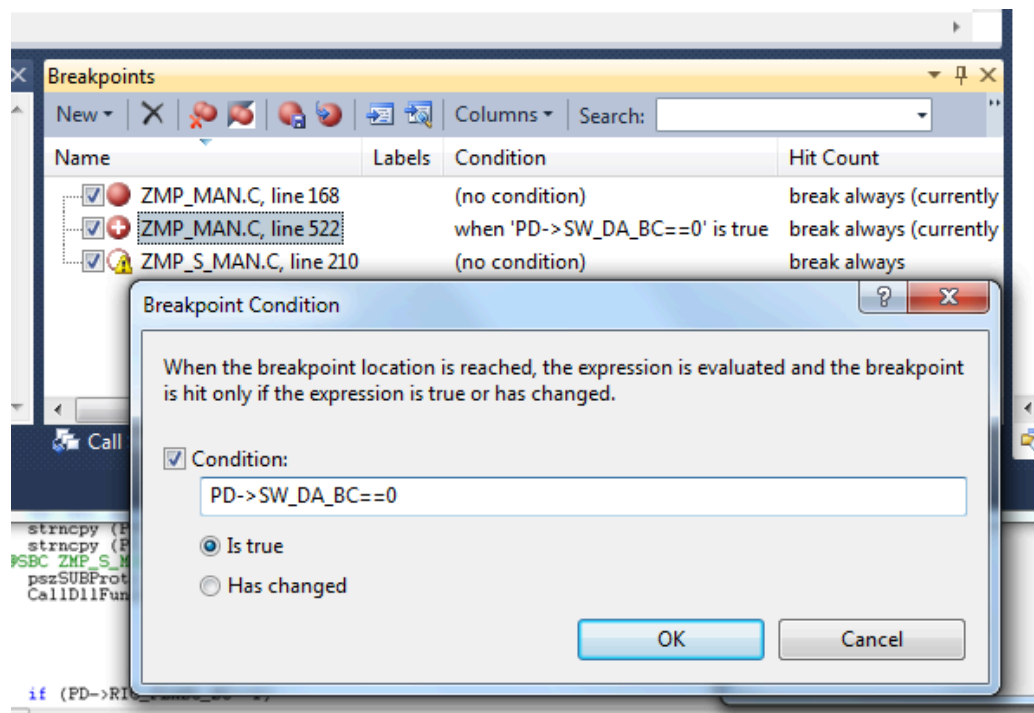
E' possibile gestire i breakpoint visualizzandoli nella parte in basso a destra di Visual Studio:



Inserire un breakpoint condizionale

Oltre che posizionali è possibile definire una condizione sui breakpoint per fare in modo di fermare l'esecuzione delle istruzioni del programma nel momento in cui si verifica la condizione ad essi associata.

Sul breakpoint è possibile cliccare con il tasto destro del mouse e selezionare *"conditions"*:



dove viene richiesto di indicare la condizione per l'attivazione del breakpoint.

Proseguendo nell'elaborazione, quando l'esecuzione passa sulla riga con il breakpoint e con la condizione verificata Visual Studio interrompe l'esecuzione del programma e si ferma al breakpoint.

Questo tipo di breakpoint è molto utile per eseguire il debug di istruzioni all'interno di cicli di elaborazione, per i quali, il problema da esaminare, si verifica all'ennesimo elemento:

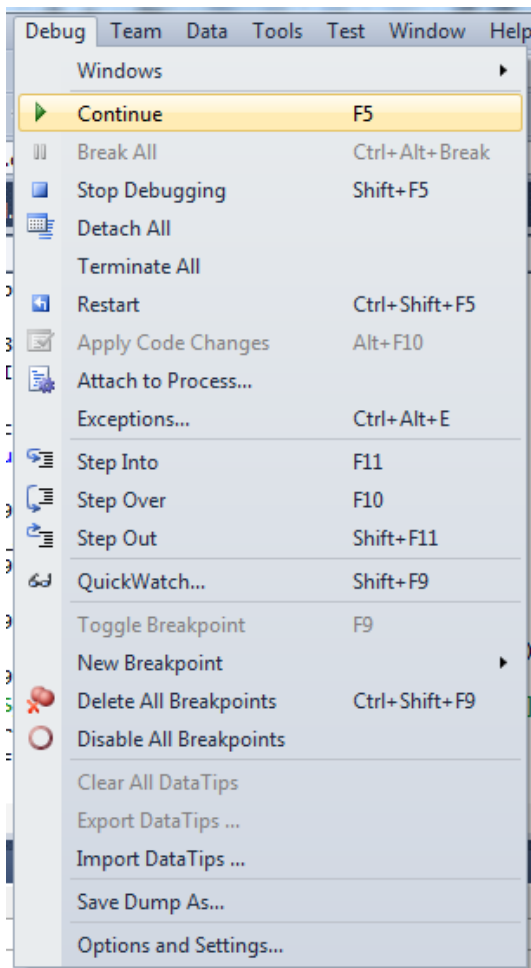
⚠ L'utilizzo di breakpoint condizionati comporta un rallentamento nell'esecuzione del sorgente.

Navigazione nel sorgente

Avviato il debug è possibile eseguire le istruzioni del sorgente in modalità "passo passo" premendo il tasto <F10>, in questo modo Visual Studio procede sull'istruzione successiva.

Se siamo posizionati sul richiamo di una subroutine e si vuole vedere le operazioni che vengono eseguite al suo interno (quindi si intende 'entrare' nella sub) premere il tasto <F11> e premere <Shift+F11> per uscire dalla sub interna; in questo modo le operazioni della sub saranno comunque eseguite tutte fino alla fine, solo che non saranno oggetto di debug.

Tutte queste operazioni sono ovviamente richiamabili dal menù "debug" di Visual Studio:



Visualizzare il contenuto delle variabili

Oltre alla navigazione nel sorgente la fase più importante del debug è quella di visualizzare il contenuto delle variabili.

Variabili di ambiente

Per quanto riguarda le variabili d'ambiente è sufficiente aggiungere il suffisso "_BC" al fondo del nome di ogni variabile d'ambiente, ma nessun prefisso. Per esempio la variabile `SYSDATEXT&` che contiene la data di sistema, diventerà `"SYSDATEXT_BC"`.

Anche in questo caso i simboli (\$, %, &, #, ?, !) che identificano il tipo della variabile, devono essere rimossi.

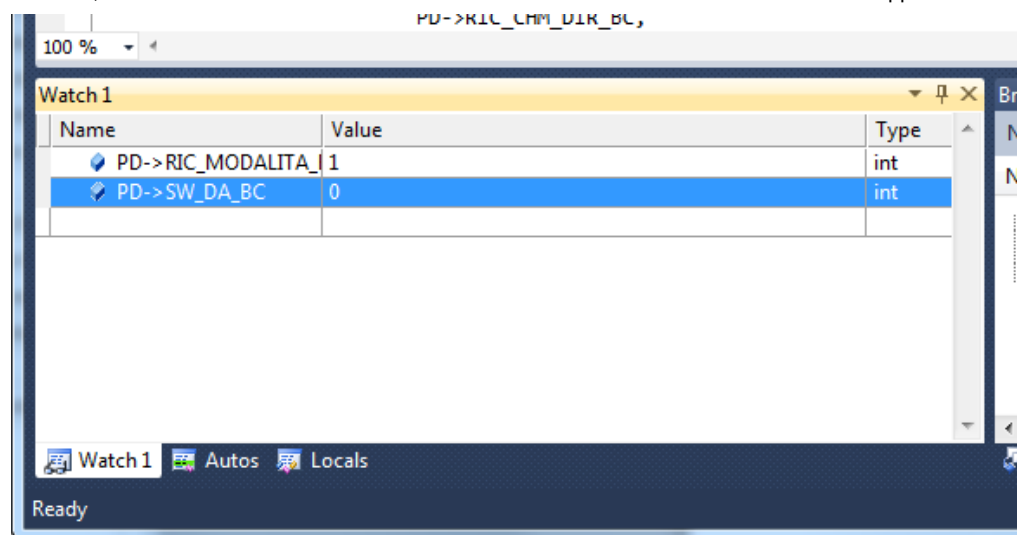
Variabili COMMON SHARED

Questo tipo di variabile è visibile in tutti gli altri programmi. Per poter visualizzare le variabili common shared è sufficiente scrivere il nome della variabile come è stato dichiarato sul sorgente BC.

Anche in questo caso i simboli (\$, %, &, #, ?, !) che identificano il tipo della variabile, devono essere rimossi.

Watch

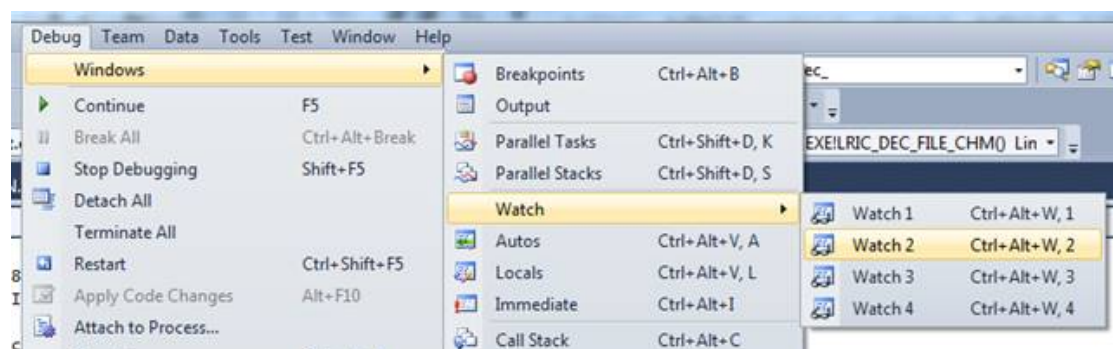
La finestra dei "Watch" consente di inserire delle variabili al suo interno per visualizzarne il contenuto durante l'esecuzione del programma.



Nella colonna "Name" è possibile inserire le variabili delle quali si desidera conoscere il valore e nella colonna "value" viene riportato il valore della variabile.

Per quanto riguarda le variabili numeriche d'ambiente e le variabili numeriche presenti in '@SBP, viene sempre visualizzato il valore dell'indirizzo di memoria nella quale è allocata la variabile, sarà sufficiente espandere la variabile utilizzando il tastino per visualizzarne il valore.

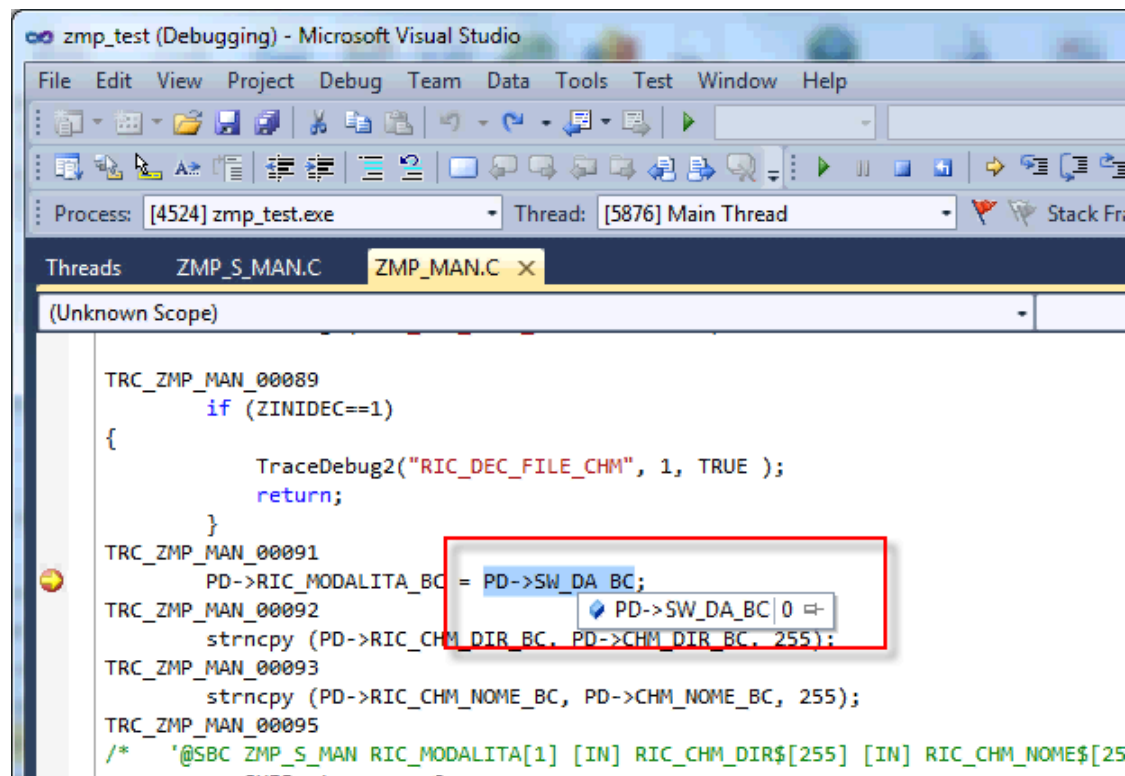
Inoltre è possibile utilizzare più watch windows per catalogare le variabili che si desidera visualizzare:



Se la finestra dei "Watch" non è visualizzata procedere nel seguente modo:

- selezionate il menù "Debug";
- selezionate il sottomenù "Windows->Watch";
- selezionate la voce "Watch1".

Un altro metodo per conoscere il valore delle variabili di un sorgente consiste nel selezionare la variabile e posizionarsi sopra con il mouse:



Il tooltip visualizzato riporta la variabile ed il suo valore in quel momento.

Variabili di testo e vettori

Per quanto riguarda le variabili di tipo testo o i vettori è possibile aggiungere alcuni caratteri alla variabile per semplificare la visualizzazione dei dati.

Name	Value
PD->STRING_VAR_BC	0x01516048 "Variabile String"
PD->STRING_VAR_BC,s	"Variabile String"
PA->VETT_STRING_BC[2],s	"Secondo valore"
PA->VETT_STRING_BC	0x01519645
[0]	0x01519645 ""
[1]	0x0151965a "Primo valore"
[2]	0x0151966f "Secondo valore"
[3]	0x01519684 "Terzo valore"
[4]	0x01519699 ""
[5]	0x015196ae ""
[6]	0x015196c3 ""
[7]	0x015196d0 ""

Da come si può notare dall'esempio, aggiungendo il suffisso ",s" al fondo della variabile "PD->STRING_VAR_BC" è possibile visualizzare il contenuto della variabile senza vedere il suo indirizzo di memoria.

Per quanto riguarda i vettori è possibile visualizzare l'intero contenuto espandendolo, oppure è possibile visualizzare un solo valore inserendo fra parentesi quadre "[numElem]" il numero dell'elemento che si desidera vedere dove il numero dell'elemento può essere una costante o il valore di una variabile del sorgente.

Variabili numeriche o presenti nell' '@SBP

Anche per le variabili numeriche o passate in '@SBP è possibile rimuovere dal "Watch" l'eventuale indirizzo di memoria.

Name	Value
PS->INT_VAR_BC	0x015108fc
	1
*PS->INT_VAR_BC	1

Per farlo è sufficiente inserire un "*" (asterisco) all'inizio della variabile.

Operazioni aritmetiche sui Watch

Oltre a visualizzare il valore delle singole variabili è possibile visualizzare il risultato di espressioni tra variabili:

Name	Value
PD->SORG_INT_VAR_BC	5
PD->SORG_DOUBLE_VAR_BC	1234.5600000000
PD->SORG_INT_VAR_BC+PD->SORG_DOUBLE_VAR_BC	1239.5600000000
(PD->SORG_INT_VAR_BC+PD->SORG_DOUBLE_VAR_BC)-0.56	1239.0000000000

In pratica è possibile scrivere all'interno del Watch la seguente espressione "PD->A_BC+PD->B_BC" è il valore che verrà visualizzato, sarà la somma delle due variabili.

In questo modo se vi fosse una condizione che testa il risultato di una espressione, sarà sufficiente inserirla nei Watch per controllare il valore restituito e attraverso l'utilizzo delle parentesi è possibile creare espressioni anche piuttosto complesse.

Variabili di ambiente BC

Ambiente - Variabili di utilizzo interno BC

Variabili per definire il contesto di esecuzione di un programma

Variabile	Descrizione
WS\$[2]	Stazione di lavoro
WSP\$[2]	Stazione di lavoro principale.
PHP\$[59]	Directory di collocazione dei programmi.
PHB\$[59]	Directory di collocazione dati base.
PHD\$[59]	Directory di collocazione dati.
PHE\$[59]	Altra directory di collocazione dati.
PHF\$[59]	Altra directory di collocazione dati.
PHG\$[59]	Altra directory di collocazione dati.
PHT\$[59]	Directory per file temporanei.
PHTEX\$[255]	Directory per file temporanei (estesa).
PHDLL\$[255]	Directory di collocazione della DLL (impostato a PHP\$ in lettura del profws).
PHDLLP\$[256]	Directory di collocazione della DLL personalizzata.
PHSTD\$[256]	Directory di collocazione tabelle standard.
PHDB\$[500]	Stringa di connessione al database definita di default in base alle variabili d'ambiente ZDBSORGENTE, ZDBSERVER\$, ...
GR\$[2]	Gruppo selezionato.
ZGRUPPI\$[255]	Elenco dei gruppi visualizzabili dall'operatore connesso (ZOPERATORE\$). I gruppi vengono restituiti sotto forma di stringa separati dal carattere ",". Se la stringa è vuota significa che l'operatore è abilitato alla visualizzazione di tutti i gruppi.
SYSDATEX&	Data di sistema nel formato GGMMAAAA.
SYSANNOEXT!	Anno di sistema nel formato AAAA.
ANNOEXT!	Anno IVA definito per il gruppo, nel formato AAAA.
SYSDAT[6]	Data di sistema nel formato GGMMAA.
ANNO[2]	Anno IVA definito per il gruppo, nel formato AA.
DITTA[5]	Codice ditta eventualmente selezionato per il gruppo.

MN\$[6]	Percorso del Menu in cui è collocato il programma (?).
PR\$[29]	Procedura in uso (es.: "DR4" per DR 740).
PRP\$[29]	Procedura in uso (es.: "PRF" per PROFIS, "SQ" per Spring, ecc.).
XRS1\$[30]	Ragione sociale 1.
XRS2\$[30]	Ragione sociale 2.
ZNOMEXE\$[8]	Nome del Main (senza alcuna estensione) attualmente in esecuzione. Se più lungo di 8 caratteri viene troncato.
ZNOMEEXE\$[256]	Nome esteso del Main (senza alcuna estensione) attualmente in esecuzione.
ZNOMMOD\$[30]	Nome del Modulo (senza alcuna estensione) attualmente in esecuzione.
ZNOMDLL\$[255]	Nome della DLL che contiene la sub attualmente in esecuzione.
ZNOMSBC\$[255]	Nome della sub di una DLL attualmente in esecuzione.
ZSRCSTACK\$[32000]	Stack dei sorgenti in esecuzione (/SISMENU4/SM_S_MENU_TESTATA/SM_S_MENU_RIGHE).
PROGC\$[29]	Nome del prossimo modulo che si intende eseguire in uscita da un modulo (la variabile è valorizzata dalla specifica CHAIN).
ZOPERATORE\$[20]	Operatore con cui si è fatto accesso alle procedure.
ZPROFILO\$[8]	Profilo utilizzato dall'operatore.
ZPROFW[1]	Se uguale ad 1 indica che è stato letto il file di configurazione PROFWS<stazione>.
ZOPERATOREDOM[9]	Dominio selezionato
ZCONTESTO\$[25]	Contesto selezionato

Variabili per gestione esportazione dati

Variabile	Descrizione
SNPLW%[1]	Esportazione dati: flag di attivazione: 0 - No, 1 - Sì.
ZEXPATH\$[60]	Esportazione dati: Percorso del file.
ZEXDEFPATH\$[60]	Esportazione dati: Percorso di default del file.
ZEXDEV\$[40]	Esportazione dati: Nome del file.
ZEXNAME\$[101]	Esportazione dati: Nome, completo di percorso, di esportazione.
ZEXFRMT%[1]	Esportazione dati: Formato del file di esportazione (1 - TXT, 2 - XLS, 3 - XML, 4 - FIX).
ZEXDSEP\$[1]	Esportazione dati: Carattere separatore per le cifre decimali
ZEXFSEP\$[1]	Esportazione dati: Carattere separatore tra campi.
ZEXDSIGN%[1]	Esportazione dati: Posizione segno: 0 - Avanti, 1 - Dietro.
ZEXAPPEND%[1]	Esportazione dati: Flag accodamento (0)/sovrascrittura (1) del file di esportazione.
ZEXID%[2]	Esportazione dati: Gruppo di dati da esportare.
ZEXNUM%[2]	Esportazione dati: Numero gruppi di esportazione.
ZEXGRUP\$[40](10)	Esportazione dati: Descrizione gruppi esportazione.
ZPRCHKNUMS%[1]	Esportazione dati: Flag abilitazione controllo lista variabili (0 - No, 1 - Sì).
ZEXSTRLIMIT\$[1]	Delimitatore di stringhe.
ZEXDATASEP\$[1]	Separatore di data.
ZEXFINEPAR\$[1]	Fine paragrafo.
ZEXNOTRIMTXT[1]	Esportazione dati: Flag per NON eliminare gli spazi a destra nelle stringhe, in fase di esportazione su file testo: 0 = No: le variabili di tipo stringa saranno trimmate a destra; 1 = Sì: le variabili di tipo stringa saranno esportate con gli eventuali spazi finali, per la dimensione indicata sul formato di esportazione; 2 (a partire da BC/X v. 17.4) = Sì, per le variabili alfanumeriche vengono inseriti a destra tanti spazi quanti ne sono necessari per far sì che l'occupazione della variabile sul file sia pari alla dimensione indicata sul formato della variabile. Per le variabili alfanumeriche non valorizzate (uguali a ""), gli spazi non vengono inseriti.

Variabili per gestione Euro

Variabile	Descrizione
ZEXEURO[1]	Indica se si stanno utilizzando importi in Lire (0) o in Euro (1).
ZDECEURO[1]	Numero di decimali gestiti per gli importi espressi in Euro.

ZEXVALUTA\$[4]	Descrizione della valuta corrente. Può assumere i valori: "Lire" se ZEXEURO = 0, "Euro" se ZEXEURO = 1.
----------------	---

Variabili d'ambiente

Variabile	Descrizione
ZSYSTEM%[1]	Indica il sistema su cui si stanno eseguendo le procedure. Può assumere i seguenti valori: 1 - Dos, 2 - Unix, 3 - WGS, 4 - AS400, 5 - Windows.
ZSYSOSVER\$[100]	Indica la versione (descrittiva) del sistema operativo, completa di Service Pack e numero di Build. Può assumere i seguenti valori: Windows 95, Windows 98, Windows Me, Windows NT 3.51, Windows NT 4.0, Windows NT 4.0 Workstation, Windows NT 4.0 Server, Windows 2000, Windows 2000 Workstation, Windows 2000 Server, Windows XP, Windows XP Workstation, Windows XP Server, Windows Vista, Windows Server "Longhorn", Windows 7, Windows 8.
<u>ZSYSOSVERN</u> %[3]	Indica la versione del sistema operativo, in formato numerico, con la seguente codifica: 1 - Windows 3.1 2 - Windows 95 3 - Windows 98 4 - Windows ME 5 - Windows NT 3.52 6 - Windows NT 4 7 - Windows 2000 8 - Windows XP 9 - Windows Server 2003 10 - Windows Vista 11 - Windows Server 2008 12 - Windows Server 2008 R2 13 - Windows 7 14 - Windows 8 15 - Windows Server 2012 16 - Windows 8.1 17 - Windows Server 2012 R2 18 - Windows 10 19 - Windows Server 2016 20 - Windows Server 2019 21 - Windows Server 2022 22 - Windows 11
ZSYS64BITPROC%[1]	Indica se il sistema operativo ha un processore a 64 bit. Può assumere i valori: 0 = No; 1 = Sì.
ZSYSPCNAME\$[15]	Indica il nome del computer.
<u>ZSYSIPADDRESS</u> \$[254]	Indica l'indirizzo IP associato al computer.
<u>ZSYSTEMINALSERVER</u> [1]	(A partire da BC/X v.22.5) Indica se il programma è eseguito in un sistema Terminal Server. Può assumere i valori: 0 = No, 1 = Sì
<u>ZSYSWINDIR</u> \$[255]	(A partire da BC/X v.18.0) Indica la direttiva di Windows (es. C:\WINDOWS" se l'applicazione viene eseguita sul computer locale o in rete, "C:\Documents and Settings\<Utente> \WINDOWS" per le applicazioni eseguite in Terminal Server)
ZSYSDIR\$[256]	Indica la direttiva di sistema (system o system32).
ZSYSUSERNAME\$[256]	Indica il nome dell'utente.
ZSYSDIRSTART\$[100]	Indica la directory di collocazione dell'eseguibile avviato.
<u>ZSYSDIRTEMP</u> \$[255]	Indica la directory temporanea di sistema, definita tramite la variabile di ambiente TEMP (es.: C:\Documents and settings\utente\Impostazioni locali\Temp).
ZSYSPROCID[5]	Contiene il "process-id" del programma esecuzione.
ZSPID[5]	(A partire da BC/X v.25.3) Contiene il process-id del menu che ha avviato il programma

Variabili per gestione protezioni

Variabile	Descrizione
ZPKP%[2](40)	Informazioni sulle procedure abilitate (Punti abilitazione da 1 a 40).
ZPKQ%[2](40)	Informazioni sulle procedure abilitate (Punti abilitazione da 41 a 80).

ZPKR%[2](200)	Informazioni sulle procedure abilitate (Punti abilitazione da 101 a 300).
ZDEMO%[1]	Indica se il programma è eseguito in versione DEMO.
IDSPROC\$[1](80)	Tabella delle procedure abilitate dal disco IDS.
IDSWIN32%[1](80)	Tabella delle procedure abilitate dal disco IDS per l'ambiente Windows.
IDSOK%[1]	Flag di controllo IDS terminato con esito positivo (1) o negativo (0).
IDSPRODOTTI%[1](80)	
IDS_QUADRO%[1]	Tipologia di IDS per Quadro.
IDS_SPRING%[1]	Tipologia di IDS per Spring.
ZSC%[1]	Posizione del bit di abilitazione della procedura SC.
ZDR4%[1]	Posizione del bit di abilitazione della procedura DR4.
ZDR5%[1]	Posizione del bit di abilitazione della procedura DR5.
ZDR6%[1]	Posizione del bit di abilitazione della procedura DR6.
ZGR%[1]	Posizione del bit di abilitazione della procedura GR.
ZBIL%[1]	Posizione del bit di abilitazione della procedura BIL.
ZCSP%[1]	Posizione del bit di abilitazione della procedura CSP.
ZGSP%[1]	Posizione del bit di abilitazione della procedura GSP.
ZGSC%[1]	Posizione del bit di abilitazione della procedura GSC.
ZIV1%[1]	Posizione del bit di abilitazione della procedura IV1.
ZGP%[1]	Posizione del bit di abilitazione della procedura GP.
ZDR3%[1]	Posizione del bit di abilitazione della procedura DR3.
ZCOB%[1]	Posizione del bit di abilitazione della procedura COB.
ZCWE%[1]	Posizione del bit di abilitazione della procedura CWE.
ZACB%[1]	Posizione del bit di abilitazione della procedura ACB.
ZAWE%[1]	Posizione del bit di abilitazione della procedura AWE.
ZVEB%[1]	Posizione del bit di abilitazione della procedura VEB.
ZVWE%[1]	Posizione del bit di abilitazione della procedura VWE.
ZMAB%[1]	Posizione del bit di abilitazione della procedura MAB.
ZMWE%[1]	Posizione del bit di abilitazione della procedura MWE.
ZCOE%[1]	Posizione del bit di abilitazione della procedura COE.
ZACE%[1]	Posizione del bit di abilitazione della procedura ACE.
ZVEE%[1]	Posizione del bit di abilitazione della procedura VEE.
ZMAE%[1]	Posizione del bit di abilitazione della procedura MAE.
ZMRP%[1]	Posizione del bit di abilitazione della procedura MRP.
ZMRW%[1]	Posizione del bit di abilitazione della procedura MRW.
ZCI%[1]	Posizione del bit di abilitazione della procedura CI.
ZCIW%[1]	Posizione del bit di abilitazione della procedura CIW.
ZCL%[1]	Posizione del bit di abilitazione della procedura CL.
ZCLW%[1]	Posizione del bit di abilitazione della procedura CLW.
ZQIS%[1]	Posizione del bit di abilitazione della procedura QIS.
ZQSH%[1]	Posizione del bit di abilitazione della procedura QSH.
ZGSM%[1]	Posizione del bit di abilitazione della procedura GSM.
ZJT%[1]	Posizione del bit di abilitazione della procedura JT%.
ZGSA%[1]	Posizione del bit di abilitazione della procedura GSA.
ZQCO%[1]	Posizione del bit di abilitazione della procedura QCO.
ZQVE%[1]	Posizione del bit di abilitazione della procedura QVE.
ZQAC%[1]	Posizione del bit di abilitazione della procedura QAC.
ZQMA%[1]	Posizione del bit di abilitazione della procedura QMA.
ZQIS%[1]	Posizione del bit di abilitazione della procedura QIS.
ZQIP%[1]	Posizione del bit di abilitazione della procedura QIP.
ZQIC%[1]	Posizione del bit di abilitazione della procedura QIC.
ZQCL%[1]	Posizione del bit di abilitazione della procedura QCL.
ZQPR%[1]	Posizione del bit di abilitazione della procedura QPR.
ZBCO%[1]	Posizione del bit di abilitazione della procedura BCO.
ZBVE%[1]	Posizione del bit di abilitazione della procedura BVE.
ZBMA%[1]	Posizione del bit di abilitazione della procedura BMA.

Variabili di uso generale

Variabile	Descrizione
ZNUMDEC%[1](10)	Tabella numero di decimali per categoria.
ERRSOP%[2]	Indica se una lettura con @SOP è andata a buon fine o no.
ZERRCOPY%[2]	Indica un eventuale errore verificatosi durante una COPY.
ZBCERR%[2]	Codice di errore, restituito dall'ambiente insieme a ZBCMSG\$, in seguito al verificarsi di una condizione di errore controllata dal programma.
ZBCMSG\$[257]	Messaggio di errore, restituito dall'ambiente insieme a ZBCERR%, in seguito al verificarsi di una condizione di errore controllata dal programma.
ZBCMSGEX\$[511]	Messaggio di errore esteso, restituito dall'ambiente insieme a ZBCERR%, in seguito al verificarsi di una condizione di errore controllata dal programma.
FMES\$[80]	Contiene un eventuale messaggio di errore per codice fiscale sbagliato, restituito da @TSCF o @TSCFE.
ZNDIR%[2]	Numero di files trovati dalla specifica @DIR.
ZNPRINTERS%[2]	Numero di stampanti restituite dalla specifica @GETPRINTERS.
ZERRDIR%[2]	Errore verificatosi durante l'esecuzione della specifica @DIR.
ERRSOPOUT%[2]	Flag errore verificatosi nel salvataggio/ripristino del dispositivo di emissione (@SAVEOUTPARMS, @RESTOREOUTPARMS).
ZDIR_FNOME\$[64]	Specifica @GETDIRFILEINFO - Nome del file.
ZDIR_FDIM[12]	Specifica @GETDIRFILEINFO - Dimensione in Byte.
ZDIR_FDATA&	Specifica @GETDIRFILEINFO - Data ultima modifica.
ZDIR_FORA[6]	Specifica @GETDIRFILEINFO - Ora ultima modifica.
ZDIR_FTPO[1]	Specifica @GETDIRFILEINFO - Tipo file (0 - File, 1 - Direttiva).
ZSILENTE[1]	Indica se il programma è eseguito in modalità silente 0=No, 1=Si
ZOUTERR\$[255]	Direttiva di collocazione dei files di errore
ZVERSIONESUB[3]	In caso di versionamento sub, indica quale è la versione in esecuzione
ZSTDFORCED[1]	Indica se è stato forzato il richiamo alla versione standard di una sub (sintassi STD::). Può assumere i valori: 0 = No, 1 = Si

Variabili per gestione del Merge

Variabile	Descrizione
ZMRGWPDA\$[100]	Nome file riferimenti ai dati.
ZMRGSRCS\$[100]	Nome file RTF sorgente.
ZMRGTGT\$[100]	Nome file RTF destinazione.
ZMRGCHAN%[2]	Canale di apertura file WPDA.
ZMRGTQUAD%[2]	Flag scrittura file di quadratura.
ZMRGTAPPEND%[2]	Accodamento variabile al file RTX.
ZMERGE%[2]	Tipo di merge: 0 - Merge da RTB; 1 - Merge da RTF.
ZMRGERR%[2]	Errore durante il merge.

Variabili per trasferimento file UNIX/DOS

Variabile	Descrizione
ZPRTRASFDOS%[1]	Stampa: Flag trasferimento a DOS.
ZPRDELUNIXFILE%[1]	Stampa: Flag cancellazione del file in UNIX.
ZPRDOTRASF%[1]	
ZPRDIRDOS\$[60]	Stampa: Direttiva DOS di trasferimento.
ZEXDELUNIXFILE%[1]	Esportazione: Flag cancellazione del file in UNIX.
ZEXTRASFDOS%[1]	Esportazione: Flag trasferimento a DOS.
ZEXDOTRASF%[1]	
ZEXTRASFMODE%[1]	Esportazione: Definisce la modalità di trasferimento con la specifica @SND. Può assumere i valori: 0 - Trasferimento ASCII, 1 - Trasferimento Binario.
ZEXDIRDOS\$[60]	Direttiva DOS di trasferimento.

Variabili gestione versioni

Variabile	Descrizione
ZPROFIS_INSTAL[1]	Tipo di installazione PROFIS: 1 - Studio, blank o 2 - Azienda.
ZPROFIS_GENERAZ[1]	Tipo di generazione PROFIS: 0 - Non rilevante; 2 - PROFIS 2000; 3 - Profis 2006.
ZVER_ANAGRAFICA[1]	Versione anagrafica: blank o 1 - Ageba, 2 - Anage.
ZVER_CONFIG[1]	Versione tabelle configurazione: blank o 1 - Aws, 2 - Sistaz.
ZVER_MENU[1]	Versione tabelle menù: blank o 1 - MENU<pr>.BTR, 2 - BCMENUT/BCMENUR; 3 - .SXMNU.
ZVER_FLIBERI[1]	Versione tabelle Formati liberi: blank o 1 - FRM, 2 - .SXF.

Variabili strutture dinamiche

Variabile	Descrizione
ZDSELEM_TOT[2]	Numero elementi totale della struttura.
ZDSELEM[2]	Elemento corrente della struttura.
ZDYN_VALELEMENTO[1]	Risultato della validazione dell'elemento della struttura letto da una GetDynStruct.
ZDSNODECOD[1]	Disabilitazione meccanismo di decodifica della FindDbData su struttura.

Variabili per applicazioni Web e .NET

Variabile	Descrizione
ZDOTNET[1]	Indica l'esecuzione di un programma .NET (0 - No, 1 - Si).
ZWEB[1]	Indica l'esecuzione di un programma in contesto WEB (0 - No, 1 - Si).

Variabili per contesto SIR

Variabile	Descrizione
ZSIR[1]	Indica la tipologia di esecuzione in contesto SIR. I valori ammessi sono i seguenti: 0=Contesto NON SIR; 1=Contesto SIR

Variabili per la conservazione legale

Variabile	Descrizione
<u>ZCONSLEGALE</u> [2]	Permette di definire quale versione delle dll d'ambiente, relative alla conservazione legale, caricare. 0 - Carica la libreria sisicCrypto.dll versione 4.1.0.3 (PHB) 1 - Carica la libreria sisicCrypto.dll versione 3.1.4.0 (PHB/icCrypto/v3140)

ZGRUPPI\$ - Elenco gruppi a cui può accedere l'operatore

La gestione operatori consente di definire una limitazione sui gruppi ai quali l'operatore può accedere.

In particolare, un operatore può:

- Accedere a tutti i gruppi
- Accedere ad gruppo specifico
- Accedere ad un elenco di gruppi

La limitazione sui gruppi è stata finora applicata solo dal menu delle procedure.

Esistono però contesti applicativi (come ad esempio la Gestione documentale) nei quali è necessario applicare delle selezioni per Gruppo in fase di accesso alle tabelle, selezionando solo i gruppi a cui l'operatore ha accesso. Per permettere questo è essenziale definire una query di tipo MULTIGRUPPO impostando la selezione per il campo DBGruppo a seconda delle impostazioni definite per operatore.

La variabile d'ambiente ZGRUPPI\$ è disponibile al programma proprio per eseguire questo tipo di interrogazioni. Viene valorizzata dall'ambiente con la seguente modalità:

Gruppi a cui può accedere l'operatore	Esempio di Valore ZGRUPPI\$	Descrizione e Note
Tutti	""	L'operatore può accedere a tutti i gruppi
Gruppo specifico	"GR"	L'operatore può accedere esclusivamente al gruppo GR
Elenco gruppi	"G1,G2,G3" "<Gruppo1>,<Gruppo2>,...,<GruppoN>"	I gruppi a cui può accedere l'operatore sono separati dal carattere ,

La variabile ZGRUPPI\$ può utilizzata all'interno di una query per impostare un condizionamento tramite la specifica @DEFQUERYWHERE:

Esempio applicazione della variabile ZGRUPPI\$ all'interno di una DEFQUERYWHERE
<pre> ' IF ZGRUPPI\$<>"" THEN '@DEFQUERYWHERE IDQUERY _ ESPR[INLISTSTR (EMMAG.*GR*,ZGRUPPI\$)>0] ENDIF '</pre>

Nell'esempio, se la variabile ZGRUPPI\$ è <> "", quindi se esistono dei condizionamenti gruppi-operatori, viene applicato il filtro INLISTSTR sul campo *GR*.

- ▼ Il campo *GR* è utilizzabile esclusivamente se la query è di tipo MULTIGRUPPO. Quando si utilizza la variabile ZGRUPPI\$ è indispensabile utilizzare il criterio IN (INLISTSTR): non conoscendo il contenuto della variabile (se formata da uno o più elementi), si utilizza un criterio capace di applicare il filtro in entrambi i casi.
- ▼ L'utilizzo della variabile ZGRUPPI\$ comporta l'esecuzione di una funzione e
- ▼ conseguentemente di una operazione su DB. Se in un programma occorre riferirsi a tale variabile più volte, occorre copiare il contenuto in una variabile di comodo e utilizzare questa, in modo da ottimizzare le operazioni.

Archivi/DB - Variabili di utilizzo interno BC

Variabili configurazione Database

Variabile	Descrizione
ZDBSORGENTE[1]	Tipo di accesso al gestore DBMS: blank o 0 - Btrieve/Pervasive, 1 - MSSQLserver, 2 - Oracle, 3 - Pervasive SQL.
ZDBSERVER\$[40]	Nome Server.

ZDBTBLPROD\$[40]	Nome DB tabella Prodotto.
ZDBTBLIS\$[40]	Nome DB tabella Sistema (non più supportata dalla versione di ambiente 14 - Luglio 2006).
ZDBORIGINE[1]	Tipo DB utilizzato: blank o 0 - Prodotto, 1 - Sistema.
ZDBAUT[1]	Windows Autenticazione (0 - No; 1 - Si).
ZDBUSER\$[40]	Nome utente per il collegamento al database Server; se "" si utilizza l'utente "sa" per SQLServer.
ZDBPWD\$[40]	Password dell'utente per il collegamento al Server.
ZDBAPPROLE[1]	Application Role in uso (0 - No; 1 - Si).

Variabili per gestione tabelle

Variabile	Descrizione
ZDB_VALRECORD%[2]	GetDBData: Indica il risultato della validazione del record letto (eseguita dal programma). Può assumere i valori: 0 - Record valido, 1 - Leggi record successivo, 2 - Termina lettura.
ZDB_OPER_ ...	GetDBData: Indica l'operazione di lettura che deve essere eseguita. Può assumere i seguenti valori: 0 - Leggi record successivo, 1 - Leggi record precedente, 2 - Leggi il primo record, 3 - Leggi l'ultimo record.
ZDB_INDICE_ ...	GetDBData: definisce l'indice da usare per la lettura. Deve essere utilizzato nel caso in cui siano indicate più chiavi sulla specifica, per stabilire con quale ordinamento eseguire la lettura. Per la lettura con il primo indice definito la variabile dovrà avere valore 1, per il secondo indice 2, e così via. GetQueryData: definisce l'indice di ordinamento da usare per la lettura. Deve essere utilizzato nel caso in cui siano indicate più chiavi sulla specifica, per stabilire con quale ordinamento eseguire la lettura. Per la lettura con il primo indice definito la variabile dovrà avere valore 1, per il secondo indice 2, e così via.
ZDB_POS_...	GetDBData: definisce se deve essere o meno eseguito il posizionamento in base alla chiave definita. Può assumere i seguenti valori: 0 - Non deve essere eseguito il posizionamento (essendo già stato eseguito da una operazione precedente), 1 - Deve essere eseguito il posizionamento con la chiave indicata sulla specifica, 2 - Deve essere eseguito il posizionamento con position-block.
ZDB_ORDLET_...	GetDBData: definisce l'ordine di lettura dei record (in base all'indice). Può assumere i seguenti valori: 0 - Lettura dei record mediante l'indice in ordine crescente, 1 - Lettura dei record mediante l'indice in ordine decrescente.
ZDB_GRUPPOCORR_ ...	GetQueryData: contiene in ingresso delle call back il progressivo di identificazione del gruppo di record.
ZDB_POSBLOCK_<dfx>[9]	GetDBData: contiene il position block di un record di quel determinato DFX.
ZGESTVEGT%[2]	Indica se deve essere gestito il controllo sulla versione di un record In fase di lettura.
ZGESTVEPT%[2]	Indica se deve essere gestito il controllo sulla versione di un record In fase di scrittura.
ZRCLN%[2]	Nel caso di tabella con lunghezza record definita in esecuzione contiene la lunghezza dei record della tabella.
BUZRK\$[]	Nel caso di tabella con lunghezza record definita in esecuzione contiene il record letto dalla tabella.
ZNOFIL%[1]	Apertura: Flag - File non trovato.
ZNODIR%[1]	Apertura: Flag - Nome direttiva di collocazione non valido.
ZNOPERVASIVE%[1]	Apertura: Flag - Non è stato installato Pervasive o ha restituito errori di configurazione.
ZDSKFUL%[1]	Apertura: Flag - Disco pieno.
ZMAXFIL%[1]	Apertura: Flag - Raggiunto numero massimo file aperti da un programma.
ZNOPERM%[1]	Apertura: Flag - Mancanza dei permessi per l'apertura del file. Profilazione: Flag - Mancanza dei permessi per accedere al record.
ZNOKEY%[1]	Lettura: Flag - Chiave inesistente.
ZENDFIL%[1]	Lettura (sequenziale): Flag - Raggiunta fine/inizio file.
ZDUPKEY%[1]	Scrittura: Flag - Chiave duplicata.
ZMODKEY%[1]	Scrittura: Flag - Chiave modificata.

ZMAXREC%[1]	Lettura (diretta): Flag - Record inesistente (oltre la fine del file).
ZEXOPEN%[1]	Apertura: Flag - File aperto in modo esclusivo da un'altra stazione.

Videate - Variabili di utilizzo interno BC

Variabili per gestione videate

Variabile	Descrizione
ABC%[2]	Codice di un tasto funzione premuto su una videata.
ABCPRB%[2]	Codice di un tasto funzione premuto su una @PRB.
CANVID%[2]	Definisce la modalità di "pulizia" di una videata: • 0: viene eseguita la pulizia sia del video che del reticolo di input; • 1: non viene eseguita la pulizia del video ma viene comunque eseguito l'azzeramento del reticolo di input; • 2: viene eseguita la pulizia del video ma non viene azzerato il reticolo di input.
ZMIGL%[1]	Indica se abilitare il meccanismo di inserimento automatico dei puntini delle migliaia nei campi di output di una videata. Impostato a 1 di default.
NOIVA%[2]	Se messo a 0 dopo il ripristino di una videata con @IVR W forza la cancellazione e il disegno della videata ripristinata.
SLN%[2]	Definisce a quale riga eseguire la visualizzazione dei dati riportati su una specifica @PRL o @IVL R[].
ZVIFCO%[1]	Condizionamento di videate: consente di determinare se i dati in input di una videata devono essere usati come tali (valore 0), oppure se la videata deve essere proposta con tutti i dati in output (valore 1).
ZINIDEC%[2]	Decodifiche: indica se la decodifica in corso è relativa alle operazioni di creazione della videata oppure se è relativa alle operazioni interattive: • 0 - decodifica su gestione videata; • 1 - decodifica di ingresso videata. La variabile deve essere utilizzata esclusivamente all'interno della @SR di decodifica.
ZVIDFORMINIDEC%[1]	Decodifiche relative ad una videata con @DEFVIDFORM. Consente di controllare l'esecuzione delle decodifiche di ingresso videata eseguite dalla specifica @DEFVIDFORM: • 0 - (default) esegue le decodifiche di ingresso videata (decodifiche con valorizzato ZINIDEC=1); • 1 - disabilita l'esecuzione delle decodifiche in ingresso videata. Dopo l'esecuzione della @DEFVIDFORM la variabile viene azzerata e riportata al valore di default (0).
ZERRDEC%[1]	Se = 1 indica che si è verificato un errore su un'azione di call back per cui deve essere annullato lo spostamento del fuoco sulla videata (uscita riga, spostamento con tab, ecc.).
ZRISOLUZIONE%[1]	Indica la risoluzione del video: 0 - Bassa, 1 - Alta.
ZPALMARE%[1]	Indica se la procedura è abilitata per la visualizzazione su palmare (0 - No, 1 - Sì) (legge CFG<PRP>.ini "Configurazione/Visualizzazione palmare").
ZVIDRESTORE%[1]	Indica se il meccanismo di salvataggio/ripristino delle posizioni delle dimensioni delle videate è attivo: 0 - Non attivo, 1 - Attivo.
ZHELPCONTEXT\$[10]	Indica il contesto da applicare per la ricerca della scheda di help relativa alla videata che segue.
ZVIDTITLESUFFIX\$[255]	Definisce il suffisso del titolo delle finestre.

Variabili per gestione ricerche a barre di scorrimento

Variabile	Descrizione
RCINP%[2]	Se settato a 1 all'ingresso di una videata, fa sì che non si esegua una lettura su tabella.
TRIC%[1]	Indica con quale delle chiavi indicate su @RES eseguire la lettura di una tabella. Viene presa in considerazione con TRIC% = 0 la prima chiave, con TRIC% a 1 la seconda e così via.
RRIS%[1]	Indica quale intestazione usare tra quelle definite in una videata di ricerca.
RRFS%[1]	Indica quale formato usare tra quelli definiti in una videata di ricerca.

ZNSEL%[3]	All'uscita da una ricerca con selezione multipla contiene il numero di elementi selezionati nella ricerca. La variabile può essere utilizzata anche su una lista a selezione singola: in questo caso se è presente una riga selezionata avrà valore 1, viceversa (caso di lista vuota) avrà valore 0.
ZRCVIS%[1]	Se settato a 1 prima di una videata, fa sì che nella visualizzazione della videata vengano preselezionati gli elementi eventualmente selezionati.
ZNMAXSEL%	Numero massimo di elementi selezionabili attraverso la specifica @RES con selezione multipla.
ISDOWN%	Se settato a 1 indica che si sta leggendo il file in ordine inverso.

Variabili per gestione controlli Windows estesi

Variabile	Descrizione
ZVIDNAME\$[63]	Nome della videata (id interno), disponibile dopo la @IVX.
ZVIDCTRL\$[63]	Nome del controllo su cui era definito il fuoco prima di uscire dalla videata.
ZVIDMOD[1]	Videata: flag videata modificata.
ZTNODODES\$[20]	Albero: descrizione del nodo corrente.
ZTNODO[2]	Albero: identificatore del nodo corrente dell'albero.
ZTNODOPADRE[2]	Albero: identificatore del padre del nodo corrente dell'albero.
ZTNODI_TOT[2]	Albero: numero totale dei figli del nodo.
ZGRIGA[2]	Griglia: numero della riga corrente.
ZGCOL[2]	Griglia: numero della colonna corrente.
ZGRIGHE_TOT[2]	Griglia: numero totale delle righe.
ZGCOL_TOT[2]	Griglia: numero totale delle colonne.
ZGCOL_ORDNUM[2]	Griglia: numero della colonna per cui forzare l'ordinamento.
ZGCOL_ORDDIR[1]	Griglia: direzione dell'ordinamento (0 - Ordinamento Crescente; 1 - Ordinamento Decrescente: default 0).
ZGCOL_ORDCURNUM[2]	Griglia: numero della colonna corrente di ordinamento.
ZGCOL_ORDCURDIR[1]	Griglia: direzione corrente dell'ordinamento (0 - Ordinamento Crescente; 1 - Ordinamento Decrescente: default 0).
ZGRIGAMOD[1]	Griglia: flag riga griglia modificata.
<u>ZBUTTONGRIDCLICK[1]</u>	Griglia: indica se l'azione che si sta eseguendo si trova all'interno di un contesto esecutivo richiamato dal click di un pulsante della griglia standard. (Disponibile da BC/X 17.1.1)
<u>ZROWGRIDDBLCLICK[1]</u>	Griglia avanzata: indica che l'uscita dalla videata è derivata da un doppio click su una riga della griglia. La variabile viene valorizzata, esclusivamente, dalla @RUNVID, e può essere utilizzata solamente all'uscita di una videata. (0-Uscita dalla videata generata dalla pressione di un tasto; 1-Uscita dalla videata generata dal doppio click sulla griglia) Esempio d'utilizzo: <u>ZROWGRIDDBLCLICK - Rubrica - Selezione contatto</u> (Disponibile da BC/X 18.0)
<u>ZSELALLGRIDCLICK[1]</u>	Griglia avanzata: indica che l'azione <u>SELRIGA</u> , richiamata in fase di selezione di una riga della griglia, è stata scaturita dal click del pulsante <Seleziona tutti>. La variabile è utilizzabile esclusivamente all'interno dell'azione <u>SELRIGA</u> . (0-Azione richiamata dalla singola selezione di un elemento; 1-Azione richiamata dalla selezione di tutti gli elementi attraverso il pulsante <Seleziona tutti>). Esempio d'utilizzo: <u>ZSELALLGRIDCLICK - Esecuzione azione SELRIGA</u> (Disponibile da BC/X 18.8)
ZSELECTED[1]	Griglia avanzata: indica lo stato di selezione della riga corrente (0 - Non selezionata; 1 - Selezionata) (Corrisponde al campo ZSEL della struttura dinamica associata alla griglia). (Disponibile da BC/X v.22.0)
ZVIDCOMP[2]	Videate composte: indice della videata da visualizzare.
ZCURVIDCOMP[2]	Videate composte: indice della videata composta in uscita.

ZVIDCOMPVIS[2]	Videate composte: flag indicante la modalità di gestione di una videata composta. La variabile ZVIDCOMPVIS assume diversi significati, ovvero: • 0: esecuzione di una videata elementare di una videata composta o di un multipagina; • 1: creazione di una videata elementare di una videata composta (principale o dettaglio); • 2: sincronizzazione della videata secondaria di una videata composta; • 3: controllo di una videata elementare di un multipagina (parametro TASTICONTROLLO[]).
ZVIDCOMPEXIT[2]	Videate composte: forza l'uscita da una videata composta su pressione tasto.
ZVIDCOMPIVR[2]	Videate composte: consente la rimozione di una videata composta mediante @IVR W Può assumere i seguenti valori: • 0: l'esecuzione della @IVR W di una videata composta è ignorato (valore predefinito); • 1: l'esecuzione della @IVR W rimuove dallo schermo la videata composta. Il valore della variabile è azzerato dopo l'esecuzione della @IVR W.
ZSTATOBOTTONE[1]	Attributo di definizione dello stato del bottone. Può assumere i seguenti valori: • 0 - Riga espansa • 1 - Riga da espandere • 2 - Riga senza dettaglio (da non espandere).
ZVIDMPAGCLICK[1]	Videate multipagina: indica, in uscita da una videata, dopo quindi @IVX o @RUNVID, l'esecuzione dell'azione associata al tasto @IVT 10 come conseguenza di un cambio pagina, sia questo generato con il mouse (click del mouse sul selettore di pagina), sia con la tastiera (mediante CTRL+TAB o CTRL+SHIFT+TAB).
ZBCLASTCHR[1]	Ultimo carattere premuto su una griglia (evento CARATTERE di una griglia).
ZWBOOKMARK[63]	Nome bookmark corrente (editor testi WORD)

Variabili per gestione della navigazione sulla videata

Variabile	Descrizione
<u>ZNAVBUTTON</u> [1]	Navigazione sulle videate: indica quale pulsante della toolbar di navigazione è stato premuto: 1 = Riga successiva 2 = Riga precedente 3 = Prima riga dell'elenco 4 = Ultima riga dell'elenco
<u>ZNAVSTATE</u> [1]	Navigazione sulle videate: indica lo stato di abilitazione dei pulsanti della toolbar di navigazione: 1 = Riga iniziale (Disabilita i pulsanti Precedente e Prima riga) 2 = Record 3 = Riga Finale (Disabilita i pulsanti Successivo e ultima riga) 4 = Unica riga (Disabilita tutti i pulsanti)
<u>ZNAVSAVE</u> [1]	Navigazione sulle videate: indica, in uscita dalla videata, che l'uscita è stata forzata dalla pressione di uno dei tasti di navigazione della toolbar. Di solito è utilizzata per rieseguire la videata dando la possibilità di richiamare la callBack di navigazione.

ZROWGRIDDBLCLICK - Rubrica - Selezione contatto

L'esempio riporta l'utilizzo, all'interno di una videata, della variabile ZROWGRIDDBLCLICK.

La videata è quella della gestione della rubrica, selezionando il pulsante "Per", il programma prende gli elementi selezionati dalla griglie e li inserisce nell'albero.

Lo scopo è far fare la medesima cosa, anche al doppio click su una riga, inibendo l'uscita videata.

BC

```

'=====

'@SR VID_RUBRICA1

'=====

'@BS REVID

'@DEFVID DIM[18,LARG_VID#] TIT[ETICHETTA_TITOLO$]
POSBOTTOMI[BASSODESTRA]

'@DEFLABEL VAR["Rubrica"] POS[1.5,2] DIM[1,11] FMT[TESTO]
'@DEFEDIT SELRUBRICA$ RET[1,1] POS[1.5,12] DIM[1,36] FMT[TESTO[100]] _
    DISABILITATO[NRUBRICA=1] DECOD[GOSUB DECOD_SEL1] _
    COMBO ALTCOMBO[10] VALS[ELENCO_RUBRICHE$()] _
    OPZIONI[ELENCO_RUBRICHE$()]

'@DEFLINE POSINIZIO[2,2] POSFINE[2,LARG_LINE#]
'@DEFGRID GRID_ADDR AVANZATA RET[1,1] POS[3.5,2] DIM[15.5,LARG_GRID#] _
    QUERY[ZAD_QUERY] INSCOLDINAMICHE[GOSUB INS_COL_GRID_ADDR]

    SELRIGA[GOSUB SELEZIONA_RIGA] _
    MULTISEL[1] RESETCOLDINAMICHE[1] RIDIMENSIONABILE[TUTTO]

'@DEFBUTTON VAR[ETICHETTA_PER$] POS[4,START_BUTTON#] DIM[1,9] _
    AZIONE[GOSUB ADD_TO] NASCOSTO[NASCOSTO_PER]
'@DEFBUTTON VAR[ETICHETTA_CC$] POS[5.5,START_BUTTON#] DIM[1,9] _
    AZIONE[GOSUB ADD_CC] NASCOSTO[NASCOSTO_CC]
'@DEFBUTTON VAR[ETICHETTA_CCN$] POS[7,START_BUTTON#] DIM[1,9] _
    AZIONE[GOSUB ADD_CCN] NASCOSTO[NASCOSTO_CCN]

'@DEFTREE DEST RET[2,2] POS[3.5,POS_TREE#] DIM[14.5,LARG_TREE#] _
    INSNODI[GOSUB IND_NODI_DEST] _
    NODOCORRENTE[NODOCORRENTE] _
    PADRECORRENTE[PADRECORRENTE]
V[DESTINATARIO$,INDIRIZZO$,IDENTIFICATIVO?]

'@DEFBUTTON VAR["Rimuovi"] POS[18,POS_BOTTON1#] DIM[1,13] _
    AZIONE[NODO=NODOCORRENTE:GOSUB REMOVE]
'@DEFBUTTON VAR["Rimuovi Tutti"] POS[18,POS_BOTTON2#] DIM[1,13] _
    AZIONE[GOSUB REMOVE_ALL]

```

```
'@DEFFUNCKEY TASTO[F1]
'@DEFFUNCKEY TASTO[F10]

'@RUNVID

IF ABC=10 AND ZROWGRIDDBLCLICK=1 THEN
  GOSUB ADD_TO
  CANVID=1
  GOTO REVID
ENDIF

IF ABC=10 THEN
  GOSUB COMPONI_INDIRIZZI
  EMAIL_TO? =TMPEMAIL_TO?
  EMAIL_CC? =TMPEMAIL_CC?
  EMAIL_CCN?=TMPEMAIL_CCN?
  EMAIL_IDTO? =TMPEMAIL_IDTO?
  EMAIL_IDCC? =TMPEMAIL_IDCC?
  EMAIL_IDCCN?=TMPEMAIL_IDCCN?
ENDIF

RETURN
```

Come si può notare alla pressione del tasto "Per" (VAR[ETICHETTA_PER\$]), viene eseguita l'azione ADD_TO.

Dopo la '@RUNVID, quindi all'uscita della videata, viene testato se **ABC=10** e **ZROWGRIDDBLCLICK=1**, questo perché il doppio click simula sempre l'F10 ma valorizza anche la variabile d'ambiente. Se è stato eseguito il doppio click (quindi il controllo è stato verificato), viene eseguita l'azione ADD_TO, e si ritorna alla videata senza aggiornarla.

Riferimenti

Variabili d'ambiente - ZROWGRIDDBLCLICK

'@DEFGRID - Definizione griglia (avanzata) - DOPPIOCLICK

ZSELALLGRIDCLICK - Esecuzione azione SELRIGA

L'esempio riporta l'utilizzo, all'interno di una videata, della variabile d'ambiente ZSELALLGRIDCLICK.

La griglia in questione elenca tutti gli articoli presenti all'interno della base dati, nel momento in cui si seleziona un elemento, questo viene automaticamente bloccato su DB, per evitare modifiche successive.

BC


```
'=====
'@SR DEFINIZIONE_QUERY
'=====
```

```
IDQUERY[5]=0
```

```
'@DEFQUERY IDQUERY
```

```
' Tabella EARTIC
```

```
'@DEFQUERYFROM IDQUERY TABELLA[EARTIC]
```

```
' Relazione tra EDFAM (EQDitteFamiglie) - Ditte: Famiglie
```

```
' e EARTIC (EQArtAnagrafiche) - Articoli: Anagrafiche
```

```
'@DEFQUERYJOIN IDQUERY TABELLA[EDFAM] TABELLAPADRE[EARTIC] _
REL[EDFAM.DIT = EARTIC.DIT AND EDFAM.COD = EARTIC.CFA]
```

```
' Relazione tra EPOLISA (EQListiniArticoliRig) - Listini: Articoli
```

```
' e EARTIC (EQArtAnagrafiche) - Articoli: Anagrafiche
```

```
'@DEFQUERYJOIN IDQUERY TABELLA[EPOLISA] TABELLAPADRE[EARTIC] _
REL[EPOLISA.DIT = EARTIC.DIT AND EPOLISA.ART = EARTIC.COD]
```

```
'@DEFQUERYCOLUMN IDQUERY _
```

```
ESPR[EPOLISA.PRL AS [PrezzoListino]] VAR[LIAPRL#] _
```

```
ESPR[EPOLISA.SC1 AS [Sconto1]] VAR[LIASC1] _
```

```
ESPR[EPOLISA.SC2 AS [Sconto2]] VAR[LIASC2] _
```

```
ESPR[EPOLISA.SC3 AS [Sconto3]] VAR[LIASC3] _
```

```
ESPR[EARTIC.COD AS [CodArticolo]] VAR[ARCOD$] _
```

```
ESPR[EARTIC.DES AS [DesArticolo]] VAR[ARDES$] _
```

```
ESPR[EDFAM.DES AS [DesFamiglia]] VAR[FADES$]
```

```
RETURN
```

```
'=====
'@SR VIDEATA_GRIGLIA
'=====
```

SELALLBUTTON[1]=0

'@DEFVID DIM[20,70]

'@DEFGRID IDGRIGLIA RET[1,1] POS[1,1] DIM[20.19,70] QUERY[IDQUERY] _
AVANZATA SELETTORERIGA MULTISEL _
SELRIGA[GOSUB SELRIGA]

'@INSCOLGRID IDGRIGLIA LARG[10] TIT["Codice"]
'@INSCOLGRID IDGRIGLIA LARG[20] TIT["Articolo"]
'@INSCOLGRID IDGRIGLIA LARG[20] TIT["Famiglia"]
'@INSCOLGRID IDGRIGLIA LARG[10] TIT["Prezzo"]
'@INSCOLGRID IDGRIGLIA LARG[10] TIT["Sconto1"]
'@INSCOLGRID IDGRIGLIA LARG[10] TIT["Sconto2"]
'@INSCOLGRID IDGRIGLIA LARG[10] TIT["Sconto3"]

'@DEFCELLGRID IDGRIGLIA COL[1] VAR[ARCOD\$] FMT[TESTO[20]]
'@DEFCELLGRID IDGRIGLIA COL[2] VAR[ARDES\$] FMT[TESTO[20]]
'@DEFCELLGRID IDGRIGLIA COL[3] VAR[FADES\$] FMT[TESTO[20]]
'@DEFCELLGRID IDGRIGLIA COL[4] VAR[LIAPRL#] FMT[NUMERICO[5,2]]
'@DEFCELLGRID IDGRIGLIA COL[5] VAR[LIASC1] FMT[NUMERICO[5,2]]
'@DEFCELLGRID IDGRIGLIA COL[6] VAR[LIASC2] FMT[NUMERICO[5,2]]
'@DEFCELLGRID IDGRIGLIA COL[7] VAR[LIASC3] FMT[NUMERICO[5,2]]

'@RUNVID

RETURN

Viene definita una query che interroga la base dati estraendo tutti gli articoli in anagrafica, visualizzando l'indirizzo, il prezzo e i vari sconti.

Attraverso la multi selezione la griglia deve bloccare o meno gli articoli selezionati.

BC

```

=====

'@SR SELRIGA

=====

IF ZSELALLGRIDCLICK=1 AND ZGRIGA=ZGRIGHE_TOT THEN
SELALLBUTTON=0:EXITSR

IF ZSELALLGRIDCLICK=1 AND SELALLBUTTON=1 THEN EXITSR

IF ZSELALLGRIDCLICK=0 THEN

'@UPDATEDBDATA TABELLA[EARTIC] _
    IND[0 \DIT=0\COD=""\] _
    SELEZIONI[ARCOD$=ARCOD$] _
    ASSEGNA[ARBLO=ZGRIGASEL] _
    ERRORE[#NOKEY]

ELSE

SELALLBUTTON=1

'@UPDATEDBDATA TABELLA[EARTIC] _
    ASSEGNA[ARBLO=ZGRIGASEL] _
    IDQUERY[IDQUERY] _
    ERRORE[#NOKEY]

ENDIF

RETURN

```

Alla selezione della singola riga, la variabile ZSELALLGRIDCLICK corrisponderà a 0, quindi l'update sarà sempre relativo al singolo articolo.

Se viene premuto il tasto <Seleziona tutti>, viene fatto un update che va a coinvolgere tutti gli elementi visualizzati.

Molto importante l'utilizzo della variabile locale SELALLBUTTON.

La callBack viene, comunque richiamata per tutte le volte quindi bisogna utilizzare una variabile di controllo che impedisca l'esecuzione ripetuta dell'azione.

La variabile SELALLBUTTON viene valorizzata a 1 all'esecuzione dell'update e azzerata quando viene richiamata la callBack dell'ultima riga della griglia.

Riferimenti

Variabili d'ambiente

'@DEFGRID - Definizione griglia (avanzata)

ZGRIGA_GRUPPOSEL - Sincronizzazione videate

Nell'esempio viene gestita una videata composta elenco + dettaglio. Nell'elenco vengono visualizzati tutti i clienti raggruppati per provincie, nella videata composta di dettaglio è necessario visualizzare tutti i dati anagrafici del cliente stesso.

E' necessario, nel momento in cui si sincronizza videata principale e secondaria, distinguere se il fuoco è sul cliente o sul gruppo, in caso ci si trovi sul gruppo non si deve vedere nessun dettaglio.

BC

```
'=====
'@SR VIDEATA
'=====

'@DEFVID DIM[20,120] TIT[""]

'@DEFVIDCOMP GES[GOSUB VIDEATA_GRIGLIA] _
      GES[GOSUB VIDEATA_SECONDARIA] SINCRONIZZA

'@RUNVID

RETURN

'=====
'@SR VIDEATA_GRIGLIA
'=====

'@DEFVID DIM[20,70]

'@DEFGRID IDGRIGLIA RET[1,1] POS[1,1] DIM[20.19,70] QUERY[IDQUERY] _
      AVANZATA MULTISEL SELRIGA[GOSUB SELRIGA] _
      RAGPREDEFINITO\[VAR\[1\] ELENCO\[AILOCS\]\] \_
      COMPRIMIRAGGRUPPAMENTI

'@INSCOLGRID IDGRIGLIA LARG[10] TIT["Codice"]
'@INSCOLGRID IDGRIGLIA LARG[20] TIT["Cliente"]
'@INSCOLGRID IDGRIGLIA LARG[20] TIT["E-mail"]
'@INSCOLGRID IDGRIGLIA LARG[20] NASCOSTA TIT["Indirizzo"]
'@INSCOLGRID IDGRIGLIA LARG[20] NASCOSTA TIT["Indirizzo"]
'@INSCOLGRID IDGRIGLIA LARG[20] NASCOSTA TIT["C.A.P"]
'@INSCOLGRID IDGRIGLIA LARG[20] NASCOSTA TIT["Comune"]
'@INSCOLGRID IDGRIGLIA LARG[20] NASCOSTA TIT["Frazione"]
'@INSCOLGRID IDGRIGLIA LARG[20] NASCOSTA TIT["Provincia"]
'@INSCOLGRID IDGRIGLIA LARG[20] NASCOSTA TIT["Telefono"]
'@INSCOLGRID IDGRIGLIA LARG[20] NASCOSTA TIT["Telefono"]
'@INSCOLGRID IDGRIGLIA LARG[20] NASCOSTA TIT["Cellulare"]
'@INSCOLGRID IDGRIGLIA LARG[20] NASCOSTA TIT["Numero Fax"]
'@INSCOLGRID IDGRIGLIA LARG[20] NASCOSTA TIT["Sito internet"]
```

```
'@INSCOLGRID IDGRIGLIA LARG[20] NASCOSTA TIT["Ruolo"]
'@INSCOLGRID IDGRIGLIA LARG[20] NASCOSTA TIT["Persona di riferimento"]

'@DEFCELLGRID IDGRIGLIA COL[1] VAR[CFIPART] FMT[NUMERICO[5]]
'@DEFCELLGRID IDGRIGLIA COL[2] VAR[RAGSOC$] FMT[TESTO[30]]
'@DEFCELLGRID IDGRIGLIA COL[3] VAR[AREMAIL$] FMT[TESTO[20]]
'@DEFCELLGRID IDGRIGLIA COL[4] VAR[AIIND$] FMT[TESTO[80]]
'@DEFCELLGRID IDGRIGLIA COL[5] VAR[AIIND2$] FMT[TESTO[80]]
'@DEFCELLGRID IDGRIGLIA COL[6] VAR[AICAP$] FMT[TESTO[80]]
'@DEFCELLGRID IDGRIGLIA COL[7] VAR[AILOC$] FMT[TESTO[80]]
'@DEFCELLGRID IDGRIGLIA COL[8] VAR[AILOC2$] FMT[TESTO[80]]
'@DEFCELLGRID IDGRIGLIA COL[9] VAR[AIPR$] FMT[TESTO[80]]
'@DEFCELLGRID IDGRIGLIA COL[10] VAR[ARTEL1$] FMT[TESTO[80]]
'@DEFCELLGRID IDGRIGLIA COL[11] VAR[ARTEL2$] FMT[TESTO[80]]
'@DEFCELLGRID IDGRIGLIA COL[12] VAR[ARCELL$] FMT[TESTO[80]]
'@DEFCELLGRID IDGRIGLIA COL[13] VAR[ARFAX$] FMT[TESTO[80]]
'@DEFCELLGRID IDGRIGLIA COL[14] VAR[ARSITO$] FMT[TESTO[80]]
'@DEFCELLGRID IDGRIGLIA COL[15] VAR[ARDEV$] FMT[TESTO[80]]
'@DEFCELLGRID IDGRIGLIA COL[16] VAR[ARRIF$] FMT[TESTO[80]]

'@RUNVID

RETURN
```

La videata principale elenca i clienti e sincronizza la secondaria.

Nella griglia avanzata è importante notare il raggruppamento predefinito per Località e la compressione automatica di tutti i raggruppamenti.

BC

```

'=====
'@SR VIDEATA_SECONDA
'=====

IF ZGRIGA_GRUPPOSEL=1 THEN ZVIDCOMP=1
IF ZGRIGA_GRUPPOSEL=0 THEN ZVIDCOMP=2

'@DEFVID DIM[20,120] TIT[""]
'@DEFVIDMPAG TIT["Vuota"] GES[VIDEATA_VUOTA] _
TIT["Dettaglio"] GES[VIDEATA_DETTagLIO] _
NASCOSTOMPAG

'@RUNVID

RETURN

```

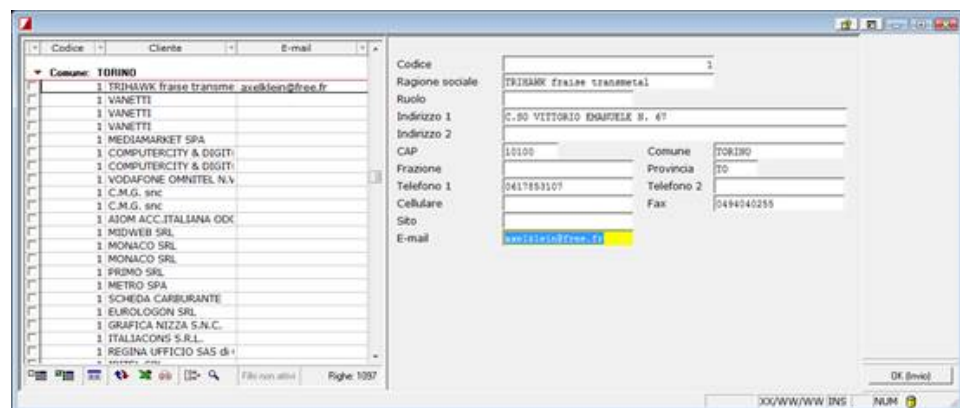
La videata secondaria è gestita da un multipagina nascosto dove la prima videata è completamente vuota, mentre la seconda gestisce il dettaglio dei clienti.

La scelta della pagina viene pilotata dalla variabile ZGRIGA_GRUPPOSEL: se 1, cioè se la selezione della griglia è sulla banda del raggruppamento allora verrà visualizzata la pagina vuota.

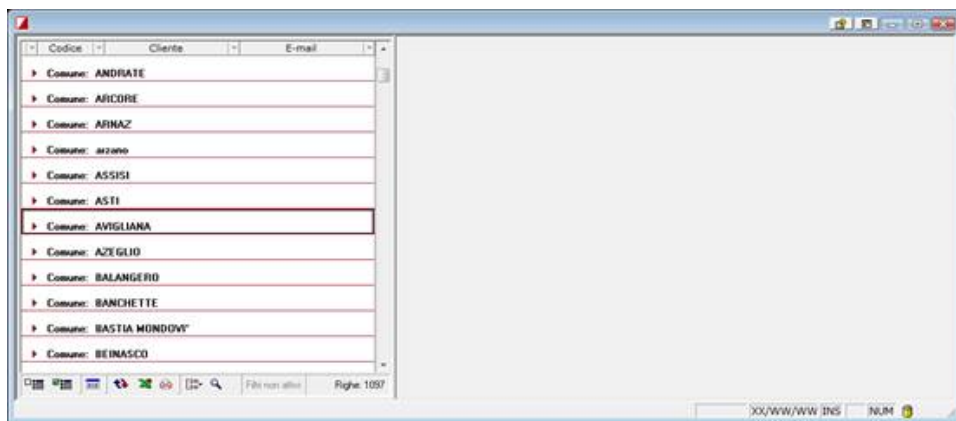
In caso contrario, cioè se la selezione della griglia è su una riga qualsiasi si visualizzerà la pagina contenente il dettaglio dell'e-mail.

Risultato in esecuzione

Selezione della griglia su una riga:



Selezione sulla banda del raggruppamento:



Riferimenti

Videate - variabili d'utilizzo interno BC

Griglia avanzata - Definizione griglia

Stampe - Variabili di utilizzo interno BC

Variabili per gestione stampe

Variabile	Descrizione
ZPOSIZ%[2]	Colonna di posizionamento della 'X' emessa da '@PLP.
NR[2]	Numero di righe stampate nella pagina corrente.
LPAG[3]	Lunghezza pagina di stampa, espressa in numero di righe.
ZTST[2]	Stampe con TAB: Se settato a 1 fa sì che vengano solo stampate le parti variabili di un TAB di stampa.
ZPLF[2]	Serve qualora sia necessario eseguire una stampa tra più moduli che si richiamano in CHAIN o '@SBC. Prima di chiamare il modulo, è necessario settare ZPLF a 1 per fare sì che la stampante non venga chiusa.
ZPRMIGL[1]	Indica se abilitare il meccanismo di inserimento automatico dei puntini delle migliaia nei campi di output di una videata Impostato a 1 di default.
ZPRPATH\$[60]	Path file di stampa (nel caso di stampa su File).
ZPRDEV\$[40]	Nome dispositivo di stampa.
ZPRMODL\$[8]	Nome del modello da usare per la stampa (solo per stampe in modalità testo).
ZPRMODPATH\$[60]	Directory on cui sono contenuti i modelli stampante.
ZPRQUEUE\$[8]	Nome della coda di stampa.
ZPRNAME\$[101]	Nome "completo" dispositivo di stampa (Path + Device).
ZPRCH\$[2]	Carattere di default della stampa: 'C' - Compresso; 'N' - normale; 'D' Dodicesimi.
ZPRNEWL\$[2]	Carattere o sequenza per eseguire il NewLine.
ZPRTMOD\$[1]	Tipo modulo: 'C' - Continuo; 'S' - Singolo.
ZPRNOM\$[5]	Nome identificativo modulo (corrisponde al parametro M[] di '@PLF).
ZPRDES\$[21]	Descrizione della stampa (deprecato: è stato sostituito da ZPRDESEXT\$).
ZPRDESEXT\$[48]	Descrizione della stampa.
ZPRLN%[3]	Ampiezza della riga di stampa (in numero di caratteri).
ZPRTOP%[2]	Margine superiore della pagina (in numero di righe).
ZPRBOTM%[2]	Margine inferiore della pagina (in numero di righe).
ZPRLEFT%[2]	Margine sinistro della pagine (in numero di caratteri).
ZPRRIGHT%[2]	Margine destro della pagine (in numero di caratteri).
ZPRORIEN%[1]	Orientamento della stampa: 0 - Verticale; 1 - Orizzontale (solo per stampe in modalità testo).
ZPRFORCEORIEN%[1]	Orientamento della stampa imposto da programma: 0 - Non forzato (determinato in automatico); 1 - Verticale, 2 - Orizzontale.
ZPRCOPIE%[2]	Numero di copie (solo per stampe Laser).
ZPRINTL%[1]	Interlinea di stampa (in numero di righe).
ZPRHOLD%[1]	Indica se congelare o meno la stampa nel caso di stampe in Spool 0 - Non congela stampa; 1 - Stampa congelata; 2 - Prima di stampare richiede l'inserimento di un modulo.

ZPLP%[1]	Viene settato a 1 da '@PLP o nel caso di stampe con F.L. con posizionamento, per notificare al programma l'avvenuto posizionamento.
ZPRBOZZA%[1]	Flag stampa bozza/definitiva. Nel caso in cui si stia stampando un TAB con fincature fatte con caratteri semigrafici, la stampa in modalità Bozza fa sì che vengano stampati i simboli '+', '-' e ' ' anziché i caratteri semigrafici.
ZPRGRAPH%[1]	Se impostato a 1, abilita la stampa dei caratteri semigrafici.
ZPRJUST%[1]	Se impostato a 1, abilita la giustificazione del testo in stampa.
ZPRLAS%[1]	Indica se la stampante selezionata è laser (valore 1) o ad aghi (valore 0).
ZPRTIPO%[1]	Tipo di dispositivo di stampa. Può assumere i valori: 0 - Nessuna emissione, 1 - Stampa su dispositivo (stampante); 2 - Anteprima, 3 - Stampa su file, 4 - Stampa su modulo laser (DF); 5 - Stampa grafica Windows (con driver).
ZPRTIPOEX%[1]	Dettaglio per la tipologia di dispositivo di stampa. Per stampante, può assumere i valori: 1 - Dos, 2 - Dos DR, 3 - Windows per file, può assumere i valori: 1 - Txt, 2 - Word, 3 - PDF, 4 - File per DR.
ZWPXNUMPAG%[1]	Numero di pagine di un testo WPX.
ZPRMODE%[1]	Tipo di dispositivo stampante (solo se ZPRTIPO = 1). Può assumere i valori: 0 - Stampa diretta su dispositivo, 1 - Stampa su terminale (UNIX), 2 - Stampa in SPOOL Sistemi; 3 - Stampa in SPOOL su coda di sistema operativo; 5 - Stampa DOS con PRINT.com/Windows.
ZPRFRMT%[1]	(deprecata) Formato del file di stampa.
ZPRAPPEND%[1]	Flag per accodamento (valore 0) sovrascrittura (valore 1) applicato al file di stampa.
ZTBRK%[1]	Modalità di interruzione della stampa: 0 - Immediata, 1 - Differita.
ZBRK%[1]	Flag interruzione stampa avvenuta: 0 - No, 1 - Sì.
ZPRLASMOD%[1]	Flag che identifica la stampa laser su modulistica fiscale.
ZPRWINGRAPH%[1]	Indicare se si tratta di stampa con driver windows (valore 1) oppure una stampa in modalità "testo".
ZPRTABNAME\$[8]	Nome della tabella ministeriale per le stampe PC.
ZPRDELPRINT%[1]	Flag Eliminazione file di spool dopo la stampa: 0 - No, 1 - Sì.
ZPRTRK%[1]	Stampe 770 su modulo: forza la troncatura degli importi da stampare alla cifra delle migliaia (in Lire) o all'unità di Euro.
ZPRNUMPAG%[1]	Numero di pagine che compongono la stampa.
ZPRFIRSTPAG[1]	Flag per stampa oggetti formati liberi su prima pagina (0 - No, 1 - Sì).
ZPRNOFIRSTPAG[1]	Flag per stampa oggetti formati liberi NON su prima pagina (0 - No, 1 - Sì).
ZPRLASTPAG[1]	Flag per stampa oggetti formati liberi su ultima pagina (0 - No, 1 - Sì).
ZPRNOLASTPAG[1]	Flag per stampa oggetti formati liberi NON su ultima pagina (0 - No, 1 - Sì).
ZPRINTLOGO\$[255]	Nome del file di immagine destinato a contenere il logo, da applicare alle stampe con formato libero o REPORTER.
ZREPORTER[1]	Flag che identifica se la sub che si sta eseguendo è stata richiamata dall'esecuzione di REPORTER (SISRPTW o BCRPTW) o dall'esecuzione di un report attraverso la specifica '@RUNREPORT. (0 - No, 1 - Sì). Può essere utilizzata per condizionare l'assegnazione delle variabili di contesto di prodotto, in base al fatto che queste siano utilizzate su una stampa formato semplice o su una stampa di un report.

Variabili per gestione stampe con F.L.

Variabile	Descrizione
ZPM%[2]	Riga di posizionamento.
ZICM%[2]	Riga di inizio corpo di stampa F.L.
ZFCM%[2]	Riga di chiusura corpo di stampa F.L.
ZRCM%[2]	Numero righe di corpo della stampa F.L.
ZPRTIPOFL%[1]	Tipo di formato: 0 - Pagina unica, 1 - Testata/Corpo/Piede.
ZPRMAXNDEC%[1]	Numero di decimali per F.L. (?).
ZPRFLFRMT\$[5]	Indica i parametri di formattazione del dato relativo alla variabile del formato di stampa.
ZPRPOSSEZ%[1]	Piede stampato a posizione Fissa (0) o Variabile (1).
ZPRNOT\$[8]	Nome del file elenco variabili (.NOT).
ZPRFLEXIST%[1]	Indica se la variabile indicata sul formato esiste (1) o no (0).
ZPRPRINTEDSECT%[1]	Indica se la sezione stampata dalla precedente PRINTFORM è stata effettivamente stampata oppure no a causa di condizionamenti (0 - No, 1 - Sì).
ZPRFLGRAPH%[1]	Flag stampa con formato libero Sisform: -1 = Stampa TAB (no formato libero); 0 = Stampa con f. libero testo (FRM); 1 = Stampa con f. libero grafico (SXF).

ZPRFLAGHI[1]	Flag stampa con formato libero Sisform ad Aghi • 0 = No, Stampa con f. libero Sisform grafico; • 1 = Sì, Stampa con f. libero Sisform ad aghi.
ZFORMPERS [1]	Flag stampa con formato libero Sisform standard o personalizzato • 0 = Non definito (se esiste il personalizzato, ha prevalenza sullo standard) • 1 = Formato standard • 2 = Formato personalizzato
ZPRVISTEXT@[1]	Flag per forzatura ricerca di formato testo in specifica '@PER.

Variabili per gestione stampe di modulistica fiscale

Variabile	Descrizione
ZPLC[1]	Flag abilitazione posizionamento automatico in stampa.
ZMOD[2]	Numero pagine sul modulo di stampa.
ZPPAG[2]	Progressivo pagina relativo al modulo.
ZPNR[2]	Numero righe di stampa riferite alla pagina.
ZPDISP@[1]	Flag controllo salti pagina GEMMA.
ZLASTIPO@[1]	Tipo dispositivo di stampa (copia di ZPRTIPO%).
ZLASMODE@[1]	Tipo di modalità di stampa (copia di ZPRMODE%).
ZLASPATH\$[60]	Path file di stampa (copia di ZPRPATH\$).
ZLASDEV\$[40]	Nome dispositivo di stampa (copia di ZPRDEV\$).
ZLASMODL\$[8]	Modello stampante (copia di ZPRMODL\$).
ZLASQUEUE\$[8]	Coda di stampa (copia di ZPRQUEUE\$).
ZPRTABVAR@[1]	Flag per gestire la stampa dei caratteri in grassetto.

Variabili per gestione telematico da stampe fiscali

Variabile	Descrizione
ZPRPC@[1]	Se settata a un valore diverso da 0, fa sì che venga attivato il meccanismo di esportazione dati. Può assumere i valori: 0 - Non è abilitata la predisposizione per la stampa del modulo PC; 1 - Abilita la predisposizione per la stampa del modulo PC ed esegue la stampa su device dei diversi quadri; 2 - Abilita la predisposizione per la stampa del modulo PC e non esegue la stampa dei quadri su device.
ZPRINIPAG@[1]	Numero di pagina alla quale ci si deve posizionare
ZPRPCPROG@[1]	Nelle situazioni in cui una stessa riga del file '.TAB' è utilizzata per generare più righe di stampa nel modulo, a ciascuna riga di stampa corrispondono codici diversi sul modulo PC: questa variabile consente al programma di identificare univocamente ogni riga di stampa tramite un progressivo (settato prima della '@PLX).
ZPRPCPAG@[1]	Numero di pagine per quadro (quando per un quadro esistono più pagine).
ZPRPCCOD\$[5]	Contiene il codice della dichiarazione. È una variabile "globale" che deve essere valorizzata prima della specifica '@PLI.
ZPRPCCODEX\$[6]	Contiene il codice della dichiarazione (per codici di 6 cifre). È una variabile "globale" che deve essere valorizzata prima della specifica '@PLI.
ZPRPCUPPER@[1]	Se = 1, fa sì che i valori delle variabili alfanumeriche siano stampati/esportati tutti in maiuscolo
ZPRTABNAME\$[8]	Nome della tabella ministeriale per le stampe PC.
ZPRPCID\$[1]	Contiene un carattere per identificare la dichiarazione in base al modello (740, 750, ...). Può assumere i seguenti valori: "4" - UPF, "5" - USP, "6" - USC, "B" - USE (es.: 760 bis), "7" = 770, "O" = 770/O
ZPRPCMIN\$[8]	Nome del file contenente la tabella ministeriale di riferimento. Deve essere valorizzata prima dell'esecuzione della specifica '@PLI.
ZPRDELPRINT@[1]	Flag Eliminazione file di spool dopo la stampa.
ZPRNPR1@[1]	Progressivo ad uso dei programmi di stampa, per gestire l'ordinamento in stampa dei diversi Quadri.
ZPRNPR2@[1]	Progressivo ad uso dei programmi di stampa, per gestire l'ordinamento in stampa dei diversi Quadri.
ZPRNPR3@[1]	Progressivo ad uso dei programmi di stampa, per gestire l'ordinamento in stampa dei diversi Quadri.
ZPRPC_2007[1]	Flag di attivazione della nuova modalità di gestione dei telematici con tabellone su file SXDAT (0=No, 1=Sì, 2=Sì, e telematico XML). E' una variabile "globale" che deve essere valorizzata prima della specifica '@PLS.

ZPRPC_SXDAT\${63}	Nome del file SXDAT contenente la parametrizzazione (completo di estensione). E' una variabile "globale" che deve essere valorizzata prima della specifica '@PLS.
ZPRPCCODEX\${6}	Contiene il codice della dichiarazione (a 6 caratteri). E' una variabile "globale" che deve essere valorizzata prima della specifica '@PLI.
ZPRPC_DENYCHARS\${255}	Elenco dei caratteri non ammessi per il telematico XML
ZPRPC_REPLACE_DENYCHARS\${255}	Elenco dei caratteri da sostituire, uno ad uno, ai caratteri inseriti nella variabile ZPRPC_DENYCHARS\$
ZPRPC_ALLOWCHARS\${255}	(a partire da ambiente v. 18.13) Elenco dei caratteri ammessi per il telematico, da convertire per l'XML. I caratteri devono essere separati da virgola (in questo caso, ad ogni carattere particolare può essere sostituita una stringa di più caratteri)
ZPRPC_REPLACE_ALLOWCHARS\${255}	(a partire da ambiente v. 18.13) Elenco delle stringhe da sostituire ai caratteri inseriti nella variabile ZPRPC_ALLOWCHARS\$. Anche questi sono separati da virgola

E-Mail - Variabili d'utilizzo interno BC

Variabili per gestione rubrica

Variabile	Descrizione
ZADDRESSBOOK\${255}	La variabile contiene il nome della rubrica selezionata. Il nome della rubrica corrisponde al parametro NOME[] inserito all'interno del sottoparametro RUBRICA della specifica @DEFMAILADDRESSBOOK. La variabile è utilizzabile all'interno delle callback di editazione dei dati visualizzati (@DEFMAILADDRESSBOOK ... TOOLBAR[]), per conoscere su quale rubrica si deve compiere l'azione d'inserimento dati. Viene valorizzata a fronte del richiamo di una <u>rubrica d'ambiente</u> attraverso queste specifiche: • <u>@SENDMAIL</u> - Invio di una e-mail • <u>@RUNMAIL</u> - Avvio del gestore di posta • <u>@RUNMAILLIST</u> - Invio di un elenco di e-mail • <u>RUNMAILADDRESSBOOK</u> - Esecuzione di una rubrica

Variabili per gestione e-mail e gestore di posta

Variabile	Descrizione
ZMAIL_ID	Identificativo univoco assegnato all'e-mail (64 caratteri)
ZMAIL_IDB	Identificativo univoco assegnato all'e-mail (Blob)
ZMAIL_FROM	Indirizzo mittente e-mail
ZMAIL_TO	Indirizzo destinatario e-mail
ZMAIL_CC	Indirizzo destinatario e-mail in contro conoscenza
ZMAIL_CCN	Indirizzo destinatario e-mail in contro conoscenza nascosta
ZMAIL_SUBJECT	Oggetto e-mail
ZMAIL_BODY	Testo dell'e-mail
ZMAIL_HTMLBODY	Testo in formato html dell'e-mail
ZMAIL_RRIT	Flag ricevuta di ritorno
ZMAIL_SIZE	Dimensioni dell'e-mail
ZMAIL_DATE	Data d'invio dell'e-mail
ZMAIL_TIME	Ora d'invio e-mail
ZMAIL_ATTACH	Numero di allegati dell'e-mail
ZMAIL_FROMNAME	Nome del mittente dell'e-mail
ZMAIL_REPLYTO	Mittenti a cui rispondere all'e-mail
ZMAIL_SIGNED	Flag che identificare se l'e-mail è firmata
ZMAIL_ENCRYPTED	Flag che identifica se l'e-mail è criptata

<u>ZMAIL_DATEFROM</u>	Data di gestione per l'aggiornamento dei collegamenti tra entità e e-mail. Viene valorizzata in fase di sincronizzazione. Corrisponde al range (DA) di controllo e-mail.
<u>ZMAIL_DATETO</u>	Data di gestione per l'aggiornamento dei collegamenti tra entità e e-mail. Viene valorizzata in fase di sincronizzazione. Corrisponde al range (A) di controllo e-mail.
<u>ZMAIL_FIRMA?</u>	Permette di definire la firma da inserire all'interno dell'e-mail che si sta generando. L'utilizzo della variabile funziona indiscriminatamente sia con il gestore di posta sistemi, che con l'invio di un'e-mail semplice (<u>@RUNVIDMAILMSG</u> , <u>@SENDMAIL</u>). La firma non è gestita in caso ci si trovi di fronte a modelli e-mail, o in caso di gestore Outlook Express

Codici di errore variabile ZBCERR

- [Ambiente - Codici di errore variabile ZBCERR](#)
- [Tabelle - Codici di errore variabile ZBCERR](#)
- [Compressione/Decompressione Files](#)
- [Stampe - Codici di errore variabile ZBCERR](#)
- [Gestione comunicazioni - Codici di errore variabile ZBCERR](#)
- [Strutture Linguaggio - Codice di errore variabile ZBCERR](#)
- [Interfacce OLE - Codici di errore variabile ZBCERR](#)
- [Gestione testi Word - Codici di errore variabile ZBCERR](#)

Ambiente - Codici di errore variabile ZBCERR

'@CHECKFORMULA

ZBCERR	ZBCMSG
22501	"Caratteri in formula non contigui"
22502	"Mancano operandi"
22503	"Operando errato in formula"
22504	"Errore in digitazione operatori"
22505	"Numero parentesi errato"
22506	"Nome variabile ... non valido"
22507	"Errata sintassi operatore []"
22508	"Nessuna formula indicata"

'@RUNFORMULA

ZBCERR	ZBCMSG
22511	"Divisione per zero"
22512	"Calcolo non eseguito: nella formula non e' presente il campo ..."

'@EXECPROCESS

ZBCERR	ZBCMSG
25500	"Impossibile eseguire il processo: ... a causa dell'errore NNN"
25504	"Impossibile eseguire il processo: ... con le credenziali dell'utente 'XXX' (dominio 'YYY'). La versione del sistema operativo (libreria Advapi32.dll) non lo consente.

'@SETREGISTRYVALUE/ '@GETREGISTRYVALUE/ '@DELREGISTRYKEY

ZBCERR	ZBCMSG
25700	"Impossibile creare la chiave di registro ..."
25710	"Impossibile aprire la chiave di registro ..."
25720	"Impossibile impostare il valore ... per la chiave di registro ..."
25730	"Impossibile leggere il valore ... della chiave di registro ..."
25740	"Il tipo del dato del valore ... della chiave di registro ... non compatibile con il tipo della variabile ..."
25750	"Impossibile cancellare la chiave di registro ..."

'@GETDIRFILEINFO

ZBCERR	ZBCMSG
26100	"Struttura dinamica ... non definita."
26101	"Direttiva ... non trovata."

Tabelle - Codici di errore variabile ZBCERR

'@DEFQUERY

ZBCERR	ZBCMSG
22010	"Attenzione! Il file ... non esiste! Impossibile avviare il parsing"
22800	"Attenzione! il file XML ... non contiene alcuna definizione di query"

'@DEFQUERYFROM

ZBCERR	ZBCMSG
22801	"Attenzione! L'identificazione della query non è definito"
22803	"Attenzione! La query contiene già la tabella ..."
22805	"Attenzione! La relazione ... non è definita nella query"
22806	"Attenzione! La tabella padre ... non è presente nella query"

'@DEFQUERYREL

ZBCERR	ZBCMSG
22801	"Attenzione! L'identificazione della query non è definito"
22803	"Attenzione! La query contiene già la tabella ..."
22805	"Attenzione! La relazione ... non è definita nella query"
22806	"Attenzione! La tabella ... non è presente nella query"

'@DEFQUERYCOLUMN

ZBCERR	ZBCMSG
22806	"Attenzione! La tabella ... non è presente nella query"
22807	"Attenzione! La query contiene già il campo ..."

'@DEFQUERYORDER

ZBCERR	ZBCMSG
22801	"Attenzione! L'identificazione della query non è definito"

ZBCERR	ZBCMSG
22806	"Attenzione! La tabella ... non è presente nella query"
22808	"Attenzione! Il campo ...non è presente nella query"

'@DEFQUERYWHERE

ZBCERR	ZBCMSG
22801	"Attenzione! L'identificazione della query non è definito"

'@DEFQUERYCONDGRID

ZBCERR	ZBCMSG
22801	"Attenzione! L'identificazione della query non è definito"

'@SETQUERYCONDGRID

ZBCERR	ZBCMSG
22801	"Attenzione! L'identificazione della query non è definito"
22806	"Attenzione! La tabella ... non è presente nella query"
22808	"Attenzione! Il campo ...non è presente nella query"

'@DEFQUERYPARAM

ZBCERR	ZBCMSG
22801	"Attenzione! L'identificazione della query non è definito"
22814	"Attenzione! La query contiene già il parametro ..."

'@GETDBENGINEINFO

ZBCERR	ZBCMSG
22850	"Impossibile leggere le informazioni sul motore di database specificato."

'@SETDBLICENZA

ZBCERR	ZBCMSG
22851	"Errore generico occorso durante l'operazione di inserimento della licenza."
22852	"La licenza specificata risulta già installata."
22853	"La licenza specificata non è valida."

'@BINDQUERYCOLUMN

ZBCERR	ZBCMSG
22801	"Attenzione! L'identificazione della query non è definito"
22806	"Attenzione! La tabella ... non è presente nella query"
22808	"Attenzione! Il campo ...non è presente nella query"

'@BINDQUERYPARAM

ZBCERR	ZBCMSG
22801	"Attenzione! L'identificazione della query non è definito"
22815	"Attenzione! Il parametro ... non è presente nella query"

'@CHECKQUERY

ZBCERR	ZBCMSG
22809	"Attenzione! La query non ha colonne di output"
22812	"Attenzione! Le query di aggregazione devono specificare una funzione di aggregazione per ogni campo di output"
22813	"Attenzione! il campo ... non può essere una colonna di ordinamento perché non è di output"

'@SAVEQUERY

ZBCERR	ZBCMSG
22020	"Attenzione! L'elemento ... non è stato definito tramite @DEFXMLDATA, e non verrà quindi restituito!"
22021	"Impossibile avviare la scrittura su file XML: libreria MSXML4.dll non individuata"
22022	"Scrittura su file XML non eseguita. Il file esiste e non viene ricoperto."
22023	"Attenzione! Impossibile creare il file ..."

'@RUNQUERY

ZBCERR	ZBCMSG
22561	"Errore nella lettura del file XML del Query"
22562	"File Query non trovato"

'@PHA

ZBCERR	ZBCMSG	SIGNIFICATO
0	""	Path corretto
21400	""	Errore sulla funzione di conversione del path in formato UNC (funzione a basso livello)
21401	""	Il path UNC ha dimensione superiore alla variabile passata come argomento

'@GETDRIVE

ZBCERR	ZBCMSG	SIGNIFICATO
0	""	Lettura drive terminata con successo.
21600	""	Impossibile acquisire l'elenco dei drive logici configurati per il sistema.
21601		Impossibile acquisire il drive logico relativo alla direttiva corrente. Potrebbe trattarsi di un percorso UNC (\\nomeserver\disco).

'@SETFILEATTRIB / '@GETFILEATTRIB

ZBCERR	ZBCMSG
25001	"In fase di accesso al file si è verificato l'errore NN"
25002	"Il file indicato non esiste"
25003	"In fase di accesso al file si è verificato l'errore NN - Descrizione"

'@SETCURDIR

ZBCERR	ZBCMSG
25002	"La direttiva indicata non esiste"
25003	"In fase di accesso al file si è verificato l'errore NN - Descrizione"

'@CHECKDBPARMS

ZBCERR	ZBCMSG
22890	gestore DBMS (sorgente dati) non supportato
22891	server non valido

ZBCERR	ZBCMSG
22892	password o utente non validi
22893	database non valido

'@POPOLATETABLE

ZBCERR	ZBCMSG
22900	File sorgente '...' non trovato.
22901	Modalità NON gestita
22902	Differenze riscontrate tra la tabella passata e la tabella letta dal file SXDAT
22903	Errore nella creazione della Struttura Dinamica di appoggio

'@RENAME

ZBCERR	ZBCMSG
27200	Impossibile rinominare il file ' .. '. Errore '...'
27201	Impossibile rinominare la direttiva ' .. '. Errore '...'

'@COPYDIR

ZBCERR	ZBCMSG
26110	Errore XX durante la copia da '.. ' a '.. '.
26111	La direttiva sorgente '.. ' non esiste

Compressione/Decompressione Files

'@ADDFILETOZIP

ZBCERR	ZBCMSG
27300	"Errore su operazione file zip, Descrizione [...] Ultimo file elaborato"

'@EXTRACTFILEFROMZIP

ZBCERR	ZBCMSG
27300	"Errore su operazione file zip, Descrizione [...] Ultimo file elaborato"
27301	"Il file [...] e' correntemente in uso"
27302	"Errore in fase di estrazione del file [...]: la password indicata non e' corretta. Il file non puo' essere estratto."
27303	"Errore in fase di estrazione del file [...]: Il file zip [...] non contiene il file specificato."

'@GETZIPFILELIST

ZBCERR	ZBCMSG
27300	"Errore su operazione file zip, Descrizione [...] Ultimo file elaborato"

Stampe - Codici di errore variabile ZBCERR

Stampe - Codici di errore variabile ZBCERR

'@SETOUTPARMSPRINTER

ZBCERR	ZBCMSG
20050	"Controllo parametri obbligatori. Non è stato indicato il tipo stampante"
20060	"Controllo correttezza parametri. Tipo stampante non corretto"
20070	"Controllo correttezza parametri. La stampante richiesta è inesistente"
20075	"Controllo correttezza parametri. La stampante richiesta esiste, ma con differenti Modello e/o Dispositivo"
20080	"Controllo correttezza parametri. Non sono stati indicati correttamente nome, modello, e device della stampante"
20085	"Controllo correttezza parametri. Modalità duplex non corretta"
20170	"Errore su argomenti. Attenzione! Il Lavoro di stampa indicato 'NNNNN' non esiste. I parametri del dispositivo (stampante) non vengono attivati."
20300	"Controllo parametri. È stata richiesta la stampa differita, ma non è stata indicata la coda di spool."
20341	Attenzione! Il codice del cassetto da selezione non è presente tra quelli definiti per la stampante XXXXX. La stampa non verrà inviata

'@SETOUTPARMSFAX e '@SETOUTPARMSCOVERFAX

ZBCERR	ZBCMSG
20327	'@SETOUTPARMSFAX - Attenzione! Non è stato indicato il numero di fax. Il fax non verrà inviato
20328	'@SETOUTPARMSFAX - Il dispositivo fax non è stato trovato. Il fax non verrà inviato
20330	'@SETOUTPARMSFAX - Attenzione! Il file XXXX non esiste. Il fax non verrà inviato
20329	'@SETOUTPARMSCOVERFAX - Attenzione! Non sono stati indicati la direttiva e/o il nome del file della copertina. Il fax non verrà inviato

'@GETPRINTERS

ZBCERR	ZBCMSG	SIGNIFICATO
0	""	Ok
21501	""	Il numero di stampanti configurate è superiore alla dimensione dei vettori utilizzati

'@CHECKPRINTER

ZBCERR	ZBCMSG	SIGNIFICATO
0	""	Ok
21502	""	La coda non esiste, e non può quindi essere utilizzata

'@GETOUTPARMS

ZBCERR	ZBCMSG
20350	"La variabile di programma destinata a contenere l'informazione XXXXXXX è sottodimensionata. Il valore viene restituito troncato"

'@GETOUTPARMSFILE, '@GETOUTPARMSPRINTER, '@GETOUTPARMSEMAIL, '@GETOUTPARMSSPOOL, '@GETOUTPARMSFAX

ZBCERR	ZBCMSG
20290	"Attenzione! I parametri relativi all'emissione sul dispositivo XXXXX non sono stati definiti!"
20291	"Attenzione! I parametri relativi all'emissione sul dispositivo XXXXX sono stati definiti ma la relativa emissione non è abilitata!"

'@RUNREPORT

ZBCERR	ZBCMSG
22551	"Errore nella lettura dei file XML del report"
22552	"File report non trovato"

'@GETPRINTERINFO

ZBCERR	ZBCMSG
20340	Attenzione! Il vettore che deve contenere l'elenco dei cassette è stato dimensionato di X elementi mentre il numero di cassette definiti per la stampante XXXX è Y. Verranno caricati solo X cassette.

Gestione comunicazioni - Codici di errore variabile ZBCERR

'@OPENSERCOM / '@WRITESERCOM / '@READSERCOM

ZBCERR	ZBCMSG
24000	"Porta seriale non aperta"

'@OPENFTP/ '@CLOSEFTP/ '@GETFILEFTP/ '@PUTFILEFTP/ '@DELFILEFTP/ '@GETATTRIBFILEFTP/ '@DIRFTP

ZBCERR	ZBCMSG
24100	'@GETFILEFTP - Errore durante la copia in locale del file <i>NomeFile</i> . Errore <i>Errore</i> "
24110	'@PUTFILEFTP - Errore durante la copia sul Sever del file <i>NomeFile</i> . Errore <i>Errore</i> "
24120	'@DELFILEFTP - Errore durante l'eliminazione del file <i>NomeFile</i> sul Server. Errore <i>Errore</i> ".
24130	'@DIRFTP - Errore durante la lettura del contenuto della direttiva <i>NomeDirettiva</i> sul Server. Errore <i>Errore</i> "
24140	'@GETFILEHTTP - Errore durante la copia in locale del file <i>NomeFile</i> . Errore <i>Errore</i> ."
24141	'@GETFILEHTTP - Impossibile stabilire una connessione HTTP con il Server <i>NomeServer</i> . Errore <i>Errore</i> "
24142	'@GETFILEHTTP - Impossibile terminare la connessione HTTP con il Server. Errore <i>Errore</i> "
24143	'@GETFILEHTTP - L'indirizzo Internet specificato non contiene il prefisso del protocollo, specificare 'http://'"
24150	'@GETFILEFTPINFO - Errore durante la lettura delle informazioni sul file <i>NomeFile</i> . Errore <i>Errore</i> ."
24200	"Errore generico durante la comunicazione FTP con il Server. Errore <i>Errore</i> ."
24210	'@OPENFTP - Impossibile stabilire una connessione FTP con il Server <i>NomeServer</i> . Errore <i>Errore</i> ."
24220	'@CLOSEFTP - Impossibile terminare la connessione FTP con il Server. Errore <i>Errore</i> ."
24230	"Impossibile stabilire una connessione ad Internet. Errore <i>Errore</i> ."
24240	"Errore durante la chiusura del collegamento a Internet. Errore %d."

'@GETFILEHTTP

ZBCERR	ZBCMSG
24140	Errore durante la copia in locale del file Errore <Cod>: <Descrizione>
24141	Impossibile stabilire una connessione http con il Server Errore <Cod>: <Descrizione>
24142	Impossibile terminare la connessione http con il Server Errore <Codice>: <Descrizione>
24143	L'indirizzo Internet specificato non contiene il prefisso del protocollo, specificare 'http://'

Strutture Linguaggio - Codice di errore variabile ZBCERR

'@DEFDYNSTRUCT

ZBCERR	ZBCMSG
25800	Strutture dinamiche - La specifica '@DEFDYNSTRUCT- di gestione delle strutture dinamiche ha causato un errore imprevisto
25802	'@DEFDYNSTRUCT - La struttura "NomeStruttura" risulta già definita nell'array delle strutture AllStructs"

'@ADDYDYNSTRUCT

ZBCERR	ZBCMSG
25800	Strutture dinamiche - La specifica '@ADDYDYNSTRUCT- di gestione delle strutture dinamiche ha causato un errore imprevisto
25801	'@ADDYDYNSTRUCT- La struttura non è presente nell'array delle strutture AllStructs
25803	'@ADDYDYNSTRUCT - Il nuovo elemento che si aggiungendo/inserendo nella struttura non è coerente con la definizione del suo formato
25804	'@ADDYDYNSTRUCT - Riferimento ad una variabile non presente nel dizionario dati.
25806	'@ADDYDYNSTRUCT - Campo non presente nella definizione del formato della struttura
25811	'@ADDYDYNSTRUCT - Chiave duplicata

'@INSYDYNSTRUCT

ZBCERR	ZBCMSG
25800	Strutture dinamiche - La specifica '@INSYDYNSTRUCT- di gestione delle strutture dinamiche ha causato un errore imprevisto
25801	'@INSYDYNSTRUCT- La struttura non è presente nell'array delle strutture AllStructs
25803	'@INSYDYNSTRUCT - Il nuovo elemento che si aggiungendo/inserendo nella struttura non è coerente con la definizione del suo formato
25804	'@INSYDYNSTRUCT - Riferimento ad una variabile non presente nel dizionario dati
25805	'@INSYDYNSTRUCT - Non è possibile inserire/riferire un elemento nella posizione "x" della struttura. Essa contiene solo "y" elementi
25806	'@INSYDYNSTRUCT - Campo non presente nella definizione del formato della struttura
25811	'@INSYDYNSTRUCT - Chiave duplicata

'@GETDYDYNSTRUCT

ZBCERR	ZBCMSG
25800	Strutture dinamiche - La specifica '@GETDYDYNSTRUCT- di gestione delle strutture dinamiche ha causato un errore imprevisto
25801	'@GETDYDYNSTRUCT- La struttura non è presente nell'array delle strutture AllStructs
25804	'@GETDYDYNSTRUCT - Riferimento ad una variabile non presente nel dizionario dati
25805	'@GETDYDYNSTRUCT - Non è possibile inserire/riferire un elemento nella posizione "x" della struttura. Essa contiene solo "y" elementi
25806	'@GETDYDYNSTRUCT - Campo non presente nella definizione del formato della struttura
25808	'@GETDYDYNSTRUCT - Incoerenza tra il tipo di un campo della struttura e la variabile del dizionario dati corrispondente

'@SETDYDYNSTRUCT

ZBCERR	ZBCMSG
25800	Strutture dinamiche - La specifica '@SETDYDYNSTRUCT- di gestione delle strutture dinamiche ha causato un errore imprevisto
25801	'@SETDYDYNSTRUCT - La struttura non è presente nell'array delle strutture AllStructs
25803	'@SETDYDYNSTRUCT - Il nuovo elemento che si aggiungendo/inserendo nella struttura non è coerente con la definizione del suo formato
25804	'@SETDYDYNSTRUCT - Riferimento ad una variabile non presente nel dizionario dati

ZBCERR	ZBCMSG
25805	'@SETDYNSTRUCT - Non è possibile inserire/riferire un elemento nella posizione "x" della struttura. Essa contiene solo "y" elementi
25806	'@SETDYNSTRUCT - Campo non presente nella definizione del formato della struttura

'@DELDYNSTRUCT

ZBCERR	ZBCMSG
25800	Strutture dinamiche - La specifica '@DELDYNSTRUCT- di gestione delle strutture dinamiche ha causato un errore imprevisto
25801	'@DELDYNSTRUCT- La struttura non è presente nell'array delle strutture AllStructs
25807	'@DELDYNSTRUCT - Non è possibile eliminare l'elemento in posizione . La struttura contiene elementi

'@FINDDYNSTRUCT

ZBCERR	ZBCMSG
25800	Strutture dinamiche - La specifica '@FINDDYNSTRUCT- di gestione delle strutture dinamiche ha causato un errore imprevisto
25801	'@FINDDYNSTRUCT- La struttura non è presente nell'array delle strutture AllStructs
25804	'@FINDDYNSTRUCT - Riferimento ad una variabile non presente nel dizionario dati
25806	'@FINDDYNSTRUCT - Campo non presente nella definizione del formato della struttura

'@ZAPDYNSTRUCT

ZBCERR	ZBCMSG
25800	Strutture dinamiche - La specifica '@ZAPDYNSTRUCT- di gestione delle strutture dinamiche ha causato un errore imprevisto
25801	'@ZAPDYNSTRUCT- La struttura non è presente nell'array delle strutture AllStructs

'@KILLDYNSTRUCT

ZBCERR	ZBCMSG
25800	Strutture dinamiche - La specifica '@KILLDYNSTRUCT- di gestione delle strutture dinamiche ha causato un errore imprevisto
25801	'@KILLDYNSTRUCT- La struttura non è presente nell'array delle strutture AllStructs

Interfacce OLE - Codici di errore variabile ZBCERR

'@DEFOLEOBJ

ZBCERR	ZBCMSGEX
25950	Il CLSID indicato non è valido o non esiste sul registro: 'XXXXX'.
25951	Impossibile attivare il collegamento all'oggetto OLE tramite il CLSID: 'XXXXX'.
25952	Impossibile attivare il collegamento all'oggetto OLE tramite il ProgID: 'XXXXX'.
25955	Controllo correttezza parametri. Non è stato indicato il nome da assegnare all'oggetto OLE (parametro NOME[]).
25956	Impossibile allocare la struttura elenco oggetti OLE Automation.
25957	Impossibile attivare il collegamento all'oggetto OLE, né tramite il CLSID: 'XXXXX' né tramite il ProgID: 'XXXXX'.

'@SETOLEOBJPROP

ZBCERR	ZBCMSGEX
20960	L'oggetto OLE indicato come parametro: 'XXXXX' non è stato definito tramite '@DEFOLEOBJ. La specifica non viene eseguita.
25961	La sintassi del parametro di tipo DISPATCH non è corretta. La specifica non viene eseguita.
25965	Controllo correttezza parametri. Non è stato indicato il nome da assegnare all'oggetto OLE (parametro OGGETTO[]).
25966	Controllo correttezza parametri. Non è stato indicato il nome della proprietà da aggiornare.

ZBCERR	ZBCMSGEX
25967	Impossibile accedere alla struttura oggetti OLE. Controllare la definizione dell'oggetto OLE ('@DEFOLEOBJ).
25968	Impossibile eseguire l'operazione: l'oggetto OLE non è stato definito tramite '@DEFOLEOBJ'.
25969	Non è stato indicato neanche un parametro. Impossibile aggiornare il valore per la proprietà.
25971	L'oggetto OLE non è stato correttamente inizializzato.
25972	L'oggetto OLE non prevede la proprietà richiesta.
25973	Impossibile eseguire l'operazione a causa dell'errore seguente: XXXXXX.

'@GETOLEOBJPROP

ZBCERR	ZBCMSGEX
20960	L'oggetto OLE indicato come parametro: 'XXXXX' non è stato definito tramite '@DEFOLEOBJ'. La specifica non viene eseguita.
25961	La sintassi del parametro di tipo DISPATCH non è corretta. La specifica non viene eseguita.
25965	Controllo correttezza parametri. Non è stato indicato il nome da assegnare all'oggetto OLE (parametro OGGETTO[]).
25966	Controllo correttezza parametri. Non è stato indicato il nome della proprietà da leggere.
25967	Impossibile accedere alla struttura oggetti OLE. Controllare la definizione dell'oggetto OLE ('@DEFOLEOBJ).
25968	Impossibile eseguire l'operazione: l'oggetto OLE non è stato definito tramite '@DEFOLEOBJ'.
25970	Non è stato indicato neanche un valore di ritorno. Impossibile leggere il valore della proprietà.
25971	L'oggetto OLE non è stato correttamente inizializzato.
25972	L'oggetto OLE non prevede la proprietà richiesta.
25973	Impossibile eseguire l'operazione a causa dell'errore seguente: XXXXXX.

'@EXECOLEOBJMETHOD

ZBCERR	ZBCMSGEX
20960	L'oggetto OLE indicato come parametro: 'XXXXX' non è stato definito tramite '@DEFOLEOBJ'. La specifica non viene eseguita.
25961	La sintassi del parametro di tipo DISPATCH non è corretta. La specifica non viene eseguita.
25965	Controllo correttezza parametri. Non è stato indicato il nome da assegnare all'oggetto OLE (parametro OGGETTO[]).
25966	Controllo correttezza parametri. Non è stato indicato il nome del metodo da eseguire.
25967	Impossibile accedere alla struttura oggetti OLE. Controllare la definizione dell'oggetto OLE ('@DEFOLEOBJ).
25968	Impossibile eseguire l'operazione: l'oggetto OLE non è stato definito tramite '@DEFOLEOBJ'.
25971	L'oggetto OLE non è stato correttamente inizializzato.
25972	L'oggetto OLE non prevede il metodo richiesto.
25973	Impossibile eseguire l'operazione a causa dell'errore seguente: XXXXXX.

'@DELOLEOBJ

ZBCERR	ZBCMSGEX
25968	Impossibile eseguire l'operazione: l'oggetto OLE non è stato definito tramite '@DEFOLEOBJ'.

Gestione testi Word - Codici di errore variabile ZBCERR

Tutte le specifiche

L'errore seguente è comune a tutte le specifiche che di gestione testi Word.

ZBCERR	ZBCMSG\$ / ZBCMSGEX\$	Scrittura SISERR.LOG	Visualizza se SUBARG=1
27000	Errore generico: Impossibile completare l'operazione. Codice: NNN Descrizione: <Descrizione Errore> TS=0/1 - Processi Word attivi=MMM - ID processi: XXX, YYY	Si	

'@GETWDOCINFO

Errore	Descrizione	Scrittura SISERR.LOG	Visualizza se SUBARG=1
27005	Acquisizione informazioni da documento Word - - Conteggio delle righe del documento non corretto.	Si	No

'@PRINTWDOC

Errore	Descrizione	Scrittura SISERR.LOG	Visualizza se SUBARG=1
27002	'@PRINTWDOC ID[NNN] - Impossibile creare la direttiva 'XXX' La stampa non puo' essere eseguita.	Si	No
27003	'@PRINTWDOC ID[NNN] - Impossibile eseguire la copia del documento: 'XXX' in 'YYY' La stampa non puo' essere eseguita.	Si	No
27004	'@PRINTWDOC - Stampa del documento sul dispositivo 'XXX'. La stampa documento da Word e' fallita a causa dell'errore seguente: <Descrizione errore>	Si	
27006	'@PRINTWDOCFILE - Stampa del documento 'XXX' sul dispositivo 'YYY'. La stampa documento da Word e' fallita a causa dell'errore seguente: <Descrizione errore>	Si	
27007	'@PRINTWDOC ID[NNN] - Errore in acquisizione nome file: XXX	Si	

'@FINDWMARK

Errore	Descrizione	Scrittura SISERR.LOG	Visualizza se SUBARG=1
27001	Attenzione! Bookmark <XXXX> inesistente	No	

'@INSWMARK

Errore	Descrizione	Scrittura SISERR.LOG	Visualizza se SUBARG=1
27017	Inserimento bookmark 'XXX': le posizioni di inserimento richieste (inizio = NNN, fine = MMM) non sono coerenti. Il bookmark non viene inserito.	No	
27018	Inserimento bookmark 'XXX': i parametri HEADER[NN] e FOOTER[MM] sono alternativi, occorre indicarne uno solo. Il bookmark non viene inserito.	No	No
27030	Impossibile completare l'operazione. Codice: 800a1483 - Descrizione: Operazione non valida per la fine di una riga. TS=0/1 - Processi Word attivi=MMM - ID processi: XXX, YYY	Si	No
27031	Impossibile completare l'operazione. Codice: 800a11fd - Descrizione: Il metodo o la proprietà non è disponibile perché il documento è protetto dalle modifiche. TS=0/1 - Processi Word attivi=MMM - ID processi: XXX, YYY	Si	No
27032	Impossibile completare l'operazione. Codice: 800a17ec - Descrizione: La selezione è protetta. Impossibile modificarla TS=0/1 - Processi Word attivi=MMM - ID processi: XXX, YYY	Si	No

'@DELWMARK, '@SETWMARK, '@RESETWMARKCOLOR, '@INSWTEXT, '@REPLACEWTEXT, '@SETWHEADER, '@LOCKWDOCREFRESH

Errore	Descrizione	Scrittura SISERR.LOG	Visualizza se SUBARG=1
27030	Impossibile completare l'operazione. Codice: 800a1483 - Descrizione: Operazione non valida per la fine di una riga. TS=0/1 - Processi Word attivi=MMM - ID processi: XXX, YYY	Si	No
27031	Impossibile completare l'operazione. Codice: 800a11fd - Descrizione: Il metodo o la proprietà non è disponibile perché il documento è protetto dalle modifiche. TS=0/1 - Processi Word attivi=MMM - ID processi: XXX, YYY	Si	No
27032	Impossibile completare l'operazione. Codice: 800a17ec - Descrizione: La selezione è protetta. Impossibile modificarla TS=0/1 - Processi Word attivi=MMM - ID processi: XXX, YYY	Si	No

'@FINDWTEXT

Errore	Descrizione	Scrittura SISERR.LOG	Visualizza se SUBARG=1
27015	Attenzione! Testo 'XXX' non presente nella selezione richiesta (%d,%d)	No	No
27019	Ricerca del testo 'XXXX': i parametri HEADER[n] e FOOTER[m] sono alternativi, occorre indicarne uno solo. Il testo non viene cercato.	No	No

'@PROTECTWRANGE

Errore	Descrizione	Scrittura SISERR.LOG	Visualizza se SUBARG=1
27020	'@PROTECTWRANGE ID[NNN] - Protezione=1 - Le posizioni richieste (inizio = MMM, fine = PPP) non sono coerenti. L'operazione non vien eseguita.	Si	No
27021	Le posizioni richieste (inizio = MMM, fine = NNN) non sono coerenti con il documento (inizio doc = PPP, fine doc = RRR). L'operazione non vien eseguita..	Si	No

'@GETWRANGETEXT

Errore	Descrizione	Scrittura SISERR.LOG	Visualizza se SUBARG=1
27021	Le posizioni richieste (inizio = MMM, fine = NNN) non sono coerenti con il documento (inizio doc = PPP, fine doc = RRR). L'operazione non vien eseguita..	Si	No
27022	Le posizioni richieste (inizio = MMM, fine = NNN) non sono coerenti. L'operazione non vien eseguita.	Si	No

'@DELWRANGE, '@SETWRANGE

Errore	Descrizione	Scrittura SISERR.LOG	Visualizza se SUBARG=1
27021	Le posizioni richieste (inizio = MMM, fine = NNN) non sono coerenti con il documento (inizio doc = PPP, fine doc = RRR). L'operazione non vien eseguita..	Si	No
27022	Le posizioni richieste (inizio = MMM, fine = NNN) non sono coerenti. L'operazione non vien eseguita.	Si	No
27030	Impossibile completare l'operazione. Codice: 800a1483 - Descrizione: Operazione non valida per la fine di una riga. TS=0/1 - Processi Word attivi=MMM - ID processi: XXX, YYY	Si	No
27031	Impossibile completare l'operazione. Codice: 800a11fd - Descrizione: Il metodo o la proprietà non è disponibile perché il documento è protetto dalle modifiche. TS=0/1 - Processi Word attivi=MMM - ID processi: XXX, YYY	Si	No
27032	Impossibile completare l'operazione. Codice: 800a17ec - Descrizione: La selezione è protetta. Impossibile modificarla TS=0/1 - Processi Word attivi=MMM - ID processi: XXX, YYY	Si	No